

Secure Integration of Trusted Execution Environments into Distributed Applications

Gianluca Scopelliti

Dissertation presented in partial fulfillment
of the requirements for the degree of
Doctor of Engineering Science (PhD):
Computer Science

Supervisors:
Prof. dr. ir. W. Joosen
Prof. dr. J. T. Mühlberg
Dr. ir. C. Baumann

January 2025

SECURE INTEGRATION OF TRUSTED EXECUTION ENVIRONMENTS INTO DISTRIBUTED APPLICATIONS

Gianluca SCOPELLITI

Supervisors:

Prof. dr. ir. W. Joosen

Prof. dr. J. T. Mühlberg

Dr. ir. C. Baumann

Members of the

Examination Committee:

Prof. dr. ir. J. De Schutter, chair

Dr. E. Truyen

Prof. dr. ir. V. Naessens

Prof. dr. P. Papadimitratos
(KTH Sweden)

Prof. dr. S. Shinde
(ETH Zurich)

Dissertation presented in partial
fulfillment of the requirements for
the degree of Doctor of Engineering
Science (PhD): Computer Science

January 2025

© 2025 KU Leuven – Faculty of Engineering Science
Uitgegeven in eigen beheer, Gianluca Scopelliti, Leuven (Belgium)

Alle rechten voorbehouden. Niets uit deze uitgave mag worden vermenigvuldigd en/of openbaar gemaakt worden door middel van druk, fotokopie, microfilm, elektronisch of op welke andere wijze ook zonder voorafgaande schriftelijke toestemming van de uitgever.

All rights reserved. No part of the publication may be reproduced in any form by print, photoprint, microfilm, electronic or any other means without written permission from the publisher.

Preface

The formal guidelines for writing a Ph.D. thesis mention that the preface should summarize the general aim of the work. Well, to be short and concise, the aim of my work was to obtain a Ph.D. and get to travel and see new places, and I believe I can confidently say that I have now reached both goals. However, this experience was quite challenging and I could never have made it without the people whom I have met and worked with during these years. Every single one of them, in different ways, “nudged” me to the right direction, allowing me to eventually succeed. For this reason, I must take enough time and space to properly express them my gratitude.

Actually, this journey started one year before my Ph.D. began, when I did an Erasmus experience in Leuven during my master’s degree. In that occasion, the very first person I got in touch with was Frank Piessens. To be completely honest, I had no idea who Frank was at the time, but I saw from the KU Leuven website that he worked in security and had a friendly profile picture, so I had decided to contact him asking for some master thesis opportunities. This turned out to be the very beginning of my adventure. So I must say thank you, Frank, for being so friendly and for connecting me with all the other people in your group. Without you, my last four years would not have been possible.

During my Erasmus I met two very important people that made a huge impact on my subsequent work. The first person that I should mention is of course JT, my main supervisor during all these years. Thank you for being so encouraging. Your mere presence during the meetings always made me feel safe, knowing that you would always make sure that we would not go off-track. Thank you for that, and I will make sure to bring you some *nyponsoppa* every time I travel to Sweden. Next, I wanted to thank Fritz, first my daily advisor during my master thesis, then my colleague during most of my Ph.D., and now a very good friend. You have been a source of inspiration for me, and I hope our paths will cross again in the future.

Of course, I should also mention the people I have met since my Ph.D. started. Moving to Sweden in November 2020 during the pandemic was a real challenge, especially after seeing the sun setting at 3pm. Thankfully, though, I was in good company. First and foremost I should thank Christoph, my supervisor at Ericsson. I have really appreciated our relationship, which was more between equals rather than supervisor and student. I particularly enjoyed our brainstorming sessions to design the V2X self-revocation schemes. Second, I would like to express my sincere gratitude to my managers Hans and Ulf, who always did everything they could to make me feel as much comfortable as possible at Ericsson. You really made me feel part of the team. Third, I must mention the other Ph.D. students involved in the 5GhOSTS project: Gerald, Merve and Mykyta. It was a pleasure meeting and working with you all. In particular, I would like to thank Merve, with whom I shared the most during these years. Thank you for being such a good friend. I look forward to your defense and I will make sure to remind you of your remaining milestones. Finally, thank you to all the other people with whom I partially shared my working days: The Ericsson Research Security team in Kista, the DistriNet people and especially the TEE group at KU Leuven. It was great to share the office spaces and have coffee breaks with such nice people, and I was honestly impressed by your brilliance and talent.

But it is not only the colleagues that I should mention in this preface. Even though it was exciting to experience different environments during my Ph.D., moving between one place and another took a toll on my personal life. However, I was lucky enough to find amazing people in Leuven with whom I shared many good memories. It is not possible to mention everybody here, but I want to especially thank the people from my football and beach volleyball groups. It has been so fun to spend time with you all and, especially during this last year, I have really felt at home. I hope we can keep going for some more time. I must also mention my friends in Italy, from both Oulx and Torino. I regret that we could not see each other very often in these last years, but it was always great to spend time together every time I had the chance to come back. Of course, I need to particularly mention my family, and especially my parents. I know that it has not been easy for you to have me far away for such a long time, but thank you for understanding my needs and supporting my decisions. I really appreciate that.

Lastly, the formal guidelines also mention that I should thank whomever have awarded my doctoral scholarship. So, even though I should have asked for your consent before writing this, I just wanted to say thank you, European Union, for your support.

Gianluca

Popularized Abstract

Modern applications consist of several interconnected services deployed on shared infrastructures such as public clouds, where physical resources such as processor and memory are shared between multiple applications to optimize resource usage and reduce costs. With this paradigm, strong isolation techniques are needed to preserve the confidentiality and integrity of data and computations. Trusted Execution Environments (TEEs) tackle this problem by relying on hardware extensions to the underlying computing architecture to provide strong isolation and access control both in the main processor and in memory. Code and data running inside a TEE are then shielded from the rest of the system, and even privileged software such as the hypervisor is not allowed to access the secure environment, thereby significantly reducing the overall attack surface. In recent years, the community has adopted the term *confidential computing* to define TEE workloads.

In this Ph.D. dissertation, we analyze TEEs from a practical standpoint, focusing on how to securely and effectively integrate them into novel or existing applications and protocols. We study two use cases, namely smart homes and Vehicle-to-Everything (V2X) credential management, where TEEs can be utilized to enhance the security posture of the system while providing additional benefits compared to traditional approaches, such as increased efficiency and scalability. Furthermore, we raise the attention to non-obvious practical challenges of this technology: We investigate the interface between an application running in a TEE and the untrusted world, showing how attackers can manipulate the notion of time perceived inside the TEE to compromise the integrity of the application. Besides, we analyze the trust relationships between customers who want to leverage TEEs in the cloud and their providers, highlighting some inconsistencies with the TEE threat model. We conclude that this technology can become a cornerstone of modern applications, but its integration requires careful design choices from developers and administrators, as well as proper support from the infrastructure providers.

Gepopulariseerde Samenvatting

Moderne computertoepassingen bestaan uit onderling verbonden *services* die worden uitgerold op computers die gedeeld worden door meerdere gebruikers, zoals in de cloud. Door computers op deze manier te delen, kan het gebruik geoptimaliseerd en de kosten verlaagd worden. Dit model vereist echter robuuste beveiligingsoplossingen om vertrouwelijke gegevens te beschermen. Zogeheten Trusted Execution Environments (TEEs) beloven een oplossing door gebruik te maken speciale hardware voor sterke veiligheidsgaranties, zelfs wanneer gevoelige gegevens zich in het geheugen of de processor bevinden. TEEs schermen programma's namelijk af van de rest van het systeem terwijl ze worden uitgevoerd. Zelfs systeemsoftware, zoals de hypervisor of het besturingssysteem, heeft geen toegang tot de beveiligde omgeving. Hierdoor worden de mogelijkheden om de beveiliging te omzeilen aanzienlijk beperkt. Deze aanpak wordt sinds kort omschreven als *confidential computing*.

In dit proefschrift analyseren we TEEs vooral vanuit een praktisch perspectief. We onderzoeken hoe TEEs gebruikt kunnen worden in zowel bestaande als nieuwe toepassingen. We bestuderen twee scenario's waarbij TEEs kunnen worden ingezet om de beveiliging van het systeem te verbeteren: smart homes en certificatenbeheer bij Vehicle-to-Everything (V2X)-toepassingen. Daarnaast bekijken we of deze technologie, naast veiligheid, ook andere voordelen kan bieden, zoals verbeterde efficiëntie en schaalbaarheid. We besteden ook aandacht aan de praktische uitdagingen van deze technologie: we onderzoeken namelijk hoe een applicatie die binnen een TEE draait, interageert met het niet-beschermde deel van het systeem. We tonen aan hoe "tijd" binnen een TEE gemanipuleerd kan worden om de integriteit van applicaties te ondermijnen. Verder analyseren we ook de vertrouwensrelatie tussen de klant van een cloud-TEE en de cloudprovider, en bespreken we tekortkomingen van het huidige beveiligingsmodel van TEEs. Deze technologie heeft het potentieel een hoeksteen

te worden voor de beveiliging van moderne toepassingen. Het gebruik ervan vereist echter zorgvuldige keuzes van zowel ontwikkelaars als beheerders, evenals passende ondersteuning van cloudproviders.

Abstract

With the rise of cloud computing and the Internet of Things (IoT), modern applications have been revolutionized from monolithical units running on local premises to a multitude of smaller, interconnected services deployed on shared infrastructures used by multiple tenants. Sharing resources among multiple applications improves scalability and efficiency, reducing costs at the same time. However, it also poses novel security challenges compared to traditional systems.

Protecting the confidentiality, integrity and authenticity of data and computations is paramount to ensure the correct operation of an application. While there already exist established techniques to protect data at rest and in transit, protecting data while in use is still an open research area. Trusted Execution Environments (TEEs) tackle this problem by relying on hardware-based mechanisms to isolate code and data during execution, leveraging extensions to the underlying computing architecture to provide strong isolation and access control both in the main processor and in memory. Code and data running inside a TEE are then shielded from the rest of the system, and even privileged software such as the hypervisor is not allowed to access the secure environment, thereby significantly reducing the overall attack surface.

In this Ph.D. dissertation, we analyze TEEs from a practical standpoint, focusing on how to securely and effectively integrate them into novel or existing applications and protocols, taking into account possible shortcomings and limitations of this technology. First, we propose a framework to facilitate the development and management of distributed applications in shared, heterogeneous infrastructures. To do so, we employ process-based TEEs such as Intel Software Guard Extensions (SGX) and Sancus to minimize the attack surface, leveraging secure I/O capabilities to guarantee strong end-to-end integrity of application data. To showcase the effectiveness of our work, we develop and evaluate a prototype based on a smart home system.

As a second step, we study recent architectures such as AMD Secure Encrypted Virtualization (SEV)-Secure Nested Paging (SNP) and Intel Trust Domain Extensions (TDX), which allow running fully-fledged virtual machines inside a TEE. While this significantly reduces the development effort, it also causes an increased attack surface and additional operational complexity, since a virtual machine comprises several software components that are executed in sequence during boot. Thus, it is essential to verify each stage of the boot process to ensure that the virtual machine is set up correctly. However, our findings reveal that, in current cloud settings, users have limited knowledge about the software executing within the TEE, since cloud providers have significant control over the boot process of virtual machines and could potentially access sensitive user data stored inside the TEE. Therefore, it is essential to clarify trust relationships between cloud providers and customers when using TEEs in the cloud.

Third, we investigate the interactions between an application running within a TEE and the underlying untrusted environment. In particular, we examine the problem of getting a stable and reliable notion of time inside a TEE, assessing the capabilities of various architectures to provide a secure time source and evaluating common use cases and applications that rely on time. Then, we use this knowledge to propose a novel revocation scheme in the context of Vehicle-to-Everything (V2X) credential management, which relies on TEEs embedded inside vehicles. Our formal verification work shows that our scheme gives a guaranteed upper bound on revocation time, and our evaluation highlights the efficiency and scalability of our solution.

In summary, this dissertation shows how TEEs can be utilized to enhance the security of existing applications and design novel solutions that can provide additional benefits compared to traditional approaches. Furthermore, we also raise the attention to non-obvious practical challenges that arise when deploying TEE workloads to untrusted infrastructures such as public clouds.

Beknopte samenvatting

Met de opkomst van cloud computing en het Internet of Things is ook de architectuur van moderne applicaties ingrijpend veranderd. Waar deze vroeger bestonden uit monolithische programma's die op lokale systemen in eigen beheer draaiden, is er nu een duidelijke verschuiving naar een architectuur gebaseerd op kleinere, onderling verbonden *services*. Deze worden uitgerold op infrastructuur die gedeeld wordt met meerdere gebruikers. Het delen van resources met andere applicaties biedt namelijk diverse voordelen, zoals verbeterde schaalbaarheid, hogere efficiëntie en lagere kosten. Daartegenover staan echter ook nieuwe uitdagingen, met name op het gebied van cybersecurity.

Voor een correcte werking van een applicatie is het essentieel om de confidentialiteit, integriteit en authenticiteit van gegevens en programmacode te waarborgen. Hoewel er diverse technieken bestaan voor veilige opslag en verzending van gegevens, blijft de bescherming van gegevens terwijl ermee gerekend wordt een actief onderzoeksdomein. Trusted Execution Environments (TEEs) trachten dit probleem op te lossen door gebruik te maken van hardware-gebaseerde mechanismen. Deze mechanismen beschermen zowel de programmacode als de gegevens van een programma in uitvoering, of deze nu aanwezig zijn in de CPU of in het geheugen, en schermen deze dus af van andere programma's die binnen hetzelfde systeem worden uitgevoerd. TEEs bieden zelfs bescherming tegen geprivilegieerde software, zoals de hypervisor of het besturingssysteem, waardoor het risico op kwetsbaarheden aanzienlijk wordt verkleind.

In dit proefschrift analyseren we TEEs vooral vanuit een praktisch perspectief en onderzoeken we hoe deze veilig en effectief kunnen worden toegepast in zowel bestaande als nieuwe toepassingen, rekening houdend met de beperkingen van deze technologie. Allereerst stellen we een kader voor dat de ontwikkeling en het beheer van gedistribueerde toepassingen in gedeelde, heterogene omgevingen vereenvoudigt. Hiervoor maken we gebruik van procesgebaseerde TEEs, zoals Intel Software Guard Extensions (SGX) en Sancus, die de attack surface

aanzienlijk verkleinen en veilige I/O mogelijk maken. Dit biedt sterke end-to-end integriteitsgaranties. Om de effectiviteit van ons werk te demonstreren, ontwikkelen en evalueren we een prototype dat gebaseerd is op een smart home systeem.

In een tweede bijdrage bestuderen we meer recente architecturen, zoals AMD Secure Encrypted Virtualization (SEV)-Secure Nested Paging (SNP) en Intel Trust Domain Extensions (TDX), die het mogelijk maken volledige virtuele machines binnen een TEE uit te voeren. Hoewel deze technologieën het ontwikkelen van toepassingen voor een TEE vereenvoudigen, vergroten ze zowel de attack surface als de operationele complexiteit. Een virtuele machine bestaat immers uit meerdere softwarecomponenten die tijdens het opstartproces worden uitgevoerd en het is essentieel is om elke fase van dit proces zorgvuldig te verifiëren. Onze bevindingen tonen echter aan dat gebruikers in huidige cloudomgevingen beperkte kennis hebben van deze software. Bovendien hebben cloudproviders vaak aanzienlijke controle over hoe virtuele machines worden opgestart en kunnen ze mogelijk zelfs toegang krijgen tot gevoelige gebruikersgegevens die door de TEE worden beschermd. Het is daarom van cruciaal belang om de vertrouwensrelatie tussen cloudproviders en klanten te duidelijk te stellen bij het gebruik van TEEs in de cloud.

Als derde bijdrage onderzoeken we de interacties tussen een applicatie die binnen een TEE wordt uitgevoerd en het onderliggende, mogelijk gecompromitteerde systeem. We bestuderen in het bijzonder hoe een stabiel en betrouwbaar tijdsbegrip binnen een TEE kan worden gerealiseerd. Daarbij beoordelen we verschillende oplossingen en bespreken we veelvoorkomende toepassingen die afhankelijk zijn van tijd voor een correcte werking. Op basis van deze analyse ontwikkelen we een innovatieve methode voor de revocatie van digitale certificaten in Vehicle-to-Everything (V2X)-toepassingen, waarbij we TEEs gebruiken in de voertuigen. We tonen formeel aan dat ons voorstel een bovengrens voor de revocatietijd garandeert. Onze evaluatie bevestigt bovendien de efficiëntie en schaalbaarheid van deze aanpak.

Samengevat, dit proefschrift toont aan hoe TEEs kunnen worden gebruikt om enerzijds de beveiliging van bestaande applicaties te verbeteren en anderzijds om nieuwe, innovatieve oplossingen te ontwikkelen die bijkomende voordelen bieden in vergelijking met de meer traditionele oplossingen. Daarnaast hebben we ook aandacht voor de praktische uitdagingen die ontstaan bij het implementeren van TEE-toepassingen in minder veilige omgevingen zoals publieke clouds.

Contents

Popularized Abstract	iii
Gepopulariseerde Samenvatting	v
Abstract	vii
Beknopte samenvatting	ix
Contents	xi
List of Figures	xv
List of Tables	xvii
List of Abbreviations	xxi
1 Introduction	1
1.1 Core Concepts	3
1.2 Trusted Execution Environments	5
1.2.1 Hardware-based Isolation	6
1.2.2 Remote Attestation	9
1.2.3 Threat Model	10
1.3 Motivation and Contributions	12
1.3.1 Main Contributions	13
1.3.2 Other Contributions	17
1.4 Dissertation Outline	19
2 End-to-End Security for Distributed Event-Driven Enclave Applications on Heterogeneous TEEs	21
2.1 Introduction	23
2.2 Running Example, Infrastructure & Objectives	25

2.2.1	The Shared Infrastructure	27
2.2.2	Modules & Applications	29
2.2.3	Attacker Model	29
2.2.4	Security Objective	30
2.3	Authentic Execution of Distributed Applications	31
2.3.1	Underlying Architecture: TEEs	31
2.3.2	Mapping Source Modules to SMs	32
2.3.3	Untrusted Software on the Nodes	34
2.3.4	Physical Input and Output Channels	36
2.3.5	Deployment	38
2.3.6	Security Argument	39
2.3.7	Software Updates and Re-Deployment	40
2.4	Implementation	42
2.4.1	Sancus	43
2.4.2	Intel SGX	49
2.4.3	ARM TrustZone	53
2.4.4	Comparison: SMs on Different TEEs	58
2.4.5	Deployment	59
2.4.6	Attestation and Key Management	61
2.5	Evaluation	63
2.5.1	Smart Homes	63
2.5.2	Testing Environment	64
2.5.3	Performance Benchmarks	66
2.5.4	Adapting Burden	72
2.6	Discussion	75
2.6.1	Integrity versus Confidentiality	75
2.6.2	Availability	75
2.6.3	Hardware Attacks and Side Channels	76
2.7	Related Work	77
2.7.1	TEE Frameworks	77
2.7.2	Mutually Authenticated Enclaves	78
2.7.3	Secure I/O	78
2.8	Summary & Conclusions	81
3	Understanding Trust Relationships in Cloud-Based Confidential Computing	83
3.1	Introduction	86
3.2	Background	87
3.2.1	VM-based TEEs	87
3.2.2	Measured Boot and Attestation	88
3.3	Problem Statement	89
3.4	System Model	90
3.4.1	The Importance of Remote Attestation	91

3.4.2	Attestation Levels	92
3.4.3	Attestation Strategies	93
3.4.4	Partial Attestation	95
3.5	CVM Attestation in Public Clouds	96
3.6	Exploring the Cloud Landscape	97
3.6.1	Results	98
3.6.2	Discussion	99
3.7	Conclusions	101
4	About Time: On the Challenges of Temporal Guarantees in Untrusted Environments	103
4.1	Introduction	105
4.2	System Model	106
4.3	Notions of Time	106
4.4	Intel SGX	110
4.5	Other Architectures	113
4.6	Use Cases	114
4.6.1	Time-Based Policies	114
4.6.2	Rate Limiting and TOTP	115
4.6.3	Resource Accounting	116
4.6.4	Credential Expiration and DRM	117
4.7	Conclusions	117
5	Efficient and Timely Revocation of V2X Credentials	119
5.1	Introduction	121
5.2	Background	123
5.3	Related Work	125
5.4	Motivation and System Model	127
5.4.1	Vehicles and Trusted Components	128
5.4.2	The Vehicle Lifecycle in a V2X Network	129
5.4.3	Attacker Model	131
5.4.4	Objectives	132
5.5	Design	132
5.5.1	Properties	135
5.5.2	Extensions	136
5.6	Formal Verification	138
5.6.1	Introduction on TAMARIN	138
5.6.2	Model	138
5.6.3	Lemmas	140
5.7	Evaluation	142
5.7.1	Revocation Time	143
5.7.2	Size of PRL	147
5.7.3	Choice of T_v	149

5.8	Discussion	151
5.8.1	Qualitative Comparison with Related Work	151
5.8.2	Compatibility with Existing Protocols	153
5.8.3	Security Considerations	154
5.8.4	Privacy Discussion	154
5.9	Summary & Conclusions	155
6	Conclusion	157
6.1	Summary of Contributions	158
6.2	Discussion and Future Work	160
6.3	Concluding Remarks	164
A	Additional Resources for End-to-End Security for Distributed Event-Driven Enclave Applications on Heterogeneous TEEs	167
A.1	List of Assumptions and Requirements	167
A.1.1	Assumptions	168
A.1.2	Implementation Requirements	171
B	Additional Resources for Efficient and Timely Revocation of V2X Credentials	173
B.1	Flowcharts	173
B.2	Tamarin Models	175
B.2.1	Mapping our Design to TAMARIN	175
B.2.2	Trusted Time Source in TCs	176
B.3	Complete Simulation Results	178
B.4	Calculating Expected Size of the PRL	181
B.4.1	Utilized Probabilities	181
B.4.2	Markov Model	182
B.4.3	Calculating the Expected PRL Size	183
B.4.4	Probabilities and Expected Revocations	184
	Bibliography	187
	Curriculum Vitae	211
	List of Publications	213

List of Figures

1.1	Hardware and software stack in a modern computing system . . .	4
1.2	Main TEE architectures	6
1.3	Simplified RATS architecture	9
2.1	Example of a smart irrigation system	26
2.2	Hardware and software components of our running example . .	27
2.3	Pseudocode of our running example in a Python-like syntax . .	28
2.4	Pseudocode of the SetKey entry point	33
2.5	Pseudocode of the HandleInput entry point	33
2.6	Pseudocode of the generated output wrapper	33
2.7	Pseudocode of the Attest entry point	33
2.8	Memory layout of module M_{FloS1} in application A_{Flo}	35
2.9	A translation of module M_{FloS1} to C	44
2.10	A translation of module M_{FloS1} to Rust	50
2.11	Remote attestation process on ARM TrustZone with OP-TEE . . .	55
2.12	Excerpt from a deployment descriptor in YAML format	60
2.13	Setup of our <i>smart home</i> application	64
2.14	Average encryption timings on different TEEs	66
2.15	Control flow and timings of the smart home application	68
3.1	High-level attestation flow	91
3.2	Hierarchy of attestation levels for CVMs	92
3.3	Possible Man-in-the-Middle attack with an unverified guest OS . .	95
4.1	Visual representation of different time levels	108
5.1	Lifecycle of a V2X architecture	127
5.2	Timeline diagram showing relations between T_v and T_{eff}	135
5.3	Definition of the advance_time rule in TAMARIN	139
5.4	Definition of TAMARIN proof lemmas	142
5.5	Simulation results	145

5.6	Maximum PRL sizes in each baseline revocation scenario . . .	149
5.7	Evaluation of the impact of T_v on the V2X system	150
B.1	Flowcharts of the TC logic	174
B.2	Properties of our design with trusted time in TAMARIN	176
B.3	Simulation results for scenario A1.	179
B.4	Simulation results for scenario A2.	179
B.5	Simulation results for scenario B1.	180
B.6	Simulation results for scenario B2.	180
B.7	A graph of the Markov chain with exemplary parameters . . .	183
B.8	99th percentile PRL sizes for fixed $T_{prl} = 30$ and varying n . .	185
B.9	99th percentile PRL sizes for fixed $n = 800$ and varying T_{prl} . .	186

List of Tables

2.1	Comparison of TEE hardware features	58
2.2	End-to-end measurements of the smart home application	69
2.3	Average timings to perform a software update on different TEEs	71
2.4	Software TCB size in our evaluation scenario	73
3.1	AMD SEV-SNP offerings on AWS, Azure and GCP	97
4.1	Levels of time and the guarantees given by them	107
4.2	Intel SGX software projects and their use of time	112
5.1	Full list of lemmas proven with TAMARIN	141
A.1	Assumptions on Hardware	168
A.2	Assumptions on Attacker	169
A.3	Assumptions on Software	170
A.4	Assumptions on Infrastructure and Credentials	170
A.5	Requirements on Secure I/O and Driver Modules	171
A.6	Requirements on Infrastructure and Deployer	172
B.1	Full list of lemmas proven on the model with trusted time . . .	177
B.2	Scenarios evaluated in our simulation	178

List of Abbreviations

AWS	Amazon Web Services
CC	Confidential Computing
CCA	Confidential Compute Architecture
CCPA	California Consumer Privacy Act
CI	Continuous Integration
CRL	Certificate Revocation List
CSP	Cloud Service Provider
CVE	Common Vulnerabilities and Exposures
CVM	Confidential Virtual Machine
DAA	Direct Anonymous Attestation
DMA	Direct Memory Access
ECU	Electronic Control Unit
EDL	Enclave Definition Language
EDP	Enclave Development Platform
EPID	Enhanced Privacy ID
FHE	Fully Homomorphic Encryption
GCM	Galois-Counter Mode
GCP	Google Cloud Platform
GDPR	General Data Protection Regulation
HB	heartbeat
HUK	Hardware Unique Key
JWT	JSON Web Token
IAS	Intel Attestation Server

IMA	Integrity Measurements Architecture
IoT	Internet of Things
ITS	Intelligent Transport System
KDF	Key Derivation Function
LOC	line of code
MAC	Message Authentication Code
ME	Management Engine
MitM	Man-in-the-Middle
MMIO	Memory-Mapped I/O
MPC	Multi-Party Computation
MSR	Model-Specific Register
OBU	On-Board Unit
ORAM	Oblivious RAM
OSR	Order for Self-Revocation
OVMF	Open Virtual Machine Firmware
PKI	Public Key Infrastructure
PRL	Pending Revocation List
PSW	Platform Software
PTA	Pseudo Trusted Application
QE	Quoting Enclave
RA	Revocation Authority
RATS	Remote ATtestation ProcedureS
RoTM	Root of Trust for Measurement
RSU	Road-Side Unit
RTMR	Runtime Measurement Register
RTT	round-trip time
SCMS	Security Credential Management System
SDK	Software Development Kit
SEV	Secure Encrypted Virtualization
SGX	Software Guard Extensions
SM	Software Module
SMM	System Management Mode

SNP	Secure Nested Paging
SP	Secure Processor
SVSM	Secure VM Service Module
TC	Trusted Component
TCB	Trusted Computing Base
TD	Trusted Domain
TDISP	TEE Device Interface Security Protocol
TDX	Trust Domain Extensions
TEE	Trusted Execution Environment
TEEP	Trusted Execution Environment Provisioning
TOTP	Time-based One-Time Password
TPM	Trusted Platform Module
TSC	TimeStamp Counter
UTC	Coordinated Universal Time
V2I	Vehicle-to-Infrastructure
V2P	Vehicle-to-Pedestrian
V2V	Vehicle-to-Vehicle
V2X	Vehicle-to-Everything
VM	Virtual Machine
VMM	Virtual Machine Monitor
VMPL	Virtual Machine Privilege Level
vTPM	virtual TPM

Chapter 1

Introduction

In the last decade, computing systems have become an integral part of our society, with all kinds of devices being physically present in the world to perform a multitude of tasks, ranging from everyday activities like browsing social media and playing video games to sensitive operations such as storing bank and health data, controlling robots in industry systems, or driving vehicles. Furthermore, boosted by the advent of cloud computing and the Internet of Things (IoT), many applications that originally were composed of a single monolithical unit have been split into several parts that are geographically distributed and interconnected by a network, continuously exchanging data to provide their services. Making traditional systems and appliances intelligent does have several advantages that can, among others, improve quality of life and reduce costs and waste. In agriculture, for example, devices such as smart irrigation systems can dramatically improve efficiency by saving large amounts of water, which not only reduces costs but also has a positive impact on the environment [177]. Similarly, drones can be used to monitor the growth of crops and create prescription maps to optimize the application of products such as fertilizers or pesticides [104]. From another point of view, assisted and automated driving can increase the safety on the road and significantly reduce the number of accidents and fatalities [158].

However, with an increased number of connected devices and applications, several concerns have arisen. On the one hand, exposing services and appliances on the Internet has made it much easier for attackers to breach into them, which has led to several incidents in the past: For example, in 2016 the Mirai botnet caused the largest distributed denial of service attack ever recorded, thanks to a malware that spread over thousands of vulnerable IoT devices connected to

the Internet [22]. More recently, in 2021, the Colonial Pipeline oil company was victim of a ransomware attack that caused the disruption of the fuel supply chain in the United States for several days, due to an insecure VPN interface [34]. On the other hand, applications might process sensitive information such as health data or bank details, which raises some interesting privacy challenges: Guidelines like the NIST cybersecurity framework [160] and regulations such as the General Data Protection Regulation (GDPR) in Europe and the California Consumer Privacy Act (CCPA) in the United States have been adopted to ensure that companies comply with data protection principles to prevent leaks of users' personal information. Nevertheless, the number of data breaches in the last years still shows a rising trend¹. To deal with this increased attack surface, several defense mechanisms have been proposed over the years, some of which have been adopted as standard countermeasure in all systems. Cryptographic techniques such as symmetric and asymmetric encryption are now the standard way to protect the confidentiality and integrity of data, both at rest (e.g., when stored on a hard drive) and in transit (e.g., during communication between two services). Besides, a mechanism to manage identities and certificates, such as a Public Key Infrastructure (PKI) [139], can provide additional authenticity guarantees, e.g., to make sure that a user is communicating with a legitimate server. Instead, protecting data "in use", i.e., while being processed within a device, is still an open research area that has gained considerable interest in recent years, as workloads are being offloaded more and more from local, trusted machines to remote servers managed by third parties and whose resources are shared among multiple processes.

In this Ph.D. thesis, we focus on a specific solution called Trusted Execution Environments (TEEs) [182], a hardware-based mechanism that allows the processor to isolate portions of code and data during execution on a system, preventing even privileged code from accessing them. Despite being a relatively new technology, TEEs are getting a lot of traction in recent years, especially since the release of Intel Software Guard Extensions (SGX) in 2015 [143] that made this technology available in commodity x86 processors. Nowadays, TEEs are more popular in the context of cloud computing where physical resources are shared among multiple tenants and strong workload isolation is needed. Besides, TEEs can shield data even from the infrastructure provider, which allows moving applications to the cloud while still retaining the privacy of user data, which is crucial for highly sensitive tasks. This paradigm is known as *confidential computing*, and it is the main focus of recent architectures like AMD Secure Encrypted Virtualization (SEV)-Secure Nested Paging (SNP) [15] and Intel Trust Domain Extensions (TDX) [108], which became available in 2021 and 2023, respectively. TEEs can provide additional security guarantees to

¹<https://www.statista.com/statistics/273550/data-breaches-recorded-in-the-united-states-by-number-of-breaches-and-records-exposed/>

existing applications and enable novel use cases and protocols that were not possible before [13].

However, integrating TEEs into distributed applications poses several technical and operational challenges. Indeed, running an application inside a TEE requires careful redesign and some code changes to isolate the sensitive code and data that need to be protected. Even though recent TEEs such as AMD SEV-SNP and Intel TDX can run unmodified binaries, developers might still need to make some adjustments to avoid using resources that may still be influenced by attackers, either directly or indirectly. Besides, there must be a mechanism in place to actually *verify* that the application is running in such an isolated environment without being compromised, so that users can *trust* that any data exchanged with the application will be not accessible by unauthorized parties. This process is called *remote attestation* and, while it is a feature offered by all TEEs, it requires some development effort and expertise to implement it correctly. Remote attestation can also be mediated by third party services or even cloud providers, though this requires an in-depth analysis of the threat model and the trust relationships between different entities.

In this dissertation, we advance the state of the art in TEE research on two different dimensions: First, we demonstrate how integrating trusted computing technologies into common protocols and applications can provide several advantages in terms of security, and in some cases even increased efficiency and scalability compared to traditional approaches. To do so, we showcase two novel use cases in the context of distributed IoT and Vehicle-to-Everything (V2X) settings. Second, we uncover some operational challenges and limitations that should be taken into account when using TEEs in practice. In particular, we discuss how getting a stable notion of time inside a TEE is a non-trivial task, and we shed some light on trust relationships between tenants and cloud providers when deploying confidential computing solutions to public clouds.

1.1 Core Concepts

In this section, we provide a short summary of basic principles of computing and security that are relevant for this dissertation. Here, we do not aim to be comprehensive, and we make a few simplifications for the sake of brevity.

Fig. 1.1 shows the main hardware and software layers and components of a modern computing system. At the bottom, the hardware layer includes the main CPU and firmware, memory and disks, network interfaces, and peripheral devices such as clock, co-processors, GPU, and so on. Here, we may also find security modules such as the Trusted Platform Module (TPM) [201]. Going up there is

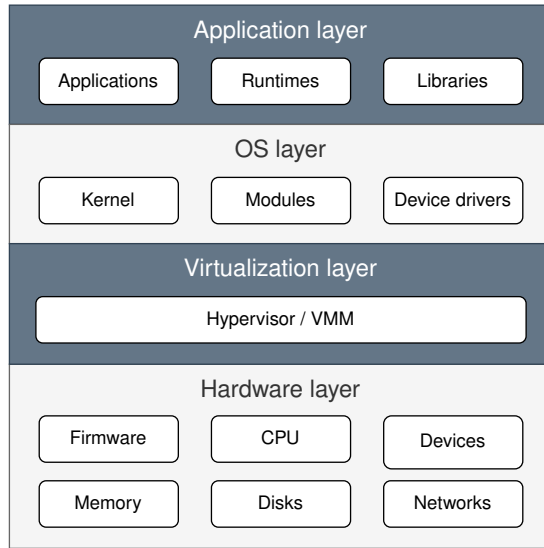


Figure 1.1: Overview of the hardware and software stack in a modern computing system. The security of each layer depends on the layers below, meaning that a compromise at a certain layer might break the confidentiality and integrity not only in that layer, but also all the layers above.

the virtualization layer, where a hypervisor or Virtual Machine Monitor (VMM) is responsible for managing one or more Virtual Machines (VMs). Physical resources such as CPU cores and memory are then *virtualized* and can be assigned to multiple VMs, enabling concurrency. Note that the virtualization layer is optional, though it is widely used in cloud settings to run multiple workloads on the same physical node, effectively optimizing resource utilization and reducing costs. Next there is the OS layer, which includes components such as the kernel, kernel modules and device drivers. This layer is responsible for bridging the gap between applications and the underlying hardware resources (which may be virtualized if running within a VM), and for providing isolation, communication and scheduling mechanisms. Lastly, the application layer contains applications, libraries and runtimes that run the actual workloads.

In a traditional system, the security of a certain layer is enforced by the layer below, providing mechanisms such as resource separation and virtualization to isolate different applications (enforced by the OS layer) and VMs (enforced by the virtualization layer). This also means that lower layers in the stack have full control on the upper layers and can access all their code and data. Therefore, a compromise at a certain layer might break the confidentiality and integrity both

in that layer and all the ones above. To this end, the term Trusted Computing Base (TCB) is defined as the sum of all hardware and software components that are critical to guarantee the confidentiality and integrity of a certain application. To build robust systems, it is therefore important to keep the TCB as small as possible.

1.2 Trusted Execution Environments

With the term Trusted Execution Environment we refer to a set of *hardware* extensions that can be used to protect an application while executing within an (untrusted) system. While there are technical differences between different architectures, all TEEs aim to provide two main features:

1. *Hardware-based isolation* of logical resources to give confidentiality and integrity guarantees to application code and data. This is enforced in various ways depending on the specific TEE design, but in general it involves ISA extensions to provide instructions for enabling/disabling protections and an access control mechanism to prevent illegal access to protected memory and CPU registers. The latter can be purely logical (e.g., allow or deny access based on the current program counter), cryptographic (e.g., encrypt and authenticate TEE memory), or both. Some architectures may also bring additional elements to provide e.g., some form of protection against side channels and physical attacks.
2. *Remote attestation*, a procedure to verify that an application is indeed running within a TEE as expected. Here, a remote *verifier*, which can be the application owner, can obtain a piece of *evidence* containing information about the platform itself, such as firmware and microcode version, as well as information about the running workload, such as a measurement of the memory layout of the application at boot time that can be compared to good, known reference values. It is important to protect the integrity and authenticity of the evidence using some cryptographic function. In most cases, a digital signature made over the evidence by the TEE itself gives proof of authenticity and is verified through a public key infrastructure managed by the TEE manufacturer. The information included in the evidence, instead, is used to determine if the workload is running on an up-to-date platform and has not been tampered with during the provisioning phase.

Together, these two features can give strong confidentiality, integrity and authenticity guarantees to an application running within a TEE. Other

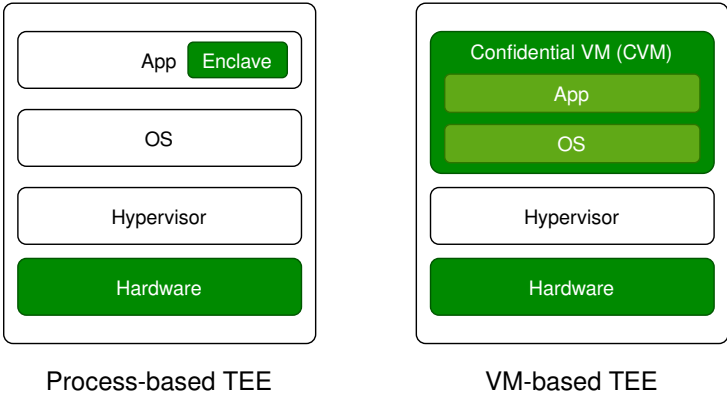


Figure 1.2: Main TEE architectures. The colored boxes represent trusted elements that are part of the TCB. Software components, as well as some parameters related to the TEE hardware such as the firmware version, can be verified via remote attestation.

mechanisms offered by some TEEs can additionally provide some support for secure storage (e.g., *data sealing*) and secure I/O. In contrast, availability is typically out of scope of TEE designs. Even though some academic proposals do exist to provide some form of trusted scheduling [11], commercial TEEs do not give any guarantees about the execution of applications, which is still managed via traditional means by the untrusted operating system or hypervisor.

Another important property of TEEs is a significant reduction of the TCB, since large software components such the host operating system and the hypervisor are kept outside of the secure environment, thus leaving a much smaller attack surface. This “breaks” the layered model of traditional systems (cf. Sect. 1.1), allowing to isolate core workloads from the rest of the software stack. Besides, part of the TCB, such as application code and TEE version, can be verified using remote attestation to ensure that they are in an expected state, giving additional guarantees about the current state of the workload. Here, it is worth mentioning that the entire attestation infrastructure is also included in the TCB.

1.2.1 Hardware-based Isolation

Existing TEEs can be divided in two main categories: (1) *Process-based*, where the isolation happens at the application layer, while the underlying operating

system remains untrusted, and (2) *VM-based*, where an entire virtual machine is protected, including guest firmware and operating system, while the host hypervisor is untrusted. Both architectures are illustrated in Fig. 1.2 and discussed in more detail below.

Process-based TEEs Process-based TEEs allow to isolate a single application, or part of it, inside an isolated environment that is commonly known as *enclave*. Enclaves are loaded in a protected memory region that is inaccessible by neither the “untrusted” application nor the operating system. Instead, they have a well-defined entry point that can be called from the outside and used to request some services, such as executing a function or storing some data in secure memory. Likewise, enclave code can call untrusted code to, e.g., execute system calls. In both cases, the processor performs a context switch to save or restore enclave state. The lifecycle of enclaves is fully managed by the untrusted application, which is responsible for calling the CPU instructions to create a new empty enclave, load pages in memory, initialize the enclave, and finally calling its functions.

In the industry, the most popular process-based TEE is Intel SGX [143], available on high-end x86 processors. On ARM devices, TrustZone [12] allows partitioning memory into a “normal” and “secure” world. While not being a TEE by itself, it can be paired with a trusted OS such as OP-TEE [131] to provide enclave-like execution environments in the secure world. Finally, although many academic solutions exist, in this dissertation we only focus on Sancus [161], a TEE designed for low-end microprocessors based on the OpenMSP430 architecture.

Process-based TEEs require applications to be partitioned into a secure and non-secure component. To do so, most TEEs provide a Software Development Kit (SDK) to manage the enclave lifecycle, call enclave functions, handle system calls and data transfer between secure and non-secure code, and perform remote attestation. This gives high flexibility and allows isolating sensitive code from the rest of the (possibly big) codebase, which helps minimizing the TCB and thus the attack surface. At the same time, however, it requires a significant development effort to carefully design enclave applications, locate and isolate sensitive functions, manage and sanitize data that goes in and out of the enclave. This can be prohibitively expensive, especially when redesigning existing (legacy) applications that consist of thousands or millions lines of code, and it also requires additional expertise in application developers. Several solutions exist to mitigate these issues, such as frameworks like Open Enclave [75] that facilitate the development of enclave applications by abstracting the TEE layer away, or library OSes and runtimes like Graphene-SGX [49] (now called Gramine) that provide a middleware layer offering OS-like functionalities inside the enclave.

In the latter case, it is sometimes possible to port existing applications to enclaves with little to no changes to the codebase, which significantly reduces the development effort but at the same time it increases the TCB.

VM-based TEEs Given the complexity and the development effort to port applications to enclaves, coupled with a rising need to protect and isolate sensitive VM workloads in the cloud, recent TEE architectures take a different approach, offering isolation mechanisms not at the application but at the virtualization layer. In this scenario, entire VMs are loaded into secure memory, which includes guest firmware (not shown in Fig. 1.2), kernel, and applications. In the industry, this functionality is offered by AMD SEV-SNP [15], Intel TDX [108], and ARM Confidential Compute Architecture (CCA) [27].

VM-based TEEs do not require any changes to applications because there is no TEE boundary between the application layer and the guest operating system. Thus, applications can execute system calls as usual, without incurring a context switch to go in or out of the TEE. Not only does this greatly reduce the burden to port applications to TEEs, but it can also improve performance in some cases. However, this comes at the cost of a significantly bigger TCB compared to enclaves.

Even though no changes are needed in the application layer, the guest VM still needs to be TEE-aware in order to execute, e.g., to properly configure page metadata to select which pages should be encrypted and which should be shared with the hypervisor. Depending on the case, these changes can be either implemented in the guest firmware or in the guest operating system. In the open-source community, support for AMD SEV-SNP and Intel TDX is in active development and some patches have already been upstreamed in the Linux kernel, as well as in QEMU and Open Virtual Machine Firmware (OVMF). Nevertheless, it is also possible to run “legacy” (i.e., non-TEE-aware) VMs by adding a privileged *paravisor* layer inside the VM responsible for the correct configuration of the workload. This is, for example, supported in AMD SEV-SNP via the Linux Secure VM Service Module (SVSM) [53]. Apart from compatibility support for legacy workloads, a paravisor can also enable additional services such as a virtual TPM (vTPM) [156].

In the cloud, VM-based TEEs can offer strong hardware-based workload isolation, which can be used either as a defense-in-depth approach to isolate different tenants (together with hypervisor-based isolation), or as a mechanism to prevent unauthorized access to a tenant’s sensitive data by a malicious cloud provider. In this latter scenario, workloads running in VM-based TEEs are commonly called Confidential Virtual Machines (CVMs).

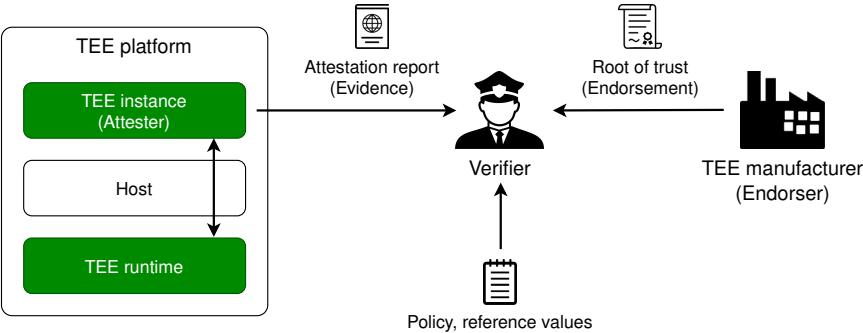


Figure 1.3: Simplified version of the RATS architecture [38] applied to TEEs. The colored boxes represent the elements whose trustworthiness is assessed by the verifier when checking the attestation report. Verifier, policy and reference values are proper RATS terms, whereas for other relevant components a mapping to the official terminology is reported between brackets.

1.2.2 Remote Attestation

Even though each TEE implements remote attestation differently, the key concepts are common to all technologies and have been standardized in the IETF RFC 9334 under the Remote ATtestation ProcedureS (RATS) architecture [38]. Fig. 1.3 shows a simplified version of this architecture.

Attestation is typically initiated by the *verifier*, who requests a report (or *evidence* in RATS terms) from the TEE instance (*attester*). Such a report is retrieved from the TEE runtime, e.g., the Quoting Enclave (QE) in Intel SGX and TDX, or the AMD Secure Processor (SP) in SEV-SNP. Freshness can be guaranteed by attaching some custom data to the report, such as a nonce or a timestamp, which may be provided by the verifier itself. To verify the authenticity of the report, the verifier has to retrieve the root of trust (*endorsement*) from the TEE manufacturer (*endorser*). This root of trust can vary according to the TEE and attestation scheme: In recent TEEs, the root of trust is a PKI root certificate that can be used to verify the signature and certificate chain of the report. Older or academic TEEs use different techniques, such as the Enhanced Privacy ID (EPID) scheme in Intel SGX [60] or a symmetric key derivation in Sancus. After verifying that the report is authentic, i.e., coming from a legitimate TEE, the verifier checks its content against some predefined policy and reference values to make sure that the TEE is in the expected state, which involves both the platform and the instance itself. Regarding the former, the verifier checks if the TEE firmware and runtime are up-to-date, and if the host has the desired features (e.g., some BIOS settings

are enabled or disabled). Concerning the latter, instead, the verifier checks the launch policy, which contains information such as whether debugging is enabled or not or if instance migration is allowed, and finally compares the *launch measurement* against its reference value. This measurement consists of a hash over the initial memory layout of the TEE instance and may include additional information such as page metadata. Successfully verifying the launch measurement gives the verifier a guarantee that the TEE instance is running the expected code. At this point, the attestation process is complete and, if all checks are successful, the TEE instance can be trusted.

It is important to note that the attestation report only reflects the *initial state* of the running TEE instance, i.e., the initial memory layout and parameters. For VM-based TEEs, as will be clear later on in Chapter 3, only the guest firmware is reflected in the initial measurement, thus the verifier needs to employ additional techniques to “extend” this measurement to upper layers such as the guest OS and applications. Yet, measuring binary files only guarantees integrity at boot time, but cannot detect runtime attacks such as memory corruption. For detecting this class of attacks, TEE attestation is out of scope and other techniques such as static/dynamic analysis and control-flow attestation should be used instead [8, 189].

1.2.3 Threat Model

To better understand the concepts of this dissertation, it is important to give a general overview of the TEE threat model. A more detailed discussion of assumptions and attacker capabilities is then provided separately for each contribution, in the respective chapters (cf. Sect. 1.4).

Isolation As mentioned above, TEEs give confidentiality, integrity and authenticity of code and data running within them. This is achieved by means of hardware extensions to enforce isolation, software libraries, tools and SDKs for the development and management of TEE applications, and an attestation infrastructure that gives the possibility to verify security claims. These elements are provided by the TEE manufacturer and are thus *trusted* by assumption (although components that are open-source can be manually reviewed). Furthermore, we also assume that all cryptography is secure and cannot be broken, including encryption algorithms, hashes, digital signatures and public key infrastructure. Instead, all other software running outside of the TEE such as host hypervisor, host operating system and other applications, along with actors such as the infrastructure provider, local tenants in the system and remote users are by design untrusted and possibly malicious.

In this model, TEEs can protect the confidentiality and integrity of applications even when the underlying infrastructure is fully compromised, but cannot help if the application itself is vulnerable. Attackers that are able to exploit a bug in the code can easily break TEE isolation and access or modify data. Similarly, an adversary might try to exploit the interface between the code running inside the TEE and the untrusted world, e.g., sending malicious inputs or modifying outputs. In both cases, it is often up to the software developer to prevent such attacks from happening, e.g., by keeping the TCB small and sanitizing interfaces.

Attacks on TEEs Despite the isolation guarantees provided by TEEs, there are some attacks that current technologies may be vulnerable to. First, even though TEEs protect memory from unauthorized access, a privileged adversary can observe access patterns and infer secret data via side channels [137, 45, 190]. Since TEEs assume a fully compromised host, attackers can leverage techniques to precisely control the execution of TEE applications even at instruction-level granularity (i.e., *single-stepping*) [226, 207, 223]. Side-channel exploitation on TEEs has been repeatedly demonstrated in the past, especially on Intel SGX [210]. Recently, techniques such as AEX-Notify have been implemented to mitigate single-stepping on enclaves [57]; Such mechanisms, while not addressing the underlying root cause of side channels, can make it much harder to perform this class of attacks. However, motivated attackers might still be able to succeed, as demonstrated by a recent attack on Intel TDX [222]. In general, side-channel attacks are considered out of scope by current commercial TEEs and should be mitigated by means of software mechanisms such as constant-time programming [47] and/or oblivious computation [6, 179].

Second, attackers might be able to perform physical attacks to induce faults during some sensitive computation and break the confidentiality and/or integrity of TEE applications [155, 50, 133]. As with side channels, software-based mitigations might also be used here to deal with these attacks [220], combined with hardware-based mechanisms to restrict the usage of certain features such as the voltage scaling interface.

Third, the TEE implementation itself may present design flaws and bugs that can break the confidentiality and integrity of applications. This has been shown by researchers on several occasions [152, 54, 61, 181, 180, 124, 210]. It is worth mentioning that most of these vulnerabilities do not apply to all TEEs in general but are specific to a particular architecture. While most of them have been already patched and can be mitigated by running applications on up-to-date platforms (which can be verified using remote attestation), it is also possible that new vulnerabilities are discovered in the future. If attackers are able to

exploit a zero-day vulnerability on a particular architecture, they might be able to break the isolation provided by the TEE. Unfortunately, there is little that administrators and application developers can do to deal with these attacks, apart from applying traditional defense-in-depth techniques and reducing the attack surface as much as possible.

All the aspects mentioned above are crucial when deploying a TEE workload in the real world, but they are out of the scope of this dissertation. Nevertheless, we briefly touch upon them in our concluding chapter (cf. Sect. 6.2).

1.3 Motivation and Contributions

Recent trends in cloud computing shifted the paradigm from running workloads on local premises to shared infrastructures where multiple tenants share the same physical resources for higher scalability and lower costs. In addition, with the Internet of Things and edge computing, some trusted services need to be offloaded to devices that may be in control of a malicious user, like a car or a smartphone. Here, TEEs can play an important role to isolate sensitive workloads even on compromised nodes, and indeed this technology is becoming more and more ubiquitous in many different fields and use cases. Compared to other approaches such as Fully Homomorphic Encryption (FHE) [3] and Multi-Party Computation (MPC) [132], which are active areas of research but do not still have many real-world applications, TEEs are a more accessible solution that can be integrated into existing workloads with relatively low effort and performance overhead.

However, there are some operational aspects that should be considered when using TEEs, such as how to securely provision and attest workloads, how to protect and authenticate communication between different services, and how to deal with peripherals such as sensors and actuators. Besides, there are differences between running an application on a local, closed system and running it within a TEE on a third-party infrastructure such as a public cloud. For example, applications that rely on system resources, such as a clock, may become vulnerable to attacks if running on a shared system, because a privileged adversary can control these resources and may thus influence the behavior of the application. Therefore, moving workloads inside TEEs requires careful study and, possibly, some application-level changes. This dissertation delves deeper into these challenges, presenting solutions to securely integrating TEEs into new use cases and acknowledging limitations of this technology.

Below, we describe the contributions of this Ph.D. trajectory. All the code developed during our work has been open-sourced for further research

and reproducibility of results. This involves proof of concepts, frameworks, prototypes and tools. For each contribution summarized below, we link to the public codebase (if any).

1.3.1 Main Contributions

This dissertation revolves around four main publications that I contributed to during my Ph.D. trajectory, either as first or second author. A summary of the core concepts of each work and personal contributions is provided below, while the full texts are reported in the next chapters. Note that these contributions do not follow a chronological order, e.g., date of publication, but rather they are ordered to best fit the overall storyline described in this introductory chapter.

End-to-End Security for Distributed Event-Driven Enclave Applications on Heterogeneous TEEs As a first contribution, Chapter 2 describes a framework to simplify the development and management of distributed applications on shared infrastructures. This work focuses on process-based TEEs to keep the TCB small, and allows developing heterogeneous applications based on Intel SGX, ARM TrustZone and Sancus. We design an event-driven programming model where each service (or *module*) defines a set of inputs and outputs that consume or produce *events*, and inputs and outputs of different services can be connected together to provide various functionalities. Unlike related work such as Open Enclave [75] and Google Asylo [90], our framework automates the deployment and attestation of each module and makes sure that each link between an output and an input is mutually authenticated with dedicated keys. Besides, we are the first to leverage Sancus’ secure I/O functionality to also protect the communication between a service and a peripheral such as a sensor or an actuator. In this way, we achieve a notion of *end-to-end security* with a strong focus on the integrity of computations and data. Our framework aims to give a guarantee that, if a physical or logical output is observed in the application, then its execution can be “traced back” to the actual piece of code and sequence of events that originated such output, including any potential physical inputs. We evaluate our work on a prototype based on a smart home system: Our results show that the performance overhead of our solution does not affect user experience for near real-time applications, while the effort to develop such a prototype in terms of lines of code is very small.

This work resulted in thousands of lines of code partially written from scratch mainly in C/C++, Rust and Python, along with other adaptations to existing repositories. Besides, we made extensive use of Docker and Continuous Integration (CI) pipelines to streamline the development, testing and evaluation

of our framework. All code, scripts and examples can be found in the AuthenticExecution organization on GitHub².

I am the first author of this paper, and my personal contributions involved a revised design and re-implementation of the framework based on the earlier workshop publication [162], as well as the full implementation of all the new code except for the ARM TrustZone part that was developed in collaboration with Sepideh Pouyanrad. In addition, I was also responsible for the complete evaluation of the framework and a substantial part of the writing process.

G. Scopelliti, S. Pouyanrad, J. Noorman, F. Alder, C. Baumann, F. Piessens and J. T. Mühlberg. “End-to-End Security for Distributed Event-Driven Enclave Applications on Heterogeneous TEEs”. In: *ACM Transactions on Privacy and Security (TOPS)*. Volume 26, Issue 3, pp. 1–46. 2023

Understanding Trust Relationships in Cloud-Based Confidential Computing

Chapter 3 presents our second contribution which, as opposed to the first, focuses on VM-based TEEs as the emerging technology in recent years for cloud-based workloads. Indeed, the reduced effort to run applications inside CVMs, coupled with new technologies and protocols to get exclusive access to accelerators such as GPUs [164], has dramatically increased the popularity of TEEs in the cloud. Nowadays, all the biggest cloud providers offer solutions to run confidential workloads on demand by leveraging AMD SEV-SNP and Intel TDX, and many companies are now selling their solution based on CVMs, such as confidential Kubernetes clusters or confidential AI applications. Our research analyzes this trend and goes into detail on the provisioning and attestation process of CVMs. As mentioned in Sect. 1.2, bootstrapping a CVM is a staged process involving several software layers, but only the very first piece of code (i.e., the guest firmware) is measured by the TEE. Therefore, in our work we discuss existing techniques to extend this chain of measurements, and we propose a categorization of *attestation levels* that indicate how much of the CVM boot process is attested by a verifier, ranging from zero (no attestation) to four (the entire boot chain is measured and verified). This categorization is then utilized in our evaluation, where we assess the three major cloud providers, i.e., Amazon Web Services (AWS), Microsoft Azure and Google Cloud Platform (GCP) and we assign a score indicating the attestation level that can be achieved on their infrastructure. However, we found that cloud providers have some influence on the provisioning of a CVM and, for example, customers cannot use their own

²<https://github.com/AuthenticExecution>

guest firmware but are limited to the firmware offered by the cloud provider, which may be closed source and thus not verifiable. This raises some questions about the trust relationships between cloud providers and their customers since, considering the threat model of TEEs, the infrastructure provider is untrusted by design (cf. Sect. 1.2.3). Therefore, our evaluation considers this scenario and highlights the differences between *nominal* attestation level, i.e., what is theoretically attestable in a cloud, and *trustworthy* attestation level, i.e., what level is reachable if we assume that the cloud provider is malicious. Our results highlight a gap between the TEE threat model and current cloud offerings. This paper received the “Best Paper Award” at the SysTEX 2024 workshop.

I am the main author of this paper and I carried out the entire work. My supervisors Christoph Baumann and Jan Tobias Mühlberg assisted me during the evaluation and contributed to the writing process.

G. Scopelliti, C. Baumann and J. T. Mühlberg. “Understanding Trust Relationships in Cloud-Based Confidential Computing”. In: *7th Workshop on System Software for Trusted Execution (SysTEX’24)*. 2024

About Time: On the Challenges of Temporal Guarantees in Untrusted Environments

As a third contribution of this dissertation, Chapter 4 discusses the challenges to get a reliable notion of time inside a TEE. Indeed, as previously mentioned, peripherals such as a clock may be influenced by a privileged adversary in the system. In our study, we provide a classification of incremental *time levels*, which indicate how an attacker can influence the time retrieved within a TEE. These levels range from zero, meaning that an attacker has full control over the time source and the communication channel, to four, meaning that an attacker cannot influence time at all. With this classification in mind, we analyze commercial TEEs and assess the time level that each of them can guarantee. For example, in process-based TEE like Intel SGX where the operating system is untrusted by design, system calls like `time()` are obviously insecure because a privileged adversary can arbitrarily modify the return value. Yet, our findings show that many SGX frameworks and libraries provide timing information via the operating system, without even giving a warning that the returned values are insecure. In contrast, VM-based TEEs give more guarantees on time as it is possible to access the TimeStamp Counter (TSC) directly from the CPU to get some indication of time passing, but an attacker might still be able to influence the usage of time, e.g., by interrupting the execution of a running VM right after a timestamp is retrieved. In the last part of this research, we analyze several use cases that require a notion of time, such as rate limiting and credential expiration. For each use case, we determine the

minimum time level that guarantees the integrity of computations, and we assess whether such use cases are acceptable with the guarantees provided by certain TEEs or if they may require additional consideration.

Fritz Alder is the main author of this paper. I mainly contributed to the design of the time levels, the evaluation of the various TEEs based on public specifications, and part of the writing.

F. Alder, **G. Scopelliti**, J. Van Bulck and J. T. Mühlberg. “About Time: On the Challenges of Temporal Guarantees in Untrusted Environments”. In: *6th Workshop on System Software for Trusted Execution (SysTEX’23)*. 2023

Efficient and Timely Revocation of V2X Credentials Finally, our last contribution, presented in Chapter 5, takes into account the aforementioned time challenges and designs a secure revocation mechanism for V2X credentials. In V2X, vehicles communicate with each other and other peers such as pedestrians and traffic lights to exchange information about themselves and their surroundings. This information is then used in various ways, such as creating dynamic maps or feeding decision algorithms for assisted or autonomous driving. To protect such sensitive communication, industry standards propose the use of digital signatures to guarantee the integrity and the authenticity of messages, coupled with a public key infrastructure to manage identities and credentials [83, 42]. Notably, V2X relies on short-lived *pseudonym credentials* to protect the privacy of vehicle users, to make it more challenging for passive observers to track them. Besides, misbehavior detection mechanisms are in place to detect malicious activity, such that when a misbehaving vehicle is detected its credentials are revoked to preserve the safety of the network. However, revoking pseudonym credentials is not a trivial task, as vehicles might be associated to multiple credentials and therefore traditional mechanisms based on revocation lists are inefficient, while solutions based on passive revocation, i.e., letting credentials expire without actively revoking them, present some security concerns [230]. As such, academic solutions have been trying to leverage TEEs inside vehicles to enhance certificate management and make revocation more efficient and scalable. A novel revocation technique based on *self-revocation*, proposed by Förster et al. [84] and picked up in subsequent works [218, 67, 122], leverages revocation messages that are sent directly to the revoked vehicle, which is asked to delete its credentials. Since credentials are managed by a trusted (and attestable) service running in a TEE, we can ensure that, whenever a revocation message is received by this service, revocation will occur as expected. In case these messages are not delivered as intended, perhaps due to network

interruptions or malicious activity, Förster et al. propose the use of *heartbeats* and timeouts as a way to detect misbehavior and perform revocation automatically after a certain time has passed since the last received message. However, as we discussed above, managing time in TEEs is problematic and it is not always possible to compute timeouts securely. Taking this into account, we design our own version of the self-revocation scheme that does not rely on a physical time source in TEEs but relies on a logical time, or “epochs”, that is distributed over the network. We formally verify our design using the Tamarin prover [144], proving that we can guarantee revocation with a predefined upper bound on revocation time, and we extensively evaluate our scheme to demonstrate its efficiency and scalability. We therefore conclude that TEEs, if used correctly, can enable novel use cases and protocols that can bring significant advantages over traditional ones.

Both Tamarin models and evaluation code are available on GitHub³ and Zenodo [184]. The repository contains extensive documentation, scripts and CI pipelines to fully reproduce our results. As part of the NDSS 2024 artifact evaluation process, our code received all three “Available”, “Functional”, and “Reproduced” badges, and subsequently the “Distinguished Artifact Award” during the conference.

I am the main author of this paper. I developed the core design together with Christoph Baumann and I was responsible for the full formal verification work, the simulations on Kubernetes, and a significant amount of text.

G. Scopelliti, C. Baumann, F. Alder, E. Truyen and J. T. Mühlberg. “Efficient and Timely Revocation of V2X Credentials”. In: *Network and Distributed System Security (NDSS) Symposium 2024*. Internet Society. 2024

1.3.2 Other Contributions

Along with the main contributions, I contributed to other publications that are not reported in this dissertation but summarized below.

Securing Sensing in Supply Chains: Opportunities, Building Blocks, and Designs Supply chains are complex ecosystems that involve several processes and actors, each with different responsibilities and business interests. With recent advancements in cyber-physical systems and the IoT, it has become

³<https://github.com/EricssonResearch/v2x-self-revocation>

possible to use sensors to track and monitor the state of a shipment in order to get information about the state of the shipment itself and its involved products. This allows companies to efficiently manage processes and operations, as well as monitoring the integrity of shipments. However, as several stakeholders with conflicting interests might be involved in the process, there is a need for a secure and transparent supply chain monitoring system where sensor data cannot be manipulated by a malicious actor. This work tries to address this problem by presenting four incremental designs for secure sensing in supply chains, each with different assumptions and approaches to acquire, process and store sensor data. We show that, leveraging trusted computing technologies and tamperproof storage systems, we can achieve a notion of end-to-end secure sensing where each stage of the supply chain pipeline can be protected in terms of integrity, authenticity and completeness of sensor data. In addition, we demonstrate the feasibility of our designs by evaluating computational overhead, latency and hardware costs.

Jan Pennekamp, Fritz Alder and Lennart Bader are the main authors of this paper and developed the four designs of our solution. I contributed to the evaluation by implementing a prototype for a secure sensing scenario based on the authentic execution framework presented in Chapter 2. The prototype consists of Sancus nodes to perform sensor readings and an Intel SGX gateway to collect and aggregate data. The evaluation measures the computation and transmission times of data for various payload sizes. The artifacts are publicly available on GitHub⁴.

J. Pennekamp, F. Alder, L. Bader, **G. Scopelliti**, K. Wehrle and J. T. Mühlberg. “Securing Sensing in Supply Chains: Opportunities, Building Blocks, and Designs”. In: *IEEE Access*. Volume 12, 2024

SNPGuard: Remote Attestation of SEV-SNP VMs Using Open Source Tools As previously described, provisioning and attesting VM-based TEEs are complex tasks involving many processes and software components. Even though open source solutions exist, there is no unified tool that automates the whole pipeline, from building the VM image (including firmware, kernel and root filesystem) to deploying and attesting the running CVM. Furthermore, support for AMD SEV-SNP in the Linux kernel, OVMF firmware and QEMU is still in active development today, with frequent changes that may temporarily break new or existing workloads. This makes it cumbersome and time-consuming for users to get started with VM-based TEEs. To address this problem, we present

⁴<https://github.com/COMSYS/secure-sensing>

an open source tool called *SNPGuard*, which tries to simplify and automate CVM provisioning and attestation as much as possible. Our work consists of several building blocks that focus on specific stages of the pipeline, e.g., building the SEV-SNP toolchain, creating a VM image, and attesting a running CVM, as well as scripts and documentation to bring such building blocks together. We support two different workflows, depending on whether the root filesystem should only have integrity protection or, additionally, encryption. Even though our implementation focuses on AMD SEV-SNP, our concepts should apply to other VM-based TEEs with minor modifications to the code.

Luca Wilke is the first author of this paper, and we shared work equally. This contribution was published as a *tool paper*, i.e., it is not a research publication but a (peer-reviewed) description of our tool. Besides, this work was partially developed to carry out the research leading to the contribution presented in Chapter 3. As part of the SysTEX 2024 artifact evaluation process, our code received all three “Available”, “Functional”, and “Reusable” badges. The tool is publicly available on GitHub⁵.

L. Wilke and G. Scopelliti. “SNPGuard: Remote Attestation of SEV-SNP VMs Using Open Source Tools”. In: *7th Workshop on System Software for Trusted Execution (SysTEX’24)*. 2024

1.4 Dissertation Outline

The remainder of this dissertation is structured as follows. Chapter 2 focuses on process-based TEEs, presenting our framework to develop distributed enclave applications on shared and untrusted infrastructures. Chapter 3 then moves to VM-based TEEs, discussing the challenges to provision and attest CVMs in the cloud. Next, Chapter 4 highlights the problem of getting reliable and consistent timing information inside a TEE. This knowledge is then used in Chapter 5 to design a novel revocation scheme for V2X credentials that leverages a trusted component inside vehicles to guarantee timely revocation without relying on a physical time source. Finally, Chapter 6 summarizes this dissertation and introduces possible directions for future work.

Chapters 2 to 5 are based on the corresponding peer-reviewed publications, with minor layout adaptations. In addition, a preamble is added at the beginning of each chapter to give some context based on the subject of this dissertation.

⁵<https://github.com/SNPGuard/snp-guard>

Chapter 2

End-to-End Security for Distributed Event-Driven Enclave Applications on Heterogeneous TEEs

This chapter was previously published as:

G. Scopelliti, S. Pouyanrad, J. Noorman, F. Alder, C. Baumann, F. Piessens and J. T. Mühlberg. “End-to-End Security for Distributed Event-Driven Enclave Applications on Heterogeneous TEEs”. In: *ACM Transactions on Privacy and Security (TOPS)*. Volume 26, Issue 3, pp. 1–46. 2023

Preamble

This work is the continuation of a workshop paper presented at the STM workshop in 2017, with title “*Authentic Execution of Distributed Event-Driven Applications with a Small TCB*” and Job Noorman as leading author [162]. Two years later, during the academic year 2019-2020, I had the opportunity to pursue my master’s thesis at KU Leuven and work on this project under the supervision of Fritz Alder, Jan Tobias Mühlberg and Frank Piessens. The

thesis was titled “*Securing Smart Environments with Authentic Execution*” [183]. When my Ph.D. trajectory began, we decided to further develop this project in collaboration with Sepideh Pouyanrad. The result of this work is an article published in 2023 in the TOPS journal and reported in this chapter.

The core idea of this work is to provide a framework to securely instantiate distributed applications running within TEEs, with a strong focus on the integrity and authenticity of data and computations. In particular, an important objective is reducing the attack surface as much as possible, allowing each individual component to be reviewed and audited such that a verifier could not only attest that the application is running the expected code, but also (and more importantly) that each output produced by the application could be *explained* by its source code, along with the provided inputs. Clearly, such a property is only possible when the software stack, often referred to as Trusted Computing Base (TCB), is small enough to be manually reviewed and possibly formally verified. Isolating small components from a bigger software stack is one of the main goals of early TEE designs like Intel SGX and Sancus, and indeed we leverage them in this work. Recently, however, the trend has started to take the opposite direction, where huge pieces of code (e.g., entire virtual machines) are moved inside the TCB with a “lift-and-shift” approach, as we will discuss in Chapter 3. Yet, solutions such as the one presented in this chapter may still be relevant for many use cases, especially cyber-physical systems like smart homes and precision agriculture, both of which have been explored in our work.

The paper presented at STM 2017 focused on developing the main design of the framework, which we used to call “*authentic execution*”. Implementation and evaluation were based on Sancus and supported its secure I/O feature. However, the number of use cases that we could support was pretty limited, as Sancus microcontrollers are highly constrained devices that cannot perform complex operations. Thus, the main focus of my master’s thesis was to extend the implementation to Intel SGX in order to support a wider variety of use cases. Subsequently, the work done during this Ph.D. brought about additional improvements such as a revised design for secure I/O, support for ARM TrustZone and software updates, and an end-to-end evaluation based on a smart home application.

Despite being only a research prototype, the authentic execution framework has become bigger and bigger over the years, and it now consists of thousands of lines of code spread across thirty repositories, all of which open source¹. Some of our changes have also been integrated into the Sancus codebase². In spite of this large number of repositories and lines of code, significant effort was made to simplify

¹<https://github.com/AuthenticExecution>

²<https://github.com/sancus-tee>

the development, deployment and testing of workflows by relying on a high level of automation via scripts, Docker containers, and Continuous Integration (CI) pipelines. As a result, our framework is accessible to all users who are interested in experimenting with TEEs without requiring strong programming skills or a security background. Apart from the publications mentioned above, our work was also presented at FOSDEM 2021 and CCS'21 [186, 187]. We also used our framework to evaluate a scenario for secure sensing in supply chains [169].

2.1 Introduction

Trusted Execution Environments (TEEs) allow an application to execute in a hardware-protected environment, often called *enclave*. Enclaves are isolated and protected from the rest of the system, ensuring strong confidentiality and integrity guarantees. Cryptographic primitives and credential management, with keys that are unique per enclave and that can only be used by that enclave, enable secure communication and remote attestation; the latter is a mechanism to obtain cryptographic proof that an application is running under enclave protection on a specific processor. Several TEEs are available in industry and research. Open-source TEEs include Sancus and Keystone; proprietary options are, e.g., Software Guard Extensions (SGX) for Intel processors, Secure Encrypted Virtualization (SEV) for AMD, TrustZone for ARM, and others [138]. Developing applications that execute on heterogeneous TEEs is difficult, in particular for scenarios that combine Internet-of-Things, Edge, and cloud hardware: each TEE requires a platform-specific software implementation, comes with different approaches to key management and attestation, a different Trusted Computing Base (TCB) footprint, and provides different hardware features and security guarantees.

The development of a distributed application composed of multiple modules running on heterogeneous hardware is non-trivial by itself, but becomes an even bigger challenge when the application has stringent security requirements that demand the use of TEE architectures. A developer needs to make choices as to which security features are required for which components, adapt the code of each component to multiple specific platforms, arrange for different deployment and attestation strategies, and implement secure interaction between the components. Open-source projects such as Open Enclave SDK and Google Asylo aim to bridge the development gap between different TEEs. However, software engineers still need to account for the communication between different modules, which has to be properly secured with cryptographic operations for data encryption and authentication. In particular, the responsibility for deploying the distributed application, loading and attesting each enclave, establishing

session keys and secure connections between distributed components, is still left to the application developer and operator. Overall, ensuring strong security guarantees in distributed scenarios poses a challenge to the adoption of TEE technology.

This paper studies the problem of securely executing distributed applications on such a shared, heterogeneous TEE-infrastructure, while relying on a small run-time TCB. We want to provide the owner of such an application with strong assurance that their application is executing securely. We focus on (1) *authenticity* and *integrity* properties of (2) *event-driven* distributed applications. For this selection of security property and class of applications, we specify the exact security guarantees offered by our approach. We believe our approach to be valuable for any kind of distributed application (event-driven or not). In particular, our prototype supports arbitrary C and Rust code.

We distinguish physical events, such as sensor inputs or actuation, from logical events that are generated and consumed by application components. Roughly speaking, our notion of *authentic execution* is the following: if the application produces a physical output event (e.g., turns on an LED), then there must have happened a sequence of physical input events such that that sequence, when processed by the application (as specified in the high-level source code), produces that output event. This notion of security is roughly equivalent to the concept of *robust safety* in later literature [2]. Our approach has been experimented with in previous work, where we secure smart distributed applications in the context of smart metering infrastructures [153], automotive computing [206], and precision agriculture [183], which we generalize and partly formalize under the name “authentic execution” in [162], and use as a training and tutorial scenario to explain attestation and secure communication with heterogeneous TEEs [154].

The main contributions of this paper are:

- We reflect on the design and implementation of an approach for authentic execution of event-driven programs on heterogeneous distributed systems, under the assumption that the execution infrastructure offers specific security primitives – standard enclaves [197] plus support for secure I/O [163] (Sect. 2.3); we comprehensively discuss corner cases and hurdles regarding the use of secure I/O in distributed enclave applications;
- We provide a revised open-source implementation of the approach for Intel SGX, ARM TrustZone and Sancus, which supports software development in Rust and C (Sect. 2.4); we elaborate on implementation challenges towards achieving comparable security guarantees in heterogeneous TEE deployments;
- We conduct an end-to-end evaluation of a concrete application scenario, considering the performance and security aspects of our framework (Sect. 2.5), that considers dynamic system updates, and showing that our approach allows for the deployment of complex distributed software systems with a very small run-time application TCB;
- We design and implement a light-weight symmetric attestation scheme for ARM TrustZone, inspired by and providing security guarantees comparable to Sancus.

Our complete implementation of the authentic execution framework and the evaluation use case are available at <https://github.com/AuthenticExecution/main>, a formalization and proof sketch of our security guarantees is also available there.

2.2 Running Example, Infrastructure & Objectives

As a running example we discuss an automated irrigation system, as illustrated in Figure 2.1, which involves a series of light-weight sensors and actuators in the field that, e.g., monitor soil moisture and crop growth, and control water supply. The system can be connected to edge infrastructure or cloud services for centralized configuration and maintenance, to integrate reporting and billing, and to minimize water consumption based on weather predictions. Naturally, smart farming systems are security critical since malicious interactions can potentially lead to huge costs and may destroy a crop; they also feature a high level of dynamicity where equipment needs to be reconfigured for specific

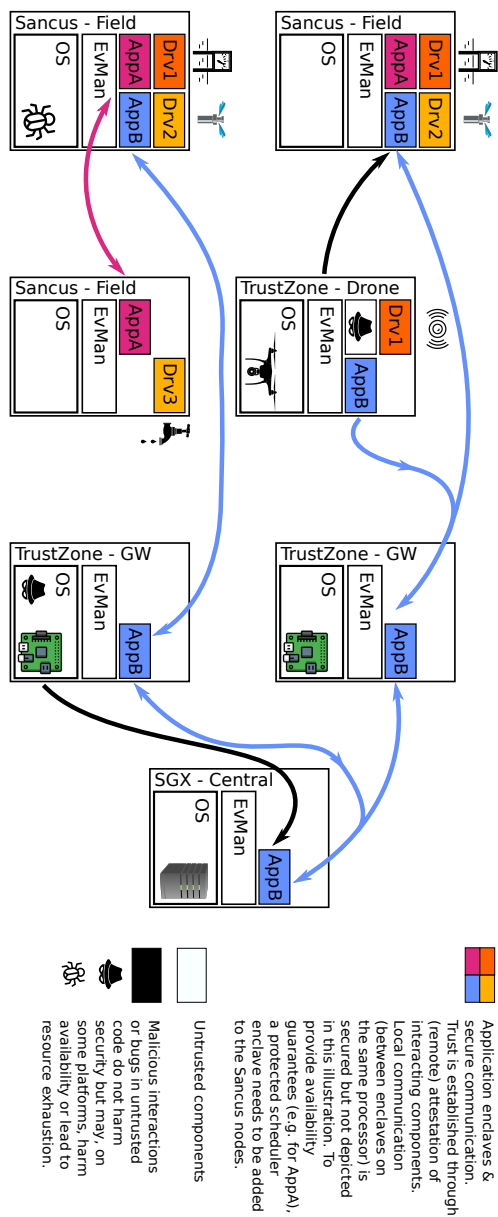


Figure 2.1: A smart irrigation system as an example for distributed application networks we can support. Light-weight sensing and actuation nodes are deployed in a field. Application *AppA* controls irrigation units (through driver *Drv2*) and water supply (*Drv3*), based on soil moisture (obtained through *Drv1*). Application *AppB* provides similar functionality but has access to additional data sources, e.g., aerial surveillance and data aggregation on central infrastructure. All application components execute in enclaves (colored) and the actual composition and configuration of components can change dynamically. Directed data flows through untrusted networks (colored arrows) and data aggregation and configuration of components can change dynamically. Attestation precedes the establishment of all data flows, and mutual authentication is established between enclaves. All other software in the scenario is untrusted regarding our security properties, which leads to a very small run-time application TCB. Guaranteeing availability properties may require a different compartmentalization strategy. The concept is also applicable across, e.g., the different control units within a car or in an autonomous robot.

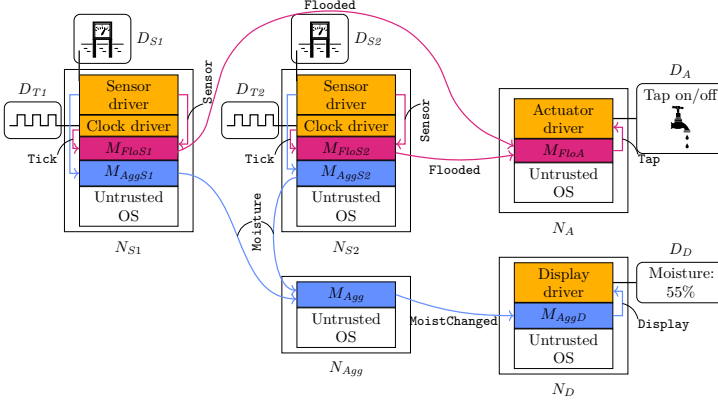


Figure 2.2: Our running example with two applications, A_{Agg} (blue) and A_{Flo} (red). Hardware (N_* and D_*) is trusted; the OS and the network are untrusted. E.g., the A_{Flo} deployer creates the three red Software Modules (SMs) (cf. Fig. 2.3a) with a trusted compiler, attests the shared sensor-, clock- and display drivers and sets-up connections between the SMs. Remote attestation assures authentic execution of A_{Agg} and A_{Flo} .

sensing and actuation tasks that depend on the type of crop [173] and, e.g., sustainability objectives [198], and demand a high level of dependability where events must be guaranteed to be processed in a timely manner.

Fig. 2.2 (source code in Fig. 2.3) zooms in on the light-weight in-field sensing and actuation on the left side in Fig. 2.1 and details application modules and event flows in an agricultural sensor network with two soil moisture sensors. The infrastructure can be reused for multiple applications which can be provided by different stakeholders. Applications include visualizing soil moisture, targeted irrigation, or detection of flooding or leakage. We show two of these applications: one (A_{Flo}) that detects flooding or leakage and disconnects the water supply in case of emergency, and another (A_{Agg}) that aggregates and displays data on soil moisture.

2.2.1 The Shared Infrastructure

The infrastructure is a collection of *nodes* (N_i), where each node consists of a processor, memory, and a number of *I/O devices* (D_i). Multiple mutually distrusting stakeholders share the infrastructure to execute distributed *applications* (A_i). For simplicity, we assume processors are simple microprocessors, such as the OpenMSP430 used in our prototype, to explain the underlying concepts and

security guarantees of our approach. As we detail in Section 2.4 and thereafter, our implementation does support the commercial TEEs TrustZone and Intel SGX, and we evaluate our approach on an integrated scenario that involves these TEEs.

An I/O device interfaces the processor with the physical world and facilitates (1) sensing physical quantities (e.g., the state of a switch), (2) influencing physical quantities (e.g., an LED), and (3) notifying the processor of state change (e.g., a key being pressed) by issuing an interrupt.

In our running example there are 5 nodes. Two of these (N_{S1} and N_{S2}) are each attached to two input devices (a clock D_{Ti} and a soil moisture sensor D_{Si}), and are installed in a field, e.g., along a row of crops. Two other nodes (N_A and N_D) are connected to actuator and display devices (D_A , D_D) to control the water supply and show application output. One node (N_{Agg}) is not connected to any I/O device but performs general-purpose computations, e.g., aggregate data from sensor nodes.

<pre> module FloS1 on Sensor(reading): if reading >= SATURATED: flooded = 1 else: flooded = 0 count = 0 Flooded(false) on Tick(): if flooded: count = count + 1 if count > MAX: Flooded(true) module FloS2 # Similar to FloS1 module FloA on Flooded1(true) f1 = true; if f2 = true: Tap(off) on Flooded2(true): # Similar to Flooded1 </pre>	<pre> module AggS1 on Sensor(reading): Moisture(reading) module AggS2 # Similar to AggS1 module Agg on Moisture1(reading): l1.append(reading) del l1[:-100] MoistChanged(reading - avg) avg = sum(l1) / len(l1) on Moisture2(reading): # Similar to Moisture1 module AggD on MoistChanged(delta) Display(delta) </pre>
---	--

(a) A_{Flo} (b) A_{Agg}

Figure 2.3: Pseudo-code of the applications from Fig. 2.2. in a Python-like syntax. Enclaved application components are declared using the `module` keyword and span until the next `module` or the end of the file. The `on` statement starts an event handler which can connect to an output of another application component, or to a physical I/O channel. Outputs are implicitly declared when invoked through a function call-like syntax.

2.2.2 Modules & Applications

We use an event-driven application model and *modules* (M_i) contain input- and output channels. Upon reception of an event on an input channel, the corresponding event handler is executed atomically and new events on the module's output channels may be produced.

An application, then, is a collection of modules together with a *deployment descriptor*. This descriptor specifies on which nodes the modules should be installed as well as how the modules' channels should be connected. Channels can be connected in two ways. First, one module's output channel can be connected to another's input channel, behaving like a buffered queue of events. Second, the infrastructure can provide a number of *physical* I/O channels which can be connected to a module's I/O channels. The infrastructure must ensure that events on such channels correspond to physical events: An event received on a physical input could correspond to a button press or, similarly, an event produced on a physical output could turn on an LED. An important contribution of this paper is a design to securely connect modules to physical I/O channels (Sects. 2.3.4 and 2.4.1).

In our example applications (Fig. 2.2), A_{Flo} consists of three modules: two (M_{FloS1} and M_{FloS2}) are deployed in a field and detect excess moisture (flooding or leakage) and one (M_{FloA}) actuates a central tap to disconnect water supply if needed. The two in-field modules have two inputs that are connected to input devices provided by the infrastructure: one that produces events for changes in soil moisture (D_{Si}) and another that sends periodical timer events (D_{Ti}). As the source code (Fig. 2.3a) shows, these modules first wait for changes in critical changes in soil moisture, then wait for a maximum number of timer events, and finally produce an output event to indicate an emergency. These output events are connected to the inputs of M_{FloA} which in turn produces output events and sends them to the output actuator D_A . A second application A_{Agg} shares access to the sensor drivers with A_{Flo} to obtain sensor readings that are then aggregated and displayed.

2.2.3 Attacker Model

We consider powerful attackers that can manipulate all the software on the nodes. Attackers can deploy their own applications on the infrastructure, but they can also tamper with the OS. Attackers can also control the communication network that nodes use to communicate with each other. Attackers can sniff the network, modify traffic, or mount man-in-the-middle attacks. With respect

to the cryptographic capabilities of the attacker, we follow the Dolev-Yao model [68].

Attacks against the hardware are out of scope. We assume the attacker to not have physical access to the nodes, neither can they physically tamper with I/O devices. We also do not consider side-channel attacks against our implementation. Physical protection and side-channel resistance are important but orthogonal and complementary to the protection offered by our approach.

2.2.4 Security Objective

The deployer uses his own (trusted) computing infrastructure to compile the application A , to deploy the modules to the nodes in the shared infrastructure, and to configure connections between modules, and between modules and physical I/O channels. At run-time, a sequence of physical input events will happen, and the deployer can observe the sequence of physical output events that return. We say that this sequence of outputs is *authentic* for an application A if it is allowed by A 's modules and deployment descriptor in response to the actual trace of input events: the source code of A explains the physical outputs on the basis of actual physical inputs that have happened. Sect. 2.3.5 will detail this sequence of physical input, processing, and lastly physical output further.

Our objective is to design a deployment algorithm such that the deployer can easily certify the authenticity of sequences. If the correctness of the deployment is verified, then our approach guarantees that any subsequent trace of output events observed by the deployer will be authentic. This security notion rules out a wide range of attacks, including attacks where event transmissions on the network are spoofed or reordered, and attacks where malicious software tampers with the execution of modules. Other relevant attacks are *not* covered by this security objective. We will explain more nuances regarding deployment in Sect. 2.3.5.

As discussed earlier, there are no general availability guarantees – e.g., the attacker can suppress network communication. However, TEE extensions such as [11] and the use of real-time operating systems can provide notions of availability that are relevant to maintain system safety in, e.g., autonomous control systems. There are also no strong confidentiality guarantees: Although this is not the focus of our design, our implementation *does* provide substantial protection of the confidentiality of application state as well as event payloads. Yet, the attacker may still observe the occurrence and timing of events in the system, and specifically on the sensing-and-actuation side of our systems, this information may very well reveal the state of the system. While it is technically possible to close these side channels, e.g., with artificially generated noise, this is

infeasible for our light-weight TEEs and under constrained energy consumption (cf. Sect. 2.6.1 for details).

Many systems similar to our running example from Figure 2.1 exist, e.g., in the context of the Internet of Things, smart building, supply chain management, or intelligent transport systems. These may come with different requirements regarding security and privacy, potentially beyond the baseline guarantees of our framework. In Section 2.5.1 we look specifically at a smart-home scenario and discuss performance characteristics and security guarantees (cf. Section 2.6).

2.3 Authentic Execution of Distributed Applications

We outline our requirements for the infrastructure wrt. security features, and show how these features are used effectively to accomplish our security goals.

2.3.1 Underlying Architecture: TEEs

Given the shared nature of the infrastructure assumed in our system model, we require the ability to isolate source modules from other code running on a node. Since an important non-functional goal is to minimize the TCB, relying on a classical omnipotent kernel to provide isolation is ruled out. Therefore, we assume the underlying architecture is a TEE [197].

While details vary between TEEs, isolation of software modules is understood as follows: A module must be able to specify memory locations containing data that are accessible by the module's code only (*data isolation*). The code of a module must be immutable and a module must specify a number of *entry points* through which its code can be executed (*code isolation*). For simplicity we further assume that both a module's code and data are located in contiguous memory areas called, respectively, its *code section* and its *data section*. We expect TEEs to implement a $W \oplus X$ policy so as to prevent code modification and code generation at runtime.

We also expect the availability of a compiler that correctly translates Software Modules (SMs) to the underlying architecture. The input to this compiler is as follows: (1) a list of entry point functions; (2) a list of non-entry functions; (3) a list of variables that should be allocated in the isolated data section; and (4) a list of constants that should be allocated in the isolated code section. The output of the compiler should be an SM suitable for isolation on the underlying architecture.

Besides isolation, we expect the TEE to provide a way to *attest* the correct isolation of an SM. Attestation provides proof that an SM with a certain identity has been isolated on the node, where the *identity* of an SM, usually based on a measurement of the deployed binary code, should give the deployer assurance that this SM will behave as the corresponding source code module.

After enabling isolation, the TEE should be capable of establishing a confidential, integrity protected and authenticated communication channel between an SM and its deployer. Although the details of how this works may differ from one TEE to another, for simplicity we assume the TEE establishes a shared secret between an SM and its deployer and provides an authenticated encryption primitive. We refer to this shared secret as the *module key*. The authentication property of the communication channel refers to an SM’s identity and hence to attestation. Thus, the TEE ensures that if a deployer receives a message created with a module key, it can only have been created by the corresponding, correctly isolated, SM.

2.3.2 Mapping Source Modules to SMs

To map a source module to an SM, we use the following procedure. First, each of the source module’s inputs and outputs is assigned a unique *I/O identifier*, and each of the connections between source modules is assigned a unique *connection identifier*. The format of these identifiers is unimportant, provided that they uniquely specify a particular input/output or a particular connection, respectively. By having a clear distinction between I/O identifiers and connection identifiers, many-to-many connections are supported, which means that: (1) a source module’s output can be connected to multiple inputs of other source modules (e.g., a keyboard that sends key press events to both a key-logger and an LCD screen), and (2) a source module’s input can be reached by multiple outputs of other source modules (e.g., a key-logger that records key presses of two different keyboards).

Second, a table (**ConnectionTable**) is added to the SM’s variables that maps connection identifiers to a **Connection** data structure, such that every connection has one entry associated with it. These entries will be initialized to all zeros by the underlying architecture, which is interpreted as an unestablished connection. For establishing a connection, an entry point is generated (**SetKey**). This entry point takes a connection identifier, an I/O identifier and a key – encrypted using the module key – as input and updates the corresponding mapping in **ConnectionTable** (Fig. 2.4). Since every connection needs to be protected from reordering and replay attacks, the **Connection** structure also includes a nonce.

```
def SetKey(payload):
    try:
        conn_id, io_id, key = Decrypt(payload)
        conn = Connection(conn_id, io_id, key)
        ConnectionTable[conn_id] = conn
    except: pass
```

Figure 2.4: Pseudocode of the `SetKey` entry point. Note that `Decrypt` uses the module key to decrypt the payload and throws an exception if the operation failed (i.e., the payload’s MAC is incorrect).

```
def HandleInput(conn_id, payload):
    try:
        conn = ConnectionTable[conn_id]
        if conn != 0:
            cb = CbTable[conn.io_id]
            key = conn.key
            nonce = conn.nonce
            cb(Decrypt(nonce, payload, key))
            conn.incrementNonce()
    except: pass
```

Figure 2.5: Pseudocode of the `HandleInput` entry point. Erroneous accesses to the tables as well as errors during `Decrypt` cause exceptions. Thus, these events, as well as those for which no input key has been set, are ignored. `Decrypt` takes a key and the expected associated data as arguments.

```
def HandleOutput(conn_id, data):
    conn = ConnectionTable[conn_id]
    if conn != 0:
        nonce = conn.nonce
        key = conn.key
        payload = Encrypt(nonce, data, key)
        conn.incrementNonce()
        HandleLocalEvent(conn_id, payload)
```

Figure 2.6: Pseudocode of the generated output wrapper. Since the compiler generates calls to this function that cannot be called from outside the module, the connection identifier is always valid and no error checking is necessary.

```
def Attest(challenge):
    try:
        ev = GenAttestationEvidence(challenge)
        return ev
    except:
        return None # attestation failed
```

Figure 2.7: Pseudocode of the `Attest` entry point. Since the attestation of an SM is TEE-specific, we use a high-level function `GenAttestationEvidence` to retrieve the attestation evidence. The actual implementation of this function depends on the TEE used.

Third, all the module's event handlers are marked as non-entry functions. A callback table (`CbTable`) is added to the SM's constants, mapping input identifiers to the corresponding event handlers. This table is used by the entry point `HandleInput`, which is called when an event is delivered to the SM. `HandleInput` takes two arguments: a plain-text connection identifier and an encrypted payload. If `ConnectionTable` has a connection for the given identifier, its key is used to decrypt the payload (using the *expected* nonce as associated data), which is then passed to the callback function stored in `CbTable`, retrieved using the I/O identifier stored in the `Connection` structure. If any of these operations fails, the event is ignored (Fig. 2.5). From a programmer's perspective, an input callback will only be called for events generated by entities with access to valid connection keys.

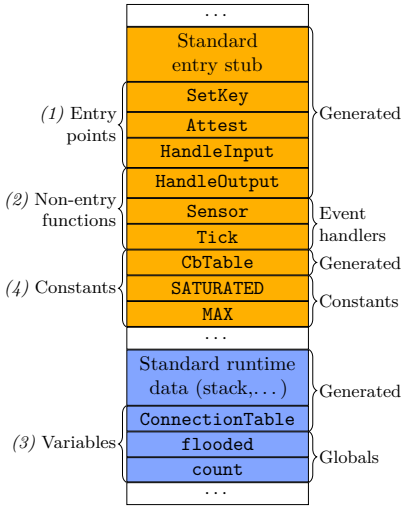
Fourth, each call to an output is replaced by a call to a non-entry wrapper function `HandleOutput`. This function takes a connection identifier and a payload, encrypts the payload together with the current nonce using the corresponding connection key, then publishes the encrypted event (via `HandleLocalEvent`, cf. 2.3.3). If the output is unconnected, the output event is dropped (Fig. 2.6).

Fifth, an entry point `Attest` is generated, which will be called by the deployer to attest the SM. This function takes a challenge as input and returns an *attestation evidence* as response. The attestation procedure and the content of the attestation evidence may vary from one TEE technology to the other; nonetheless, the input challenge is typically used as a freshness parameter to prevent replay attacks. On some TEEs, the attestation process also includes the generation of a shared secret between the SM and the deployer, which will be used as the module key (Fig. 2.7).

To conclude, the following SM definition is given as input to the TEE compiler: (1) `SetKey`, `Attest` and `HandleInput` as entry points; (2) input event handlers and `HandleOutput` as non-entry functions; (3) `ConnectionTable` as a global variable; and (4) `CbTable` as a constant. Fig. 2.8 shows the compiled memory layout of one of the example modules.

2.3.3 Untrusted Software on the Nodes

To support the deployment of modules and the exchange of events between modules, untrusted (and unprotected) software components need to be installed on each node, as outlined below.



Module Loader

The module loader is an untrusted software component running on every node. The module loader provides services for external entities to interact with modules on a node. To this end, the module loader listens for two types of remote requests: **LoadModule** and **CallEntry**. **LoadModule** takes a compiled SM as input, loads it into the TEE and returns the module’s unique identifier together with all information necessary for attestation and module key establishment. What exactly this information is and how the attestation and key establishment is performed is specific to the used TEE. **CallEntry** takes an SM’s identifier, the identifier of an entry point and potentially some arguments and calls the entry point with the given arguments.

Event Manager

The event manager is another untrusted software component running on every node that is used to route events from outputs to inputs. It recognizes three types of requests: **AddConnection**, **HandleLocalEvent** and **HandleRemoteEvent**. A deployer can invoke **AddConnection** remotely to connect the output of a module to the input of another. How exactly inputs and outputs are identified is implementation specific but it will in some form involve specifying (1) a node address (e.g., an IP address); (2) an SM identifier; and (3) a connection identifier. As will become clear later, **AddConnection** only needs to be called on the event manager of the node where the output source module is deployed.

`HandleLocalEvent` is used by modules to publish an event; i.e., inside the output wrappers. The arguments are the module and connection identifiers and the event payload. Based on the identifiers, the event manager looks up the destination event manager and invokes its `HandleRemoteEvent` API, providing the identifiers of the input to which the request should be routed. For a `HandleRemoteEvent` request, the event manager will check if the destination module exists and, if so, invoke its `HandleInput` entry point, passing the connection identifier and payload as arguments.

Implementation

Although we introduced the module loader and the event manager as two separate software components, in our prototype we merged the functionalities of both in the same application, which we simply call the *Event Manager*. This design choice was made for the sake of simplicity: the actual implementation of such logical components is unimportant as long as each node provides the functionalities described above.

2.3.4 Physical Input and Output Channels

We assume that the infrastructure offers physical input and output channels using *protected driver modules* that translate application events into physical events and vice versa. Driver modules control these physical events – sensing and actuation – within the boundaries of the processor executing the driver. For input channels, these modules generate events that correspond to physical events and provide a way for application modules to authenticate the generated events.

For output channels, a driver module (M_D) must have exclusive access to its I/O device (D) and allow an application module (M_A) to take exclusive access over the driver. That is, the driver will only accept events – and hence translate them to physical events – from the application module currently connected to it. Below, we put explicit requirements on the implementation of secure I/O and mark them for later reference.

First, we demand that **IO1**: *the infrastructure provider configures the physical I/O devices as expected*, i.e., the desired peripherals are connected to the right pins and thus mapped to the correct Memory-Mapped I/O (MMIO) addresses in the node. Since misconfiguration in the physical world cannot be detected by remote attestation, we need to require that I/O devices are set up correctly.

However, **IO2**: *the infrastructure must provide a way for the deployer of M_A to attest that it has exclusive access to the driver module M_D and that M_D also has*

exclusive access to its I/O device D . The deployer must be able to attest M_D to ensure that it indeed only accepts events from the module currently having exclusive access and that it does not release this exclusive access without being asked to do so by the module itself.

We also need to require that, **IO3**: *after a microcontroller is turned on, only authenticated driver modules may take control of I/O devices. Once control is taken it is exclusive and never released.* This prevents attackers from taking direct control of output devices after a node resets. Driver modules are thus part of the trusted computing base and their module keys are only known to the infrastructure provider who authorizes exclusive connections to driver modules upon request from the deployer and keeps track of the “ownership” of the driver modules.

Finally, we demand that **IO4**: *output driver modules do not produce outputs unless requested by the application module, while input driver modules only generate outputs to application modules that correspond to physical inputs.*

To summarize, we have modeled physical *output* channels in such a way that:

1. At startup, authenticated output driver modules take ownership over the (correct) output devices. Unauthorized access and connection attempts from other software are forbidden;
2. Application modules can take ownership of an output driver module only if the latter is not already connected to other modules, and only if authorized by the infrastructure provider;
3. After connecting to an output driver module, an application module retains exclusive access to the output device until explicitly released. Thanks to point (1), exclusive access is retained even across resets, although all connections need to be re-established.

For input modules, exclusive access is not strictly required. However, exclusive access to input devices can be configured in the same way as for outputs, if desired. These conditions lead to the following security properties:

- *Assume that for a time interval T the infrastructure provider only ever authorizes application module M_A to take control of device D via driver module M_D . If the deployer attests within T that M_A has taken exclusive access over output driver module M_D , then from that moment until the end of T every output from D can be explained alone by the code semantics of M_A and M_D in relation to the inputs received by M_A , i.e., no other module may cause outputs of D .*

- *If the deployer attests that M_A has taken access over an input driver module M_D , then from that moment on until the access is released any output from M_A can be explained alone by the code semantics of M_A and M_D in relation to the physical inputs to D and any other inputs received by M_A , i.e., an attacker cannot spoof inputs to D for M_A .*

In practice, I/O devices may produce initialization outputs when a driver module takes control of them after a reset. Handling these outputs is parts of the specificities of a driver implementation, which we ignore here for simplicity.

2.3.5 Deployment

Deployment is the act of installing application modules on their nodes and setting up connections between outputs and inputs. As a requirement for deployment, **D1**: *the channel between deployer and infrastructure provider is assumed to be secure*, whereas communication with modules is performed over an untrusted network.

In phase 1, the deployer requests access to the driver modules connected to physical I/O devices from the infrastructure provider who provides these modules. For output devices, the deployer requests *exclusive* access, while for input devices the deployer can choose between exclusive or shared access. Before granting such access, **D2**: *the infrastructure provider needs to ensure the authenticity of the driver module controlling the I/O device*, e.g., via attestation. If, for any reason, the infrastructure provider cannot give access to some I/O devices (e.g., because an output device is already taken by someone else), the deployment is aborted. Otherwise, the deployer receives connection keys from the infrastructure provider (one for each I/O device), along with the guarantee that driver modules have correctly taken control of the physical I/O devices (Sect. 2.3.4).

Phase 2 consists of deploying the application on the infrastructure. First, the deployer starts by compiling each source module into a loadable image. Second, the deployment descriptor is used to find the node on which the module should be deployed and sends its module loader a **LoadModule** request. The deployer then performs the TEE-specific method of attestation calling the **Attest** entry point of each SM, eventually setting up the module key. At the end of this step, the deployer has a secure communication channel with each of its deployed source modules. Finally, the deployer sets up the connections between modules, as well as the connections to the driver modules. Regarding the former, the deployer generates a unique connection key and sends it to both endpoints of the connection; for each endpoint, such key is encrypted using the module key

and passed to the `SetKey` entry point using the `CallEntry` API. Concerning the latter, instead, connections are established via a similar interface, using the keys obtained in phase 1.

Security Discussion

During phase 1, the infrastructure provider plays an important role as a trusted party, providing exclusive access to driver modules. **D3:** *It is required that the provider does not leak the driver module keys and that exclusive access to a device is reserved for the deployer until they request to release it or a time T has passed (to which both the deployer and the provider have agreed upon).* In other words, no one else can acquire credentials to connect to the driver module during that time.

If a reset on a node N occurs, then all driver modules belonging to that node need to be re-initialized, and exclusive access re-configured (if previously set). Concerning the latter, **D4:** *the infrastructure must provide replay protection for messages exchanged in the protocol to establish exclusive driver access*, otherwise replay attacks might cause exclusive access to be unintentionally set up for modules that formerly had access, but should not have it anymore.

Concerning application modules instead, a reset on node N would cause all modules of that node to be re-deployed, re-attested, and connections for them to be re-established. Here, an attacker could technically mount a sophisticated replay attack to restore a previous configuration of an application; however, if replay protections for driver modules are correctly in place (as discussed above), the attacker would not be able to trigger any illicit or stale outputs.

Note that the strong requirements on physical output drivers do not apply for virtual outputs to software endpoints, such as secure databases or nodes belonging to a trusted party like the deployer. Such receivers can verify the authenticity and freshness of outputs via cryptographic means and reject replayed messages automatically, which is not possible for physical outputs to the real world.

2.3.6 Security Argument

Our goal is to ensure that all physical output events can be explained by the application's source code and the observed physical input events. More precisely: *Consider a time frame starting at the end of phase 1 of deployment (Sect. 2.3.5), and ending at a point where the deployer releases exclusive access to any of the output devices they control. Assume the deployer has observed a*

specific sequence of physical output events on one of these devices D_O in the considered time frame, then there have been contiguous sequences of physical input events on the input devices connected to the application such that the observed outputs follow from these inputs according to the application source code semantics.

As an example, consider A_{Flo} (Fig. 2.3a). If, after the application has been deployed, the water supply is turned off, then there must have been physical input events that caused the field to flood.

Output events can only be produced by the application’s SMs; the assumption of a correct compiler and the successful attestation then lead to the desired property. Due to requirements **IO1-IO4**, (1) a physical output event can only be produced by the corresponding device (D_O); (2) output drivers have exclusive access to their device; and (3) an SM (M_O) has exclusive access to the driver; thus only M_O can initiate physical outputs on D_O . Even if exclusive access is lost, e.g., due to a reset of the corresponding node triggered by an attacker, the infrastructure prevents that any other module can take control of D_O and produce rogue outputs, thanks to requirements **D1-D4**. The construction of SMs ensures that a module can only be invoked through its two entry points. Of these, only **HandleInput** can result in output events (Figs. 2.4 and 2.5). Since **HandleInput** authenticates its input, output events are always the result of correct input events. Finally, as our deployment scheme only allows for two types of correct input events, physical input events and outputs from other modules, our security property follows.

2.3.7 Software Updates and Re-Deployment

In Sect. 2.3.5, we described a *static* deployment where the whole application is deployed at once; however, it is also possible to deploy application modules dynamically. Specifically in scenarios where the configuration of an application may change during its lifetime due to the reconfiguration, addition, or removal of sensors or processing nodes, or the deployment of additional applications on existing infrastructure, software updates become essential. In our smart irrigation system from Figure 2.1, this may, e.g., happen with changing environmental conditions or when plots are re-allocated for different crops.

Two common scenarios that may occur at runtime are software updates and re-deployments. The former consists of deploying an updated version of an SMs that, e.g., provides new functionality or fixes some bugs; the latter does not necessarily require code changes but may be needed when an SM crashes or needs to be migrated to another node. Our design for software updates and re-deployments consists of three steps:

1. *Deployment of the new SM.* In the first step, the new SM is compiled, deployed on the infrastructure alongside the already running SMs, and finally attested. If any of these operations fails, the update is aborted. Note that the new SM will not process any events until connections are configured through its `SetKey` entry point (step 3).
2. *Deactivation of the old SM.* In this step, the old SM is removed from the infrastructure. This is the point in time from which the application will start suffering from connectivity loss, until the new SM will be completely configured. Optionally, state may be transferred to the new SM before taking down the old one.
3. *Re-establishment of connections.* Here, all the connections associated to the old SM must be re-configured in order to point to the new SM. This process involves calling the `SetKey` and `AddConnection` entry points on the involved SMs and event managers, as for a static deployment (Sect. 2.3.5); each connection retains its connection identifier, but keys are rotated for security reasons. Similarly, connections between the SM and I/O devices must be re-configured as well, if any (Sect. 2.3.4). At the end of this step, the update will be completed.

We evaluate the impact of software updates in Section 2.5.3. Note that actions 2 and 3 can also be executed in reverse order, if desired. Re-establishing the connections before disabling the old SM would result in a smaller availability disruption, but it may also cause inconsistent state if events are generated during the update. For example, some events might be processed by the old SM and some other events by the new SM, causing undefined behavior. As our framework mainly focuses on integrity rather than availability, we should prevent such inconsistencies from occurring, for example by implementing a sort of synchronization mechanism between old and new SMs during the update. However, this would increase the complexity of our framework without providing substantial benefits compared to our original update strategy. Hence, we decided not to investigate this option further.

Transferring the state from the old to the new SM may be done in several ways. For instance, the *sealing* feature of certain TEEs (such as Intel SGX) allows an SM to securely store data persistently on disk, although it requires the new SM to be deployed on the same node as the old one. Our design already provides some basic support for state transfer by creating a *temporary connection* between the two SMs. In particular, a *transfer* output in the old SM may be connected to a *restore* input in the new SM, then the state migration can be triggered by calling a *save* entry point in the old SM that generates the *transfer* event. This would allow state to be securely transferred between

the two modules, even in different nodes. Of course, the deployer still needs to manually implement the *save* and *restore* functions in order to specify what data needs to be migrated. This protocol, however, requires the old SM to be up and running at the time of the update, and therefore cannot be applied if the module was disabled before, e.g., due to a crash or a reset on the node.

2.4 Implementation

Originally [162], we created a fully functional prototype of our design based on the hardware-only embedded TEE Sancus [161]. In later work [183], we extended our support to the commercial Intel SGX [143], a TEE for high-end x86 processors. Lately, we added support for a third TEE, based on ARM devices equipped with TrustZone technology [12].

As the implementations for all TEEs are compatible with each other, we can support heterogeneous deployments in which an SM can send protected events to any other SMs, regardless of the actual TEE technologies used. This enables a wide variety of use cases that combine sensing and actuation with cloud and edge processing. As such, our running example in precision agriculture is scalable from small local applications such as an automated irrigation system to more complex deployments that feature, e.g., a wide range of sensors and actuators, and that involve complex data aggregation and processing. For example, we could have an authentic trace that starts from a Sancus sensor that leverages Secure I/O to collect measurements, then continues on a TrustZone gateway that forwards the measurements to an Intel SGX aggregator in the cloud, and finally ends with a command to actuate an output I/O device, sent back to a Sancus output driver through the same TrustZone gateway.

This chapter is structured as follows: Sect. 2.4.1, 2.4.2 and 2.4.3 describe each of these three implementations separately, emphasizing on the features of their underlying TEEs. For each implementation we describe (1) the underlying security architecture, including data and code isolation as well as the root of trust, (2) the process of compiling applications into security modules and the processing of inputs and outputs, (3) the untrusted runtime, in particular the realization of the *LoadModule* and *CallEntry* requests, (4) the attestation protocol including the generation of attestation evidence and the establishment of module keys, (5) the configuration of secure communication endpoints, and (6) where available, the realization of secure I/O channels, protected driver modules, and relevant implementation requirements (**IO1-D4**). The main differences of the three implementations are then summarized in Sect. 2.4.4.

Next, Sect. 2.4.5 illustrates the deployment phase. Finally, Sect. 2.4.6 introduces an optional component used to facilitate attestation and key management.

2.4.1 Sancus

Sancus [161] is an OpenMSP430-based TEEs designed for low-cost and low-power embedded applications. As described in Sect. 2.3.1, Sancus divides SMs in two sections, the (public) code section and the (private) data section, enforcing strict access rules for both. An SM's code section can only be entered through a *single* entry point: its first instruction. The compiler assigns each user-defined entry point an integer identifier and adds an entry stub that evaluates such an identifier, dispatching to the correct entry point. Besides, private data is only accessible by the module's code.

Enclave development

The Sancus toolchain comes with a C compiler, libraries, and other tools that are needed to develop and build applications composed of one or more Sancus enclaves. Developers can define enclaves by annotating code and data using special macros defined in the Sancus trusted library: functions can be marked as either entry points or internal functions, using the `SM_ENTRY` and `SM_FUNC` macros respectively. Besides, variables annotated with `SM_DATA` will be allocated in the private data section, making it accessible only from inside the enclave.

Furthermore, Sancus provides an untrusted library to initialize enclaves and call their entry points. Pointers passed to an entry point must be properly sanitized to ensure that they do not point to enclave memory. Return values, instead, are copied from trusted to untrusted memory. The untrusted library also provides a function to dynamically load new enclaves at runtime; in our framework, this functionality is used to deploy application SMs.

At compile time, the Sancus compiler builds the application into a single binary that includes both untrusted runtime (e.g., the `main` function) and static enclaves. Such binary can then be loaded into a Sancus microcontroller using the dedicated loader. Besides, the Sancus toolchain also provides a tool to retrieve the module key of an enclave given a binary file, which is essential in order to attest the enclave from outside the microcontroller.

```

SM_OUTPUT(FloS1, Flooded);
SM_DATA(FloS1) int flooded, count;

SM_INPUT(FloS1, Sensor, data, len) {
    if (len > 0) {
        if (data[0] >= SATURATED) {
            flooded = 1;
        } else {
            flooded = count = 0;
            Flooded(&flooded, sizeof(flooded));
        }
    }
}
}
SM_INPUT(FloS1, Tick, data, len) {
    if (flooded && ++count > MAX) {
        Flooded(&flooded, sizeof(flooded));
    }
}
}

```

Figure 2.9: A translation of module M_{FloS1} (Figs. 2.2 and 2.3a) to C using the annotations understood by our compiler.

APIs

Our implementation is a literal translation of the design outlined in Sect. 2.3.2. All modifications to the Sancus compiler are *extensions*, and all original Sancus features are still available to programmers (e.g., calling external functions or other SMs). On top of the existing annotations we added two new ones: `SM_INPUT` and `SM_OUTPUT` for specifying inputs and outputs. Fig. 2.9 shows an example module written in C using our annotations.

`SM_OUTPUT` expects a name as argument (more specifically, a valid C identifier). For every output, the compiler generates a function with the following signature: `uint16_t name(char* data, size_t len)`. This function can be called to produce an output event. For input handlers, `SM_INPUT` generates a function with the same signature as above. In this function, the programmer has access to a buffer containing the (unwrapped) payload of the event that caused its execution. For both inputs and outputs, the names provided in annotations are used in the deployment descriptor.

Untrusted Runtime

The untrusted runtime consists of a single application, the event manager, which implements all the untrusted API requests described in Sect. 2.3.3. The ELF binary is compiled together with a ported version of RIOT OS [32], an operating system specifically designed for devices with limited resources such as our OpenMSP430-based Sancus.

Our event manager was initially developed for an experimental version of Sancus RIOT [11]. Future work will adapt the event manager to the latest Sancus hardware and RIOT, in order to fully leverage common OS features as well as the protected scheduler, a fundamental component for providing availability guarantees to Sancus enclaves, as discussed in 2.6.2.

Attestation and secure channels

Sancus uses a three-level key hierarchy for remote attestation and secure communication. Every node contains a *node key*, which is only known by the node's owner, the *infrastructure provider*. Every vendor who is to install SMs on a particular node is assigned a unique identifier. The second level of keys, *vendor keys*, is derived from the node key and these vendor identifiers. Finally, *module keys* are derived from a vendor key using an SMs *module identity*. This module identity – the concatenation of the contents of the module's code section and the load addresses of both its sections – is used for authentication and attestation. The module key is calculated by the Sancus hardware when a module is loaded and can also be calculated by the module's vendor. Since the hardware ensures that module keys can only be accessed by the corresponding SM, it is guaranteed that the *use* of a certain module key (e.g., by creating a Message Authentication Code (MAC)) implicitly attests the module's identity.

The **Attest** entry point (Sect. 2.3.2) leverages a challenge-response protocol to verify that a Sancus SM is up and running on a specific node at a certain time. The challenge consists of a random sequence of bytes generated by the deployer, long enough to prevent replay attacks. The Sancus SM computes a MAC over the challenge using its module key, and sends the result back to the deployer. The latter performs the same operation: attestation can be considered successful if the result matches the response provided by the SM.

Sancus' crypto engine uses SPONGEWRAP [35] as encryption algorithm with SPONGENT [39] as the underlying hash function to calculate MACs. The interface to the crypto engine is provided by two instructions: **Encrypt** takes a plaintext buffer, associated data (which will be authenticated but not encrypted), and a key and produces the ciphertext and an *authentication tag* (i.e., a MAC); **Decrypt**, given the ciphertext, associated data, tag and key, produces the original plaintext or raises an error if the tag is invalid. For both instructions, the key is an optional argument with the calling module's key as a default value. As with the original version of Sancus, this is the *only* way for a module to use its key.

Secure I/O

This section describes how protected drivers can be implemented using Sancus. Remember that, for output channels, we want an application module to have exclusive access to a driver (Sect. 2.3.4). This, in turn, implies that the driver should have exclusive access to the physical I/O device. Although for input channels the requirements are less strict – we only need to authenticate a device – for simplicity, we also use exclusive device access here.

Exclusive Access to Device Registers. Sancus, being based on the OpenMSP430 architecture, uses Memory-Mapped I/O (MMIO) to communicate with devices. Thus, providing exclusive access to device registers is supported out of the box by mapping the driver’s private section over the device’s MMIO region. There is one difficulty, however, caused by the private section of Sancus modules being contiguous and the OpenMSP430 having a fixed MMIO region (i.e., the addresses used for MMIO cannot be remapped). Thus, a Sancus module can use its private section either for MMIO or for data but not for both. Therefore, a module using MMIO cannot use *any* memory, including a stack, severely limiting the functionality this module can implement.

We decided to solve this in software: Driver modules can be split in two modules, one performing only MMIO (`mod-mmio`) and one using the API provided by the former module to implement the driver logic (`mod-driver`). The task of `mod-mmio` is straightforward: for each available MMIO location it implements entry points for reading and writing this location, and ignores calls by modules other than `mod-driver`. This task is simple enough to be implemented using only registers for data storage, negating the need for an extra data section.

This technique lets us implement exclusive access to device registers on Sancus without changing the hardware representation of modules. Yet, it incurs a non-negligible performance impact because `mod-mmio` has to attest `mod-driver` on *every* call to one of its entry points. Doing the attestation once and only checking the module identifier on subsequent calls is not applicable because it requires memory for storing the identifier. We address this by hard-coding the *expected* identifier of `mod-driver` in the code section of `mod-mmio`. During initialization, `mod-driver` checks if it is assigned the expected identifier and otherwise aborts. `mod-driver` also attests `mod-mmio`, verifying module integrity and exclusive access to the device’s MMIO registers. On failure, `mod-driver` aborts as well. The expected identifier of `mod-driver` can be easily deduced as Sancus assigns identifiers in order: the first loaded module takes identifier 1, the second 2, and so forth.

Sancus did not support caller authentication [161], which we require for `mod-mmio` to ensure invocation by `mod-driver` only. We added this feature by storing the identifier of the *previously* executing module in a new register, and added instructions to read and verify this SM identity.

Secure Interrupts. On the OpenMSP430, interrupt handlers are registered by writing their address to the interrupt vector, a specific memory location. Thus, handling interrupts inside SMs is done by registering a module's entry point as an interrupt handler. If the SM also supports "normal" entry points, a way to detect whether the entry point is called in response to an interrupt is required.

More generally, we need a way to identify *which* interrupt caused an interrupt handler to be executed. Otherwise an attacker might be able to inject events into an application by spoofing calls to an interrupt handler. To this end, we extended the technique used for caller authentication. When an interrupt occurs, the processor stores a special value specific to that interrupt in the new register to keep track of the previously executing module. This way, an interrupt handler can identify by which interrupt it was called in the same way modules can identify which module called one of their entry points. The processor ensures that these special values used to identify interrupts are never assigned to any SM.

Interfacing with Applications. Our implementation follows the design in Sect. 2.3.4 to connect an application SM to a driver SM via connection keys. As we assume that driver modules are provided by the infrastructure, a deployer needs to interact with the infrastructure provider for the distribution of connection keys to the drivers. In fact, these keys need to be encrypted and authenticated using an SM's module key which, for driver modules, are only known by the infrastructure provider.

In our implementation, driver and MMIO modules are static enclaves embedded in the application binary loaded in the Sancus microcontroller. At startup, these modules are immediately initialized to gain exclusive access to their I/O devices (**IO3**). To change this behavior, an attacker would require physical access to the microcontroller in order to upload a malicious binary; however, this is out of scope (Sect. 2.2.3).

The infrastructure provider can then keep track of which deployer has exclusive access to a driver SM at a certain time, denying requests coming from other deployers for the whole duration of the exclusive access period, or until the deployer explicitly gives up access (**D3**).

In our case, giving exclusive access means encrypting and authenticating the connection key (provided by the deployer) with the driver’s module key, sending the result data back to the deployer; the communication between deployer and infrastructure provider should be done via a secure channel. The deployer would then be able to gain exclusive access to the driver SM by calling its **SetKey** entry point (or equivalent) using the encrypted payload as argument. Regarding input drivers, the infrastructure provider may allow non-exclusive access by simply accepting requests from multiple deployers at the same time; if needed, a deployer may ask for exclusive access by adding an additional bit of information to the request.

To prevent replay attacks, the payload passed to the **SetKey** entry point contains a unique nonce that the driver SM can verify against a reference stored in memory (**D4**). This nonce must be rotated every time exclusive access is reconfigured.

In summary, our design for Secure I/O in Sancus consists of the following protocol:

1. The deployer sends an initial request to the driver SM asking for its current nonce. The driver retrieves the nonce from secure memory and sends it back to the deployer.
2. The deployer then sends the nonce and the connection key to the infrastructure provider via a secure channel (**D1**). Due to the static driver setup in Sancus, the provider knows that only authentic drivers have control of I/O devices (**D2**). The provider only ensures that the driver SM is not already taken by someone else and, if so, may grant the deployer exclusive access. To this end, the provider encrypts and authenticates the input data with the driver’s module key, returning it to the deployer.
3. The deployer then forwards the encrypted payload to the driver SM, which decrypts and authenticates it. If the nonce matches with the one stored in memory, the driver establishes exclusive access by storing the connection key into memory and using said key to decrypt future payloads. Finally, the driver sends an authenticated confirmation to the deployer using the same connection key, then generates a new nonce for the next run of the protocol.

Since only authentic drivers can decrypt the data using their module keys and complete the protocol illustrated above, the reception of the confirmation message allows the deployer to conclude that they indeed have acquired exclusive access to the desired I/O device (**IO2**). If the protocol does not complete for any reason (e.g., an attacker blocks one of the messages between driver and

deployer), the deployer would never receive a confirmation from the driver SM; as such, the deployer would not trust outputs generated by that driver.

In case of node resets, the deployer would not lose exclusive access to the driver because the infrastructure provider would not complete step (2) of the protocol with another deployer (**D3**). However, the original deployer would have to run the protocol again to restore connectivity, rotating connection keys to prevent replay attacks.

An important aspect to consider is that monotonic counters as nonces are not enough to prevent replay attacks, because in case of resets on the node the driver module would lose information about the current nonce. This problem can be solved by either (1) storing the current nonce in secure persistent storage, or (2) generating a random nonce each time. As Sancus currently provides neither persistent storage nor an RNG, we leave this issue to future work.

2.4.2 Intel SGX

Intel SGX is a TEE included in commercial Intel processors, consisting of hardware primitives and a set of instructions that can be used to isolate code and data of an application in protected memory regions called *enclaves*. Access to enclave memory is restricted at runtime and only accessible by code within the enclave, and the enclave can only be entered through specific entry points. Thus, neither the host OS nor other software can access enclaves' code and memory, which results in a reduced TCB.

The architecture of Intel SGX provides an enclave with an enclave measurement called MRENCLAVE. This reflects the *enclave identity* and consists of a SHA-256 hash over the content of the enclave's code and data at initialization time. This enclave identity is complemented by the *author identity*, called the MRSIGNER, which reflects the hash of the enclave's author's public key. This author is the entity who signs the enclave before distribution.

These two identities are then used as reference values for both local and remote attestation, as well as to generate *sealing keys* that allow the enclave to securely store persistent data on disk.

Enclave development

To develop an Intel SGX application, Intel provides its own SDK written in C/C++. Here, the application is partitioned into two sections: the untrusted code and the enclave. Communication between trusted and untrusted

```

/* global variables definition omitted
   for brevity */

/*@ sm_output(Flooded)

/*@ sm_input
pub fn Sensor(data : &[u8]) {
    if data.len() > 0 {
        if data[0] >= SATURATED {
            flooded = true;
        } else {
            flooded = false;
            Flooded(&[0]);
        }
    }
}
}
/*@ sm_input
pub fn Tick(_data : &[u8]) {
    if flooded {
        count += 1;
        if count > MAX {
            Flooded(&[1]);
        }
    }
}
}

```

Figure 2.10: A translation of module M_{FloS1} (Figs. 2.2 and 2.3a) to Rust using the annotations understood by our Intel SGX parser.

applications is made through a specific interface, defined by the developer using a C-like syntax called Enclave Definition Language (EDL). Calls from the untrusted application to the enclave are made via well-defined entry points named ECALLs, while untrusted functions are made available to the enclave via OCALLs.

Recently, new frameworks have been introduced that allow developers to write enclaves in modern languages and with reduced effort. In our framework, we leverage Fortanix Enclave Development Platform (EDP) [85], an SDK written entirely in the memory-safe Rust. EDP abstracts the Intel SGX layer away from a developer, who can write an enclave similar to native application; the necessary bindings are automatically added at compile time. EDP is seamlessly integrated with the Rust standard library, although some functionalities are not available for security reasons.

APIs

Our implementation of the design illustrated in Sect. 2.3.2 aims to reduce the development effort as much as possible. As described above, Fortanix EDP already gives huge benefits to a developer by abstracting the Intel SGX layer away, taking care of secure argument passing, data and code isolation, etc. In addition, our framework provides a simple mechanism to declare outputs

and inputs by using *code annotations*, which are parsed by the framework at deployment time in order to retrieve information about the module's endpoints and to inject required code. These annotations are analogous to the Sancus implementation described in Sect. 2.4.1, although we rely on special single-line comments instead of using macros (Fig. 2.10).

Output and input events are designed to be *asynchronous*: after an output event is generated, the SM resumes its execution immediately, and no return values are expected. Asynchronous events work well if they are generated from physical events (e.g., a button press), as the only purpose of the associated SM is to notify one or more other SMs to which it has active connections. However, in some other cases *synchronous* events might be necessary, for example for querying a database. Here, a return value is expected and the SM that generated the output event might want to block the execution until the value is received. To address this need, the Intel SGX implementation comes with two additional annotations: `//@ sm_request` and `//@ sm_handler`. Request-handler events are similar to output-input events, however a request blocks the execution until the connected handler (if exists) returns a value.

Untrusted Runtime

The untrusted runtime consists of an untrusted Linux process that implements the logic illustrated in Sect. 2.3.3. Concerning the execution of new SMs, Fortanix EDP provides a default runner for Intel SGX enclaves called `ftxsgx-runner`, responsible for loading and executing an enclave. Thus, when a new `LoadModule` request arrives, the event manager spawns a new process using the Rust's `std::process::Command` API, executing `ftxsgx-runner` and passing as argument the enclave's binary and signature. To realize `CallEntry` requests, our SGX compiler adds a simple TCP server as a front-end to the SM. At compile time, each SM is assigned a free port in the same network namespace as the event manager, so that they can exchange events with each other via *localhost*.

Remote attestation and secure channels

In short, the remote attestation of an Intel SGX enclave consists of the following steps: (1) a remote entity (the *challenger*) sends an attestation request to the enclave. The request should contain a challenge that will be included in the attestation evidence to provide freshness; (2) the enclave (the *prover*) generates an attestation evidence, also called *Quote*, that is returned to the challenger. This Quote includes information about the identity of the enclave

and the platform on which the enclave is running (hardware information, microcode version, etc.); (3) the challenger verifies the quote by first ensuring its authenticity and second verifying that the enclave and hardware are in the expected state.

A quote is generated by a dedicated enclave called Quoting Enclave (QE), which resides on the same platform as the enclave to be attested. The quote is protected using cryptographic keys unique to that particular platform. A trusted third party is then responsible for the quote verification, to ensure that (1) the quote is authentic, and (2) the hardware and firmware of the platform are up to date. After that, the challenger verifies the identity of the enclave, ensures that the quote is fresh (i.e., the challenge is included in the quote), and finally decides whether to trust or not the enclave.

Our framework supports the Enhanced Privacy ID (EPID) attestation scheme³ [114], for which a Rust implementation already existed⁴. The scheme also involves a mutually-authenticated Diffie-Hellman key exchange between challenger and enclave; we leverage this protocol to establish a shared secret between a module and its deployer, which will be used as the *module key*.

Secure I/O

Unfortunately, SGX does not support secure I/O channels out of the box. In fact, it is by design impossible to map DMA devices into enclave memory [60]. While academic solutions have been proposed to overcome these issues (Sect. 2.7.3), they usually require additional trusted hardware [227] and software [130, 170, 216] to establish tamper-proof, exclusive channels to I/O devices. Hence, our framework does not support physical I/O channels for SGX-based SMs. Nevertheless, in the IoT settings we envision, SGX-based modules will likely be deployed in edge or cloud computing platforms where physical inputs and outputs are less relevant. Instead, such modules may be used for monitoring and control of a number of IoT gateways or devices. As such, they would be connected to virtual endpoints in the deployer's trusted back-end system using credentials that are securely stored in enclave memory.

³We implemented EPID for simplicity, though other attestation schemes could be supported as well. From a security perspective, all these schemes would provide similar guarantees.

⁴<https://github.com/ndokmai/rust-sgx-remote-attestation>

2.4.3 ARM TrustZone

TrustZone is a very common TEE implemented in the different flavors of recent ARM processors, which applies strong system-level isolation by separating both system hardware and software resources into two domains, namely the Normal World, and the Secure World, protecting the code and data in the secure world from being directly accessed or modified by the normal world [171].

To perform the secure context switching between worlds, ARM Cortex-A processors introduced a new CPU mode called monitor mode, which runs at the highest execution level of the secure world. The monitor mode can be entered via an interrupt, external abort, or explicit call of a special privileged instruction: *Secure Monitor Call*. Then, the value of the least significant bit of the Secure Configuration Register, known as the Non-Secure bit is changed and propagated through the AXI system bus to the memory and peripherals to preserve the process security state. On ARM Cortex-M processors, hardware is responsible for the transition between worlds, which optimizes switching overhead [171, 157]. In this paper, we only focused on the features of TrustZone for Cortex-A processors.

The normal world runs a general-purpose operating system, such as Linux or Android, and untrusted (client) applications, while a lightweight TEE operating system and trusted applications (TAs) run in the secure world. In this paper we rely on Open Portable Trusted Execution Environment (OP-TEE) [131], which implements a secure operating system (the OP-TEE OS), a secure monitor, a non-secure user-space library called OP-TEE client exposed to the client applications, and a set of build toolchains to facilitate the development of TAs.

TrustZone relies on a secure boot process to prevent a system from being attacked during the booting process and enforce a strong system integrity policy. Secure boot forms a Chain of Trust by leveraging a signature scheme based on RSA to verify system images before their execution [138]. Concretely, the boot process operates as a sequence of stages, each former boot stage loads and checks the signatures of the next follow-on stage, usually initiated from the root of trust (RoT). RoT is usually a public key that is stored in the trusted root-key storage registers and cannot be modified [28].

TA development

To develop a TA, OP-TEE provides its own SDK to both build and sign the TA binary with a private RSA key, then the signature is verified by OP-TEE OS upon loading of the TA to check its integrity. A TA must implement a couple of mandatory C-like syntax entry points to allocate some session resources

and execute the target services. Communication with a TA is established by initiating a request from the client application through some defined entry points in the OP-TEE client to load the intended TA binary and its libraries into the secure memory. To identify a TA to be used, the client provides a unique 16-byte value called UUID that is generated by the developer. Then, the desired services in the TA are invoked from the client application by passing a so-called command ID which is also defined at compile time.

API

In our framework, we intend to provide a high-level abstraction over the TEE layer that allows developers to focus only on application logic. Further to the abstraction provided by GlobalPlatform TEE Internal Core API in OP-TEE, we define annotations `SM_ENTRY`, `SM_INPUT`, and `SM_OUTPUT` similar to the mechanism described in Sect. 2.4.1 to specify respectively entry points, inputs, and outputs of each TA. The provided annotations take a name as an argument that is chosen by the developer. Then, our framework produces the desired functions using some defined macros at deployment time. Likewise, the remaining required codes for developing a TA such as communication with a client application and cryptographic operations are injected by our framework automatically. Thereby, the provided abstraction enables the developer to implement complex logic without knowledge of the TA implementation in OP-TEE.

Untrusted Runtime

The untrusted runtime involves an unprotected application running on Linux which is called the event manager to implement all the components outlined in Sect. 2.3.3. Our event manager leverages the GlobalPlatform TEE Client API implemented in OP-TEE client library to communicate with TAs. This API provides a set of functionalities for initializing and running TAs in the secure world as well as mechanisms for passing data to them. Therefore, when a new `LoadModule` request arrives, the event manager first establishes a connection with TEE through `TEEC_InitializeContext(...)` function which returns a context object *ctx*. Second, a logical connection with a specific TA within the scope of *ctx* is created by passing UUID of the desired TA to `TEEC_OpenSession(...)` function. When a session is opened, the TA's binary and its libraries are loaded into the secure memory. This session is then used to invoke the TA's entry points for `CallEntry` requests using `TEEC_InvokeCommand(...)`, which takes as parameters a command ID defined for the intended entry point as well as the expected payload. Our event manager uses

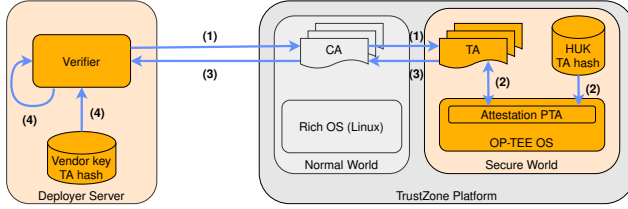


Figure 2.11: Our remote attestation process for ARM TrustZone with OP-TEE. Colored components are considered trusted in our architecture.

this operation payload to exchange events with a TA through shared memory. It is noteworthy to mention that the entry points verify the type of parameters before using them to check its value according to the expected parameter. In addition, the data passed to entry points is encrypted and authenticated. Thereby, any malicious attempts to call an undesired entry point or modify the payload would result in a failed cryptographic operation.

Remote attestation and secure channels

Remote attestation is a crucial security feature for our framework that provides an authorized party with cryptographic proof that a known and benign TA is initially loaded on a valid TrustZone device. In contrast to Sancus and Intel SGX, the TrustZone specification offers no mechanism for remote attestation. Some existing approaches leverage hardware components such as a hardware Trusted Platform Module (*TPM*) or a firmware-based TPM (e.g., Microsoft’s *fTPM* [174]) that provides security guarantees similar to a hardware TPM, to perform attestation. In our work we avoid the added complexity and substantial increase of the TCB, and introducing no additional hardware. Instead we propose an attestation mechanism based on native TrustZone features.

Our design is based on Sancus’ remote attestation scheme (Sect. 2.4.1). Analogously, our approach relies on symmetric cryptography and a three-level key hierarchy. We build upon an OP-TEE proof-of-concept implementation for remote attestation that requires minimal hardware features and assumptions⁵, avoiding the use of an TPM/*fTPM* as they are not available on many platforms. We implemented an attestation protocol derived from the OP-TEE proof-of-concept to address the requirements of our framework, while minimizing the run-time TCB.

⁵See https://github.com/OP-TEE/optee_os/issues/3189/ and https://github.com/OP-TEE/optee_os/pull/4011/. Very recently, attestation is being picked up and included in release 3.17 (April 2022) by the OP-TEE team again.

Our scheme assumes that each TrustZone device is equipped with a Hardware Unique Key (HUK) (also called *endorsement key* in other technologies), which can be created at manufacture time and stored either in hardware fuses or the secure eMMC partition. Similar to hardware devices like the TPM or Sancus, the HUK is protected from unauthorized access and can be used as root-of-trust of the TrustZone device. Since the secure boot process guarantees system integrity, OP-TEE can be treated as the trusted base for accessing the HUK, and the isolation properties provided by TrustZone protect secure-world code at run-time. Thus, we get a strong guarantee that the HUK can only be accessed by privileged code in the secure world.

To leverage the HUK and provide attestation primitives to the TAs, we developed an attestation Pseudo Trusted Application (PTA)⁶ that is statically built into the OP-TEE core and runs at the same privileged execution level as the OP-TEE OS. Analogous to Sancus attestation, this PTA uses a Key Derivation Function (KDF) to retrieve symmetric vendor and module keys from the HUK. In particular, the vendor key is computed as follows:

$$\text{vendor_key} := \text{KDF}(\text{HUK} \parallel \text{vendor_id})$$

Identically to Sancus (cf. Section 2.4.1), **vendor_id** is a unique identifier assigned to the software vendor, and the **vendor_key** needs to be securely communicated out-of-band. The **vendor_id** is then embedded in the TA code and passed to the PTA during the attestation process. The module key, instead, is computed as:

$$\text{module_key} := \text{KDF}(\text{vendor_key} \parallel \text{TA_hash})$$

The TA hash is used by the OP-TEE OS to verify the integrity of the TA at load time⁷. We made minimal changes to the OP-TEE OS to store the hash in secure memory after loading a TA, so that it could be retrieved by the attestation PTA for computing the module key. The module key can be also used to establish a secure communication channel between the TA and the remote party.

Our remote attestation process shown in Fig. 2.11 is illustrated in detail as follows:

1. The verifier sends an attestation request to the client application running in the normal world on the target device. The request contains a randomly

⁶PTAs are an OP-TEE concept. They provide interfaces that are exposed by the OP-TEE Core to client TAs. See https://optee.readthedocs.io/en/latest/architecture/trusted_applications.html for more details.

⁷The TA's binary structure and loading process are described in detail at https://optee.readthedocs.io/en/latest/architecture/trusted_applications.html.

- generated *challenge* to provide freshness. Then, the client application passes the received *challenge* to the intended TA through shared memory.
2. The TA leverages the attestation PTA to obtain the module key K . The attestation PTA performs a double derivation to retrieve the module key from the HUK, as explained above. To do so, the vendor identifier is passed as parameter by the TA itself, whereas the TA hash is fetched from secure memory. After receiving K , the TA calculates a MAC D of the provided *challenge*.
 3. The TA sends D to the client application through shared memory and then D is forwarded to the remote verifier.
 4. The verifier derives the same module key K from the vendor key and TA hash. Then, the verifier uses K to compute a MAC of *challenge* and then compares it with the received MAC. If the two values match, the target device is authenticated and the TA integrity is verified.

The KDF used in our proof-of-concept is a simple SHA-256 hash, though we truncate the module key to 128 bits to align its size with the other TEE implementations. For MAC function we leveraged the AES-128-GCM authenticated encryption scheme, using the challenge as associated data.

Similarly to Sancus, attestation binds a TA to a specific node because the module key is (indirectly) derived from the HUK. Deploying the TA on a different node would result in a different module key. Since the TA's UUID is hard-coded, multiple instances of the same TA would have different hashes and consequently different module keys, provided that each instance uses a different UUID.

This attestation scheme gives strong guarantees that a TA is running expected code on a particular TrustZone node. The freshness of the challenge prevents replay attacks, ensuring that the TA is up and running at the time of the attestation. Adversaries cannot learn the module key by sniffing the network traffic, nor can they impersonate a TA to compute the response to the challenge. Besides, any attempt to modify the messages between TA and verifier would only result in a failed attestation. However, the scheme relies on the security of the HUK and vendor key: if a HUK is leaked, all TAs deployed on its corresponding TrustZone node are compromised, while leaks of a vendor key only compromises the TAs of its corresponding software vendor deployed on its corresponding node. Hence, a device manufacturer should provide software vendors with a secure interface (e.g., an authenticated API) to retrieve the vendor key for a specific TrustZone node.

	Isolation	SW Attestation	TCB Attestation	Memory Protection	Sealing	Code Confidentiality	Secure I/O	HW-Only TCB	Upgradeable TCB	Preemption	Dynamic Layout	Target ISA
Sancus	●	●	●	○	○	●	●	●	○	●	○	MSP430 (16-bit)
SGX	●	●	●	●	●	●	○	○	●	●	●	x86_64 (64-bit)
TrustZone ^a	●	●	●	○	●	●	○	○	●	●	●	ARM (32-bit)

● = Yes; ◐ = Possible; ○ = No

Table 2.1: Comparison of TEE hardware features and the resulting security guarantees and application capabilities in our framework. This table is derived and adapted from [138]. *Footnotes* — ^a: We consider TrustZone with the OP-TEE OS and our extensions for attestation.

Secure I/O

TrustZone leverages dedicated hardware components to enforce hardware isolation to the I/O devices. Specifically, TrustZone introduces the TrustZone Protection Controller to define the access restrictions for peripherals and configure them as secure and non-secure dynamically or statically. This is achieved by the reflection of Non-Secure bit into the respective peripheral. Several works take the advantage of TrustZone to establish a trusted I/O path for different purposes [127, 62, 199, 232] (Sect. 2.7.3). Our framework does not support physical I/O channels for TrustZone-based modules yet but rather interfaces Sancus-enabled I/O devices.

2.4.4 Comparison: SMs on Different TEEs

Sect. 2.4.3 summarizes the features of each TEE and highlights their main differences. This table is derived and adapted from [138]. Note that for TrustZone we consider OP-TEE OS with our attestation extensions. The key points that distinguish our TEEs are *a*) software isolation which is supported by all three TEEs and where we provide common interfaces to control isolation and event handling; *b*) attestation primitives, which are not available on all TEEs and we provide attestation support as necessary and establish common Attest entry point and event handler abstracting for our framework; and *c*) secure I/O, which is only natively supported with Sancus and where we design generic scheme that can hopefully be instantiated by other TEEs, in particular by ARM TrustZone.

Software isolation and attestation are strict requirements in our design (Sect. 2.3.1), which are fully satisfied by Intel SGX and Sancus. For TrustZone, instead, we relied on OP-TEE OS for isolation, whereas we designed and implemented our own attestation protocol based on Sancus (Sect. 2.4.3). Other than code integrity and authenticity, attestation may also provide additional information, such as the security configuration of the underlying TCB (Intel SGX), or the guarantee that an SM is running on a specific node (Sancus and TrustZone). The latter is particularly useful when dealing with driver modules (Sect. 2.3.4). Besides, only Intel SGX protects the integrity and authenticity of data from physical attacks, thanks to its Memory Encryption Engine (MEE).

Both Intel SGX and TrustZone support sealing, i.e., storing data securely on permanent storage. However, our framework does not currently provide any abstraction over sealing primitives, meaning that developers need to manually implement code to call such primitives. Similarly, code confidentiality (i.e., deploying encrypted SMs in order to protect sensitive code and static data) is a feature supported by all architectures, though not essential for the use cases we consider. Nevertheless, both sealing and code confidentiality could be addressed in future work. Besides, Sancus fully supports secure I/O (Sect. 2.4.1), while on TrustZone and Intel SGX some extensions are needed, as proposed in related work (Sect. 2.7.3).

Concerning architectural features, only Sancus provides an hardware-only trusted computing base. In fact, Intel SGX relies on software parts such as microcode and attestation enclaves, while TrustZone includes the whole OP-TEE operating system in its TCB. However, software can be easily upgraded to provide new functionalities or fix bugs, which is why Intel SGX and TrustZone support TCB upgrades. Furthermore, both TrustZone and Intel SGX have full support for interrupts and dynamic layout (i.e., virtual memory). Sancus, being an embedded TEE, does not support virtual memory, whereas for interrupts we offer partial support (Sect. 2.4.1).

2.4.5 Deployment

We developed a Python script called `reactive-tools` to ease the deployment and initialization of a distributed application. In essence, this script builds the SMs and then communicates with the remote event managers to bootstrap enclaves and establish connections (Sect. 2.3.3). `reactive-tools` takes as input a *deployment descriptor*, a configuration file that contains a high-level description of the application to deploy.

```

nodes:
- type: sancus
  name: node_sancus
  host: node-sancus
  vendor_id: 4660
  vendor_key: 0
    b7bf3ae40880a8be430d0da34fb76f0
  reactive_port: 6000
modules:
- type: sgx
  name: webserver
  node: node_sgx
  vendor_key: cred/vendor_key.pem
  ra_settings: cred/settings.json
connections:
- from_module: button_driver
  from_output: button_pressed
  to_module: gateway
  to_input: button_pressed
  encryption: spongent

```

Figure 2.12: Excerpt from a deployment descriptor in YAML format. Three main sections are defined: **nodes**, **modules** and **connections**. Each node and module has a **type** field indicating the TEE technology used; besides the fields that are common to all TEEs, each platform also requires other specific parameters. The fields of a connection, instead, do not change. The complete deployment descriptor (in JSON format) used in this example can be found at <https://github.com/AuthenticExecution/examples/blob/main/button-led/descriptor.json>.

Deployment descriptor

The deployment descriptor contains three main sections: **nodes**, **modules** and **connections**. Fig. 2.12 shows an excerpt from one of our sample applications.

As shown in the figure, a module is statically assigned to a node and a specific TEE technology. This is a limitation of our framework as each TEE implementation uses its own API and programming language. A possible direction for future work could be to use an unified language for defining modules, which can be an existing programming language such as Rust or a custom language such as the pseudocode we used in Fig. 2.3; then, the framework could automatically select the right compiler/toolchain according to the node in which the SM is going to be deployed. However, it is worth noting that not all TEEs provide the same features: for example, Intel SGX supports data sealing, whereas Sancus does not support persistent storage at all.

In the **connections** section, the deployer declares connections between SMs. A connection links together two endpoints, each of them identified by the pair $\langle SM_name, endpoint_name \rangle$. Moreover, each connection must indicate the authenticated encryption algorithm to use for protecting events in transit. Currently, both Intel SGX and TrustZone support AES-Galois-Counter Mode (GCM) and SPONGENT (both with 128 bits of security), whereas Sancus only supports SPONGENT.

Deploying a new application

`reactive-tools` provides separate commands for the deployment and attestation of modules, as well as for the establishment of connections. To this end, the deployer would run the `deploy`, `attest`, and `connect` commands in sequence, each time providing as input the same deployment descriptor. If all these commands succeed, the deployment is complete and the application is ready to process events.

Interfacing with the application

At runtime, the deployer might want to interact with the application in a secure way, e.g., to initialize a web server or to get some metrics from a database. However, the `CallEntry` API provided by the event managers to call modules' entry points is not secure, as for this kind of message the data is transmitted unencrypted over the network. Hence, the deployer would need to manually implement encryption and/or authentication mechanisms for data exchanged with the application.

Nevertheless, to facilitate the secure communication between a deployer and their SMS, our framework provides a convenient feature called *direct connections*. Unlike normal connections between two modules, a direct connection uses the deployer's machine as source endpoint. Since connection keys are generated and distributed at deployment time by our script, it is possible to also use such keys to generate output (or request) events and send them directly to running modules.

These connections can be defined in the deployment descriptor together with the others, and are characterized by the additional `direct:true` field. The deployer can then either generate new events manually or automatically using `reactive-tools`; either way, the receiving module processes these events just as any other event. Since the deployer's machine is assumed to be trusted, the same security guarantees as normal connections apply for direct connections as well.

2.4.6 Attestation and Key Management

To ease the attestation of modules and the storage of encryption keys, we developed an optional component called the *Attestation Manager*. It consists of an Intel SGX enclave that is deployed on the infrastructure together with the

applications, and to which the deployer and `reactive-tools` can send requests through a dedicated command-line interface.

When the attestation manager is deployed, it first needs to be attested using the usual Intel SGX attestation process. Similarly to the attestation of SMs, a shared secret is exchanged between the challenger (i.e., the deployer) and the enclave, which is then used to establish a secure channel.

After the initialization is complete, the attestation manager is ready to receive **Attest** requests from the deployer; the attestation manager then takes care of executing the actual attestation logic, based on the TEE technology used by the module. If the attestation succeeds, the attestation manager stores the module key securely in the enclave memory, and it can be fetched (or used indirectly) by the deployer at any time.

Advanced usages

The attestation manager, when used, may enable additional use cases. Below, we shortly describe some of them. Note that we only aim to provide an informal and high-level outline of such use cases, to motivate the usage of a centralized attestation/key management component. As such, we will not go into details on the design nor perform a security analysis.

- *Key management.* The simplest use case is to merely store credentials (i.e., module and connection keys), which can be accessed by the deployer or any other authorized entities when needed. Credentials can be stored in enclave memory or persisted in storage using sealing capabilities of TEEs such as Intel SGX. Note that the latter case may introduce vulnerabilities against rollback attacks [9].
- *Distributed confidential computing.* Certain applications process privacy-sensitive information, such as health-related data. TEEs already protect against honest-but-curious infrastructure providers, while attestation ensures the authenticity of application code and proves that the isolation mechanisms of a TEE are correctly in place. However, in some cases it may be desirable to conceal sensitive data even from the deployer, who normally has access to connection keys and could potentially decrypt data in transit between modules. Therefore, the attestation manager could be used to generate and distribute connection keys to modules without the deployer learning any information about their value.
- *Root of Trust in edge devices.* In some edge scenarios such as automotive/Vehicle-to-Everything (V2X), nodes may be part of a local

network that does not have continuous access to Internet services. In such cases, it might be useful to deploy an attestation manager on the network responsible for the initialization and attestation of local enclaves. For instance, we could envision a TrustZone gateway in a car that initializes all Sancus' Electronic Control Units (ECUs) when the engine is turned on, refusing to start the car if any of the attestations fails [206]. In this scenario, the gateway acts as the *root of trust* of the system, and its remote attestation would be possible as soon as the system comes back online.

2.5 Evaluation

To evaluate our framework, we developed a prototype for a Smart Home application that combines together Intel SGX, ARM TrustZone and Sancus. In this chapter, we introduce our Smart Home environment and motivate security requirements (Sect. 2.5.1); then, we describe the hardware and software setup of our evaluation (Sect. 2.5.2). After that, we provide a detailed performance analysis of our application (Sect. 2.5.3), and finally discuss TCB size and development effort (Sect. 2.5.4).

2.5.1 Smart Homes

A smart home is a residential property integrated with technology to remotely control appliances and systems, such as lighting, heating and cooling, and entertainment. The adoption of smart home devices has increased rapidly, with an estimated 250 million smart homes worldwide in 2021, expected to reach 350 million by 2023⁸. Despite the benefits such as enhanced comfort and energy efficiency, security concerns arise from inadequate security measures in many smart home devices. Studies like [63] have shown that many vulnerabilities come from poor authentication, missing encryption, insecure software updates, and insufficient access control, leading to potential privacy violations and unauthorized access to personal information and control of home devices.

Thus, our framework can be utilized in the context of smart homes to provide robust security guarantees. Through the use of TEEs, both code and data are protected in use, and attestation ensures code integrity. Additionally, our deployment approach supports secure software updates and guarantees that all connections are encrypted and mutually authenticated. This eliminates a broad range of attacks while minimizing the development and deployment effort.

⁸<https://www.statista.com/topics/2430/smart-homes>

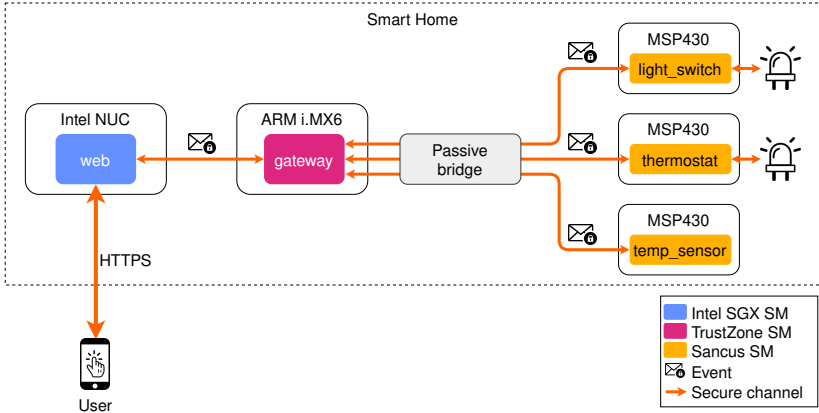


Figure 2.13: Setup of our *smart home* application. Each SM is deployed on a different node, either running Intel SGX, TrustZone or Sancus. All communication between modules is encrypted and authenticated by our framework, while the communication between *web* and users relies on HTTPS and mutually authenticated. Sancus nodes are connected via UART to a passive bridge, which converts between UART streams and TCP/IP.

2.5.2 Testing Environment

To evaluate our framework, we developed a prototype for a small smart home application consisting of three simulated IoT devices: a temperature sensor, a smart thermostat and a light switch. A smart home gateway, similar to existing projects such as Home Assistant⁹, manages the IoT devices and enforces some custom-defined logic, e.g., to automatically turn on or off the heating if the temperature goes below or above predefined thresholds. Furthermore, a web application is made available to local and remote users, to monitor the house and perform some operations on demand, e.g., to switch the lights on or off. Both smart thermostat and light switch are connected via Secure I/O to an LED, indicating whether heating and lights are on or off at a certain time. The source code of our prototype is publicly available on Github¹⁰.

We implemented an application such as described above using five SMs:

- *web*: Exposes a web application to allow external users to interact with the smart home;

⁹<https://www.home-assistant.io/>

¹⁰<https://github.com/AuthenticExecution/examples/tree/main/smart-home>

- *gateway*: Manages all the IoT devices and enforces a user-defined logic, while at the same time interacting with *web* to send status data and receive commands from external users;
- *temp_sensor*: Simulates a sensor that provides the current temperature in the house;
- *thermostat*: Simulates a smart thermostat, to enable or disable the heating system;
- *light_switch*: Simulates a smart light switch, to enable or disable the lights.

As depicted in Fig. 2.13, we deployed our smart home application as follows:

- *web* as an Intel SGX SM, on an Intel NUC7i3BNHXF with Intel Core i3-7100U;
- *gateway* as a TrustZone SM, on a BD-SL-i.MX6 board with ARM Cortex-A9 running at 1GHz;
- *temp_sensor*, *thermostat* and *light_switch* as Sancus SMs, on three different 16-bit OpenMSP430 microcontrollers running at 8MHz.

In our setup, all nodes communicate over TCP/IP in the same local network; However, multiple deployment strategies may be adopted, e.g., deploying *web* in a public cloud. Since our Sancus microcontrollers can only communicate through UART, we wrote a Python script that acts as a passive bridge, converting UART streams to TCP/IP packets and vice versa without being able to decrypt or manipulate the content of events.

All communication is end-to-end protected from the user to the IoT devices. As shown in the figure, communication between SMs is carried over encrypted and authenticated *events*, leveraging our framework (Sect. 2.3). Instead, the communication channel between *web* and the user is encrypted using HTTPS. User and *web* mutually authenticate each other: *web* is authenticated by the user during the TLS handshake, during which *web* presents an ephemeral X.509 certificate generated after successful attestation; the user, instead, authenticates by providing a *secret token*, similarly to session cookies or JSON Web Tokens (JWTs).

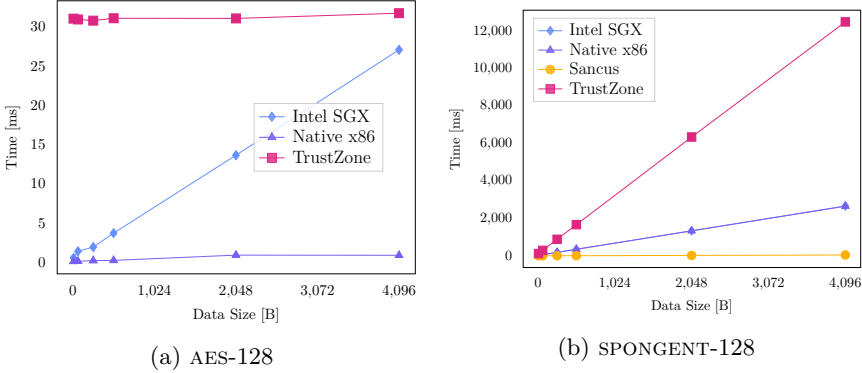


Figure 2.14: Average timings to perform encryption using AES-128 and SPONGENT-128 on different TEEs. As a reference, we also measured the average time spent on a Linux x86 process without TEE protection (i.e., the *Native x86* line). All measurements are in milliseconds.

2.5.3 Performance Benchmarks

We analyzed the performance of the Smart Home application introduced above. Cryptographic overhead was assessed per node and an end-to-end evaluation was conducted by measuring the round-trip time of an event sent from *web* to *light_switch* and return. The impact of a module update was evaluated by measuring application downtime during the update.

Cryptographic operations

Fig. 2.14 shows the average time calculated on each TEE to perform crypto operations using either AES-GCM or SPONGENT as authenticated encryption library, with 128 bits of security. We used different sizes for the data to encrypt, ranging from 8 to 4096 bytes. To provide a reference, we also carried out the same experiments on a simple Linux x86 process without TEE protection, running on the Intel SGX node (*Native x86* in the figure). The plots show average timings computed over 110 iterations, except that Sancus values have been extrapolated from previous experiments [163].

Modern x86 and ARM processors include AES instructions in their instruction set, allowing them to perform cryptographic operations in hardware for improved performance and security. As shown in Fig. 2.14a, a single AES encryption is extremely fast natively, taking around 878 μ s to encrypt 4096 bytes of data. The overhead caused by the enclaved execution in Intel SGX significantly slows

down the execution of these functions with a factor that increases linearly with the size of the data: encrypting 4096 bytes takes up to 31 times more than natively. Regarding TrustZone, instead, it can be observed from Fig. 2.14a that the payload size had little impact on the encryption time, with values between 30 and 31 ms and standard deviation of 0.3 ms. We noticed that TrustZone has a fixed overhead due to some system calls that need to be called to initialize each cryptographic operation. In our case, the TA has to call `TEE_ResetOperation`, `TEE_AEInit` and `TEE_AEUpdateAAD`¹¹. Instead, we could not provide any data for Sancus, as it does not include an AES module in its architecture.

The lack of an AES engine in Sancus is a serious issue for our framework, as it prevents Sancus modules to communicate securely with modules of other TEEs and the deployer. To circumvent this problem, a software implementation of the SPONGENT crypto library was implemented in previous work, in both C/C++ and Rust. Thus, both Intel SGX and TrustZone modules can leverage such library to exchange protected events with Sancus modules. However, the overhead for performing such operations in software is significant: as shown in Fig. 2.14b, while for small amount of data the difference is less prominent, the performance heavily degrades for bigger data. Compared to AES and for payloads ranging between 8 and 4096 bytes, SPONGENT is up to 3026 times slower on native x86, up to 99 times slower in Intel SGX, and up to 395 times slower in TrustZone. Instead, experiments show that SPONGENT is significantly faster in hardware¹². Unfortunately, this poor performance may completely compromise the real-time requirements of certain use cases, and replacing SPONGENT with AES might be appropriate, depending on the acceptable processing time and power consumption on Sancus nodes for a specific use case.

End-to-end measurements and RTT

We performed end-to-end experiments on our Smart Home application to measure execution times and round-trip time (RTT). In particular, we evaluated the scenario where an external user manually enables the lights by sending an HTTP request to *web*. This event triggers the following flow:

1. *web* sends an event to *gateway* containing 2 bytes of payload that encodes the desired action (i.e., enable the lights);

¹¹<https://optee.readthedocs.io/en/latest/architecture/crypto.html>

¹²The SPONGENT [39] family of light-weight hash functions are optimized for hardware implementation. We have confirmed in independent experiments that the implementation in Sancus indeed outperforms a software implementation on other architectures by several orders of magnitude.

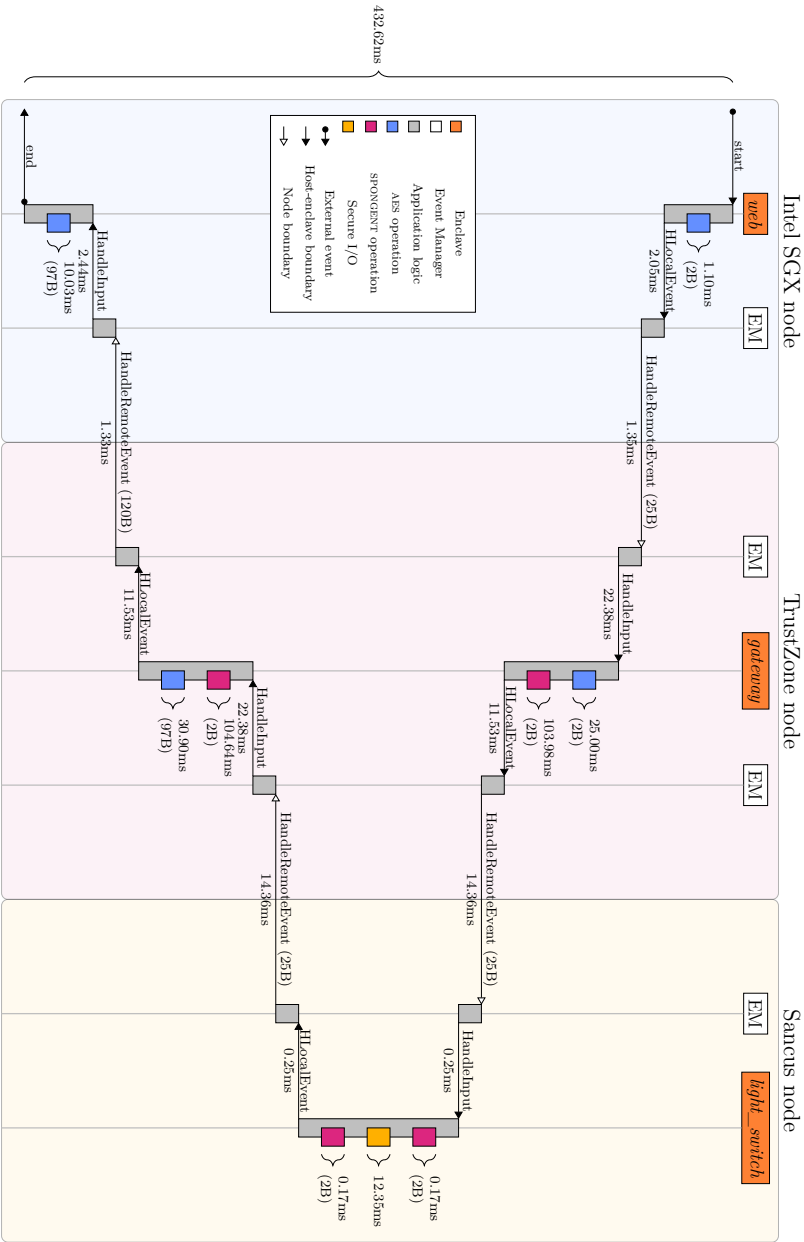


Figure 2.15: Sequence diagram showing the control flow and timings of the Smart Home application in Fig. 2.13. The diagram illustrates a scenario where the user manually switches the lights; bar lengths are *not* to scale.

Operation	Time (ms)	% of RTT
AES instructions	67.03	15.49
SPONGENT HW instructions	0.34	0.08
SPONGENT SW instructions	208.62	48.22
Host-enclave boundary	72.81	16.83
Secure I/O	12.35	2.86
Network delay	31.40	7.26
Other	40.07	9.26
RTT	432.62	100.00

Table 2.2: End-to-end measurements of the Smart Home application, aggregating the average time spent for performing specific tasks. The last row shows the round-trip time (RTT), which consists of the sum of all the intermediate times. In the third column, we show the impact of each task on the RTT.

2. *gateway* decrypts the event and generates a new one for *light_switch*, using the same payload;
3. *light_switch* receives and decrypts the event, then dispatches an internal event to turn the LED on. After that, it sends an event back to *gateway* as a notification that the lights have been turned on, using the same 2 bytes of payload as before;
4. *gateway* receives the event and updates its internal state. Then, it generates an event for *web* containing the current status of the smart home in JSON format, whose size is 97 bytes;
5. Finally, *web* receives the updated status. Starting from this moment, the user will be able to see in the web application that the lights have been successfully turned on.

Fig. 2.15 shows the sequence diagram of the flow just described. Time was recorded to measure the performance impact of the most important steps: encryptions/decryptions, host-enclave boundary crosses, Secure I/O, and transmission times. We also measured the end-to-end RTT, consisting of the difference in time between the *start* and *end* points in *web*. The sequence diagram shows average timings over 110 iterations. The average RTT was 432.62 milliseconds, which includes 8 encryptions/decryptions (4 using AES and 4 using SPONGENT), 8 host-enclave boundary crosses and 4 network transmissions between nodes. To better understand the performance impact of each operation, timing values have been aggregated and shown in Tab. 2.2.

Around 64% of the RTT was spent on encrypting/decrypting events: On the one hand, cryptographic operations performed in hardware were generally fast,

taking a total of 67.03 ms for AES and 336 μ s for SPONGENT. The former, however, was heavily affected by the fixed TrustZone overhead for initializing cryptographic operations, which may be optimized in future releases of OP-TEE. On the other hand, SPONGENT operations performed in software caused huge performance penalties on the application, taking a total of 208.62 ms ($\sim 48.2\%$ of the RTT).

Crossing the boundary between host and enclave accounted for the 16.83% of the RTT (72.81 ms); in particular, we observed that the slowest transitions between untrusted and trusted domains were registered in the TrustZone node, with an average of 17 ms for each transition. Domain transitions in Sancus proved to be extremely fast, each of them requiring only around 250 μ s. In Intel SGX, instead, it took roughly 2.2 ms to enter/exit the enclave.

Poor network performance in Sancus had a non-negligible impact on our measurements. In fact, the average total transmission time was 31.40 ms, the 91% of which spent on the transmission of events to and from Sancus microcontrollers. As described in Sect. 2.5.2, this is due to the low baud rate (57600 bps) of the UART interface and the overhead caused by the passive device used to communicate with the Sancus microcontrollers.

From our experiments, we observed that the Secure I/O overhead was on average 12.35 ms which, according to Sect. 2.4.1, includes: (1) sending an event from *light_switch* to the LED driver module (`mod-driver`); (2) sending an event from the LED driver module to the LED MMIO module (`mod-mmio`); (3) executing the logic to enable the LED. We did not measure such steps independently in this evaluation, since the main objective of our evaluation was to assess and evaluate the inter-operation between different TEEs and measure the communication overhead across different implementations. However, we extensively evaluated Sancus in our previous work [162].

Finally, an average of 40.07 ms ($\sim 9.26\%$ of the RTT) was due to other application logic that was not measured during the experiments. In the sequence diagram, this consists of the grey rectangles minus the time spent for performing cryptographic operations and Secure I/O. For example, this logic includes the time spent for the look up of a `Connection` structure given a specific connection ID, or for finding the correct SM to which deliver an event. The performance of such operations is implementation-specific, and may be improved by using more efficient logic and data structures.

In summary, our experiments demonstrate that the overhead of our framework does not affect user experience for near real-time applications such as Smart Home. Considering our setup, after sending out a command, the user receives a feedback in less than 500 milliseconds. Our micro-benchmarks also show that the

Module	TEE	Build	Deploy	Attest	Connect	Total
<i>web</i>	Intel SGX	1.199	0.247	1.794	0.775	4.027
<i>temp_sensor</i>	Sancus	0.443	2.510	2.576	0.251	5.833
<i>gateway</i>	TrustZone	0.976	0.438	0.329	1.277	3.063

Table 2.3: Average time to perform a software update on different TEEs, all measurements are in seconds. We carried out the experiments on the Smart Home application, using the same setup described in Fig. 2.13. Build, deployment, and attestation times are highly application-specific; in our case, all modules were re-deployed without code changes and no state transfer between old and new instances was implemented.

RTT can be further improved by adopting different deployment strategies, e.g., by merging *web* and *gateway* together in a single SM. For real-time applications, however, this may not be sufficient and additional capabilities might be needed, such as hardware acceleration for all cryptographic operations. We also note that all experiments were performed on a prototype that is not fully optimized, and all modules were built in debug mode; Thus, we expect that the overall performance would improve in a production environment.

Module update

Real-world applications require changes at runtime, e.g., software updates or module migration from one node to another. This process impacts on the availability of the whole application, because updating the application unavoidably results in temporary loss of connectivity. Therefore, we evaluated the overhead of a software update in our framework, for all our supported TEEs. Again, we carried out our experiments on the Smart Home application (Figure 2.13); results are shown in Tab. 2.3, each value representing the average over 10 iterations.

In our experiments, we re-deployed the original SMs without any code changes. As shown in Tab. 2.3, the build time was rather small for all TEEs, ranging from nearly 450 ms for Sancus to around 1.2 seconds for Intel SGX. However, building times highly depend on the amount of changes made to the original code, and whether the compiler’s cache was used or not. Hence, these measurements are not really interesting because highly variable. Similarly, deploying the new SM on the infrastructure depends on the size of the SM itself and the bandwidth of the communication medium. The table shows that the deployment of a Sancus module is a slow process, taking an average of 2.510 seconds; this can

be explained by the slow communication channel between the Sancus node and the deployer, as explained above. The deployment process for Intel SGX and TrustZone takes only 247 ms and 438 ms, respectively; Binary sizes are shown in Tab. 2.4.

Our experiments show that the attestation time greatly varies according to the TEE used. Attesting an Intel SGX enclave took an average of 1.794 s; in fact, the modified sigma protocol used in our framework consists of several steps, including the involvement of the Intel Attestation Server (IAS) to decrypt the quote (Sect. 2.4.2). For TrustZone, instead, we use a simple challenge-response protocol, as described in Sect. 2.4.3. Thus, attesting a TrustZone module is very fast and takes an average of 329 ms. Sancus uses a similar mechanism, though its attestation takes much longer, i.e., 2.576 s on average. However, we observed that the majority of this time was actually spent on the deployer’s side to compute the module key from the module’s binary; the actual challenge-response protocol only took less than 100 milliseconds.

Finally, after the new module is ready (i.e., deployed and attested), all connections need to be redirected from the old to the new instance. Following the update strategy illustrated in Sect. 2.3.7, this is the time in which the application would suffer from temporary connectivity loss. However, our experiments show that this process only takes a very short amount of time, ranging from 251 ms for Sancus to 1277 ms for TrustZone. It has to be noted, though, that these values only represent the time to *deliver* all the events to the SMs and event managers (**SetKey** and **AddConnection** events, respectively), and do not include the processing time in the nodes. Hence, the effective time would be a bit higher. Besides, this time is linearly dependent on the number of connections to re-establish; in our case, we had five connections for *web*, nine for *gateway*, and two for *temp_sensor*.

In conclusion, the overall time for a software update is highly variable and application-dependent. Nevertheless, the connectivity loss is only perceived during the Connect phase, which roughly takes around 150 milliseconds for each connection to re-establish.

2.5.4 Adapting Burden

We performed a code evaluation of our Smart Home application, results of which are shown in Tab. 2.4. Our analysis focuses on two main aspects: first, the TCB size in relation to the whole software stack; second, an estimation of the developer’s effort. Concerning the TCB, we calculated both lines of code and binary sizes, while for the developer’s effort we only focused on the code.

Component	Src (LOC)	Bin (B)	Component	Src (LOC)	Bin (B)	Component	Src (LOC)	Bin (B)
RIOT OS	9713		Host OS ^a	>700M	314.3M	Host OS ^a	>700M	314.3M
Event Manager	1312		Event Manager	623	1.2M	Event Manager	1448	17.3K
Total untrusted	11025	35638	Total untrusted	>700M	315.5M	Total untrusted	>700M	314.3M
Stub code and libraries	797		Fortanix EDP	>20K		OP-TEE OS	286K	244K ^b
<i>thermostat</i>	10	9001	Stub code	3412		Stub code	1432	
<i>temp_sensor</i>	21	9423	3rd party libraries	~20K		<i>gateway</i>	101	111.3K
<i>light_switch</i>	10	9079	<i>web</i>	372	4.2M	Total trusted	287K	355.3K
Total trusted	838	27503	Total trusted	>43K	4.2M	TCB reduction (%)	99.9	99.9
TCB reduction (%)	92.9	56.4	TCB reduction (%)	99.9	98.7	Dev. effort (%)	6.6	
Dev. effort (%)	4.9		Dev. effort (%)	9.8				

(a) Sancus

(b) Intel SGX

(c) TrustZone

Table 2.4: Size (“Src”: source code, “Bin”: binary size) of the software for running the evaluation scenario. Shaded components are part of the run-time software TCB. For a fair comparison, we only consider source code (e.g., C/C++, Rust files), and not build scripts or other similar files. Besides, our evaluation does not include compilers, standard libraries, and other software layers such as hypervisors. TCB reduction is calculated as untrusted code over total code, while developer’s effort as application logic (e.g., *gateway*) over total trusted code, which includes stub code but excludes third party libraries. Module binaries have been built in debug mode. *Footnotes* — ^a: We use Debian 11 (bullseye) as reference [64]; ^b: As of 2016, according to the official documentation.

We note that application modules were built in debug mode, which may have caused slightly bigger binaries.

One of the most important benefits of process-based TEEs is a substantial reduction of the TCB, leading to a considerably reduced attack surface on each node. Tab. 2.4 shows that the reduction in Sancus is 92.9% in lines of code (LOCs) and 56.4% in binary size, while on SGX this reduction is more prominent with 99.9% reduction in code and 98.7% in binary size. Interestingly, TrustZone achieves a 99.9% reduction in both, which means that OP-TEE is only a thin software layer compared to a classic operating system. It has to be noted that the code reduction in SGX and TrustZone is enhanced by the fact that our reference host OS (Debian 11) consists of more than 700 million C/C++ LOCs, and more than one billion LOCs overall [64].

Our framework strives to simplify the development of distributed enclave applications. As shown in Sects. 2.3 and 2.4, our API allows a developer to only specify the core logic of a software module, while the rest of the code is added at compile time by the framework. Our evaluation of the developer’s effort consists of the ratio between the LOCs written by the developer and the total LOCs of a module, including stub code but excluding third party libraries. Results show that the amount of code written by the developer is minimal: only 4.9% for Sancus, 9.8% for SGX and 6.6% for TrustZone. However, these numbers may greatly vary: our evaluation was carried on a simple prototype with a few modules; More complex applications would require higher effort.

The above evaluation can only provide rough estimates of the benefits of our framework: Performing a precise code evaluation is not a trivial task. Kernels and operating systems typically make extensive use of conditional compilation to enable/disable specific features or to target different hardware architectures. Thus, even though an operating system may have millions or more LOCs, most of them are not used during compilation. Besides, the number of LOCs is often magnified by auxiliary files such as build scripts, Makefiles, etc. It is debatable whether these files should be included in the calculation or not: they are not part of the core logic of a software, but they can nevertheless introduce vulnerabilities at compile time. Applications, instead, may include several third-party libraries which size might not be easy to calculate: examples are closed-source and dynamic libraries. Finally, our evaluation did not consider software components such as bootloaders, firmware/BIOS, hypervisors, compilers, standard libraries, etc. While some of these are untrusted, others are part of the TCB and must be taken into account for a complete evaluation.

2.6 Discussion

2.6.1 Integrity versus Confidentiality

We have focused our security objective on integrity and authenticity, and an interesting question is to what extent we can also provide confidentiality guarantees. It is clear that, thanks to the isolation properties of protected modules and to the confidentiality properties of authenticated encryption, our prototype already provides substantial protection of the confidentiality of both the state of the application as well as the information contained in events. However, providing a formal statement of the confidentiality guarantees offered by our approach is non-trivial: some information leaks to the attacker, such as for instance when (and how often) modules send events to each other. This in turn can leak information about the internal state of modules or about the content of events. The ultimate goal would be to make compilation and deployment fully abstract [1] (indicating roughly that the compiled system leaks no more information than can be understood from the source code), but our current approach is clearly not fully abstract yet. Hence, we decided to focus on strong integrity first, and leave confidentiality guarantees for future work.

There is also a wealth of orthogonal research aiming to protect network information flows from being used by attackers to learn the system state by means of, e.g, covert channels or routing protocols for mix networks (cf. [191] for an overview). The applicability of these approaches depends on application configuration and available system resources and may heavily impact availability guarantees. On light-weight TEEs such as Sancus, and specifically in the presence of constraints on system resources and power consumption, these additional protections are not applicable.

2.6.2 Availability

Our authentic execution framework focuses on the notion that if a valid input to a module is received, we can deduce that it must have originated from an authentic event at the source. By definition, this notion does *not* give any availability guarantees: only if we receive such a valid input can we make claims about the authentic event, whereas nothing can be claimed otherwise.

However, there has been recent work on enforcing availability guarantees for TEEs, specifically for Sancus [11]. At its core, this related work enables strict availability guarantees to enclaves on Sancus, such as that an enclave will be executed periodically or be enabled to serve an interrupt within a predictable

time frame. This allows to make claims about how specific enclaves will behave and *when* these enclaves will react upon events they receive. At the same time, it is still not possible to make any availability claims about events that exceed the device boundary, i.e., events that cross beyond the device itself and that are communicated over a shared untrusted medium such as a network. Overall, it may be possible to make strong claims about when and how fast a received event will be processed by an enclave, but it is not possible to guarantee that an event will necessarily arrive over the untrusted network.

Thus, we see availability as a complementary guarantee for authentic execution: If an event arrives at the device boundary, earlier work such as Aion [11] can guarantee that this event can be handled within a strict time frame, depending on the scenario's needs. Furthermore, the same work can guarantee that if *no* event is received within the expected time, the enclave will also be able to react within a guaranteed time window. This may be useful for a specific set of scenarios where disrupted availability could lead to unwanted or dangerous outcomes. By employing availability as a safeguard, the absence of events could, e.g., be used to trigger mode change and activate a local control loop to emergency-shutoff equipment until an authentic event is received.

While we do believe that this complementary notion of availability can have a benefit to certain scenarios that utilize authentic execution, we leave the combination of these two orthogonal research directions for future work.

2.6.3 Hardware Attacks and Side Channels

Although hardware attacks and side channels are explicitly ruled out by our attacker model (Sect. 2.2.3), it is necessary to discuss the impact in case an attacker would have given access to such techniques. We leave an analysis of our implementation for side channels for future work.

An attacker that successfully circumvents the hardware protections on a node would be able to manipulate and impersonate all modules running on *that node*. That is, the attacker would be able to inject events into an application but only for those connections that originate from the compromised node. The impact on the application obviously depends on the kind of modules that run on the node. If it is an output module, the application is completely compromised since the attacker can now produce any output they want. If, on the other hand, it is one among many sensor nodes, that get aggregated on another node, the impact may be minor.

Depending on the node type different attack vectors apply. A wealth of literature targets Intel processors and SGX enclaves [41, 95, 217, 205, 209] that enable

the extraction of cryptographic keys or other application secrets. On embedded TEE processors such as Sancus, many side channels such as cache timing attacks [135, 100] or page fault channels [226] are not applicable. Interrupt-based side channels [208] do pose a challenge and have been mitigated in orthogonal research in hardware [44] and in software [224].

2.7 Related Work

2.7.1 TEE Frameworks

In recent years, TEEs have gained more and more popularity in both research and industry, due to their properties and the security guarantees they provide. However, developing TEE applications is not trivial as software developers are normally required to have some expertise in the field; moreover, more and more TEEs are emerging nowadays, which makes it more difficult to write heterogeneous applications or simply to port an application from one TEE to another.

To this end, Keystone [123] is an open source framework for building customized TEEs; it provides generic primitives such as isolation and attestation, and hardware manufacturers and programmers can tailor the TEE design based on what they actually need. More software-based frameworks such as OpenEnclave [75] and Google Asylo [90] aim to provide a general API for writing enclave applications, allowing developers to write code that is not tied to a specific TEE and can therefore be easily ported to multiple platforms. Our framework uses a different approach, which consists of abstracting the TEE layer away from developers in order to write enclave applications just like normal ones. Other projects such as Enarx [73], follow the same principles as ours, although they mainly focus on high-end TEEs such as Intel SGX, TDX, and AMD SEV. To the best of our knowledge, our framework is the only one that support heterogeneous architectures, combining high-end systems, IoT devices, and small embedded microcontrollers.

Process-based TEEs such as Intel SGX strive in minimizing the TCB, as only the critical part of an application can be moved to the enclave. However, this comes with significant performance penalties, especially when crossing the boundary between the enclave and the untrusted domain. Instead, VM-based TEEs such as AMD SEV [16], Intel TDX [108] and ARM CCA [128] are substantially better in terms of performance, but have a much bigger attack surface as the entire virtual machine is part of the TCB. As future work, we might investigate

the use of VM-based TEEs combined with lightweight operating systems such as the formally-verified seL4 microkernel [118].

2.7.2 Mutually Authenticated Enclaves

At deployment time, our framework leverages symmetric connection keys to establish secure and mutually-authenticated channels between modules. Modules do not need to explicitly attest each other, but rather they rely on the fact that connection keys are securely distributed to modules *only* after successful attestation. The implicit assumption here is that such keys are known only by the modules and their deployer, and are always kept in secure memory and transmitted over encrypted channels. A similar idea was applied to the automotive scenario by creating a mesh of mutually-authenticated Sancus enclaves to provide integrity and authenticity of CAN messages exchanged between the ECUs of a vehicle [206].

In [231], multiple Intel SGX enclaves of an application establish mutual trust during the initialization of TLS sessions, using certificates previously generated by central servers upon successful attestation. Marblerun [200] uses the same principle to create a service mesh between enclaves in a Kubernetes cluster. Our framework, instead, relies on symmetric encryption for the establishment of secure channels, as public-key cryptography is not supported on small microcontrollers like Sancus. Although asymmetric encryption and client certificates seem a better choice in terms of key management and authentication, we decided to leverage symmetric connection keys even on Intel SGX and TrustZone in order to allow communication with Sancus modules.

2.7.3 Secure I/O

There are several intensive previous solutions aiming to provide a trusted path between an authorized trusted application and I/O devices that guarantees the integrity and authenticity of I/O data. McCune et al. [141] have proposed a BitE framework that leverages a trusted mobile device to establish an encrypted and authenticated input channel between I/O devices and an application running on a TPM-equipped untrusted platform. However, data inside the host platform is not isolated, hence the OS kernel must be part of the TCB. Bumpy [142], a succeeding work from the same research group, addresses this limitation by relying on the Flicker TEE [140] and using the encryption-capable keyboard to provide a safe pathway from input devices through untrusted environments. Bumpy uses dedicated hardware and hence cannot generalize to arbitrary I/O devices. Our approach to trusted I/O improves over Bumpy by greatly reducing

the size of the software TCB, from a full OS to less than 1 kLOC. By using Sancus as a TEE, we enable the integration of attestable software and I/O encryption directly into the input device. When communicating with a host that interacts with Sancus nodes, technologies such as Flicker, TrustZone or Intel SGX can be used to protect host services and to further reduce the TCB.

Several studies investigated a hypervisor-based approach. The techniques offered by Zhou et al. [233, 234] aim to establish a trusted path between an input peripheral and an application using a hypervisor to run the untrusted OS and application endpoints providing the necessary device drivers in separate VMs. DriverGuard [51] is a hypervisor-based mechanism that leverages a combination of cryptographic and virtualization techniques to protect I/O device control, I/O data flows against attacks from a malicious guest kernel. These systems rely on hardware interfaces, peripherals, and the USB bus to behave according to specification. Rogue devices on the USB bus, e.g. hardware keyloggers, can intercept messages between an I/O device and the host. Although our design focuses on integrity guarantees, our prototype protects communication channels using authenticated encryption. A more recent work, SGXIO [216], provides support for generic, trusted I/O paths for enclaves in Intel SGX by a composite of TPM, Intel SGX, and hypervisor techniques. Thus, SGXIO introduces a formally verified hypervisor (seL4 is recommended) to establish a trusted path while Intel SGX is in charge of protecting user applications from an untrusted OS. This approach improves upon existing generic trusted paths for x86 systems using Intel SGX's easy programming model. Similarly, Aurora [130] utilizes a hypervisor running in the Secure Management Mode (SMM) of Intel x86 processors as well drivers in the Driver eXecution Environment of open-source UEFI firmware to implement a concept not unlike the protected driver modules from our work. A TPM is used to store Intel Boot Guard measurements of the firmware and hypervisor, enabling mutual attestation between the SMM side and application modules in SGX enclaves. Omitting a hypervisor layer, BASTION-SGX [170] introduces a trusted Bluetooth controller and application support to establish a trusted channel between SGX enclaves and Bluetooth devices. SGX-USB [227], instead, places a proxy device between the host and an I/O peripheral connected through USB, to establish a secure path to an Intel SGX enclave. Unlike the other SGX-based solutions, SGX-USB does not rely on software modifications or additional trusted hardware in the host platform.

Fidelius [76] leverages only hardware settings with no reliance on hypervisor security assumptions to assist the establishment of a trusted path from input and output peripherals to a hardware enclave. More specifically, Fidelius utilizes a trusted dongle (e.g., Raspberry Pi) to protect the entire I/O path between the keyboard/screen and a small JavaScript interpreter that runs inside an Intel SGX enclave during web browsing sessions under a compromised OS and

browser. The trusted dongle captures user input and sends a stream of encrypted keystrokes to an attested local enclave. Then, the web enclave decrypts and updates the state of the relevant trusted input field and eventually forwards it to the remote server. In the opposite direction, a series of encrypted overlays are sent from the enclave to the display. The trusted path between a dongle and the web enclave is established through pre-shared symmetric keys. Conversely, the web enclave and remote server attest each other and establish a secure channel. Fidelius uses an LED indicator on each dongle and a status bar on the screen to ensure the establishment of the trusted paths; however, this leads to a high cognitive load on the users as they must monitor continuously different security indicators. Fidelius leverages a trusted external device to guarantee the integrity and authenticity of the transferred I/O data, while our solution for secure I/O relies only on the isolation and attestation properties in Sancus and does not suffer from micro-architectural attacks and relay attacks.

ARM TrustZone provides isolated execution combined with generic trusted I/O paths. In TrustZone, different peripherals, including parts of RAM, IO devices, and interrupts, can be dynamically configured by a set of hardware controllers to be accessible only in the secure mode. This is achieved by the reflection of the NS bit into the respective peripheral. In contrast to Intel SGX, TrustZone assumes that all secure application processes are trusted and does not isolate them in hardware. However, several prior works take the advantage of TrustZone to establish a trusted I/O path with much focus on balancing the size of TCB against functionality. For example, TrustUI [127] builds trusted paths by splitting device drivers into an untrusted backend and a trusted front-end. The two parts communicate through shared memory by leveraging proxy modules running in both worlds. Similarly, TruZ-Droid [228] runs only half part of the HTTP and SSL protocols in the secure world to reduce the TCB size. VeriUI [134] proposes a login protection mechanism by moving all layers of the communication stack and UI library into TrustZone. TrustOTP [199] relies on tiny drivers running in the secure world to protect the one-time password display. Secure I/O in TrustZone is a well-researched topic that has got a great deal of attention in the past years. Apart from the projects described above, many others exist in the literature [232, 229, 125, 126]. More recently, a notion of trusted I/O was also implemented in Android smartphones to protect user transactions [62]. In our work, we did not implement a secure I/O solution for TrustZone, but rather we interfaced Sancus-enabled I/O devices with Intel SGX and TrustZone enclaves, providing exclusive and confidential access to isolated host drivers without trusting the underlying communication channels.

Efforts have recently been made to enhance TEEs by incorporating hardware acceleration, particularly GPUs. This enables software modules to exclusively access external devices for computationally intensive tasks, such as machine

learning model training. Several academic projects propose hardware extensions to GPUs to enable isolation and strict access control, including Graviton [212], HIX [112], and HETEE [235]. StrongBox [66] utilizes TrustZone extensions already present in ARM devices to provide a solution without requiring hardware changes. This trend has also influenced commercial products, such as NVIDIA's new H100 Tensor Core GPU that supports confidential computing [164], and Intel TDX Connect, which offers extensions to the TDX architecture to securely assign I/O devices to trusted domains [107]. Our architecture can utilize hardware acceleration to enhance the use cases we are considering (Sects. 2.2 and 2.5.1). For instance, it can be used to process sensor data for computing statistics or training machine learning models.

2.8 Summary & Conclusions

In this article, we present a software architecture to securely execute distributed applications on shared, heterogeneous TEE-infrastructures. We have extended Sancus, a light-weight embedded TEE with support for secure I/O, interfaced it with the commercial off-the-shelf TEEs ARM TrustZone and Intel SGX, and implemented and evaluated a software development framework which enables the execution of reactive (event-driven) distributed applications on a shared infrastructure with strong authenticity guarantees and in the presence of capable attackers, while relying on a very small TCB. We have implemented (and foresee many more) compelling use cases in IoT and control systems. While much of our work is focused on small embedded microprocessors, we demonstrate that our solution to be generally applicable to, e.g., build secure distributed applications that integrate IoT sensing and actuation with cloud-based data processing or aggregation. Yet, implementing secure I/O is more challenging on non-embedded architectures. Extensions to our work would be to strengthen security guarantees while maintaining a small TCB, fully integrating a notion of availability based on [11], and to formalize the security argument using recently proposed logics (e.g., [33]) for reasoning about TEEs.

Chapter 3

Understanding Trust Relationships in Cloud-Based Confidential Computing

This chapter was previously published as:

G. Scopelliti, C. Baumann and J. T. Mühlberg. “Understanding Trust Relationships in Cloud-Based Confidential Computing”. In: *7th Workshop on System Software for Trusted Execution (SysTEX’24)*. 2024

Preamble

The design of the authentic execution framework described in Chapter 2 leveraged process-based TEEs like Intel SGX to minimize the attack surface as much as possible. Indeed, running small portions of code into hardware enclaves was one of the main objectives and strong points of early TEE designs. However, in recent years there has been a turnaround, where new VM-based TEEs such as Intel TDX and AMD SEV-SNP are being used to protect entire virtual machines. These workloads, also called Confidential Virtual Machines (CVMs), include several software layers ranging from the guest firmware to the operating system and applications, thereby leaving a much larger attack surface compared to process-based TEEs. Despite this, it seems that the community is progressively

moving towards these new TEEs because they are much easier to use, as CVM do not require any application-level changes and can be integrated into cloud workloads with minor modifications. In addition, VM-based TEEs provide some native support for secure I/O to leverage accelerators such as GPUs securely and efficiently. This, for example, is the case of Intel TDX Connect [107] and AMD SEV-TIO [17], both of which implement the TEE Device Interface Security Protocol (TDISP) specification [167] to create secure channels between a CVM and a trusted device.

Although VM-based TEEs make it significantly easier to deploy confidential workloads compared to their predecessors, effectively lowering the bar on confidential computing adoption, there are still some non-trivial challenges that need to be taken into account. In this chapter, we discuss the increased complexity of performing remote attestation on CVM workloads. As mentioned above, booting up a VM goes through several stages where several software components are loaded into TEE memory at different times. A compromise at any stage might break the integrity of the CVM as a whole, therefore a verifier needs to *measure* each of these components to ensure that what is being loaded into the TEE matches the expected measurements. Normally, the TEE computes a measurement of the initial state of the CVM, also called *launch measurement*, and includes it to the attestation report; However, this measurement typically only includes the very first piece of software that is loaded in the CVM (e.g., the firmware), while the rest of the boot chain is not measured and requires other methods to ensure its integrity. This behavior is different from process-based TEEs, where the whole enclave binary is loaded (and, therefore, measured) at once. In this case, the launch measurement computed by the TEE includes the entire memory layout of the enclave, making remote attestation easier.

The increased complexity on the attestation of CVMs compared to enclaves is exacerbated when deploying workloads on public clouds, where the cloud provider has some control over the software components that are being loaded into CVM memory. In fact, it is common that cloud customers are restricted to a predefined list of images that they can choose from, which are managed by the cloud provider. Even though the customer in some cases is allowed to (partially) customize a VM image, some components such as the guest firmware might be proprietary and closed-source. As a consequence, attesting the full CVM boot chain becomes extremely difficult (if not impossible) to perform without some support from the cloud provider. This raises interesting questions about trust between a cloud provider and its customers: According to the TEE threat model, the infrastructure provider is untrusted by design and, because of this, confidential computing is gaining a lot of attention in the cloud market since several customers would benefit from such a solution to protect their sensitive

data from unauthorized access. However, relying on the cloud provider for the attestation of CVMs partially defeats the purpose of using TEEs in the first place. In our work, we investigate this issue and try to shed some light on the whole CVM attestation process.

This chapter presents our findings, which were presented at the SysTEX 2024 workshop and received the Best Paper award. Our research also resulted in a *tool* paper that was published in the same workshop, thanks to a collaboration with Luca Wilke [221]. The motivation behind this second publication was that, in our experience, setting up CVMs was quite a daunting task. Among all, the biggest problems were caused by a lack of open-source tooling and documentation available, which caused some extra effort and manual work. Thus, we developed and released SNPGuard¹, a tool that greatly simplifies CVM provisioning and attestation based on AMD SEV-SNP, providing a high level of automation on every step of the process. We hope that our tool can help researchers accelerate research in the same field and lower the bar towards TEE adoption.

The evaluation presented in our paper was carried out in March 2024, but the main results presented in Tab. 3.1, to a large extent, are still valid at the time this preamble is being written (i.e., November 2024). Nevertheless, a few potential improvements to current offerings are worth mentioning: On Microsoft Azure, it should now be possible to retrieve fresh SEV-SNP attestation reports from the vTPM [145], solving the freshness problem mentioned in Sect. 3.6.1. Besides, Google Cloud Platform (GCP) customers can now verify the firmware running on CVM instances [91]; However, at the current stage only the binaries are publicly available, while the code still remains closed-source. On the other hand, Microsoft has recently released the source code of its paravisor, OpenHCL, on GitHub [146]. This would potentially allow cloud customers to measure and verify the very first piece of software loaded into Azure’s CVMs (and, from there, the entire boot process), though this is still not possible since a reproducible build process for OpenHCL is currently not implemented yet. Additionally, Azure’s DCas_cc_v5 and DCads_cc_v5 machines support nested virtualization [149], which potentially allows running a SEV-SNP CVM from within a normal VM (this process is called “CVM-in-VM”). With this approach, cloud users could install their own hypervisor in the normal VM and fully customize the boot process of CVMs (and, subsequently, attestation), thus bypassing the cloud provider. This approach, still in preview on Azure, can dramatically reduce trust towards the cloud provider.

¹<https://github.com/SNPGuard/snp-guard>

3.1 Introduction

Cloud computing is the backbone of digitalized societies, supporting today’s cloud-native web service development paradigm. Increasingly, also industries that traditionally operated their own IT environments, e.g., telecommunication, manufacturing, and healthcare, are now moving to the cloud for better scalability, manageability, and cost reduction. However, this trend is accompanied by questions about the security and trustworthiness of *Cloud Service Providers (CSPs)* along with regulatory concerns about privacy, data protection, and data sovereignty: “moving to the cloud” involves moving large amounts of sensitive data, along with some of the responsibility to protect it, to a third party.

Different forms of *Confidential Computing (CC)* have surfaced to address such concerns, with *Trusted Execution Environments (TEEs)* [182] such as Intel SGX and TDX, AMD SEV-SNP, or ARM CCA promising to effectively take the cloud provider out of the *Trusted Computing Base (TCB)*. These TEEs provide secure compartments on cloud servers that can run applications while protecting data in use from infrastructural threats and enabling the “removal of even the cloud provider from the Trusted Computing Base” [56]. At the same time they provide *attestation* mechanisms to cryptographically verify that the unmodified application is indeed running in an authentic TEE. On paper, such a design allows for a drastic reduction of customers’ dependency on cloud providers to adequately secure the cloud platform infrastructure – firmware, OS, and virtualization layers – and leaving only the hardware vendor that implements a TEE as a root of trust.

While early TEEs for server systems embedded *enclaves* into user processes, the current market trend are *Confidential Virtual Machines (CVMs)* that place an entire tenant VM into a TEE. While this approach increases the TCB within the TEE, it simplifies the deployment of software in TEEs and reduces system-call performance overheads [5] relative to enclaves, thus lowering the CC adoption hurdle for customers. Consequently, cloud providers such as Amazon Web Services (AWS), Microsoft Azure, or Google Cloud Platform (GCP) have recently started marketing CVM solutions as an easy fix for the security, privacy, and trustworthiness challenges outlined above.

However, launching and attesting a CVM is a different beast than the enclave-based CC approach. A VM consists of layers of system software, including firmware, bootloader, kernel, and guest OS, as well as user space applications, with many components usually provided by the CSP. Given this large software stack running inside the CVM, the issue of trustworthiness comes again into focus. Effectively removing the infrastructure, i.e., the CSP, from the customer’s TCB requires (1) an attestation infrastructure that allows to attest

the authenticity of both the TEE hardware and *all* the software running inside it, and (2) transparency for software components provisioned by the CSP in the CVM, e.g., by supplying the underlying source code alongside a reproducible build process.

In this paper we examine CVM offerings on public clouds along these two axes in order to evaluate the level of trust in the CSP these CC solutions still require. In particular, we take a close look at the boot process and provided attestation mechanisms involved in the setup of AMD SEV-SNP CVMs on popular cloud providers. It turns out that there are shortcomings that prevent achieving confidential computing under infrastructural threats. In many cases this is due to missing attestation infrastructure or proprietary software components that need to be included in the CVM. We make the following contributions:

- We introduce a hierarchy of *attestation levels* with increasingly stronger guarantees for CVMs and showcase what could go wrong with partial attestation at the lower levels of this hierarchy;
- We highlight that, if some components cannot be verified by the CVM owner independently, some blind trust in the CSP is still required when deploying services in commercial clouds;
- We conduct a case study into AMD SEV-SNP as provided by popular commercial CSPs to assess how products meet our attestation levels, and we explain shortcomings in current offerings.

3.2 Background

3.2.1 VM-based TEEs

Confidential VMs are enabled by recent technologies such as AMD Secure Encrypted Virtualization (SEV)-Secure Nested Paging (SNP) [15], Intel Trust Domain Extensions (TDX) [108], and ARM Confidential Compute Architecture (CCA) [27]. While the inner workings of these mechanisms may differ, they all allow to essentially wrap an entire VM, including its firmware, OS, and applications, into a TEE and also attest its authenticity to a third party via a cryptographic protocol. This contrasts greatly with enclave-based TEEs, such as Intel SGX, which provide an isolated execution environment for a trusted partition of a larger untrusted user space application.

On the other hand, VM-based TEEs are mostly transparent to the VM running inside it except for an adaptation layer, either in the guest system software or

a paravisor [53]. Hence, they can provide CC to larger applications deployed as VMs without changing application code. Moreover, unmodified CC can be provided to containers via micro-VM container runtimes such as Kata [175]. Thus, CVMs are considered to be more suitable for cloud-based deployments. Nevertheless, this ease-of-use comes with an increased TCB in the TEE. This not only increases the attack surface and risk for vulnerabilities, but also complicates attestation.

For instance, a recent study on AMD SEV found that measuring and encrypting even moderately sized portions of the initial CVM memory leads to prohibitively large boot times [99]. Also, build reproducibility becomes more complex the more different software components are combined into the initial VM image. Hence, measuring the complete VM along with the launch of the TEE is considered impractical and current CVM architectures instead follow a staged attestation approach that first creates a root of trust within the VM firmware. Subsequent attestation steps are then integrated into the VM boot process, leveraging measured/secure boot technologies to incrementally build a chain of trust for the loaded software modules.

3.2.2 Measured Boot and Attestation

Even without consideration for TEEs, booting a VM is a complex process that is composed of several stages. The first components to run in such a Linux VM are its firmware, e.g., Open Virtual Machine Firmware (OVMF) and bootloader, e.g., GRUB. In some cases, such as with Direct Linux Boot [172], it is possible to skip the bootloader altogether and boot a kernel directly from the firmware. Once booted, the kernel extracts and mounts the initial RAM disk, e.g., *initramfs*, and executes the contained *init* script. Its sole purpose is to mount the actual root filesystem for the VM containing the OS and user space applications which is provided as a block device mapped into VM memory. After mounting this filesystem, the script changes the root directory to it and launches the OS, which in turn may start user applications.

The integrity and authenticity of the boot process is paramount to ensure the trustworthiness of any system. Measured boot [204] is a common technique to produce evidence that a computer system has booted securely. While its implementation may differ between hardware architectures, measuring the boot process usually relies on a fixed and trusted initial firmware stage and a tamperproof Root of Trust for Measurement (RoTM), such as a Trusted Platform Module (TPM). The initial firmware stage may cryptographically measure itself and load subsequent boot code which, in turn, may continue the measurement process and extend the produced measurement results. The

measurement process and storage of results, typically hashes, is handled by the RoTM, and the authenticity of results can be verified at a later stage by a third party. For TEEs, the RoTM is integrated with the TEE implementation, e.g., in microcode for Intel SGX, or in a secure co-processor for AMD SEV-SNP, called AMD Secure Processor (SP). When setting up a TEE instance, the untrusted system interact with the RoTM through secure interfaces to create trustworthy measurements of the instance. For CVMs, measured boot is established by measuring the guest firmware along with the TEE itself during setup. For subsequent measurements, a local RoTM needs to be included to the firmware, e.g., via a virtual TPM (vTPM). Measurement results can then be certified by the RoTM and sent to a remote verifier for verification.

3.3 Problem Statement

This work examines trust relations involved in the CVM deployment process. To avoid potential confusion, we first delineate what we mean by “trust” and “trustworthiness” in this context.

Cloud service customers expect CSPs to run their cloud platform securely, e.g., that best practices are followed to prevent compromise by outsiders, customer data is sufficiently secured from leakage to other tenants, and policies are in place to curtail insider threats. Depending on their specific requirements, a tenant needs to perform a risk analysis before deploying their software at a given cloud platform given the information they have about it. This information is often incomplete as most CSPs run proprietary software to operate their platforms and customers must inherently rely on the promises of a CSP’s marketing department for operational aspects that are opaque to them, while also soft metrics such as reputation, track record, or popularity may factor into their decision.

Accepting any residual risk, the tenant can then be said to *trust* the cloud platform. However, after making the decision, this trust becomes mandatory as the customer simply has to rely on the CSP to hold up their end of the bargain. This trust is also perpetuated *blindly* until objective evidence surfaces that it is no longer justified.

We posit that the dual of this blind trust is *trustworthiness*, which allows for continuous risk assessment by a cloud tenant through evidence of platform security obtained at run time. Such evidence may come in the form of attestation reports for software deployed in TEEs together with the possibility to independently verify the security of the TCB. The more such run-time evidence

is provided, the greater the trustworthiness of an execution platform becomes, and the less blind trust is required.

For the CVM case this means that the CSP not only has to provide the necessary infrastructure to attest the authenticity of all boot stages, but also provide transparency for the software components provisioned by the CSP to allow security analysis of the remaining TCB.

This work aims to examine this issue more closely and proposes a hierarchy of attestation levels for the CVM that allow to obtain increasing amounts of runtime evidence for the integrity and authenticity of a CVM, thus allowing to increase trustworthiness. We then demonstrate the utility of this metric in an analysis of popular public cloud platforms offering AMD SEV-SNP functionality judging the maximum level of trustworthiness achievable by them.

3.4 System Model

In our system model, a guest owner deploys a CVM on an infrastructure where potentially other software is running at the same time, such as other VMs (confidential or not), monitoring software, etc. The infrastructure is managed by an infrastructure provider, and should provide the hardware and software primitives required to run the CVM in a TEE. Besides, a verifier performs attestation to make sure that the CVM was deployed and booted correctly; We refer to the IETF RATS architecture for a more comprehensive representation of the attestation infrastructure [38]. Furthermore, an end user interacts with the CVM to, e.g., deploy workloads and retrieve results. In general, some interaction between such actors is required and trust relationships between them may vary according to the use case. Besides, two or more logical actors can, in practice, be impersonated by the same party: For example, when running a CVM on a local machine, all actors may be played by the owner of that machine. When deploying a CVM in the cloud, however, all actors are likely unique. We will discuss this scenario in more detail in Sect. 3.5.

In this paper, we follow the standard threat model of TEEs: All software running outside of the CVM is potentially malicious, including the VMM/hypervisor, other VMs, etc. Note that this is true even when the guest owner and the infrastructure provider are the same party or trust each other, since the software running on the infrastructure might be compromised by remote attackers. Instead, the TEE manufacturer and any software and hardware components provided by them are assumed to be secure. The TEE threat model mainly focuses on confidentiality and integrity but leaves out availability. Additionally, side channels and physical attacks are also out of scope.

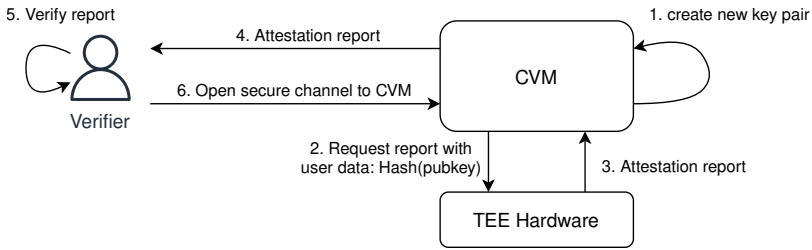


Figure 3.1: High-level attestation flow.

3.4.1 The Importance of Remote Attestation

When it comes to TEEs, remote attestation is an essential step to gain confidence in the deployed CVM, making sure that, after boot: (1) the CVM is running in a genuine and up-to-date TEE, and (2) the CVM is in an expected and good state, i.e., only trustworthy and measured software has been loaded. Moreover, attestation is often used to bind the identity of the CVM to a cryptographic credential, later used to establish a secure channel for provisioning secrets and deploying workloads, e.g., via SSH or TLS connections. Figure 3.1 shows how this is usually achieved.

After generating an ephemeral key pair (step 1), the CVM embeds some information about the credential in the attestation report requested from the TEE hardware (step 2). This could include, e.g., a hash of the public ephemeral key as well as a nonce provided by the verifier for freshness (not shown). The signed attestation report returned by the TEE hardware (step 3) then includes this data along with information about the TEE hardware and measurements of software deployed in the TEE. After receiving and verifying the report (steps 4 & 5), the verifier can now be certain that the key is *owned* by a CVM with the specific hardware and software configuration matching the attestation report.

When attestation fails, this binding becomes compromised, and the verifier (or other entities) cannot be sure that they are communicating with a legitimate CVM. The problem is exacerbated by the complexity of CVM attestation that might lead to some components being accidentally skipped during verification. In the next sections, we describe all steps of CVM attestation and show what might happen when some of these steps are missed.

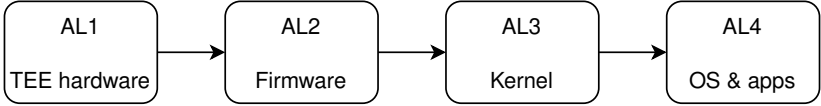


Figure 3.2: Hierarchy of attestation levels for CVMs.

3.4.2 Attestation Levels

Attesting a CVM is not an easy task since we need to ensure that every part of the boot process is measured and can be verified. This section identifies five stages of attestation of a CVM, called *Attestation Levels (ALs)*. These levels are incremental, meaning that the guarantees provided at a certain level build upon the levels below. A visual representation of such levels is provided in Fig. 3.2.

AL0: No attestation This is the baseline level where no attestation is done. As a result, the verifier cannot make any claims on the state of the CVM and the threat model is the same as for a regular VM execution environment.

AL1: Attested TEE isolation In this level, the verifier has access to the “raw” attestation report signed by the TEE manufacturer, and is able to independently verify its integrity and authenticity. This typically involves the verification of the report’s signature and certificate chain, which goes up to a root certificate that is self-signed by the TEE manufacturer. Moreover, the report also contains some information about the platform’s TCB (e.g., CPU model, microcode version, etc.), and freshness information provided by the verifier. This allows the verifier to attest that the CVM is indeed running in a genuine and up-to-date TEE, which reduces the threat surface significantly as the host firmware, OS, and virtualization layer can now be excluded from the TCB. The verifier can also be sure now that software running outside of the CVM cannot directly access data stored inside of it. However, threats specific to the CVM technology used, e.g., side-channel leakage, or to the software running inside the CVM remain.

AL2: Measured firmware As mentioned above, remote attestation is used to establish a secure channel into the CVM, but AL1 does not give any information about what software is running at the other end of that channel. A first step to alleviate this situation is verify the *launch measurement* contained in the attestation report, which reflects the memory layout of the CVM at boot time including the firmware, page metadata and CPU register state. To obtain

evidence for the correct deployment of the firmware, the verifier needs to validate the launch measurement against a trusted reference value. That measurement in itself does not tell anything about the code and data running in the CVM, hence it is important that the firmware's source code is available and can be reproducibly built. After a successful verification, it can be ruled out that the trusted firmware has been replaced by a malicious one.

AL3: Measured Kernel To rule out compromise of later boot stages, the verifier needs to validate the entire boot chain that starts from the firmware and goes up to the kernel, (optionally) through a bootloader. This also includes the initial RAM disk and kernel command-line parameters. By default, these components are not measured by the TEE and thus not part of the launch measurement. However, there are several ways to make this attestation possible, some of which are discussed in Sect. 3.4.3.

AL4: Fully measured boot The verification of the measurements done at AL3 stops at *early userspace*, i.e., right before the root filesystem is mounted to the CVM. At the very least, the root filesystem of the CVM containing OS and application data should be integrity protected, to prevent loading a malicious version. In case it contains secrets, it should also be encrypted. The challenge in this step is to securely provision keys to the CVM only after a successful AL3 attestation, to prevent leaking the keys to a compromised CVM. Again, possible implementation strategies are provided in Sect. 3.4.3.

After a successful AL4 attestation, the verifier has confidence that the desired system and application software has been deployed correctly in the CVM. At this point, the threat model is similar to running the guest system on a trusted dedicated server and run-time security of the CVM is under the guest owner's purview. In particular, the deployed software may employ security policies to sanitize untrusted inputs or prevent the loading of additional, untrusted software. Moreover, further attestation steps may be initiated by the guest owner, e.g., run-time attestation of application control-flow integrity, or attesting the authenticity of attached secure I/O devices.

3.4.3 Attestation Strategies

Typically, support for AL1 and AL2 attestation is provided by all TEE manufacturers, via a certificate chain and a public key infrastructure (for AL1) and a measurement of the initial memory layout of the CVM (for AL2). In both cases, it is often sufficient to verify the attestation report that can be

fetches by the CVM and sent to the verifier. To reach ALs 3 and 4, instead, additional steps are needed. Below, we discuss possible approaches.

AL3 A simple strategy to make this attestation possible leverages Direct Linux Boot to bind kernel measurements to the firmware, in such a way that the launch measurement in the attestation report also reflects the identity of kernel components [70]. This can be done by adding kernel measurements as firmware variables, which will be loaded into secure memory at boot and measured by the TEE. Then, the firmware will load and pass control to the kernel only if its measurements match with the expected ones stored as variables, aborting otherwise. A more involved solution involves relying on a vTPM for measuring the CVM boot chain. On AMD SEV-SNP, this functionality can be exposed thanks to Virtual Machine Privilege Levels (VMPLs) [19], where the vTPM executes in a higher privilege level than the rest of the system. To bind vTPM quotes to TEE attestation reports, the digest of the vTPM endorsement key is added as user data in the TEE report [156]. The verifier, then, has to check both the report and the quote to verify the trustworthiness of the CVM. On Intel TDX, instead, a vTPM can be exposed from a separate trust domain [105]. Additionally, Intel TDX also provides four Runtime Measurement Registers (RTMRs) whose value is reflected in the attestation report [109]. These registers are analogous to a TPM’s PCRs, and can be similarly leveraged by a TDX-aware firmware to measure boot components.

AL4 To ensure the integrity of the root filesystem, a common approach is to extend the vTPM measurements up to the user space with Linux Integrity Measurements Architecture (IMA) [178]. IMA is a kernel subsystem responsible for measuring all binaries that are loaded at runtime, and it supports remote attestation. Measurements are stored in a measurement file which itself is integrity-protected by a measurement stored in the TPM, to prevent tampering from a privileged adversary. When adopting IMA, it is important to be aware that runtime measurements may be susceptible to time-of-check-time-of-use attacks [40], and that there exist so-called *measurement gaps*, i.e., some components are not measured.

Another approach relies on protecting the integrity of the whole filesystem at rest. To this extent, a popular software solution is **dm-verity** [117], which provides integrity protection by verifying the data blocks in a filesystem against pre-computed hash values, stored as a Merkle tree on a separate disk. The root of the tree (called root hash), guarantees integrity of the whole filesystem, and must be protected from tampering. In our attestation flow, the root hash can be provided as a kernel parameter such that its integrity can be verified with a

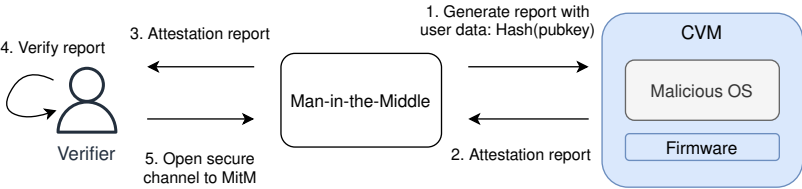


Figure 3.3: Possible Man-in-the-Middle attack with an unverified guest OS.

AL3 attestation. Filesystems using `dm-verity` are mounted as read-only; To enable both read and write, `dm-integrity` [116] can be leveraged instead to (re-)compute HMAC tags for each sector of the block device. In this case, however, a key should be separately provisioned to the kernel during early userspace. In our CVM scenario, this means that attestation should be performed during boot. Finally, `dm-integrity` can be combined with `dm-crypt` [115] to additionally provide encryption.

3.4.4 Partial Attestation

It should now be clear that attesting a CVM is not a trivial task. Unlike process-based TEEs such as Intel SGX, verifying the TEE attestation report alone is necessary but *not sufficient* to cover the whole boot process of the confidential workload. Failing to properly verify one or more boot stages might cause some attacks to go undetected, such as injecting malicious code or backdoors that would compromise the integrity and confidentiality of the CVM. These kind of attacks can be either local (e.g., a compromised hypervisor tampering with the root filesystem) or remote (e.g., a malicious kernel image downloaded from an untrusted registry). Therefore, partial attestations might give a false sense of security to guest owners and end users, who expect that their code and data is protected when, in reality, this might not be true.

To showcase the severity of partial attestations, we consider the scenario where the verifier only attests the CVM up to AL2. Here, a Man-in-the-Middle (MitM) attack is possible if the CVM loads a malicious OS (Fig. 3.3): As the CVM can produce attestation reports with arbitrary user data, a MitM that controls the CVM OS can bind the report with a credential owned by the MitM (step 1). The verifier, when receiving the attestation report (step 3), can successfully verify that the report comes from a genuine CVM with the expected firmware (step 4), but cannot make any claims on the running OS. Failing to recognize this issue might cause the verifier to bind the CVM identity to the credential owned by the MitM, leading to the verifier possibly opening a channel with the

MitM and leak secrets (step 5). What is worse, the MitM does not even need to run in a CVM, meaning that all data will be processed in clear. Therefore, although the verifier has performed a successful AL2 attestation, in practice the security guarantees obtained are basically the same as without attestation.

3.5 CVM Attestation in Public Clouds

Above, we have discussed the different attestation levels a verifier can achieve when evaluating the authenticity and integrity of a deployed CVM. However, who exactly plays the role of verifier depends greatly on the application scenario and the trust relations between the involved parties. In this work, we focus on a scenario where a tenant wants to deploy a CVM as guest owner on a cloud platform, acting as the verifier in the attestation process. Afterwards, attestation results may also be exposed to end users as well [87]. While different tenants may have different trust relations with a given CSP and lower attestation levels may suffice for them in practice, we assume here that the tenant wants to reduce the required trust in the infrastructure provider as much as possible and achieve a high level of trustworthiness for the CVM solution via attestation.

Here, the measurement reports used during attestation should ideally be verifiable independently of the CSP. In particular, attestation reports produced by the TEE hardware need to be available to the verifier. In addition, the verifier needs to be able to identify trusted software deployed in the TEE, i.e., they need to know the corresponding reference measurements for the firmware, kernel, user applications, etc. A CSP may provide these reference values so that the authenticity of the deployed software can be established. However, this results in a notion of verifiability that still relies on trust in the CSP and in the security of the deployed software. In principle, a trusted third party such as Intel Trust Authority [110] could certify the security of that software and publish the corresponding reference values, but that just shifts the required trust to a different entity.

A stronger *verifiability without trust* entails that the verifier can inspect the source code of software deployed in the TEE and obtain the reference measurements via a reproducible build process. Having access to the source code allows the verifier to conduct their own security analysis of the code and judge its trustworthiness. Of course, if the code is publicly available, this review can also be performed by the open source community, but this would again introduce required trust into the picture.

Overall, for our cloud CVM scenario, the tenant wants to achieve verifiability without trust for as high an attestation level as possible. Even if a high *nominal*

	AWS	Azure	GCP
TEE	●	◐	●
Firmware	●	◐	◐
Kernel	◐	◐	◐
Root FS	◐	◐	◐
Nominal AL	4	4	4
Trustworthy AL	2	0	1

● = Verifiable w/o trust; ◐ = Verifiable w/ trust; ○ = Not verifiable

Table 3.1: AMD SEV-SNP offerings on AWS, Azure and GCP. This table reflects attestation levels described in Sect. 3.4.2 and verifiability with or without trust, resulting in the maximum nominal AL (with trust in the CSP) and trustworthy AL (without trust). Results dated 2024-03-26.

AL can be achieved with the help of the CSP, still being forced to trust the CSP blindly for the measurement values and deployed software reduces the trustworthiness. We introduce the term *trustworthy AL* to denote the maximum, effective attestation level the tenant can achieve as a verifier without trust into the public cloud provider.

3.6 Exploring the Cloud Landscape

We investigated current CC offerings on public clouds, focusing on SEV-SNP as the most widely available VM-based TEE. Our evaluation is solely based on commercial cloud features under general availability or public preview. We found that only the three major CSPs, i.e., Microsoft Azure, GCP and AWS, offer to deploy SEV-SNP CVMs on demand. We looked for VM-based confidential computing support in other CSPs, but they either supported plain SEV or only offered single-tenant bare-metal servers with SEV-SNP hardware. While plain SEV does not provide integrity protection nor flexible attestation capabilities, bare-metal servers come at much higher costs and typically require a fixed monthly subscription. Instead, we were interested in the multi-tenant scenario where customers can deploy CVMs on demand on a shared infrastructure, with little to no control over the hypervisor.

As discussed in Sect. 3.5, our evaluation tried to determine the level of *verifiability without trust* that can be achieved on such CSPs. Besides, we also considered

the model where the verifier (partially) trusts the CSP and relies on provided services and data, such as a custom attestation report or a proprietary attestation service.

Results are shown in Tab. 3.1 and based both on public documentation and direct experiments. Regarding the former, all references in Sect. 3.6.1, except Github links, have been uploaded to the Internet Archive² to provide a stable reference. Concerning the latter, our evaluation was made on 2024-03-26. Since this is a snapshot of the current state and features of clouds, some of our findings may not hold in the future. However, our evaluation framework is still useful to evaluate future offers in the CVM area.

3.6.1 Results

In our experiments on AWS and GCP, we were able to fetch a raw attestation report from the AMD SP via the `/dev/sev-guest` device, allowing to independently verify that the guest VM is indeed running in a up-to-date SEV-SNP TEE. Azure CVMs, instead, only provide access to an attestation report generated at boot time and stored in vTPM non-volatile memory. As this report does not contain any freshness information chosen by the verifier, the latter cannot distinguish a legitimate attestation report from a replay attack and must rely on the CSP to attest that the VM is indeed using SEV-SNP. We tried to boot an SNP-aware kernel built from the AMDSEV repository [14], yet we still could not access the AMD SP.

Regarding AL2, only AWS offers an open source firmware that can be reproducibly built [31]. We successfully checked that the attestation report indeed reflects the correct measurement. Instead, Azure and GCP guests boot a proprietary firmware whose code cannot be audited. To the best of our knowledge, firmware reference measurements are also not provided, meaning that the verifier cannot even compare the measurement in the attestation report against the expected value. However, we observed that both Azure and GCP have a secure boot option that prevents CVMs from booting if the firmware is corrupted [147, 94], though we could obviously not experimentally verify this claim. Yet, tenants that are willing to trust their CSP can be confident that a running firmware was not tampered with by an outsider attacker.

All CSPs allow extending the measurements up to the kernel via a vTPM. However, on AWS [30] and GCP [92] the vTPM is implemented in the hypervisor, greatly increasing the TCB. Azure, instead, exposes a vTPM from within the

²<https://www.archive.org>

CVM firmware [151] which, as mentioned before, cannot be reviewed. Thus, no CSP currently supports trustworthy attestation of kernel components.

After kernel verification, the chain of measurements can be extended to the root filesystem via different approaches (Sect. 3.4.3). Hence, AL4 is a natural extension of AL3, provided that the CSP allows for the customization or upload of VM images by the guest owner. We experimentally verified that the former is indeed possible by installing a custom bootloader and kernel, as well as changing kernel command-line parameters, e.g., to enable Linux IMA. Uploading images seems also feasible according to public documentation [29, 150, 93], but we did not verify this. Still, for trustworthy attestation, the integrity of the root filesystem ultimately relies on the integrity of the boot chain up to the kernel: If the latter cannot be verified independently of the CSP, neither can the former.

In summary, all CSPs provide the infrastructure to perform a full attestation (*nominal AL4*), but they rely on either proprietary software or components outside the TEE TCB, or both. On Azure, a verifier can obtain an attestation report but cannot verify its freshness, hence replay attacks are possible and the CSP must be trusted for all attestation levels (*trustworthy AL0*). On GCP, one can fetch dynamic reports, but the firmware cannot be independently verified (*trustworthy AL1*). Only AWS allows to attest an auditable firmware, yet extending the chain of measurements to later boot stages uses a hypervisor-based vTPM outside the TEE (*trustworthy AL2*).

3.6.2 Discussion

We showed that guest owners still require a significant amount of trust in the CSPs when using CVM solutions in the cloud. This goes against the classic TEE threat model, where the infrastructure is considered untrusted. There are two reasons for this: (1) CSP code is running inside the TEE boundary, with no possibility for review by the remote verifier, and (2) the attestation process involves CSP-managed components. While running CSP code in the CVM is sometimes necessary (e.g., to configure network interfaces, mount devices, etc.), it is certainly possible to increase transparency towards the verifier, as demonstrated by AWS' reviewable firmware. The second problem is mostly caused by a lack of customization options offered by CSPs. This is somewhat understandable since most cloud customers are looking for ease of use and "one-click compliance", where they are willing to trust the CSP to perform security checks in their place. For advanced users, however, more customizability is desirable.

Our evaluation did not focus on userspace software that runs after booting the root filesystem, such as `cloud-init` [46] or CSP-managed software like Azure

Linux VM Agent [148]. These agents take care of VM configuration and can execute privileged commands, like installing packages or updating the trusted SSH identities. Any such operation, if compromised, can thwart the security guarantees of CVMs. Hence, it is equally important to implement hardening measures in the root filesystem to, e.g., check `cloud-init` configuration files before executing them or disable unnecessary software.

Lately, several companies like Anjuna [21], Edgeless Systems [71], and Enclave [74] have started offering their own confidential computing solutions to customers, providing fully-managed confidential workloads in form of containers, virtual machines, or both, and supporting deployments on popular CSPs. Attestation of workloads is managed by a dedicated attestation service (which can also run within a TEE) and is therefore transparent to the customer, who only receives a report indicating whether or not their deployment is verified. Such solutions can largely improve user experience and ease the adoption of confidential computing. However, when it comes to cloud deployments, they still depend on the CSPs and the confidential computing features offered by them. Therefore, besides adding another third party to the trust model, such solutions cannot increase the *trustworthy AL* shown in Tab. 3.1. Analogously, the same applies to third-party attestation services such as Intel Trust Authority [110].

A possible solution to the trust problem could leverage the live migration feature of TEEs to initialize a CVM locally and then migrate its state to a remote host, where it continues its execution. Live migration is performed over a secure channel established between the TEE modules in both hosts, preserving the confidentiality and integrity of the CVM. Since the CVM is initialized on a local platform, the guest owner has full control over its boot process and could therefore implement a full *AL4* attestation strategy. Live CVM migration is supported by AMD SEV-SNP and Intel TDX [15, 111]. However, apart from some technical and organizational challenges (e.g., the guest owner needs proper TEE hardware to initialize the CVM), this workflow is currently not supported by any CSP and thus we did not explore it further.

Finally, we note that CC is still a relatively new technology and support for recent TEEs such as SEV-SNP in the cloud has not yet reached maturity. On the evaluated CSPs, CVMs are currently available in selected regions only and, on GCP, support is in public preview. Despite existing limitations, we see many efforts from CSPs to improve their offering, driven in part by the open source community and standardization bodies. For example, Google has recently open-sourced an unofficial tool to verify the firmware of CVMs running on GCP [89], which potentially allows reaching a *trustworthy AL2*.

3.7 Conclusions

Being marketed as a solution that allows for the “removal of even the cloud provider from the Trusted Computing Base” [56], which has caught the attention of customers who need an additional layer of protection for intellectual property, sensitive data, or for regulatory compliance in data protection, confidential computing is becoming a mainstream technology for cloud-based services. However, our research shows that, today, the promised security guarantees are far from reality as cloud providers still play a crucial role in the deployment and management of confidential workloads, especially regarding the latest trend with confidential virtual machines. Yet, customers that are willing to trust the CSPs can still benefit from current offerings as a defense-in-depth mechanism for strong hardware-rooted isolation from other tenants and the hypervisor. We also notice a joint effort between cloud providers and organizations such as the Confidential Computing Consortium and the IETF, as well as the open source community, to enhance current confidential computing solutions. Here, researchers can play an important role and nudge actors towards more principled security.

Chapter 4

About Time: On the Challenges of Temporal Guarantees in Untrusted Environments

This chapter was previously published as:

F. Alder, **G. Scopelliti**, J. Van Bulck and J. T. Mühlberg. “About Time: On the Challenges of Temporal Guarantees in Untrusted Environments”. In: *6th Workshop on System Software for Trusted Execution (SysTEX’23)*. 2023

Preamble

In the previous chapter, we have seen that TEEs present some operational challenges that need to be properly addressed when deploying workloads on untrusted third-party infrastructures such as public clouds. As a follow-up, this work highlights a specific and non-obvious limitation of TEE architectures that, if overlooked, might subvert the security guarantees provided by enclaves.

This research is the result of several discussions with Fritz Alder during the design of the self-revocation scheme that will be presented in Chapter 5. As will be more clear later, our design required computing timeouts and other operations based on the current time, which initially did not seem to be a problem but then, upon closer investigation, we stumbled upon some issues: How can we provide a reliable representation of elapsed time to an enclave, and how can we synchronize time among a network of enclaves? Indeed, if we look at the TEE threat model, the infrastructure where enclaves run is untrusted, which means that privileged software layers such as the host operating system and the hypervisor are assumed to be either malicious or compromised. Hence, fetching the time from the untrusted environment is dangerous and should be avoided in favor of more reliable sources. However, some TEEs do not natively provide access to a trusted time source and, even when they do, it is difficult to safely provide time values to enclaves without being altered by an attacker, thus making such values partially or completely lose their meaning.

Therefore, measuring time inside an enclave is a delicate task that needs to take into account factors such as the TEE architecture being used, the time source from which time is read, and the specific use case considered. By contrast, however, the current trend in confidential computing is to move existing workloads to TEEs with a “lift-and-shift” approach, i.e., without making any modifications to application code. This trend has been more pronounced in recent years, with the advent of VM-based TEEs for cloud workloads (cf. Chapter 3). Unfortunately, this approach is not always a good idea as it might provide a false sense of security to application owners: Considering the usage of time, if an application uses timestamps for some security-critical operations (e.g., as an entropy source for creating a cryptographic key), then an attacker that has control over the time source could potentially exploit the application by providing it with carefully-chosen values. For instance, most libraries and frameworks for Intel SGX rely on the untrusted time provided by the operating system, thus applications running in SGX enclaves are susceptible to this class of attacks by default and might need to be redesigned to avoid using insecure time.

Based on this motivation, our publication aims at raising the attention to possible issues of TEEs that may be overlooked, highlighting that not every TEE is the same and each architecture might provide different capabilities that can influence the threat and attacker model. Thus, when moving an application into a TEE, it is important to carry out a careful security analysis, making design and implementation changes when necessary.

4.1 Introduction

Since the rise of atomic clocks, the precise progression of time is measured by examining the radioactive decay of atoms. In computing, this is often simulated by quartz clocks that trade a lower accuracy for affordability in everyday technology. To enable the use of time in human communication, standards like Coordinated Universal Time (UTC) define the precise time at a given moment as points of reference. However, what is colloquially called *wall-clock time* is an inherently human-made concept that is agreed upon by social convention. Unix systems, for example, translate UTC to *system time* as the number of passed seconds since the 1st of January 1970. As such, wall clock time in computing is a challenging task that is nowadays dealt with by utilizing time servers and running time synchronization protocols.

In the area of secure systems, recent years have seen the rise of capable confidential computing solutions that can protect data in use, such as Intel SGX, AMD SEV, Intel TDX, and ARM CCA. The key idea behind these Trusted Execution Environments (TEEs) is to protect critical code and data in so-called *enclaves* that are isolated from the rest of the system stack. Particularly, while the privileged operating system and hypervisor remain in charge of enclave resource management and availability, any direct access to enclave code or data is prevented by means of hardware-level access control logic. TEEs can, hence, effectively protect sensitive data while it resides in memory. However, enclaves may still request untrusted network communication from the potentially hostile operating system to perform computations.

The challenge of time-keeping in the context of confidential computing can be troubling in multiple ways. First, agreeing on time in a distributed context often relies on external trusted time servers, together with an approximation of the time delay between server and recipient. However, enclaves that need time information have to communicate with remote servers through an untrusted, attacker controlled environment. This potentially allows the adversary to unpredictably delay network packets. Second, even if no trusted time server is utilized and time is only communicated between local peers, adversaries can often control the execution and interruption of enclaves, which may give the attacker some control to induce further delays.

In this paper, we investigate the challenge of reasoning about time and providing temporal guarantees in the context of confidential computing. Particularly, we identify five levels of increasingly more capable time tracking approaches and overview existing timing capabilities in popular TEEs, with an explicit focus on the widespread Intel SGX architecture and software ecosystem. Our specific contributions are:

- We identify five distinct levels of tracking the passage of time from within isolated enclaves.
- Focusing on Intel SGX, we analyze which time levels can be provided to enclaves and how this is used by the software ecosystem and in selected academic work.
- We classify other TEE architectures and analyze their limitations in regards to trusted time-keeping.
- Lastly, we overview use cases and applications, highlighting the overall difficulty of guaranteeing trusted time in untrusted environments.

4.2 System Model

We consider a general TEE architecture, where a security-critical application executes in a trusted enclave protection domain that is isolated from any other software on the system, including the privileged operating system and hypervisor. The enclave may further include a shielding system that offers trusted services to the application, e.g., in form of a library OS or Software Development Kit (SDK).

This study considers applications that need some notion of “wall-clock time” from a trusted time source in order to realize their security objectives. The time source could be local, e.g., a secure timestamp counter in the CPU, or remote, e.g., a trusted server. We explicitly focus on reasoning about the wall-clock time that passes between two execution points, which does not necessarily correspond to the actual running time of the enclave. That is, while the latter may be easier to measure, e.g., by counting the number of instructions executed in the enclave, it is decidedly not a good proxy for wall-clock time in the presence of attacker-induced interrupts that may arbitrarily delay enclave execution.

Note that in case the application needs *absolute* wall-clock time, i.e., calendar time, an extra step may be needed such as an initial time synchronization with a trusted source.

4.3 Notions of Time

This section identifies five successive levels of time tracking approaches for confidential applications, summarized in Tab. 4.1. These levels are additive, so that each level represents an increasingly more capable and accurate time

Type	Added guarantee	Rollback	Freq.	Delay	Interrupt	Example time source	Discussed use case
T ₀	None					Untrusted OS	—
T ₁	Time monotonically advances	✓				Untrusted OS + check	Time-based policies
T ₂	Time moves at constant pace	✓	✓			ME, timer thread, re-mote server	Rate limiting
T ₃	Time is read with known delay	✓	✓	✓		Secure TSC, MMIO timer	Resource counting
T ₄	Use of time is atomic	✓	✓	✓	✓	Trusted scheduler	Credential expiration, DRM

Table 4.1: Levels of time and the guarantees given by them. Checkmarks identify that a level protects against this type of attack.

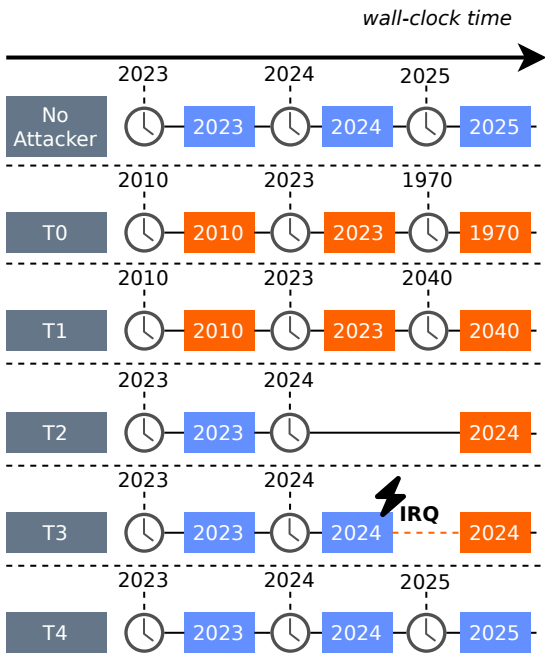


Figure 4.1: Visual representation of different time levels with attacker influences marked in orange. T_0 is controlled by the attacker, T_1 can only move forward but make jumps, T_2 can be arbitrarily delayed, and finally in T_3 the attacker can still interrupt the enclave before it utilizes the time. Instead, level T_4 gives the same guarantees as when no attacker is present.

source. However, higher levels may also place more demands on the underlying TEE architecture, and, as discussed later, lower levels may be sufficient for certain applications.

Fig. 4.1 depicts each time level, together with a sample time trace reported to and experienced by the enclave. The top row represents the ground truth, where no attacker is present and wall-clock time advances linearly and at a constant rate. The running time of the enclave application is visualized in the colored squares, whereas the clock symbols represent a query to the abstract time source. The example value returned by the time source is indicated above the clock symbol and will be used in the subsequent enclave computation, colored blue if the time is correct w.r.t. the wall-clock time, or orange if time is influenced by the attacker.

T_0 : No trusted time The default case is to have no trusted time, i.e., to fully rely on untrusted input without performing any checks on the incoming time data. As illustrated in Fig. 4.1, the observed time can be any point in the past or in the future and may vary arbitrarily. While this may suffice for testing

scenarios, e.g., annotating untrusted timestamps in a debug log, it is clear that any application-specific, security-sensitive use of time cannot rely on a T_0 timer.

T_1 : Monotonically increasing time If enclaves only have access to an untrusted time source, they can perform certain minimal checks on received timestamps before acceptance. The most important check to perform here is to ensure that received time data only ever increases, i.e., that time never winds backwards. Depending on the granularity of the untrusted time source and the sampling frequency, additional checks may be added, e.g., checking that two time queries should never return the same value or zero. While these checks are minimal and the underlying time source is still untrusted, the counter value can be stored inside the enclave and T_1 can at least prevent attackers from rewinding the internal time of an enclave. Since T_1 timers cannot rely on external data, they have to store the time locally. This makes them prone to rollback attacks where time is reverted by restarting the enclave with an obsolete local timestamp.

Fig. 4.1 shows that attackers in T_1 can advance time at will and can trigger arbitrarily large time jumps for the enclave.

T_2 : Trusted remote time To improve time-keeping, the enclave can switch from an untrusted to a remote trusted time source, guaranteed to have a fixed, known frequency. As such, this time source can be relied upon for trusted time information, when standard cryptographic measures are in place to protect and validate the integrity and authenticity of the time packets over the untrusted network (e.g., this secure time channel can be configured as part of the attestation protocol). Note that the “remote” trusted time source can be truly remote, e.g., a server reached over the Internet, but could also be provided on the same system-on-chip, e.g., by a trusted co-processor. If the monotonicity of the external time source can be trusted, rollback attacks targeting the time value are also not possible anymore since on enclave restarts, the time can be re-queried. Obviously, rollback attacks are still a threat to enclaves with a T_2 timer and above, but they will not affect the time stored in the enclave.

As the communication channel between the time source and the enclave is still under the attacker’s control in regards to availability, T_2 time requests and responses may be arbitrarily delayed. Hence, T_2 timestamps received by the enclave can only ever be trusted as a *lower bound* on the reference wall-clock time. Fig. 4.1 depicts a potential situation where the attacker would allow the enclave to query the time source, but then delay the response.

T₃: Trusted local time In order to prevent arbitrary delay attacks, enclaves must be able to access the trusted time source through a direct channel without adversary intervention, e.g., when the time source is an on-chip timestamp counter that can be queried via dedicated CPU instructions. If both time source and channel are trusted, adversaries cannot arbitrarily delay timestamps anymore before they reach the enclave. This, technically, grants enclaves access to accurate time information with both a lower and an *upper bound*.

We argue, however, that in order to utilize such accurate time information in practice, enclaves also need to be able to act *atomically*, i.e., without (undetected) attacker-induced interruptions. Fig. 4.1 depicts a possible attack that is still possible on T₃ where the adversary precisely interrupts the enclave right after it reads from the trusted time source. The adversary can then keep the enclave interrupted until it suits them and only resume the enclave at a convenient time, effectively reducing the accurate temporal guarantees back to only the lower bound offered by a T₂ time source.

Consider an enclave that only grants access to a resource if a presented certificate is valid, and denies access otherwise. If the adversary begins the validation process at a time that the certificate is still valid, the enclave will read a permissible time from the time source. Right after the time value has been read into the enclave, the adversary could however interrupt the enclave. When the enclave is later resumed, the adversary could gain access to the resource even if the initially presented certificate is not valid anymore.

T₄: Trusted and atomic local time The dangers of the above attack vector are nuanced. Many interruption attacks on T₃ timers only work if the adversary initially had access to a resource. Thus, arguably, maintaining this access until after the credentials expired can be considered as a parallel issue to denial of service attacks where output of the enclave is delayed by the adversary. We note, however, that for some applications and use cases this attack vector may not be acceptable and enclaves should only present an output at the correct time. We discuss two examples of this in Sect. 4.6.4.

4.4 Intel SGX

The Intel SGX architecture does, in itself, not provide any notion of trusted time. However, several proposals exist to provide trusted time services to SGX enclaves.

Platform service Intel provides Platform Software (PSW) to set up and use trusted time and monotonic counters [48]. Particularly, the PSW offers a T_2 time source based on a coarse-grained clock in the Intel Management Engine (ME), a chip that physically resides on the same package but is entirely separate from the main CPU. This allows timer-based policy enforcements on offline platforms that are not necessarily connected to a network. All communication between enclaves and the ME passes via the untrusted operating system and is, hence, cryptographically protected, but can be arbitrarily delayed. Enclaves can use the `sgx_get_trusted_time` API to track the amount of time (in seconds) passed since a previous read of the timer. ME services are excluded in the Intel SGX Linux PSW from version 2.8 from 2020 onwards.

Timestamp counter Initial SGX versions did not allow to execute the `x86 rdtsc` instruction to receive the processor’s internal TimeStamp Counter (TSC) in enclaves. However, more recent SGX2 processors now support the `rdtsc` instruction in enclave mode, with an explicit warning that the instruction may be trapped and the timestamp returned may be influenced by the untrusted operating system.

We experimentally confirmed that privileged adversaries can arbitrarily control the values returned by `rdtsc` on SGX2 platforms by directly manipulating the underlying `IA32_TIME_STAMP_COUNTER` Model-Specific Register (MSR). For this, we developed a sample enclave that executes two `rdtsc` instructions and returns the measured cycle difference. One would expect this difference to always be positive and reflect the elapsed time in between the two `rdtsc` instructions. However, in our proof-of-concept, we make the observed time in the enclave move “backwards” by interrupting the enclave and overwriting the TSC value using the privileged `wrmsr` instruction in between both `rdtsc` calls. Thus, we conclude that `rdtsc` on current SGX platforms is not any better than an adversary-controlled T_0 timer.

We note, however, that the TSC MSR is duplicated per logical processor and can, hence, only be changed by interrupting the enclave. That is, as long as the enclave is not interrupted, any successive `rdtsc` reads will behave as a trusted T_4 timer that monotonically increases at fixed frequency. This may be leveraged with upcoming ISA changes [59] that will make SGX enclaves interrupt-aware. At the same time, the restriction that an enclave is never interrupted may be too limiting for some real-world deployments.

Academic timers TrustedClock [129] is a system that relies on an interrupt handler in System Management Mode (SMM) to implement functionality that is similar in its security guarantees to `sgx_get_trusted_time` by the Intel

SDK	OE	EDP	Gramine	LKL	Occlum	Mystikos	Ego	Enarx
—/T ₂	T ₀ *	T ₀	T ₁	T ₁	T ₀ *	T ₀	T ₁	T ₀

Table 4.2: Intel SGX software projects and their use of time. Stars indicate that we prototyped a proof of concept to roll back time for a project. Intel SDK is shown for Linux/Win.

SGX PSW. The authors base their security argument on the handler being loaded at boot time with a secret key that is not accessible at runtime by the untrusted OS. Enclaves can then communicate with the handler by establishing a secure channel based on the pre-shared public key. Aside from the security implications of including the unattested SMM handler in the trusted computing base, TrustedClock is stated to not be completely immune to timer modifications by the untrusted OS. Instead, TrustedClock relies on combining several x86 hardware timers and asserting their monotonicity (similar to T₁), thus making it harder for a malicious OS to precisely tamper with all involved timers at the same time.

TimeSeal [23] utilizes a cohort of multiple threads that each periodically poll `sgx_get_trusted_time` to monotonically increase the time. The authors argue that by using multiple threads and nuanced synchronization policies between the threads, attacks can be limited in their scope. Nonetheless, attackers that delay or prevent scheduling of timer threads at convenient times, such as right before a thread returns a time response, will be able to skew the reported time of TimeSeal. Additionally, with the discontinuation of `sgx_get_trusted_time` on Linux systems, TimeSeal may not be implementable anymore as of today.

Software ecosystem Several shielding systems offer a “lift-and-shift” approach to port legacy applications to SGX. However, these applications were not designed for SGX’s threat model and may rely on sane OS time services.

Tab. 4.2 summarizes how existing SGX shielding systems expose the untrusted OS time to applications. Interestingly, we found that most projects include a warning, but forward OS time without any restrictions (T₀), whereas the Gramine and LKL library operating systems at least ensure that time does not move backwards (T₁). We developed two minimal proof-of-concepts to demonstrate backwards time advances in OpenEnclave and Occlum. We, hence, recommend that all projects include minimal checks to adopt the T₁ time model.

4.5 Other Architectures

Intel TDX The TDX specification [106] discusses TSC virtualization, which provides a consistent virtual TSC value to a Trusted Domain (TD) among all its vCPUs. The virtual TSC starts counting from zero when the TD is initialized and runs at a fixed frequency defined by a parameter stored in the TD configuration. Guest TDs can access the virtual TSC using the `rdtsc` instructions, and its value is calculated from the physical TSC adjusted to the TD's offset and frequency.

We analyzed the source code of the TDX module, which makes sure that the TSC is not modified before a TD enter by comparing the current `TSC_ADJUST` MSR value against a reference value stored on TD creation, raising an exception if a mismatch between the two values occurs. This allows TDs to obtain a stable and consistent TSC value across TD lifetime, guaranteeing a time level T_3 .

AMD SEV The AMD SEV-SNP [18] extensions introduced a so-called *Secure TSC* feature, which should provide enhanced protection against TSC manipulations. The Secure TSC feature can be enabled by a guest VM at boot and prevents the hypervisor from intercepting any `rdtsc` instructions called by the guest VM. Furthermore, the calculation of the TSC value is made using offset and frequency parameters stored in VM secure memory, thus not accessible by a malicious hypervisor. However, it is unclear from the specification whether the Secure TSC feature prevents the hypervisor from directly writing into the TSC via the `wrmsr` instruction, similar to our proof-of-concept on SGX2. In such case, the hypervisor could effectively tamper with the TSC while the guest VM is not running, preventing it from leveraging a stable and consistent representation of elapsed time. Without such guarantee, the TSC counter would only be a T_0 time level for SEV-SNP guest VMs.

ARM TrustZone ARM TrustZone [171] separates program execution into two protection domains, the secure world and the normal world. Both worlds are hardware-isolated, run their own OS, and normal-world software is prevented from directly accessing secure world resources. TrustZone also allows for system devices, such as a time source, to be restricted to one world. This, however, is implementation specific, with the TrustZone protection controller being an optional component. TrustZone further extends the processor's interrupt controller with prioritized secure and non-secure interrupt sources to prevent denial-of-service attacks from the normal world. Based on these features, ARM TrustZone supports up to T_4 , and has been previously leveraged to implement secure real-time systems [214] or TPM functionality in software [174].

ARM CCA ARM confidential compute architecture [25] adds additional execution worlds to the ARM architecture that allow to run multiple so-called Realms in parallel to the ARM TrustZone secure world. All Realms are all managed by a trusted realm management monitor (RMM) component [26]. While each Realm has direct access to architectural timers, the RMM does not make scheduling decisions or manage interrupts, both controlled by the untrusted hypervisor [25]. As such, ARM CCA has access to a T_3 time source.

TPM A Trusted Platform Module (TPM) is a hardware-based security component that provides a secure foundation for system integrity usually in form of a co-processor. TPMs comprise timing components and monotonic counters. According to the specification [201], a TPM includes two primary timing components referred to as *Time* and *Clock*. The *Time* component is separate hardware that provides the number of milliseconds since it has been powered on and cannot be controlled by software. The *Clock* component yields a value that software can advance but can never roll back.

Thus, by mapping this specification to our time levels, the *Clock* component only guarantees a T_1 time, since a privileged adversary can advance the *Clock* value arbitrarily, while *Time* may reach up to the T_4 level if an offset is securely provided to the TPM to align its value to the wall-clock time. However, according to [201], attackers may change the TPM clock frequency by at most $\pm 32.5\%$, which should be taken into account when checking the time.

4.6 Use Cases

This section focuses on use cases that rely on some form of time. To facilitate this discussion, we adopt an incremental approach, beginning with time-based policies that only necessitates level T_1 , and gradually progressing through each level up to credential expiration, which requires T_4 .

4.6.1 Time-Based Policies

A simple time-based policy is that a user that once lost access to a resource will not regain access to it. By utilizing T_1 , enclaves can enforce this behavior so that the attacker cannot revert time to regain access. Enclaves can be useful in systems where the OS is initially in a trusted state but may become compromised during runtime. In this case, the enclave would periodically be invoked with linearly increasing timestamps and then at some time be invoked

with an old timestamp. If the attacker timestamp precedes the last timestamp sent by the benign operating system, the enclave would be able to detect this attack and prevent access.

It is important to note that, by using T_1 , the enclave will not be able to revoke access if the OS does not wish this to happen, as progress of time is still attacker-controlled. The only guarantee is that, once revoked, access can not be reinstated. This is similar to *Clock* guarantees of TPMs [201]. Similarly, using T_1 timers assumes that rollback attacks on the enclave state can be detected by the enclave. Otherwise, the adversary could restart the enclave and provide a stale timestamp or stale sealed data to the restarted enclave to get the desired result. If rollback attacks on the time value can not properly be prevented, a T_2 timer has to be used.

4.6.2 Rate Limiting and TOTP

Rate Limiting Time-based rate limiting is a mechanism often used to narrow the impact of brute force attacks. Consider a service running in an enclave that performs password checking [119]. If time is OS-controlled, time-based rate limiting would not be effective as attackers can artificially advance time. The service would easily be tricked into processing more requests than desired. Thus, T_1 is not a sufficient time level for rate limiting. Only when the frequency of the clock is independent from the attacker can the enclave rely that at least a minimum amount of time has passed.

This use case does not have higher requirements on the time source than a T_2 time source. It does not matter whether a minute or a year has passed between two requests as long as *at least* a minimum amount of time has passed. If the untrusted OS increases the time between requests, this does not go against the security guarantees offered by rate limiting and would only impact availability of the service.

TOTP Time-based One-Time Passwords (TOTPs) [101] are commonly used in second-factor authentication systems. The algorithm takes as input a fixed parameter k (e.g., a key) and a variable parameter x , and gives as output the one-time password, which is simply computed as a HMAC truncated to the desired size. In TOTP, the variable parameter x depends on the current time t , an initial value t_0 , and a fixed time window W according to the equation $x = \lfloor (t - t_0) / W \rfloor$. Typically, x_0 is equal to 0 and W is equal to 30 seconds. This means that, within the same 30-second window, the resulting OTP will always be the same, provided that the same k is given as input. As such, attackers

who have control over the time source, as in levels T_0 and T_1 , could potentially slow down or even freeze the time perceived by the TOTP server, preventing the OTP from changing. This could be exploited by attackers to circumvent the second factor authentication provided by TOTP, either through brute-force attacks or by reusing an old OTP previously used by the legitimate user. Thus, we conclude that TOTP requires a T_2 time source.

In the context of Intel SGX, a few projects have proposed the use of TOTP authentication. For instance, SGX-UAM [225] utilizes TOTP to authenticate a client to the identity provider, while SCONE [55] employs it as a second-factor authentication for their configuration and attestation server. While the former utilizes `sgx_get_trusted_time`, now discontinued on Linux, it is unclear which time source is used by the latter.

4.6.3 Resource Accounting

When utilizing confidential computing in cloud environments, both users and cloud providers may require a trustworthy and accountable measurement of spent resources during a computation. Interesting nuances with this use case occur when code running inside the enclave is not necessarily trusted by both parties that wish to rely on the measurement. Take for example the case of Function-as-a-Service in enclaves [7]. There, workload providers run a workload and only wish pay for the resources used. If possible, workload providers may want to pay less than what they owe. Cloud providers, on the other hand, may wish to overestimate the resources consumed and demand more compensation.

The cloud provider can always make accurate estimates on enclave time, since the workload runs on their infrastructure. However, the workload provider cannot trust any time levels below T_3 : If the untrusted OS controls the time, such as in T_0 or T_1 , time can be manipulated to overestimate resource consumption, leading to higher billing for the workload provider. Furthermore, even with a T_2 clock the cloud provider could arbitrarily delay the enclave communication with the clock and artificially increase the workload time. Only if the channel to the clock is independent from the cloud provider, and the clock is independent from influence, the time source can be accurately utilized for resource accounting. Note that, without a full T_4 utilization of the clock, the cloud provider could still cheat and interrupt the workload for periods of time. In this case, the system would need to be set up in a way that punishes such behavior and undercounts time spent potentially interrupted, e.g., through means of resetting the count on interrupt reentries.

4.6.4 Credential Expiration and DRM

A common use cases for using time is to check the validity of credentials, such as PKI certificates and JSON Web Tokens (JWTs). Typically, these credentials have a validity window defined by a *Not Before* and a *Not After* field. Thus, it is important that the time source provides a reliable time to correctly validate such credentials, e.g., during the TLS-handshake or the authorization step at application level.

If time is not reliable inside the enclave, it could lead to security breaches such as allowing access to sensitive data to unauthorized users. If an untrusted T_0 or T_1 clock is used, attackers may rewind the time perceived by the enclave to fall within the validity window of an expired certificate. If instead stronger T_2 or T_3 timers are used, the adversary may still induce network delays and/or interrupts to coerce the enclave into accepting credentials that were valid at the beginning of the communication, but have expired before their actual use. This problem is exacerbated when credentials are short-lived: typically, access tokens are refreshed frequently and thus have a rather short lifetime of minutes to hours [102]. Thus, the enclave should only trust a time level T_4 in order to make correct decisions.

The open-source `rust-mbedtls` library written by Fortanix [86] allows TLS-termination inside SGX enclaves. Time-validity of certificates is only checked if the `time` feature is enabled, which means that either it is not checked at all (if disabled), or it is checked against the insecure OS time (if enabled). This shows that the absence of a trusted time source in SGX can potentially compromise the security of applications, thus software engineers should take into account such limitations when developing their applications.

The use case for DRM can be reduced to an instance of credential expiration, where a user wants to access a resource that should already be locked to them. If the enclave is interrupted after the authorization check but before the use of the resource, the latter might take place when the user has no longer permission to access it.

4.7 Conclusions

Tracking the passage of time is nuanced when relying solely on measurements that are partially controlled by an adversary. To better reason about the usage of time inside TEEs, we identified five progressive levels of time-keeping, T_0 to T_4 and classified the popular Intel SGX as well as other commonly used TEEs. Considering relevant use cases such as rate limiting or resource counting,

we show that not every application requires access to the highest level of time, for which existing solutions may already suffice. However, this classification also shows that other use cases, such as credential expiration, need stronger time guarantees that most architectures cannot yet provide. Thus, reliable time-keeping is highly application-specific and software engineers should take into account the limitations of current technologies.

Chapter 5

Efficient and Timely Revocation of V2X Credentials

This chapter was previously published as:

G. Scopelliti, C. Baumann, F. Alder, E. Truyen and J. T. Mühlberg.
“Efficient and Timely Revocation of V2X Credentials”. In: *Network and Distributed System Security (NDSS) Symposium 2024*. Internet Society. 2024

Preamble

We conclude this journey by presenting an example of a secure integration of TEEs into existing protocols and applications, taking into account the challenges presented in Chapter 4 and demonstrating how carefully-studied designs can provide benefits such as increased security, efficiency and scalability to a system. In particular, we focus on credential management systems in Vehicle-to-Everything (V2X) applications, where credentials need to be distributed to vehicles in a network in order to secure Vehicle-to-Vehicle (V2V) communication. A crucial aspect of these systems is *credential revocation*, i.e., rendering the credentials of malicious or misbehaving vehicles unusable to preserve the safety

of the V2X network. It is especially important that revocation is *timely*, i.e., it is executed within a short and predictable time, but also *efficient*, i.e., it scales well with the potentially large size of the network without incurring prohibitive costs and latencies.

Existing standards on V2X credential management raise some concerns on both the timeliness and efficiency of their revocation schemes [230]. For this reason, academic proposals have tried to integrate trusted computing technologies inside vehicles to enable a “decentralized” credential management and revocation [84, 218, 67, 98, 122]. In these schemes, each vehicle has a *trusted component* responsible for securely storing credentials and performing cryptographic operations such as message signing and verification. Other software in the vehicle, therefore, has no direct access to those credentials, which remain protected even in case the vehicle becomes compromised. However, as long as credentials are valid, an attacker is still able to request the trusted component to sign malicious V2V messages and potentially harm the system. To effectively make credentials unusable, researchers have introduced the idea of *self-revocation*, where the trusted component inside the vehicle autonomously deletes its credentials if requested by the V2X infrastructure via a revocation message. In these proposals, trusted components are typically TPMs, but TEEs like Intel SGX are another viable option [113].

Our contribution addresses a problem that was overlooked in previous work: What happens if the revocation message is never received by the trusted component, for example due to network malfunctions or malicious activity (either in the network or inside the vehicle)? To deal with this issue, researchers proposed to use keep-alive messages and timeouts to detect misbehaviour [84, 218], but did not give enough details to effectively solve the problem. Besides, they failed to consider a crucial point, i.e., that trusted computing technologies, and TEEs in particular, cannot provide strong availability guarantees nor a reliable notion of time (cf. Chapter 4). In addition, previous work did not provide a comprehensive evaluation of their design, and it is unclear whether the self-revocation approach indeed gives actual benefits compared to traditional revocation schemes. Therefore, both the timeliness and efficiency properties are somewhat questionable in earlier work.

The first time I read about credential management in V2X was in January 2022, when I came across a paper from Whitefield et al. [218]. In that occasion, I immediately recognized the potential of self-revocation, but I also quickly realized that this approach had many challenges that were not sufficiently addressed in the paper. Sadly, I noticed that even subsequent work seemed to neglect these issues [67, 122]. This, nevertheless, turned into an opportunity to contribute to this domain. Together with Christoph Baumann, we designed our secure version of the self-revocation scheme, which we refined over the

months and even submitted a patent application with title “*Secure revocation mechanisms for V2X attestation protocols*” that was filed in June 2022. The core of our work, however, was ensuring that our approach actually gives the guarantees that earlier work could not provide. To this extent, much time and effort was dedicated to formally verify our scheme with Tamarin prover [144] to satisfy the *timeliness* property, i.e., that we can actually guarantee timely credential revocation even with a strong network attacker and considering the availability limitations of TEEs. Part of the formal verification work was also the basis to evaluate the *efficiency* of our scheme where, together with Fritz Alder and Christoph Baumann, we demonstrated that our design is efficient and scalable.

Our evaluation is a significant contribution to validate our design and provide enough evidence about the feasibility of our approach. For reproducibility of results, all our code was open-sourced to allow everyone who is interested in our work to verify our claims. Our code was also peer-reviewed as an artifact alongside our paper and received all three “Available”, “Functional”, and “Reproduced” badges. Subsequently, we also received the “Distinguished Artifact Award” during the NDSS 2024 conference. This was the recognition of countless hours spent on improving and documenting the code to reduce as much as possible the effort required to reproduce our results, which was particularly appreciated by the reviewers. Our code was also presented as a demo at the Ericsson Research Security Day 2022 and the VehicleSec 2024 workshop, co-located with NDSS [185].

Lastly, this contribution led to two master’s theses aiming to perform a quantitative evaluation of revocation schemes in V2X. Indeed, due to the lack of publicly available reference implementations, our paper could only provide a *qualitative* comparison between our self-revocation scheme and traditional approaches. In order to carry out a “fair” evaluation, instead, it is necessary that each scheme is evaluated under the same conditions to give a meaningful comparison. To achieve this, our students implemented credential management on top of the Artery V2X simulation framework [176], which they used to compare active, passive and self-revocation mechanisms. Their results show that our self-revocation scheme gives the most balanced solution in terms of revocation time and network overhead [65, 166].

5.1 Introduction

Intelligent Transport Systems (ITS) are a key technology for innovation in transportation that link different modes of transportation and address safety,

congestion, emissions, and efficiency problems. Increased connectivity in vehicles, called Vehicle-to-Everything (V2X) communication, incorporates digital interactions with other road users with potentially safety-critical impacts on the vehicle and its passengers. As highlighted by Sedar et al. [188] there are many possible incentives for attackers to target V2X systems, which range from personal interest (e.g. tracking or insurance fraud) to geopolitical conflict (e.g. causing congestion or accidents).

A large body of academic research and technical standards for V2X communication focuses on privacy-preserving protocols to enroll and manage pseudonymous identities [218, 67, 122, 82, 83, 42]. The key argument for such protocols is that special care must be taken to ensure vehicle routes and patterns in their use of V2X services must not be observable by other V2X participants in the network. Yet, the central concern in transportation has always been road safety. In V2X scenarios, road safety requires that misbehaving network participants, e.g., faulty or malicious vehicles that report inaccurate road and traffic conditions, can be identified and prevented from causing harm. For example, the US Security Credential Management System (SCMS) defines revocation “*an essential design objective*” [42].

In a practical working system, it must be possible to (1) detect, and (2) punish such misbehaving participants by removing them from the network. We identify a common shortcoming in existing privacy-preserving V2X approaches, with respect to their capability to punish malicious or misbehaving actors after misbehavior connected to a participant’s pseudonym is detected: Ensuring that this participant will not be able to communicate with other vehicles in the future and effectively *revoking* all their existing credentials in a timely manner is either not guaranteed and avoidable by the attacker, or is inefficient and not scalable due to bandwidth and computational overheads. Indeed, recent surveys [230, 215] highlight that existing schemes based on active [42] or passive [83] revocation suffer from scalability issues, low reactivity, or both.

To address these challenges, we build upon decentralized revocation schemes based on self-revocation [218, 67, 122], which utilize a Trusted Component (TC) embedded in vehicles for secure credential management. Self-revocation enables vehicles to self-certify and also self-revoke pseudonymous credentials, obsoleting most of the centralized public key infrastructure of current ITS designs. This paper addresses a number of shortcomings of previous work, summarized by the following contributions:

1. We design a secure self-revocation scheme that, in contrast to earlier work, can guarantee revocation with an upper bound on revocation time, without relying on a trusted time source in TCs (Sect. 5.5);

2. We formalize and verify our design using the TAMARIN prover [144], showing that it provides strong guarantees on the revocation of credentials even in presence of a powerful adversary who drops and/or delays revocation requests (Sect. 5.6);
3. We evaluate the security, performance, and scalability of our design, showing that our approach guarantees prompt revocation and low utilization of resources, even in the presence of network malfunctions and a high number of attackers, in difference to related work (Sects. 5.7 and 5.8).
4. We discuss applicability of our revocation scheme to state-of-the-art V2X protocols, showing that it is compatible with current standards in the EU and US, as well as Direct Anonymous Attestation (DAA) (Sect. 5.8.2).

We provide the full formal specifications and an implementation of our design with this publication [184]. The most recent version of this artifact is also available on GitHub¹.

5.2 Background

In the EU, performance and security requirements for ITS and advanced use cases (such as vehicle platooning or remote driving) are laid out in [77] and [80]. For example, it is specified that ITS need to support “high connection density for congested traffic” with “3,100 to 4,300 cars per square kilometer.” Vehicles in these scenarios communicate at rates of up to 50 messages/s with an end-to-end latency of no more than 20 ms (platooning), or even via continuous data streams of up to 25 MBit/s with an end-to-end latency of no more than 5 ms (remote driving) [80].

To maintain security in these scenarios, infrastructure and vehicles must implement efficient protocols that facilitate credential management, including revocation, and signing and verifying message content while upholding privacy objectives and legal requirements on data protection. As summarized by Hicks and Garcia [98], the requirements for the management of V2X credentials involve communicating parties to verify the authenticity of messages, guarantee unlinkability, minimize the repercussions of a compromise of central trust management systems, and enable revocation of vehicles upon the detection of policy violations, malfunction, or malicious interaction. Therefore, vehicles must be capable to timely manage the validity of credentials, which involves changing

¹<https://github.com/EricssonResearch/v2x-self-revocation>

their own pseudonym certificates and validating those of communicating parties. Importantly, communications and security management protocols must also account for extended periods without connectivity, where a vehicle is unreachable or powered off.

Certificate Management and Revocation In Europe, the architecture for credential management in V2X is defined by ETSI [82, 83], while the US follows the SCMS design [42]. In both architectures, ITS participants possess two types of credentials: (1) *long-term credentials*, for authenticating a vehicle towards the ITS infrastructure, and (2) *pseudonym credentials* (*pseudonyms* in short), for secure and privacy-preserving peer-to-peer communication, such as Vehicle-to-Vehicle (V2V), Vehicle-to-Infrastructure (V2I), or Vehicle-to-Pedestrian (V2P). Long-term credentials can be either private keys stored in secure elements of vehicles at manufacture time, or certificates obtained during enrollment with the ITS. Pseudonyms are typically retrieved from the infrastructure upon successful authentication and authorization of a vehicle (via its long-term credential), and are usually short-lived (with lifetime of up to weeks [79]) and rotated regularly to prevent vehicle tracking.

Revocation aims at excluding a participant from interacting with others. We distinguish between *identity-based revocation* and *message-based revocation*, which differ in the way that the decision to revoke is made: The former is based on a vehicle's *canonical identity*, and is necessary to enforce legal/policy requirements (e.g., to have only insured vehicles on the road), in case of private-key compromise (e.g., a leak), or when a vulnerability affecting a certain component or vehicle vendor becomes public, requiring revocation of multiple vehicles at once. The latter, which is the focus of our work, is based on peer-to-peer traffic, where a misbehaviour detection system is in place to report faulty or malicious messages to a Revocation Authority (RA), which eventually mandates the revocation of the vehicle that signed such messages. When a vehicle gets revoked, its long-term credentials are blacklisted by the ITS in order to deny it future requests, e.g., for re-enrollment or for obtaining new pseudonyms. Additionally, all current vehicle's pseudonyms should be made unusable, to prevent the transmission of malicious messages to other peers. In this regard, SCMS and ETSI use two opposing strategies, i.e., active and passive revocation, respectively (cf. Sect. 5.3).

Trusted Execution and DAA A common approach to establish a root of trust in distributed systems is by relying on Trusted Components (TCs), such as Trusted Platform Modules (TPMs) or Trusted Execution Environments (TEEs) [138], which provide the means to securely store and use long-term credentials,

perform key generation, remote attestation, and sealing. Several papers propose the use of trusted components inside vehicles to manage cryptographic credentials and increase security and privacy [96, 213, 113]. Based on the security features provided by TCs, Direct Anonymous Attestation (DAA) has been developed as a privacy-preserving protocol to remotely authenticate a system [43, 201]. In difference to ongoing standardization efforts in ITS and SCMS, which rely heavily on centralized public key infrastructure [82, 83, 42], the application of DAA in an ITS context obsoletes most central PKI by relying on secure processing elements (notably TPMs), which enable a decentralized credential management. For example, some DAA schemes allow vehicles to derive pseudonyms autonomously from long-term credentials [218, 67]. These pseudonyms are self-certified using DAA signatures that can be verified by other ITS participants.

5.3 Related Work

Traditional message-based revocation schemes take two opposing directions: *Active revocation*, where the information of revoked pseudonyms is distributed to vehicles in real time, and *passive revocation*, where pseudonyms have a short lifetime and are left to expire. Additionally, recent schemes propose a novel approach based on *self-revocation*.

Active revocation Active revocation is employed by SCMS and consists in the distribution of Certificate Revocation Lists (CRLs) [58] to all vehicles in the network, containing the list of pseudonyms that have been revoked and thus should not be trusted. Vehicles use this information to decide whether or not to accept or discard a message received from other peers. CRL updates are issued regularly by the RA to indicate newly revoked pseudonyms, and must be received as quickly as possible by all vehicles to ensure the safety of the system. However, as CRLs grow large over time, the associated computation and communication overheads make these mechanisms less suitable for real-time systems. Several papers propose optimizations to reduce the size of CRLs and the verification overhead [193, 120], though recent surveys show that these optimizations are still far from ideal [215, 230]. The SCMS design relies on *linkage values* to identify all pseudonyms of a vehicle with a single value, effectively reducing the size of the CRL but significantly increasing the delay to decide whether to accept or not a network message [42].

Passive revocation As an alternative to active revocation, some schemes propose issuing pseudonym certificates that have a fixed and short lifetime, forcing vehicles to request new pseudonyms regularly to continue participating in the network [98, 211, 4]. Internal CRLs of long-term credentials are used by the infrastructure to deny such requests to malicious vehicles, thus negating the need for pseudonym revocation. Passive revocation is also adopted in the ETSI standard [83].

Since pseudonym certificates are not revoked, vehicles detected as malicious can continue their operation until the last of their pseudonyms has expired. As such, malicious vehicles still have a non-negligible time window after their detection in which they can spread false information and cause safety-related incidents [79], and has been identified as an open issue [230]. Thus, to increase security, pseudonym lifetimes would need to be quite short. However, with shorter lifetimes and increasing participants in the network, the traffic between infrastructure and vehicles would increase significantly, along with the risk of Sybil attacks [69].

Self-revocation Some DAA protocols adopt a scheme called self-revocation, where revocation is performed with the active cooperation of the vehicle to be revoked [218, 67, 122]. The concept of self-revocation was initially proposed by Förster et al. in their REWIRE protocol [84], as a privacy-preserving revocation scheme for vehicular networks that leverage a hardware-based TC in vehicles. In such a scheme, the RA broadcasts an Order for Self-Revocation (OSR), containing the pseudonym identifier to be revoked. The idea is that, upon reception of such message, the targeted vehicle will detect that the revoked pseudonym is one of its own and then proceed with the deletion of all its credentials, such that the vehicle will be unable to generate malicious messages any further. This approach eliminates the need for CRLs or short pseudonym lifetimes, providing substantial benefits in terms of performance and scalability [20].

This simple scheme works with the assumption that the TC is a trusted entity and will behave as expected, i.e., that it will delete the credentials as soon as the OSR is received. In case such messages are delayed or dropped, e.g., due to a network disruption or a malicious host, Förster et al. propose the use of *keep-alive messages* that are periodically broadcast by the RA to vehicles and used to detect network interruptions, causing the automatic deletion of all credentials in a TC if no messages are received for an extended period. However, this approach is described only informally, and does not address an attack scenario where only OSRs are dropped, but keep-alive messages are not. The REWIRE scheme was formally verified in [219], though the possible disruption

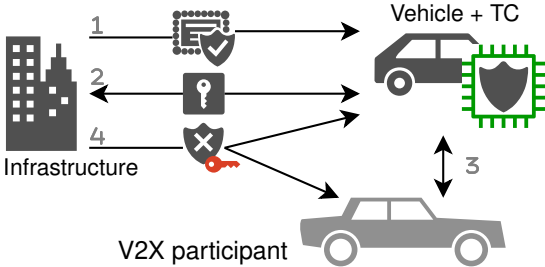


Figure 5.1: Lifecycle of a V2X architecture. An infrastructure enrolls vehicles (1) and equips them with pseudonymous credentials (2) which they can use to take part in V2X communication (3). On-demand, the infrastructure revokes pseudonyms or certificates (4).

of OSRs was not addressed. Furthermore, a similar *heartbeat* mechanism is later proposed in subsequent work [218, 67, 122], which suffers from the same limitations. Finally, all solutions seem to implicitly rely on a notion of trusted synchronized time to check the freshness of revocation requests and compute timeouts, but such a feature may not be available on all vehicles due to the limitations of current technologies [10, 24], neither do previous papers discuss possible issues related to clock drifting between vehicles and the RA.

5.4 Motivation and System Model

Section 5.3 highlights that, while active and passive revocation schemes present some drawbacks, self-revocation of pseudonyms seems a promising solution to remove misbehaving or malicious participants from the V2X network efficiently and in a timely manner. However, related work on the topic ignores several challenges posed by realistic attackers and does not provide strong guarantees or upper time bounds for revocation. None of the previous papers effectively solved the problem of attackers actively dropping/delaying revocation requests, and the *heartbeat* idea was only informally introduced but never developed [218, 67, 122]. To the best of our knowledge, there are no self-revocation schemes available that work in practice, making these protocols unsuitable for real-world deployment in V2X systems. Earlier formal verification work [219] does not address this problem, and no related paper provides an upper-bound analysis nor a comprehensive evaluation. Additionally, previous work hints at the requirement of trusted timestamps in network messages, but none provides a full design that integrates a trusted time source in TCs. In fact, under a realistic attacker model, current confidential computing technologies, and TEEs

particularly, cannot rely on a trusted time source [10]. Thus, previous works that implicitly assume trusted synchronized time cannot currently be implemented securely.

In this work, we aim to address these issues by providing a reliable, formally verified revocation mechanism that can be employed in V2X systems and beyond. We propose a design that considers the challenges of having a reliable time source in TCs, and we discuss optional extensions that may bring additional advantages. We strive to minimize the computational and communication overheads caused by revocation schemes, making our design efficient and suitable for any kind of system and network. Below, we overview our system model, discuss attacker capabilities, and introduce our objectives. Our design is then detailed in Sect. 5.5.

5.4.1 Vehicles and Trusted Components

In V2X, participants can be vehicles, pedestrians, traffic lights, and so forth. For readability, in this paper we only discuss vehicles, but our system model can be generalized to *any* participant in the network. We consider vehicles as integrated systems composed of a number of Electronic Control Units (ECUs) connected by a local in-vehicle network such as a CAN bus. These ECUs may also be connected to a wide variety of sensors and actuators, used to retrieve information from the vehicle itself or its surroundings (e.g., speed, position), or to perform actions (e.g., accelerate, brake). Typically, all such components are managed by a central unit called On-Board Unit (OBU), which is also responsible for processing network messages sent to and from the vehicle [136].

Additionally, we assume that vehicles contain a secure processing element called the Trusted Component (TC), responsible for credential management and cryptographic operations, analogously to DAA protocols (cf. Sect. 5.2). In our system model, the TC is a passive device that exposes an interface to the OBU for tasks such as enrollment/attestation, pseudonym generation, and message signing/verification. Therefore, all sensitive operations are performed on the TC itself, and the OBU cannot directly access cryptographic keys or other sensitive material. Examples of TCs are the TPM, ARM TrustZone, or TEEs such as Intel Software Guard Extensions (SGX) and ARM Confidential Compute Architecture (CCA). Nowadays, trusted components are ubiquitous in modern hardware [182], and with recent developments in the automotive domain, starting with Infineon’s automotive-certified Optiga TPM [103] (released in 2018) and automotive security processors such as the NXP NCJ38A [165] (available since 2020), TEE solutions are available and working groups within the Trusted

Computing Group and Global Platform are active in designing automotive security services based on TEEs [203, 88].

In our attacker model (cf. Sect. 5.4.3), we consider all components of the vehicle potentially malicious or compromised, except the TC. Therefore, at high level, we can define a vehicle as the sum of two main components: the *untrusted host* and the TC. Just like sensors, timing components used for precise timestamping are also part of the untrusted host, such that the TC cannot rely on a trusted notion of time by itself [10, 24]. Therefore, our design does not assume a trusted time source in vehicles to guarantee a timely revocation, but relies on trusted timestamps (or “epochs”) distributed by the ITS infrastructure to TCs (cf. Sect. 5.5).

5.4.2 The Vehicle Lifecycle in a V2X Network

The lifecycle of a vehicle in a V2X network is depicted in Fig. 5.1 and, in short, is composed of four main phases: (1) *enrollment*, (2) *pseudonym generation*, (3) *operation*, and (4) *revocation*. Although our paper mainly focuses on the last phase, every revocation scheme, to be effective, has some dependencies on the other phases. For example, vehicles that do not properly check the validity of signatures might erroneously accept messages signed with invalid or revoked pseudonym certificates, making the revocation process useless. Thus, in this section we first compile a set of functional requirements that a V2X protocol should implement in order to use our revocation scheme effectively. These requirements are based on the architectures proposed in state-of-the-art V2X credential management systems, and are discussed at *high level*: Our goal is to make our revocation scheme protocol-agnostic in order to be compatible with most existing protocols such as DAA or technical standards such as ETSI and SCMS (cf. Sect. 5.8.2).

Enrollment Each vehicle is equipped with a TC which exclusively stores credentials and performs cryptographic operations with them (cf. Sect. 5.4.1). In the initial phase, a vehicle enrolls to the V2X network by authenticating to the infrastructure, e.g., by presenting a valid certificate [83]. With trusted components such as TEEs, *remote attestation* also becomes an essential step to make sure that the TEE is configured correctly and is up to date, and the TC is running genuine software [182]. Successful enrollment should result in the issuance of a *long-term credential*, used by the vehicle as a token to authenticate subsequent requests to the infrastructure and to obtain pseudonym certificates (cf. Sect. 5.2).

Requirement 1 *Vehicles correctly enrolled to the network are issued a long-term credential, stored in the TC.*

Pseudonym generation In this phase, a vehicle uses the long-term credential obtained during enrollment to get a new pseudonym certificate. This process can be done either *online*, where the pseudonym is issued by the infrastructure upon successful authorization [83, 42], or *offline*, where the vehicle's TC performs a key derivation from the long-term credential [218, 67]. Either way, only non-revoked vehicles should be able to obtain new pseudonyms.

Requirement 2 *Pseudonym certificates are obtained using (valid) long-term credentials through online or offline protocols.*

Operation Pseudonyms are then used to authenticate messages that vehicles exchange with network peers. For readability and without loss of generality, this paper focuses on the communication between vehicles (V2V), but peers could also be pedestrians (V2P), smart city infrastructure (V2I), and so forth (cf. Sect. 5.2). V2V messages may include information about the vehicles themselves (e.g., speed, location, direction) and the surrounding environment (e.g., traffic, accidents). These messages need to be integrity-protected and authenticated to prevent message spoofing or tampering, and time-stamped to provide freshness; Both actions are performed by the TC, using pseudonym credentials for the former and its internal time *now* for the latter (we elaborate more on *now* in Sect. 5.5).

Requirement 3 *V2V messages are signed by the TC using its own valid pseudonyms and timestamped according to an internal TC variable *now*, representing the TC's current time.*

We then require that vehicles correctly check the authenticity of V2V messages to ensure that they have been signed using a valid pseudonym certificate. Furthermore, we demand that messages older than a predefined age are discarded, to prevent processing outdated information.

Requirement 4 *Vehicles should check the signature of received V2V messages to ensure that they have been generated using a valid pseudonym. A message with timestamp t_{v2v} should only be accepted by a receiving TC if not older than a predefined window T_v w.r.t. its internal time *now*:*

$$t_{v2v} \geq now - T_v \quad (5.1)$$

Revocation We expect the infrastructure to operate mechanisms for misbehavior detection and reporting, where pseudonyms can be flagged, e.g., for spreading messages that contain false information [42, 81]. Based on such reports, the RA can decide to revoke the corresponding vehicle’s credentials using the reported pseudonym. This is the concept of *message-based revocation* introduced in Sect. 5.2.

Requirement 5 *The RA makes decisions on revocation by employing misbehavior detection mechanisms in the network, to identify and report misbehaving pseudonyms.*

Moreover, we require that *only* non-revoked vehicles can enroll to the network, otherwise it would be trivial for attackers to bypass revocation by simply re-enrolling to the network. This can be achieved by keeping an internal blacklist of long-term credentials [83, 42]. Revoked vehicles may be reinstated at the discretion of the ITS infrastructure, e.g., after a factory reset.

Requirement 6 *Only vehicles whose credentials have not been revoked can (re-)enroll to the V2X network.*

We observe that a vehicle in a V2X network is operational iff two conditions are fulfilled: (1) the vehicle has *at least* one valid pseudonym, and (2) the TC’s internal time *now* is loosely synchronized with the others, i.e., within window T_v . If either of these two conditions is not met, a vehicle cannot generate valid and fresh V2V messages, according to Reqs. 3 and 4. Hence, we define *effective revocation* as follows:

Definition 1 (effective revocation) *A vehicle is considered effectively revoked when either all its pseudonyms are made unusable (or expire), or when its TC gets permanently de-synchronized, i.e., when now cannot be updated anymore.*

5.4.3 Attacker Model

We consider a powerful attacker that has control over a vehicle with valid credentials and wants to disrupt the V2X network by spreading malicious V2V messages, avoiding revocation of their pseudonyms as long as possible. Examples of attack vectors include faulty sensors, software bugs in ECUs, root access to the attacker’s own car, etc.

We assume a Dolev-Yao style attacker that controls the network stack and can tamper with, rearrange, resend, and stall network messages but not break

established cryptography [68]. The attacker is assumed to have full control over all hardware and software on vehicles except for functionality implemented by their TC, which is protected and fully trusted. Entities in the ITS infrastructure (e.g., the RA) are also trusted.

Besides the aforementioned cryptographic limitations, our attacker is not able to perform physical attacks on the TC. We stress that voltage tampering attacks against the hardware can still be a real threat and cannot be tackled by network protocols, but need to be addressed on lower levels of the stack [50, 155]. Side-channel attacks against the TC are also out of scope and will have to be mitigated through the usual means of hardware and software defenses [137, 45].

5.4.4 Objectives

Based on our system and adversary model, and the open issues of existing schemes (cf. Sect. 5.3), we define objectives for a practical and secure real-world revocation mechanism:

- O1 Guarantee of revocation:** Revocation should always succeed within a deterministic upper bound, even in case of delayed or dropped revocation requests. In particular, the *effective revocation time* is a time period starting with the revocation decision made by the RA and ending when the vehicle is effectively revoked.
- O2 No assumptions on trusted time:** Synchronizing trusted time across multiple vehicles with a strong attacker is challenging and no assumptions should be made on having access to a local trusted time source.
- O3 Scalability:** Any revocation mechanism suitable for V2X scenarios must be scalable in terms of vehicles and share of attackers in the network. This covers both computational as well as network overhead.
- O4 Adaptability:** The revocation mechanism needs to be configurable for different network characteristics and security requirements. It must be able to tolerate varying degrees of network latency.

5.5 Design

In a nutshell, our design has the RA send periodic messages containing the pseudonyms of vehicles to be revoked, as well as information used for a loose time synchronization between all participants. Vehicles need to relay these

messages to their TCs in order to be able to communicate with the network, otherwise they will eventually get de-synchronized. Hence, attackers cannot postpone revocation indefinitely, and eventually they become effectively revoked after a fixed amount of time that can be configured by system parameters (**O1**, **O4**). Below, we describe our design in detail.

Time synchronization Our definition of effective revocation introduced in Sect. 5.4, together with Reqs. 3 and 4, suggests that TCs need a reliable representation of time (i.e., *now*). Since a trusted and synchronized local time source is not always available in TCs (**O2**), we rely on a trusted server (i.e., the RA) that distributes timing information to all vehicles in order to roughly synchronize them to a reference time frame, which could be the actual time and date or an epoch counter [37]. Note that vehicles can still use a local time source for precise timestamping in real-time communication, though it is considered part of the untrusted environment just as other sensors (cf. Sect. 5.4.1).

The heartbeat The RA periodically distributes authenticated messages called heartbeats (HBs). HBs include a so-called *Pending Revocation List (PRL)*, i.e., a list of pseudonyms pending revocation. When the RA decides to revoke a pseudonym *ps*, based on Req. 5, it calls an internal REVOKE function to add the pseudonym identifier (e.g., a hash of its public key) to the PRL, such that subsequent HBs will contain this revocation request. As we will discuss in Sect. 5.5.1, each pseudonym needs to stay in the PRL only for a limited period. In addition, every HB contains a timestamp t_{hb} , which is used to provide freshness information to the HB and to distribute timing information to the vehicles, used for time synchronization. In summary, the format of a HB is the following:

$$HB := PRL \parallel t_{hb} \parallel sig_{RA}(PRL \parallel t_{hb}) \quad (5.2)$$

Here, $sig_{RA}()$ is a digital signature made by the RA. HBs can be either fetched manually by a vehicle or periodically distributed via *broadcast* to reach multiple vehicles at once. Additionally, they can also be exchanged in a peer-to-peer fashion when connectivity is limited (e.g., rural areas).

Processing a heartbeat Vehicles should forward received HBs to their TCs. Similar to V2V messages (cf. Req. 4), the TC firstly verifies its digital signature to ensure that it is an authentic message generated by the RA, and then checks its freshness by ensuring that the timestamp t_{hb} is not lower than its current time *now* minus the validity window T_v , as follows:

$$t_{hb} \geq now - T_v \quad (5.3)$$

Upon successful verification of the HB, the TC synchronizes its time as below:

$$now := \max(now, t_{hb}) \quad (5.4)$$

Then, the TC performs *self-revocation* if any of its pseudonyms (represented by the set *Certs*) appears in the PRL:

$$PRL \cap Certs \neq \emptyset \Rightarrow self_revoke() \quad (5.5)$$

Self-revocation puts the TC in a revoked state, where all subsequent signature requests made by the vehicle's OBU are denied. Credentials may also be deleted from memory. A TC can only be restored upon successful authorization by the infrastructure (e.g., via vehicle reset and re-enrollment).

Missing reception of heartbeats Self-revocation is effective as long as HBs are correctly delivered to all TCs. However, attackers in the network, as well as compromised OBUs, might prevent this from happening in order to keep TCs operational for as long as possible (Sect. 5.4.3).

We note that a TC needs to process a HB in order to synchronize its time. In other words, if no valid HBs were processed for an extended period, the internal TC time *now* would stand still at the last processed HB, according to Eq. (5.4). This would lead to vehicle de-synchronization, in which the revoked vehicle would not be able to generate *fresh* messages, even though its TC is still operational (cf. Req. 3). In such case, other vehicles would start discarding all messages coming from the revoked vehicle as soon as Eq. (5.1) is not satisfied anymore. Thus, our security objective is guaranteed even when a revoked vehicle's TC does not process HBs.

Automatic revocation To make such de-synchronization permanent, a revoked vehicle should not be able to process any more HBs without causing self-revocation. This can be trivially achieved by permanently keeping pseudonyms in the PRL. However, the PRL would become bigger and bigger, making the distribution of HBs as slow and inefficient as traditional CRLs (cf. Sect. 5.3). To prevent this, we make TCs automatically perform self-revocation should they detect de-synchronization. We call this process *automatic revocation*, which is triggered when the timestamp t_{msg} of any received message (either a HB or a V2V message) is much bigger than the current time *now*, according to the equation below:

$$t_{msg} > now + T_v \Rightarrow self_revoke() \quad (5.6)$$

That is, vehicles de-synchronized for more than T_v are automatically revoked as soon as they process any *fresh* message. This, however, may also cause benign

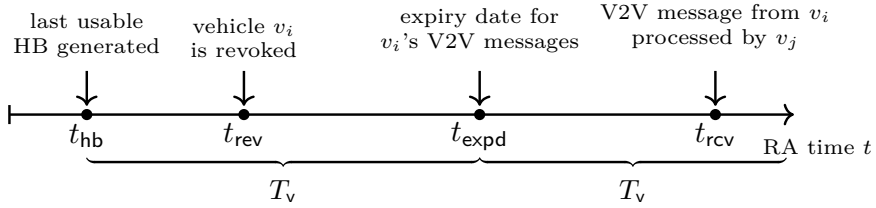


Figure 5.2: Timeline diagram showing times from Sect. 5.5.1 and their relations. When estimating the worst-case revocation time, t_{hb} would coincide with t_{rev} and t_{rcv} would occur T_v after t_{expd} , i.e., t_{rcv} is $2T_v$ after t_{rev} .

(i.e., unrevoked) vehicles to trigger revocation accidentally: If this happens, they can simply be restored by the infrastructure, though this may cause temporary service interruption and should be avoided when possible. See Sect. 5.7.3 for a deeper discussion of the issue.

Eq. (5.6) ensures that, at any time, either: (1) the TC is in sync with the RA, being behind the current time t at most T_v , or (2) the TC is behind t by more than T_v , and any received message with $t_{msg} \geq t$ will cause automatic revocation. Usually, only compromised vehicles that ignore received messages remain in case (2) without being revoked automatically. If an honest vehicle ends up being isolated from the network like that, it is most likely due to targeted jamming attacks or lacking network coverage. We discuss this issue further in Sect. 5.8.3 but rule it out here. Therefore, in what follows we assume for all honest vehicles in the network:

$$now \geq t - T_v \quad (5.7)$$

5.5.1 Properties

The design elaborated above guarantees two important properties of revocation:

- (i) *If a pseudonym ps belonging to a vehicle v_i is revoked by the RA, then after a fixed time period T_{eff} v_i will be effectively revoked;*
- (ii) *Each pseudonym can be removed from the PRL after a fixed time period T_{prl} since its insertion time.*

Property (i) is crucial as it implies effective revocation, guaranteeing that the REVOKE function always succeeds within a fixed time period, even if HBs are dropped or delayed (**O1**). Property (ii) ensures that the size of the PRL (and consequently the HBs) does not keep growing over time, guaranteeing scalability (**O3**). Below, we give an intuition of properties (i) and (ii), while in Sect. 5.6 we discuss our formal verification work to prove their correctness. Fig. 5.2 illustrates the general timeline assumed below.

Revocation time Let us assume that, at a time t_{rev} , the RA orders revocation for a pseudonym ps owned by a malicious vehicle v_i . That vehicle would drop all HBs generated after t_{rev} to prevent TC_i from locking its credentials. Given that a TC only updates its internal time when processing a HB (Eq. (5.4)), we observe that t_{rev} is the highest value that TC_i would store as internal value *now*. Thus, all V2V messages signed by TC_i would have a timestamp $t_{\text{v2v}} \leq t_{\text{rev}}$. Putting this information into Eq. (5.1) for a receiver v_j , we obtain:

$$\text{now}_j \leq t_{\text{rev}} + T_v \quad (5.8)$$

Eq. (5.8) shows that TC_j will discard all V2V messages signed by TC_i as soon as its internal time now_j exceeds $t_{\text{rev}} + T_v$. Conversely, for a time period T_v since t_{rev} , TC_j is still accepting messages coming from v_i . Assuming all honest vehicles in the network are at most T_v behind the RA time t (Eq. (5.7)) and using Eq. (5.8) in the inequation, we get:

$$t - T_v \leq \text{now}_j \leq t_{\text{rev}} + T_v \quad (5.9)$$

Estimating the upper bound for $t - t_{\text{rev}}$, we can then define the *effective revocation time* T_{eff} as follows:

$$T_{\text{eff}} = 2T_v \quad (5.10)$$

Time in PRL As (1) t_{rev} is the highest value for now_i that TC_i can have after the revocation order for ps , and (2) Eq. (5.6) ensures that HBs containing timestamp t_{hb} greater than $\text{now}_i + T_v$ would trigger revocation in TC_i , we can conclude that ps can be removed from the PRL as soon as RA time t passes $t_{\text{rev}} + T_v$. Thus, we can define the minimum period of time T_{prl} that ps needs to remain in the PRL as follows:

$$T_{\text{prl}} = T_v \quad (5.11)$$

5.5.2 Extensions

Trusted time source A trusted time source in vehicles would enhance V2V communication through precise and trusted timestamping, making **O2**

redundant. In our design, if the TC has access to a trusted time source its internal time can advance automatically and Eq. (5.4) would be unnecessary. In such case, however, the condition for automatic revocation would never occur, because Eq. (5.6) cannot be satisfied. Thus, we can define an additional condition by measuring the time that has passed since the most recent HB $max_{t_{hb}}$ that has been received: If such time is higher than T_v , the TC would trigger revocation:

$$now > max_{t_{hb}} + T_v \Rightarrow self_revoke() \quad (5.12)$$

As we will discuss in Sect. 5.6 and Appendix B.2, we demonstrated that the properties defined in Sect. 5.5.1 still hold, with the assumption that TC and RA clocks are perfectly synchronized. In reality, however, there may be a bounded clock drift Δ that needs to be added to Eq. (5.10) and Eq. (5.11).

Security enhancements Although T_{eff} is a strict upper bound on revocation time (cf. Sects. 5.5.1 and 5.6), it is still possible to improve the *average* revocation time. We devised two techniques to make it harder for attackers to postpone revocation, both are evaluated in Sect. 5.7.1 to show their effectiveness:

1. *Encrypted HBs.* we observe that the worst-case scenario illustrated in Sect. 5.5.1 can be easily achieved if a malicious OBU has complete access to the content of a HB. Here, the OBU can arbitrary delay and drop HBs in order to evade revocation and avoid de-synchronization for as long as possible. By leveraging encryption, instead, the attacker would be unable to look at the content of a HB, and would be forced to make a guess on whether or not to drop, delay, or relay to the TC a HB received from network. HB encryption can be achieved by means of a pre-shared symmetric key obtained during enrollment and rotated periodically. In case such key is leaked, it would only cause attackers to gain access to the content of HBs, but they would still be unable to forge valid ones.
2. *Active revocation.* The PRL included in HBs can be used to perform active revocation (cf. Sect. 5.3). That is, a TC could store the most recent PRL received via HB and use it as a lightweight CRL in the message verification phase (Req. 4). As the size of the PRL is generally small (cf. Sect. 5.7.2), this allows to reduce the average revocation time for malicious vehicles without causing too much overhead in the message verification phase.

5.6 Formal Verification

We formally verified the correctness of properties (i) and (ii), defined in Sect. 5.5.1, using TAMARIN prover [144], a symbolic verification tool for security protocols based on multi-set rewriting rules and first-order logic. This section summarizes our work, while Appendix B.2 contains additional details. The complete models and instructions to execute the proofs locally on TAMARIN version 1.6.1 are available online [184].

5.6.1 Introduction on Tamarin

TAMARIN is a symbolic verification tool that supports verification of security protocols, based on multi-set rewriting rules and first-order logic. Cryptography constructs, such as symmetric/asymmetric encryption, digital signatures and hashing, are built into the tool together with a Dolev-Yao adversary.

A protocol in TAMARIN is modeled as a set of *rules*, each of them defining a specific protocol step. The system's state is expressed as a multi-set of *facts*, i.e., predicates that store state information. Each rule has a pre-condition, i.e., facts that are consumed by the rule, and a post-condition, i.e., facts that are produced after its execution. In addition, *restrictions* allow to specify additional constraints on the rule's execution.

Rules can also produce special facts called *action facts*, which are not part of the system's state but rather are used to notify that a certain event has occurred (e.g., a HB has been generated). Action facts are used to define properties on the model, called *lemmas* in TAMARIN, using first-order logic. To verify a lemma, TAMARIN uses a “backwards reasoning” approach where it starts from a later state and reasons backwards to derive information about earlier states. In particular, TAMARIN starts with a state that contradicts the lemma, and tries to construct an execution trace to check whether that state is actually reachable or not via the defined rules. If a complete execution trace that reaches that state can be constructed, then the lemma is falsified, otherwise it is successfully verified.

5.6.2 Model

We modeled our revocation scheme considering both our main design and our extension when TCs have access to a trusted time source (Sect. 5.5.2); The latter is discussed in Appendix B.2. Our models consider an arbitrary T_v and include three entities: (1) the RA, which generates HBs and revokes pseudonyms,

```

restriction IsLatestTime:
  "All t #i . IsLatestTime(t)@i ==>
    not (
      Ex t2 #j . TimeIncrement(t2)@j & j<i
        & GreaterThan(t2, t)
    )"

rule advance_time:
  [ !Time(t) ]
  --[
    OnlyOnce(<'advance_time', t>)
      , TimeIncrement(t + '1')
  ]->
  [ !Time(t + '1') ]

```

Figure 5.3: Definition of the `advance_time` rule that increments the time counter by one, together with the `IsLatestTime` restriction that ensures the most recent value is used in a rule.

(2) the TC, which processes HBs received from network and generates V2V messages, and (3) a generic third entity, which receives and verifies V2V messages generated by the TC. Since our focus is on verifying revocation properties (cf. Sect. 5.5.1), we did not focus on enrollment or credential management. We did, however, implement basic cryptographic functionalities for making and verifying digital signatures, using the *signing* built-in. This is enforced on both HBs and V2V messages. Additionally, we allow the TC to obtain and use an arbitrary number of pseudonym credentials.

Notion of time Given that our goal is to measure the time since revocation, our model also requires a notion of progress. This was not trivial to implement in TAMARIN, as it does not provide any native support for time passing. Therefore, we leveraged the *multiset* built-in to model time as a logical counter that represents time steps, allowing our entities to perform any number of operations within the same time step. To increment the counter, we defined a rule that takes as pre-condition the current value and increments it by one (Fig. 5.3). The same built-in was also used to model the PRL.

Persistent facts Time steps, timeouts, PRL and pseudonyms have all been modeled using *persistent facts*, i.e., facts that remain in memory persistently, even after being consumed by a rule. In our experience, this was a better choice than using *linear facts* which, once consumed, are deleted from the system's state. For example, persistent facts allow reusing the same `!Time(t)` fact in multiple rules, which reduces the number of possible states and thus the memory used by TAMARIN. We also experienced some undesired side-effects when using linear facts, such as infinite recursion due to the fact that rules were both consuming and producing `Time(t)` facts at the same time. However, the use of persistent facts cause multiple `!Time` facts to be in the system's state at the same time. For example, after the rule `advance_time` in Fig. 5.3 is executed

for the first time, we would have both `!Time('1')` and `!Time('1'+'1')` facts in memory. This would potentially cause execution of rules using the old `!Time` fact instead of the new one. Although this inconvenience does not compromise the overall accuracy of the model, it does not represent a realistic scenario and it also causes a higher number of possible states. Hence, we added a restriction `IsLatestTime(t)` (cf. Fig. 5.3) in every rule to ensure that the latest time is always used. For the PRL, we used sequence numbers to restrict rules to use the most recent list.

Modeling T_v We modeled T_v as a parameter initialized in the setup phase (`Init`). Our goal was to model T_v as an arbitrary value to prove our properties for any value greater than zero. To do so, we defined a first rule called `init_parameters`, which creates a linear fact `TvTmp` with initial value `'1'`. Then, we defined another rule called `increment_Tv` that consumes such linear fact and produces a new one with value incremented by one. Finally, the `Init` rule takes the current value and finalizes it, creating a persistent fact `!Parameters` used in other rules.

5.6.3 Lemmas

We defined three types of lemmas: (1) *Sanity lemmas*, to ensure that all rules can be executed at least once, i.e., that we defined the model correctly; (2) *Functional lemmas*, to ensure that the model correctly implements our design in Sect. 5.5; and (3) *Proof lemmas*, to verify the properties in Sect. 5.5.1. Table 5.1 provides the full list of lemmas and a short description.

Modeling systems that have an internal state and progress over time makes it extremely challenging to verify lemmas within a reasonable amount of time and memory. In order to build an efficient proof for some of our lemmas, we gave as input to TAMARIN a Python script called *oracle*, which provides a custom ranking for *goals*. Indeed, TAMARIN proves a lemma by attempting to solve the open goals of the constraint system, which are sorted based on some heuristics. By providing an oracle as input, goals can be ranked according to a custom heuristic. This was essential for us, as for some of the lemmas TAMARIN could not terminate with the default heuristics due to an inefficient ranking of goals. For example, solving `!Time` goals needs to be postponed as much as possible because, in an unconstrained system, time can have an arbitrary value and TAMARIN would get stuck in an infinite loop. By using a specific ranking, instead, we could give priority to other goals that add more and more constraints, allowing `!Time` goals to be solved efficiently. Our oracles guarantee

Lemma	Type	Oracle	Steps	Description
<code>sign_possible</code>	S		5	Ensures that the TC can sign V2V messages
<code>generate_hb_possible</code>	S		5	Ensures that the RA can generate and distribute HBs
<code>issue_revocation_possible</code>	S		5	Ensures that the RA can revoke pseudonyms, adding them to the PRL
<code>processing_hb_possible</code>	S	✓	7	Ensures that the TC can receive and process a HB
<code>revocation_possible</code>	S	✓	10	Ensures that the TC can self-revoke when receiving a HB
<code>process_message_possible</code>	S		7	Ensures that a third entity can process V2V messages signed by the TC
<code>exists_par_tv_5</code>	S		8	Used to verify that T_v is arbitrary
<code>exists_par_tv_8</code>	S		11	Used to verify that T_v is arbitrary
<code>no_signing_after_revocation</code>	F		2	Ensures that the TC cannot make signatures after performing self-revocation
<code>all_heartbeats_processed_within_tolerance</code>	F	✓	130	Ensures that the TC correctly discards outdated HBs, i.e., older than T_v
<code>all_signatures_max_time_t_rev</code>	F	✓	156	Ensures that, if revocation is mandated at time t_{rev} , the TC cannot make valid V2V messages with timestamp greater than t_{rev}
<code>effective_revocation</code>	P	✓	1964	Verifies property (i) in Sect. 5.5.1
<code>no_heartbeats_processed_after_tolerance</code>	P	✓	100	Verifies property (ii) in Sect. 5.5.1

Table 5.1: Full list of lemmas proven with TAMARIN on our model. *Type* is either *S* (sanity), *F* (functional) or *P* (proof). *Oracle* indicates whether an oracle was needed, while *Steps* shows the number of steps required by TAMARIN to prove the lemma.

```

// property (i)
lemma effective_revocation [heuristic=o "oracle.py"]:
  "All msg ps t #i . MessageAccepted(msg, ps, t)@i ==>
    Ex tv #j . SystemInitialized(tv)@j & j<i
    & not (
      Ex ps2 t_rev #k . RevocationIssued(ps2, t_rev)@k
      & GreaterThan(t, t_rev + tv)
    )"

// property (ii)
lemma no_heartbeats_processed_after_tolerance [heuristic=o "
  oracle.py"]:
  "All prl t_hb t #i . HeartbeatProcessed(<prl,t_hb>, t)@i ==>
    Ex tv #j . SystemInitialized(tv)@j & j<i
    & not (
      Ex ps t_rev #k . RevocationIssued(ps, t_rev)@k
      & k<i
      & GreaterThan(t_hb, t_rev + tv)
    )"

```

Figure 5.4:
TAMARIN proof
lemmas that prove
properties (i) and
(ii) in Sect. 5.5.1.

that the verification only takes a few minutes in total. Table 5.1 indicates for which lemmas a custom oracle was needed.

Figure 5.4 shows the proof lemmas used to prove our properties. The lemma `effective_revocation` says that there is no state where *any* V2V message signed by a revoked ps is verified by another entity whose internal time is higher than $t_{rev} + T_v$. Hence, the attacker is permanently de-synchronized from the network and T_{eff} can be estimated assuming Eq. (5.7), as shown in Sect. 5.5.1. The lemma `no_heartbeats_processed_after_tolerance`, instead, proves that a revoked TC cannot process *any* HBs whose timestamp is bigger than $t_{rev} + T_v$. There are two possible reasons why this happens: (1) The TC has processed a HB generated since t_{rev} , meaning that it is in a revoked state since then; (2) The TC has not processed any HB generated since t_{rev} , meaning that processing a HB with timestamp bigger than $t_{rev} + T_v$ would trigger automatic revocation. Either way, this can be directly translated to our second property: Pseudonyms belonging to the revoked TC can be safely removed from the PRL after $t_{rev} + T_v$, i.e., $T_{prl} = T_v$ (cf. Eq. (5.11)).

5.7 Evaluation

In this section, we evaluate our scheme and give additional insights. Sect. 5.7.1 focuses on property (i), showing experimental results on revocation time obtained from simulating different attackers and network conditions (**O1**). Sect. 5.7.2 demonstrates that, thanks to property (ii), the PRL is kept small even with a high number of revocations, proving the scalability of our approach (**O3**).

Finally, Sect. 5.7.3 discusses how T_v should be chosen, based to the characteristics of the network and the desired security and efficiency properties (**O4**).

5.7.1 Revocation Time

We implemented a full prototype that realizes the revocation scheme from Sect. 5.5 and we evaluated it on a simulated V2X network running on Kubernetes [52]. The simulation environment allows to approximate the average revocation time for a range of network parameters in order to demonstrate the effectiveness of our scheme. Additionally, we simulated network malfunctions such as delays and packet losses, to show that our approach is resilient against real-world conditions. Code and configuration files are available online [184].

Implementation Due to the lack of open-source V2X implementations, we implemented a custom V2X protocol based on [113], in which a group symmetric key is shared among all vehicles to sign and verify V2V messages. In addition, TCs are able to autonomously generate their own pseudonyms (similar to [218, 67]), which are random identifiers attached to messages as metadata. Thus, all V2V messages are linked to the (pseudonymous) identity that signed them, allowing for the revocation of bad actors' credentials. This protocol satisfies all requirements of our revocation scheme (cf. Sect. 5.4.2), but may not be practical for real-world use cases. Still, it allows us to obtain realistic average revocation times for different attacker models, numbers of vehicles, network conditions, and validity windows T_v .

In our prototype, vehicles are grouped into one or more edge areas, while a centralized infrastructure is responsible for enrollment (via an Issuer) and revocation (via a RA). Besides, on each edge area a Road-Side Unit (RSU) is responsible for fetching the latest HB from the RA and broadcasting it to all vehicles in the area. Furthermore, an edge area is divided into groups to simulate proximity, so that a vehicle broadcasts V2V messages only to its neighbors, i.e., other vehicles currently located in the same group. Each vehicle has two components: the untrusted host, or OBU, and the TC. The latter is implemented as a passive device that exposes the following API to the OBU: JOIN to enroll in the network, CREATE to generate a new pseudonym, SIGN and VERIFY to sign and verify V2V messages, and HEARTBEAT to process a HB. In our prototype, JOIN enrolls a vehicle in the network, after which the TC obtains: (1) the RA's public key, needed to authenticate the HBs, (2) the V2V group credential, needed to sign and verify V2V messages, (3) a long-term identifier, and (4) the current timestamp used for synchronization.

V2V messages are randomly-generated data, signed and verified using the symmetric group key. Revocation is triggered by reporting pseudonyms to the RA, who eventually adds them to the PRL via the REVOKE API. A *reporter* component inspects the V2V traffic in each area and periodically reports a random pseudonym.

Attackers We implemented different behaviors for the OBU, depending on how HBs are processed and relayed to the TC. In particular, we simulated attackers trying to evade revocation by dropping or delaying the HBs. Our goal was to show that our properties actually hold in practice (Sect. 5.5.1), but also that the average revocation time can be significantly improved with some extensions (Sect. 5.5.2). To this end, we implemented three main attacker levels:

- *honest*: Simulates normal behavior as a baseline, i.e., the OBU relays all HBs to the TC as soon as they are received from the RSU;
- *smart*: The *smart* attacker delivers HBs normally as long as none of its pseudonyms are in the PRL. However, it attempts to evade revocation by dropping all HBs that contain any of its pseudonyms;
- *blind*: Simulates a scenario where HBs are encrypted (Sect. 5.5.2), thus attackers have no access to their content and do not know if and when their pseudonyms will get revoked. Therefore, the *blind* attacker has to make a guess on whether to deliver or drop a HB. We tried several logics for this attacker, and the one with best results was the attacker relaying only one HB every T_v , to allow clock synchronization and prevent revocation to be triggered automatically (Eq. (5.6)).

Experimental setup We ran our simulations on a Kubernetes [52] cluster with version 1.22.17. Up to 400 simulated vehicles are scheduled on 8 worker nodes, each with Intel(R) Xeon (R) CPU E5-2680 v4 @ 2.40GHz and two 10Gbps network interfaces, and 168 GB of total available memory.

We evaluated several scenarios with different values for T_v , with or without trusted time in TCs. In this paper we only discuss a scenario without trusted time where T_v is equal to 30 seconds, resulting in a T_{eff} and T_{prl} of 60 and 30 seconds, respectively, according to Eqs. (5.10) and (5.11). Complete simulation results are provided with our artifacts [184].

Each simulation ran for two hours and spawned 400 vehicles over a single edge area, divided into 20 groups. Of the vehicles, 10% were malicious, using

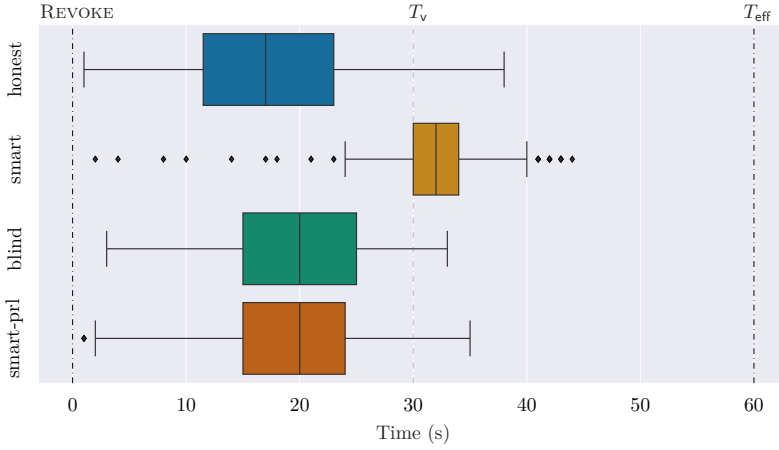


Figure 5.5: Simulation results for $T_{eff} = 60s$. The boxes show, for each revoked pseudonym, the time between REVOKE and the last VERIFY event, aggregating more than 600 revocations but excluding negative values.

one of the attacker levels described above. Each vehicle could have only two concurrent pseudonyms, disabling pseudonym rotation to not create bias on the revocation time. A new HB was generated every second by the RA, and every second the RSU fetched and distributed the latest HB to vehicles. In order to simulate network malfunctions and denial of service attacks, the RSU either dropped or delayed HBs, each with probability 0.4, such that only 20% of the HBs were delivered to vehicles without delay. Also, vehicles generated and distributed V2V messages every second. To increase the number of VERIFY events, messages from malicious vehicles had a 30% probability to be replayed. A pseudonym was reported and revoked every 10 seconds to gather sufficient experimental data (more than 600 revocations).

Results The box plots in Fig. 5.5 illustrate the distribution of revocation times for each attacker level. An additional box plot labeled *smart-prl* shows simulation results under a *smart* attacker, where TCs use the PRL to perform active revocation, as described in Sect. 5.5.2. Each value represents the time between the revocation of a pseudonym ps (REVOKE event) and the last V2V message signed with ps that was verified by a non-malicious TC (VERIFY event). Such time period represents the *effective revocation time* for that particular pseudonym (cf. Sect. 5.4.2).

It should be noted that we filtered out negative time values from the results: such values represent a situation where the last VERIFY event happened *before* the REVOKE event, for example because the TC owner of the revoked pseudonym was already in a revoked state due to the revocation of another of its pseudonyms, or simply because that pseudonym was not used to sign messages in the time after REVOKE. Either way, these values were not interesting for our evaluation.

Fig. 5.5 shows that pseudonyms belonging to *honest* vehicles are revoked rather quickly. Still, as V2V messages have a validity period of T_v and receiving vehicles' internal time may lag behind the RA within T_v , the effective revocation time varied widely, with a median of 17 seconds.

The simulation with the *smart* attacker level resulted in a median of 32 seconds and the majority of revocations between 24 and 40 seconds. This attacker is able to postpone revocation by dropping HBs, yet not relaying fresh HBs eventually causes de-synchronization in the TC. Therefore, in 99% of cases revocation was effective within $2/3$ of T_{eff} , as honest vehicles were able to stay synchronized with the infrastructure even when simulating severe network malfunctions.

As for the *blind* attacker level, we experienced a higher number of negative values in our results. This can be explained by the fact that a *blind* OBU is not able to recognize which HBs can be dropped or delayed, which ones are fresh and which are old, and which ones can be delivered to the TC without causing revocation. As a result, and considering the network malfunctions we simulated, the TC is more likely to get de-synchronized and thus automatically revoked, even before revocation is mandated by the RA. This suggests that encrypting HBs makes it more difficult for an attacker to evade revocation. In the few cases where the vehicle was still operating after the REVOKE event, instead, revocation rarely took longer than T_v .

Finally, the *smart-prl* simulation showed a significant improvement over the *smart* one, with values more evenly distributed between 1.5 and 37.5 seconds and median of 20 seconds. However, these results were negatively affected by our simulated network malfunctions. As with traditional CRLs, the ability of a TC to discard messages coming from revoked vehicles depends on how quickly HBs are received and processed; Thus, with limited connectivity the effective revocation time increases. Indeed, we experienced better results when simulating scenarios with lower drop/delay rates for HBs.

Overall, our simulations showed the feasibility of our revocation scheme in a simulated though practical scenario. In the average case, revocation takes much less than T_{eff} to be effective, even when attackers attempt to evade it. Furthermore, simulating network malfunctions did not incur permanent de-

synchronization of non-malicious vehicles, demonstrating the resiliency of our approach under real-world conditions.

5.7.2 Size of PRL

The size of a heartbeat is not constant. Instead, it depends on the size of the PRL, which varies over time according to the number of pseudonyms revoked and the parameter T_{prl} (Eq. (5.11)). If many pseudonyms are revoked at the same time, the PRL will be large until T_{prl} time has passed and they are evicted from the PRL again. To calculate the expected average PRL size, we approximate adding and removing pseudonyms from the list as a stochastic process and create a Markov model based on these parameters. The stationary distribution of this Markov chain then gives the probabilities for each given size of the PRL after an arbitrary number of steps.

Below, we elaborate on the core formulas and the expected values of the PRL size for a chosen set of parameters. Appendix B.4 contains a complete explanation of the utilized probabilities, the Markov model, and a discussion of the nuances of modelling the PRL size as a probabilistic process.

PRL as a Markov Model The process of adding and removing pseudonyms from the PRL can be seen as a finite state machine where the states are the possible sizes of the list. Transitions to a higher or lower state then happen on the addition or removal of a pseudonym, w.r.t. a base revocation probability p per pseudonym and time step. A discrete time Markov chain describes the probability of moving from state i to state j for each possible state [97]. As such, this Markov model represents the probabilities for gaining and losing pseudonyms from the PRL due to revocation or the removal of pseudonyms after T_{prl} , respectively. Assuming that the PRL contains i out of n possible pseudonyms at a time step, we can independently model the probabilities of gaining and losing k pseudonyms as two distinct binomial probabilities $G_{i,k}$ and $L_{i,k}$, respectively:

$$G_{i,k} = \binom{n-i}{k} \cdot p^k \cdot (1-p)^{n-i-k} \quad (5.13)$$

$$L_{i,k} = \binom{i}{k} \cdot \left(\frac{1}{T_{prl}}\right)^k \cdot \left(1 - \frac{1}{T_{prl}}\right)^{i-k} \quad (5.14)$$

Observe that $G_{i,k} = 0$ if $i + k > n$ and $L_{i,k} = 0$ if $i < k$. We can then combine these binomial probabilities into a Markov matrix with the following property:

$$p_{i,j} = \sum_{l=\max(i-j,0)}^i L_{i,l} \cdot G_{i,l-i+j} \quad (5.15)$$

At its core, this Markov matrix contains all probabilities of moving from list size i to list size j as a combination of losing $l \leq i$ pseudonyms from the list and adding $l - i + j \leq j$ new ones, where in each time step, a pseudonym is removed from the list with probability $\frac{1}{T_{\text{prl}}}$ and added with probability p . While this is not reasonable to calculate the effective list size after a specific number of steps, it suffices in a probabilistic model to determine the expected, average list size.

Average size of the PRL In Markov chains, a stationary distribution is a state probability that maintains its distribution even after taking a step in the Markov chain [97], represented by the stationary vector of the Markov matrix. It is thus a probabilistic equilibrium that, once reached, will be stable forever. After startup, a Markov chain will eventually reach its stationary distribution after enough steps have been taken. We can calculate the equilibrium for our Markov chain for parameters that are useful in the real world and for different revocation probabilities. By accumulating the probabilities for each given state, we can thus calculate the maximum list size as a percentile of possible states.

We evaluate the impact of a growing share of attackers on the PRL size in the context of two baseline scenarios. In these scenarios, *honest* pseudonyms, i.e., pseudonyms in the network that do not belong to attackers, may still experience a seldom revocation due to erroneous behavior, e.g., due to a faulty sensor. In the first baseline scenario, we consider a 1% probability that a pseudonym gets revoked at least once within a day, i.e., a 1% chance for an expected revocation every 86400 time steps of one second each. In the second scenario, we raise this to the extreme case of a 99% probability of being revoked at least once a day or 86400 time steps.

Fig. 5.6 depicts the 75th, 90th, and 99.99th percentiles of the PRL sizes for these two scenarios. The plot uses the parameters of $n = 800$ pseudonyms and $T_{\text{prl}} = 30$, to align with the evaluation described in Sect. 5.7.1. Each honest lifetime scenario is paired with a growing share of attackers in the network. Attackers are modeled to have a 75% chance of causing a pseudonym revocation every 30 minutes on average, which simulates a strain on the PRL that is by magnitudes larger than any honest pseudonym would normally incur.

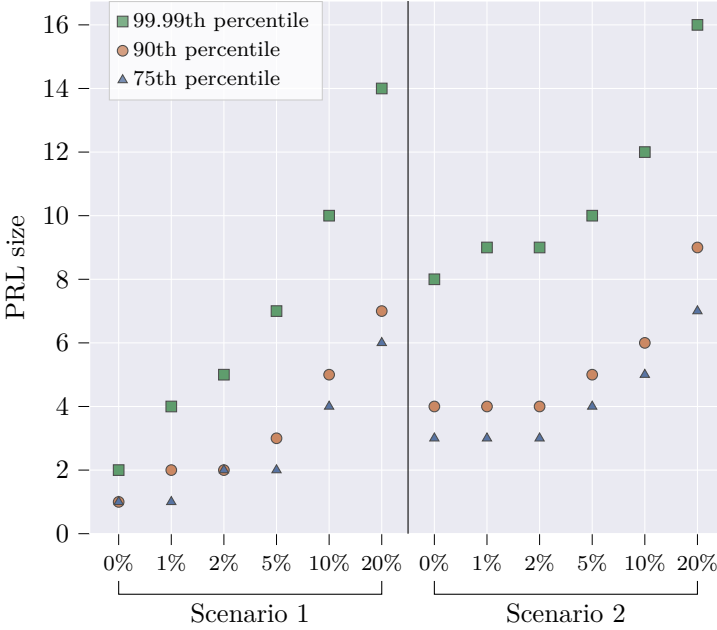


Figure 5.6: Markov model percentiles for maximum PRL sizes for $T_{prl} = 30s$, $n = 800$, and different shares of attackers in each baseline revocation scenario.

In most situations, the PRL contains fewer than 10 and 12 pseudonyms for the two baseline scenarios, respectively. Even with a share of 20% of network participants that cause revocations heavily, the 99.99th percentile of PRL size still stays below 17. This means that even for coordinated peak revocation events, the PRL size will stay in very manageable margins. For comparison, we estimated the expected number of revocations over a day: they range from roughly 8 to 10500 for Scenario 1, and 3600 to 13500 for Scenario 2 – a worst case average of one revocation every 6-8s.

5.7.3 Choice of T_v

The choice of T_v is crucial in our scheme. We evaluated four reference values of 30, 150, 300 and 900 seconds and visualize the results in Fig. 5.7. Below, we discuss the impact of T_v across multiple dimensions.

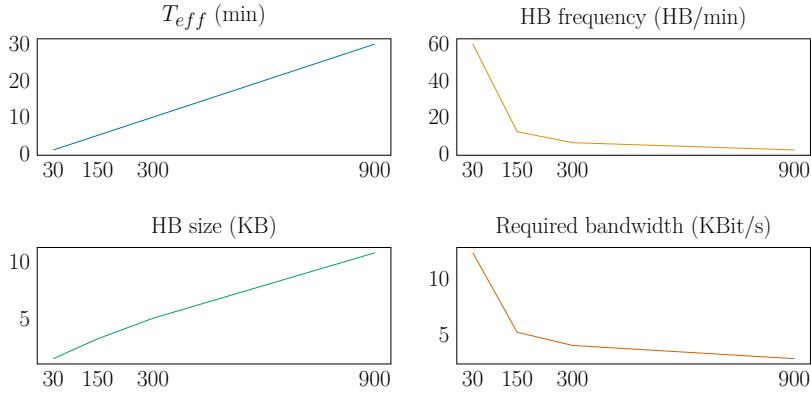


Figure 5.7: Evaluation of the impact of T_v on the V2X system. The X axes show the value of T_v in seconds.

Revocation time The first dimension is related to the revocation time, and is shown in the top-left plot of Fig. 5.7. According to Eq. (5.10), the effective revocation time T_{eff} is linearly dependent on T_v . Therefore, the smaller T_v , the shorter it takes to revoke, resulting in a safer system.

HB frequency The frequency with which HBs are generated and distributed by the RA is not fixed arbitrarily, but depends on T_v . Here, it is essential to ensure that honest vehicles remain loosely synchronized with the infrastructure during their operation. This means that, to avoid temporary service interruption, TCs need to process *at least* one HB every T_v (cf. Sect. 5.5). Thus, the frequency of HBs is inversely proportional to the value of T_v with a certain coefficient N , which accounts for network latencies and missing packets. The top-right plot of Fig. 5.7 shows the frequency with coefficient $N = 30$, i.e., a new HB is generated every $T_v/30$ seconds. With larger T_v , the number of HBs per minute drastically decreases, resulting in fewer resources used.

HB size According to Eq. (5.2), the size of a HB is linearly dependent on the size of the PRL, plus a constant value that represents the sizes of timestamp t_{hb} and digital signature $sig_{RA}()$. Additionally, as discussed in Sect. 5.7.2, the size of the PRL is directly impacted by the value of T_v . That is, the higher T_v is chosen, the longer each pseudonym needs to stay in the PRL, according to Eq. (5.11). This means that, on average, HBs are bigger when T_v increases. The bottom-left plot of Fig. 5.7 shows the HB size when varying T_v , considering the 99.99 percentile of the PRL size in relation to T_v on the worst case scenario,

i.e., Scenario 2 with 800 pseudonyms and 20% of attackers in the network (cf. Fig. 5.6). We also assume that a pseudonym identifier is 64 bytes, t_{hb} is 8 bytes, and we overestimate the size of $sig_{RA}()$ to 512 bytes, which is higher than most RSA and elliptic curve signatures [159].

Network bandwidth Lastly, it is important to evaluate the network bandwidth required for transmitting HBs. This can be computed by multiplying the size of a HB by the frequency with which HBs are distributed. Results show that the minimum bandwidth needed to transmit HBs is rather small, i.e., around 12 KBit/s in the worst-case scenario and with large N .

Conclusions The choice of T_v should be a good compromise between security and usability. The smaller T_v is, the shorter a revocation takes, but it also increases the resources used and the risk of de-synchronization in honest vehicles. Besides, a small T_v is feasible when the V2X network is reliable and low-latency, e.g., in urban areas, but may not be always practical, e.g., in rural areas. On the other hand, a large T_v results in a more resilient and efficient V2X system at the cost of a slower revocation. However, compared with passive revocation (cf. Sect. 5.8.1), even T_{eff} values of 30 minutes are a significant improvement on the revocation time, coming at negligible cost, as shown in Fig. 5.7.

5.8 Discussion

Our design advances the state of the art by providing several advantages compared to related work (Sect. 5.3), which we analyze in Sect. 5.8.1. We discuss compatibility with V2X protocols such as ETSI, SCMS and DAA in Sect. 5.8.2, and we explain a few corner cases of our design in Sect. 5.8.3. Finally, we examine privacy implications in Sect. 5.8.4.

5.8.1 Qualitative Comparison with Related Work

This section performs a qualitative analysis of network overhead, verification overhead and revocation time of our approach compared with related work.

Network overhead As discussed in Sect. 5.7.3, the bandwidth required to transmit HBs is as low as a few kilobits per second in most cases, even considering targeted attacks that cause a high number of revocations. This is a significant

improvement over active revocation schemes that leverage CRLs, since these lists are ever growing and thus their size increases over time. We even outperform schemes that combine active and passive revocation to reduce the CRL size: For example, [193] suggests periods in the CRL of 3-4 weeks for each pseudonym. Compared to our evaluation (cf. Sect. 5.7.3), this is still three to four orders of magnitude higher.

Furthermore, unlike passive revocation schemes that require frequent pseudonym changes [98], our design consumes minimal resources at runtime. Since HBs can be broadcast within an edge area, the number of messages transmitted by the infrastructure for revocation only depends on the number of edge areas and not on the number of vehicles. This makes the system extremely scalable, as the RA only needs to distribute relatively small-sized HBs at low frequency (cf. Sect. 5.7.3).

Verification overhead Active revocation schemes incur some latency during the verification of V2V messages to check the revocation status of pseudonyms. Even though some improvements exist to either reduce the CRL size or optimize the verification time, such schemes still cause a non-negligible overhead (Sect. 5.3). Our design, instead, does not require any additional verification overhead except for processing HBs, which are received more seldomly than V2V messages (cf. Sect. 5.7.3). Thus, the overhead for the TC should be comparable to passive revocation schemes. In Sect. 5.5.2 we propose to use the PRL to perform active revocation, which is considered an *optional* feature that can be enabled only when the PRL is small enough, which we demonstrated to be true in most cases (Sect. 5.7.2).

Revocation time Compared to active revocation, our design provides comparable results in the average revocation time, especially when leveraging the extensions discussed in Sect. 5.5.2 and evaluated in Sect. 5.7.1. However, active revocation does not have an upper bound on the effective revocation time, as some vehicles may receive CRL updates with unpredictable delay. Besides, our scheme outperforms passive revocation since a rotation of pseudonyms by the infrastructure every few minutes is infeasible [98], while schemes relying on pre-issued certificates with activation codes have a huge gap between revocation and actual removal of malicious actors from the network [211, 193].

5.8.2 Compatibility with Existing Protocols

ETSI ITS In line with our system model (cf. Sect. 5.4.2), vehicles following *ETSI TS 102 941* [83] obtain long-term *enrollment credentials* from an Enrollment Authority, after successful authentication of their canonical identity. These credentials are then used to obtain pseudonymous *authorization tickets* from an Authorization Authority. Timestamps are used for replay protection [82]. ETSI [83] further proposes passive revocation of pseudonyms by centrally revoking long-term credentials (cf. Sect. 5.2). Our scheme is applicable here and can largely improve on revocation time and network overhead. This, however, requires TCs in all vehicles.

SCMS Although similar to the ETSI architecture, the SCMS scheme employs active revocation (cf. Sect. 5.2). Our scheme can entirely replace CRLs in SCMS to reduce the network utilization and nullify the verification overhead that vehicles incur when processing V2V messages, which may be high due to the use of linkage values in the CRL [42]. Analogously to ETSI, the design of SCMS would need to include TCs.

DAA On the academic side, DAA-based protocols have been proposed as an approach for pseudonym generation with strong privacy properties (cf. Sect. 5.2). Vehicles use a JOIN protocol to enroll, authenticating themselves via attestation to receive a long-term credential. To obtain pseudonym certificates, some protocols require communication with an Authorization Authority [98]; others allow TCs to derive pseudonyms autonomously via an CREATE protocol [218, 67]. While many DAA protocols do not explicitly discuss replay protection, we argue that Eq. (5.1) is still a requirement of the V2X network, and thus provided. Under this assumption, even DAA protocols are compatible with our revocation scheme.

DAA protocols that already propose self-revocation can benefit from our design by having stronger guarantees on revocation time. However, protocols where TCs derive pseudonyms directly [218, 67] might not satisfy Req. 6 because the infrastructure cannot map pseudonyms to long-term identities, and therefore revoked vehicles might be able to re-enroll the network. While this has clear benefits in terms of privacy, it raises security concerns (cf. Sect. 5.8.4).

Applicability beyond V2X Our revocation scheme can be generalized to revoke *any* type of credential. Hence, it may be applied to other use cases, e.g., in scenarios where (1) real-time communication requirements hamper the use of

classic revocation checks, (2) users may generate pseudonymous credentials for enhanced privacy w.r.t. peers or the infrastructure, or (3) revocation needs to be fast and reliable to avoid serious damage by bad actors. V2X networks check all boxes but we see benefits for, e.g., smart cities, direct mobile-to-mobile communication, and other peer-to-peer applications.

5.8.3 Security Considerations

Vehicles powered off Vehicles that are temporarily powered off cannot process any HBs, meaning that potential revocations may be missed without the TC noticing, and without being able to execute automatic revocation. Thus, it is important to persist the value of the last t_{hb} processed by a TC, such that vehicles would remain synchronized when powered off for a short time. For longer offline periods, instead, vehicles would need to re-synchronize with the infrastructure and, simultaneously, check their revocation status.

Self-identification of pseudonyms The underlying assumption of our revocation scheme is that, to perform self-revocation, a TC is able to identify its own pseudonym certificates within the PRL. However, it may not be always possible to do so when the TC occasionally deletes some pseudonyms from memory, e.g., due to pseudonym rotation. A trivial solution to this problem would be to never delete any pseudonyms, though this may not be possible in resource-constrained environments (e.g., TPMs). Previous work [84, 219] discusses a more efficient solution that leverages cryptographic tokens, allowing TCs to safely delete their pseudonyms.

Denial-of-Service Attacks An attacker may cause vehicles to get de-synchronized via network attacks, preventing them from processing HBs. This could, e.g., be achieved by jamming all wireless network activity close to a victim. While not being practical at scale, jamming attacks would likely disconnect the victim from the network regardless of the credential management mechanisms employed, and may thus require re-enrollment or other connection management steps. Therefore, such attacks likely do not cause additional harm when our approach is used.

5.8.4 Privacy Discussion

Privacy requirements in V2X include that adversaries cannot link a pseudonymous signature to a canonical identity or other pseudonyms, or observe the use

of specific network resources [218, 83, 230]. Aside from adversarial interactions, data minimization towards the infrastructure is also a design goal of V2X systems to minimize the overall attack surface [42].

Our design does not compromise the privacy provisions of the underlying certificate management steps – enrollment, pseudonym generation, etc. – because HBs only contain public information and an adversary receiving PRLs only sees a list of pseudonyms (similarly to traditional CRLs) but learns no further information about specific vehicles, nor can they link pseudonyms to each other. Learning when specific pseudonyms are considered compromised by the infrastructure is unavoidable, and breaking the underlying cryptographic primitives is outside of our attacker model.

However, Req. 6 and technical standards in Europe [83] and the US [42] highlight that revoked vehicles must not be able to re-enroll. This compromises data minimization in the V2X infrastructure since it necessitates mapping a pseudonym back to the canonical identity or long-term credentials of the vehicle and then deny-listing that identity. There are DAA-based approaches that preserve unlinkability of pseudonyms towards the infrastructure, as TCs generate pseudonyms autonomously [218, 67]. However, this in turn raises security concerns since re-enrollment of revoked vehicles becomes feasible. Possible technical solutions to this conundrum depend on the V2X protocol and its credential management scheme, as well as the capabilities of the TC. Therefore they are out of scope of this work.

5.9 Summary & Conclusions

Revocation of credentials and pseudonymous identities is a critical aspect of security and trust management in V2X systems, necessary to prevent malicious actors from manipulating V2X communication for personal gain or to harm people and infrastructure. In this paper, we present and evaluate a self-revocation scheme that addresses scalability and security issues of existing approaches, and we formally prove that revocation in our scheme cannot be bypassed and completes with a configurable upper time bound. Furthermore, our approach is compatible with recent standardization efforts in the ETSI framework in Europe and the security credential management system in the US, and also with privacy-preserving pseudonym schemes that rely on Direct Anonymous Attestation. Compared to other solutions, our design requires a very limited communication bandwidth and we argue that the distributed nature of our scheme reduces computational resource utilization, specifically at the infrastructural layer. We show that our approach scales well to large

numbers of revocations and allows for revocation times in the order of seconds, outperforming even short-lived pseudonyms. We conclude that approaches that leverage trusted computing technologies, such as ours, come with improved scalability and performance characteristics that warrant inclusion in future standardization efforts in application scenarios that deal with large numbers of participants and critical low-latency communications, beyond V2X and ITS. Future work needs to investigate privacy-preserving solutions to block malicious actors from re-joining ITS networks after revocation, without incurring availability issues for well-behaving actors when losing connectivity.

Chapter 6

Conclusion

Protecting data while in use has become one of the most active research topics, especially since the rise of cloud computing and the Internet of Things. Trusted Execution Environments (TEEs) are a promising solution to achieve this goal, providing not only confidentiality and integrity of code and data while in use, but also a mechanism to verify the authenticity of computations via remote attestation. Today, TEEs are becoming a mainstream technology and all major chip manufacturers offer their own solution. At the same time, cloud providers and emerging startups have started selling “confidential computing” services to their customers, while the open source community and standardization bodies are working to make this technology more accessible to users by providing tools and documentation. On the other hand, researchers have shown that TEEs are not bulletproof and many vulnerabilities have been discovered in recent years, some of which have been already fixed, others that still remain an open issue. Despite this, TEEs are getting more and more traction among industry players, each driven by their own business interests.

In the future, confidential computing might become the new “de-facto standard” of modern computing, with all workloads running by default in hardware enclaves isolated from each other. In this dissertation, we made a step in this direction by presenting solutions to make TEEs more accessible and easy to use. At the same time, however, we highlighted some operational challenges that make it more difficult to deploy secure TEE workloads, and we showed that integrating this technology in existing protocols and applications is not trivial and requires careful analysis and redesign. In this chapter, we summarize our contributions, discuss possible directions for future work, and conclude this dissertation with final remarks.

6.1 Summary of Contributions

The focus of our initial work was on process-based TEEs such as Intel Software Guard Extensions (SGX), ARM TrustZone and Sancus, which allow to protect small portions of code in order to minimize the Trusted Computing Base (TCB). In Chapter 2, we presented a framework that allows developers and system administrators to easily develop and deploy distributed applications on heterogeneous TEEs. Our framework leverages an event-driven programming model where each individual component, called module, can declare input and output endpoints used to exchange messages between different modules and implement some custom logic in response of an event. This allows developers to focus exclusively on the application logic, while the networking, cryptographic operations, TEE management and remote attestation are automatically managed by the framework itself, greatly reducing the development effort. Another important aspect of our framework is securing interaction with peripherals thanks to Sancus' secure I/O capability, which gives strong integrity and authenticity of device drivers and the data exchanged between a module and an I/O driver. Of course, leveraging TEEs comes with a performance overhead; Nevertheless, our evaluation showed that our framework does not affect user experience in near real-time applications such as smart home systems.

In recent years, the confidential computing market has been shifting from process-based to VM-based TEEs such as AMD Secure Encrypted Virtualization (SEV)-Secure Nested Paging (SNP) and Intel Trust Domain Extensions (TDX). This makes development simpler, because applications can run inside such TEEs unmodified, but at the same time it greatly increases the TCB and makes management and attestation more challenging. Our contribution presented in Chapter 3 discusses these challenges, with a focus on the boot process of a Confidential Virtual Machine (CVM) and remote attestation. We showed that booting up a CVM involves several software layers that may be managed by different entities, and we proposed a classification of incremental *attestation levels* representing the extent of an attestation in proportion to the whole boot process. This classification helped us reason about the several steps that need to be taken to perform remote attestation correctly (and the tools that can be used to achieve it), as well as the security implications of a *partial* attestation (i.e., where not all the software components are measured and attested). Finally, we assessed confidential computing services in public clouds such as Amazon Web Services (AWS), Microsoft Azure and Google Cloud Platform (GCP), highlighting a crucial inconsistency between the threat model of TEEs, where the infrastructure provider is considered untrusted, and current cloud offerings,

where cloud providers still play an important role in the management and attestation of CVMs.

TEEs isolate code and data of an application from other software in the same system, thereby protecting their confidentiality and integrity. However, the application still relies on the untrusted environment (e.g., the host operating system and the hypervisor) to function properly. Apart from scheduling operations, applications might need to access system resources such as a clock. This not only affects availability, which is out of scope of the TEE threat model, but might also lead to confidentiality and integrity breaches depending on how these resources are actually used. In Chapter 4, we discussed possible issues related to the use of time inside a TEE. First, we provided a classification of incremental time levels that indicates, for each level, what guarantees it gives and which attacks are possible. Then, we used this classification to assess each individual TEE according to what support for trusted time they can provide, and subsequently we identified some common use cases where time plays a crucial role, such as rate limiting or credential checking. We highlighted that, for such applications, using an insecure time source may lead to vulnerabilities that can invalidate the security guarantees given by TEEs, and we recommended that developers should take this issue into account when developing TEE applications from scratch or moving existing workloads to TEEs.

Finally, Chapter 5 showcased how TEEs can be safely integrated into new protocols and use cases for strong confidentiality, integrity and authenticity guarantees, while at the same time providing additional benefits such as increased efficiency and scalability. We demonstrated this in the context of Intelligent Transport System (ITS) and Vehicle-to-Everything (V2X), where previous research had highlighted the need for a secure and efficient revocation mechanism for vehicle credentials. Learning from the lessons discussed in Chapter 4, we proposed a novel scheme for self-revocation that, unlike previous work, does not rely on a trusted time source in TEEs but leverages a synchronization protocol between V2X participants based on a logical clock (“epochs”). Compared to related work in the field, our revocation scheme is the only one that can provide a *guaranteed* upper bound on revocation time, thanks to our formal verification work. Furthermore, our evaluation showed that our solution can handle V2X networks with a large number of vehicles and/or revocations. Indeed, TEEs allow to partially decentralize credential management, moving some of the core logic to trusted services running inside vehicles. This allows novel protocols to be designed, which may provide several benefits compared to the current state of the art.

In summary, we hope that our research has given a deeper understanding on the benefits and limitations of TEEs. Working with this technology is not a trivial task and presents several technical and operational challenges.

Nevertheless, this solution can provide a robust defense-in-depth technique to isolate code and data at runtime, reducing the attack surface and the risk of compromise with relatively low overhead. Crucially, verifiability via remote attestation is paramount to the security of TEE applications to ensure that isolation mechanisms are correctly in place and the integrity of code and data has not been compromised. Unfortunately, however, this is not fully achievable with today’s cloud offerings, but we expect that future TEE solutions will make further steps towards real confidential computing.

6.2 Discussion and Future Work

TEEs are maturing and becoming a cornerstone of modern applications. In this dissertation, we illustrated how they can be securely and efficiently integrated into the design of applications and protocols, but we also highlighted some operational challenges that application developers and administrators should take into consideration. In this section, we build upon our contributions to reflect about possible directions for future work, focusing more on general aspects of integrating TEEs into applications rather than discussing specific improvements of our previous work.

Secure I/O on commercial TEEs Almost all applications require interaction with the external world to function properly, such as end-user devices, sensors and actuators for cyber-physical systems, and co-processors and accelerators for dedicated workloads like machine learning. Current TEEs protect applications running in the main CPU, while encryption in transit can protect data that goes in and out of the secure environment. When the computation moves to another device, however, additional isolation mechanisms are necessary.

In Chapter 2, we described how to perform secure I/O on Sancus microcontrollers to protect the communication between an enclave and an I/O device such as a sensor. We achieved this by leveraging memory-mapped I/O to map a device’s memory to protected enclave memory and configuring exclusive access of such memory to a corresponding *driver* enclave (cf. Sect. 2.4.1). While this solution was good enough for our small use case, it may not be suitable for more complex I/O devices whose drivers are proprietary and/or consist of a large number of lines of code. Furthermore, most commercial TEEs do not support secure I/O out of the box, making our approach unusable. A solution to this problem could be relying on proxy devices that sit between an enclave and a I/O peripheral to encrypt and authenticate all communication, as proposed in earlier work [170, 227, 76]. Other approaches to securely connect “legacy”

devices to a TEE would require hardware changes and a careful design [36]. However, these solutions do not protect the I/O devices themselves, which can still be malicious or contain vulnerabilities. For devices that can directly access protected TEE memory, e.g., via Direct Memory Access (DMA) operations, this poses a non-negligible risk.

A significant step towards fully secure I/O would be running a TEE in I/O devices and accelerators as well, which would make every computation secure (i.e., confidentiality- and integrity-protected) and attestable in every step. This is the approach of the Nvidia Hopper architecture [164], which allows running isolated GPU workloads that can be interconnected with VM-based TEEs such as Intel TDX and AMD SEV-SNP. Today, the Hopper GPU is available on Microsoft Azure in preview mode and is already being used for confidential computing applications such as confidential AI [72]. Another ongoing line of work is defining standards to securely attach a TEE-aware device to a secure virtual machine, such as the TEE Device Interface Security Protocol (TDISP) [167]. The TDISP standard is currently being implemented by Intel and AMD [107, 17] and there are academic solutions for ARM Confidential Compute Architecture (CCA) as well [194].

While the solutions mentioned above seem promising, they also make management and attestation of TEE workloads more complex. In principle, one could apply a similar approach as we proposed in Chapter 2, where each software module can run either in the main CPU (e.g., in enclaves or confidential virtual machines), or in external devices such as a GPU. Each module and the underlying hardware should then be attested and communication between modules properly protected. Future work needs to investigate mechanisms to streamline this process while keeping the attack surface as small as possible.

Increase transparency in TEE software The security guarantees of TEEs depend on several software components, such as the TEE firmware and microcode, libraries and Software Development Kits (SDKs), attestation software such as the Quoting Enclave (QE) in Intel SGX, and of course the code that runs inside the TEE. All of these components are part of the TCB, and a vulnerability in any part of the code might compromise the isolation guarantees provided by the TEE. Ideally, the TCB should be as small as possible and every software component should be verifiable to ensure that no such vulnerabilities exist. In reality, however, the code running in a TEE is often large enough to make formal verification techniques unsuitable or only usable on small parts of the codebase. The problem is exacerbated on VM-based TEEs, since their workloads consist of fully-fledged virtual machines composed of several software layers, even though there are some solutions to

dramatically reduce the size of CVM workloads, such as Gramine-TDX [121] and Mushroom [202]. It is essential that developers and administrators consider TCB reduction as a primary objective when designing a confidential application.

Furthermore, as revealed in Chapter 3, attesting VM-based TEEs is a complex process and, when using confidential computing solutions in the cloud, cloud customers have limited capabilities as some software components are closed-source and/or directly managed by the cloud providers, making verification harder or even impossible to achieve independently. Without proper attestation, it is hard to tell whether the customer’s data is really protected or not, and relying on the cloud provider for attestation defeats the purpose of using TEEs in the cloud in the first place. To make TEE workloads really *trustworthy*, therefore, it is necessary that a verifier is capable at least to independently verify the authenticity of all the code running inside the TEE. As discussed in Sect. 3.6, none of the major cloud providers offers a complete solution to achieve a transparent verifiability of the CVM boot process, although some improvements have been made in the past months (cf. preamble of Chapter 3). Future work must prioritize this issue, focusing on open-source code and reproducible builds to increase transparency and trustworthiness. Secure I/O solutions, as discussed above, should also be involved in this process.

Overcome limitations of TEEs In Chapter 4, we focused on the interface between an application running inside a TEE and the untrusted world, pointing out that some use cases require careful design choices, especially when dealing with (untrusted) system resources such as a clock. This problem becomes especially dangerous when porting existing applications to TEEs with a “lift-and-shift” approach, i.e., without any code changes. Since traditional applications have been designed with a different threat model in mind, where the host is assumed to be trusted (cf. Sect. 1.1), they might rely on resources that might be controlled by privileged attackers. For example, the Sigy attack exploits malicious signals to compromise unaware SGX applications [195]. Therefore, such issues should be acknowledged by developers when designing a TEE application, avoiding lift-and-shift when possible.

Furthermore, there are some inherent limitations of TEEs that should be taken into account. Other solutions for protecting data in use, such as Fully Homomorphic Encryption (FHE) [3] and Multi-Party Computation (MPC) [132], rely on advanced mathematics and statistics principles to transform the data before being used, such that the computations are performed on the transformed data rather than the original plaintext. With TEEs, instead, hardware-based isolation mechanisms and encryption prevent unauthorized access, but the data is still processed in the clear within the main processor. This puts significant

responsibilities and trust on TEE manufacturers, since vulnerabilities in a TEE might compromise the security of applications and leak sensitive data. Unfortunately, as highlighted in Sect. 1.2.3, this has already happened several times in the past and is likely to happen again in the future. For example, researchers at Google analyzed the implementation of AMD SEV-SNP and Intel TDX, finding several issues that resulted in more than 30 Common Vulnerabilities and Exposures (CVEs) [54]. Besides, since the data is processed in the clear, attackers might leverage techniques such as side-channel and physical attacks to bypass the isolation provided by a TEE and exfiltrate data. Although this kind of attacks is an orthogonal problem that is partially out of scope of the TEE threat model (even though certain TEEs offer defense mechanisms against some side channels), it must be taken into account when deploying a confidential application in the real world. To mitigate such attacks, as mentioned in Sect. 1.2.3, developers can rely on software-based techniques such as constant-time programming and oblivious computation: To this end, a good real-world example is the Signal contact discovery service [192], which uses an Oblivious RAM (ORAM) algorithm to hide access patterns and protect against side-channel attacks [196].

Research has shown that attacks on TEEs are a realistic threat and defenses can only reduce the risk but cannot completely remove it. As such, future work should investigate solutions to minimize the consequences of a successful attack. For example, our V2X self-revocation scheme proposed in Chapter 5 assumes that the Trusted Component (TC) securely stores all cryptographic material and, in case of revocation, makes it unusable for future communication. However, if a key leaks from the TC due, e.g., to a side-channel attack, there is currently no mechanism to revoke such key. Although we explicitly clarified that these attacks are out of scope, they would partially invalidate the properties that we formally verified (cf. Sect. 5.6). Another interesting case study is given by Van Schaik et al., who performed an attack on a blockchain network that leverages Intel SGX to provide confidentiality to smart contracts via a secret seed shared by all enclaves [210]. The authors were able to recover such seed by adding a rogue SGX enclave to the network, compromising the confidentiality of the entire blockchain. These examples demonstrate how important it is to consider TEE failures in the design phase of a real-world application, e.g., to develop “failsafe” mechanisms that take effect when such failures occur. In our V2X case, a simple solution could be adding to our design a fixed lifetime to all certificates, similar to the passive revocation scheme [83]. While most of these mechanisms may be application-specific, some concepts could be generalized to many use cases, such as credential management systems on the whole.

TEEs in industry standards In Chapter 5, and in particular in Sect. 5.8.2, we discussed how our self-revocation scheme can be integrated into existing V2X protocols, and we highlighted how industry standards like ETSI [83] cannot support our scheme out of the box but need some adaptations, i.e., to require and make use of a TC in vehicles. This showcases that, in order to increase adoption of TEEs and make them a mainstream solution, industry standards need to integrate this technology in their design. As a matter of fact, some standards do already exist, such as the IETF Remote ATtestation ProcedureS (RATS) [38] and Trusted Execution Environment Provisioning (TEEP) [168] architectures, as well as some reports like the ETSI NFV SEC 009 [78]. However, they discuss TEEs in a generic way without applying them to real use cases yet.

In order to facilitate the integration into standards, research needs to demonstrate how TEEs can fit into modern use cases and which benefits they provide compared to traditional solutions. Proposing a novel design is not enough, as it needs to be accompanied by an extensive evaluation to demonstrate its feasibility, as well as a fair comparison with the state of the art. Additionally, formal techniques can be used to give strong arguments about the security posture of the new design. In V2X credential management systems, for example, we noticed a general lack of these fundamentals, both in standards and research, although with some exceptions [219].

6.3 Concluding Remarks

Integrating TEEs into distributed applications requires careful consideration of the attacker model and the limitations of this technology. Simply deploying a workload inside an enclave or confidential virtual machine, in a “lift-and-shift” manner, might create a false sense of security where users expect that their workload is secure while, in reality, it might still be exposed to attacks. Looking at the TEE threat model, infrastructure providers are considered untrusted entities and, with this in mind, the term “confidential computing” has been adopted in the market to describe workloads running inside TEEs, with the promise that providers *cannot* look at or tamper with applications running on their own infrastructure.

However, our research shows that, in some cases, cloud providers could actually break this promise if they wanted to, since they play a significant role in the provisioning and attestation of TEE workloads. Even though there is a good possibility that this problem will be solved in the future, cloud providers can still rely on side channels, physical attacks, and potential vulnerabilities in the TEE implementation to achieve their malicious intent. These attacks can be partially

mitigated in software, but researchers have demonstrated that some of them are still a realistic threat. Thus, it is debatable whether cloud providers should really be considered untrusted in the TEE threat model, or if they should rather be regarded as “honest-but-curious” entities that in theory have the capabilities to compromise a TEE, but in practice will not perform active attacks. Yet, it is possible that these attacks might also be carried out by remote adversaries who somehow succeeded in taking control of a cloud server. Here, users should also trust cloud providers to adopt adequate security mechanisms to prevent such compromise from happening.

Even considering their limitations, TEEs significantly increase security by making it much harder for an adversary to compromise the confidentiality and integrity of applications. To break into a TEE instance, malicious cloud providers, co-resident tenants or remote attackers would need to perform sophisticated attacks that would require time, effort and expertise to succeed. Thus, this technology is an important building block towards making more robust systems, coming with relatively low cost and overhead. As TEEs are being used in production environments today, it is important to provide tools and support for using this technology correctly, while at the same time raising awareness on its challenges and limitations.

Appendix A

Additional Resources for End-to-End Security for Distributed Event-Driven Enclave Applications on Heterogeneous TEEs

A.1 List of Assumptions and Requirements

This work relies on a number of assumptions on hardware, software, and the environment. At the same time, we put specific requirements on the realization of concepts, entities, and mechanisms that we abstract from here, e.g., secure I/O, the infrastructure provider, and the deployment process. Below we list and justify these assumptions and requirements for easier reference.

A.1.1 Assumptions

Table A.1: Assumptions on Hardware

Ref.	Reproduction	Comment
2.1	<i>The execution infrastructure offers specific security primitives – standard enclaves plus support for secure I/O.</i>	These hardware technologies are the base for authentic execution.
2.3.1	<i>We assume the underlying architecture is a TEE.</i>	TEEs allow us to isolate source modules from other code running on a node and to minimize the TCB.
2.3.1	<i>The TEE provides an authenticated encryption primitive.</i>	This cryptographic scheme allows for integrity-protected and confidential communication between a module and other modules or the deployer.
2.3.4	<i>The infrastructure offers physical input and output channels using protected driver modules that translate application events into physical events and vice versa.</i>	Without a secure I/O mechanism, authentic execution is impossible. Here we assume hardware support for some form of protected driver modules.
2.4.3	<i>Our scheme assumes that each TrustZone device is equipped with a Hardware Unique Key (HUK).</i>	A trust anchor is needed for any remote attestation scheme. For our TrustZone implementation of remote attestation we assume a HUK.

Table A.2: Assumptions on Attacker

Ref.	Reproduction	Comment
2.2.3	<i>Attackers can manipulate all the software on the nodes, including OS, and can deploy their own applications on the infrastructure.</i>	We assume a strong attacker that can mess with all software except what is protected by the hardware, e.g., TEEs and driver modules.
2.2.3	<i>Attackers can also control the communication network that nodes use to communicate with each other, can sniff the network, modify traffic, or mount man-in-the-middle attacks. With respect to the cryptographic capabilities of the attacker, we follow the Dolev-Yao model</i>	This is a common assumption for a network-based attacker or an attacker that has complete control over at least one of the networking stacks of a connection.
2.2.3	<i>The attacker does not have physical access to the nodes.</i>	Otherwise this would allow, e.g., for physical side-channel and bit faulting attacks, which are out of scope here.
2.2.3	<i>Neither can they physically tamper with I/O devices.</i>	Doing so would allow an attacker to mess with physical inputs or the connection of devices to I/O ports, undermining the requirements on secure I/O and the authentic execution guarantees.
2.2.3	<i>We also do not consider side-channel attacks against our implementation.</i>	This could leak secrets such as a module key which would allow to bypass the authentic execution mechanism. Countermeasures against such attacks exist but are highly hardware-dependent.

Table A.3: Assumptions on Software

Ref.	Reproduction	Comment
2.3.1	<i>Both a module's code and data are located in contiguous memory areas called, respectively, its code section and its data section.</i>	This is mainly a simplification to support TEEs that do not allow complex memory management, e.g., Sancus.
2.3.1	<i>We assume a correct compiler.</i>	Without this assumption one could not deduce that the measured binaries of secure modules implement the expected source code functionality.

Table A.4: Assumptions on Infrastructure and Credentials

Ref.	Reproduction	Comment
2.2.4	<i>The deployer's computing infrastructure is assumed to be trusted.</i>	While this can be alleviated by using TEEs, eventually a trusted system is needed to verify attestations on behalf of the deployer.
2.7.2	<i>Connection keys are known only by the modules and their deployer.</i>	Leaking Connection keys might allow an attacker to forge messages and bypass the authentic execution mechanism.
2.3.4	<i>Driver modules are part of the trusted infrastructure</i>	As they take exclusive access of I/O devices to process physical inputs and outputs, they are controlled by the trusted infrastructure provider.
2.3.4	<i>Driver module keys are only known to the infrastructure provider.</i>	Otherwise, attackers could take control of driver modules on their own.

A.1.2 Implementation Requirements

Table A.5: Requirements on Secure I/O and Driver Modules

Ref.	Reproduction	Comment
IO1	<i>The infrastructure provider configures the physical I/O devices as expected, i.e., that the desired peripherals are connected to the right pins and thus mapped to the correct Memory-Mapped I/O (MMIO) addresses in the node.</i>	If this was not the case, the inputs registered by the drivers would not correspond to the actual physical inputs, and vice versa for physical outputs.
IO2	<i>The infrastructure must provide a way for the deployer of M_A to attest that it has exclusive access to the driver module M_D and that M_D also has exclusive access to its I/O device D.</i>	Without these attestation and exclusive access capabilities, it could not be guaranteed that inputs registered and outputs produced by the drivers are genuine.
IO3	<i>As soon as a microcontroller is turned on, driver modules take exclusive access to their I/O devices and never release it.</i>	This prevents an attacker from taking control of I/O devices before the infrastructure provider after a node reset.
IO4	<i>No outputs are produced by output driver modules unless requested by the application module.</i>	Otherwise outputs could not be attributed to corresponding inputs up the chain.
IO4	<i>Input driver modules do not generate outputs to application modules that do not correspond to physical inputs.</i>	Otherwise physical outputs may be generated that are not justified by any physical inputs. There is a caveat on automatic initialization messages and other meta-data outputs of the drivers.

Table A.6: Requirements on Infrastructure and Deployer

Ref.	Reproduction	Comment
D1	<i>The channel between deployer and infrastructure provider is assumed to be secure.</i>	Without a secure channel between these entities, attackers could launch man in the middle attacks between drivers and the deployer's secure modules.
D2	<i>Before granting such access, the infrastructure provider needs to ensure the authenticity of the driver module controlling the I/O device, e.g., via attestation</i>	As driver modules are part of the TCB, it has to be ensured that the correct, trusted driver module is running on the node where exclusive access to a device driver has been requested.
D3	<i>It is assumed that the provider does not leak the driver module keys and that exclusive access to a device is reserved for the deployer until they request to release it or an agreed-upon time T has passed.</i>	If the keys are leaked, an attacker may take control of said device driver at will. Releasing exclusive access prematurely is more of an availability problem as connection keys distributed to the driver module are never leaked, i.e., attackers could in that case take over a device driver but not insert themselves into a previous communication session with that driver.
D4	<i>The infrastructure must provide replay protection for messages exchanged with the driver module in the protocol to establish exclusive driver access.</i>	Otherwise an attacker may repeatedly obtain exclusive access to a driver module without involving the infrastructure provider.

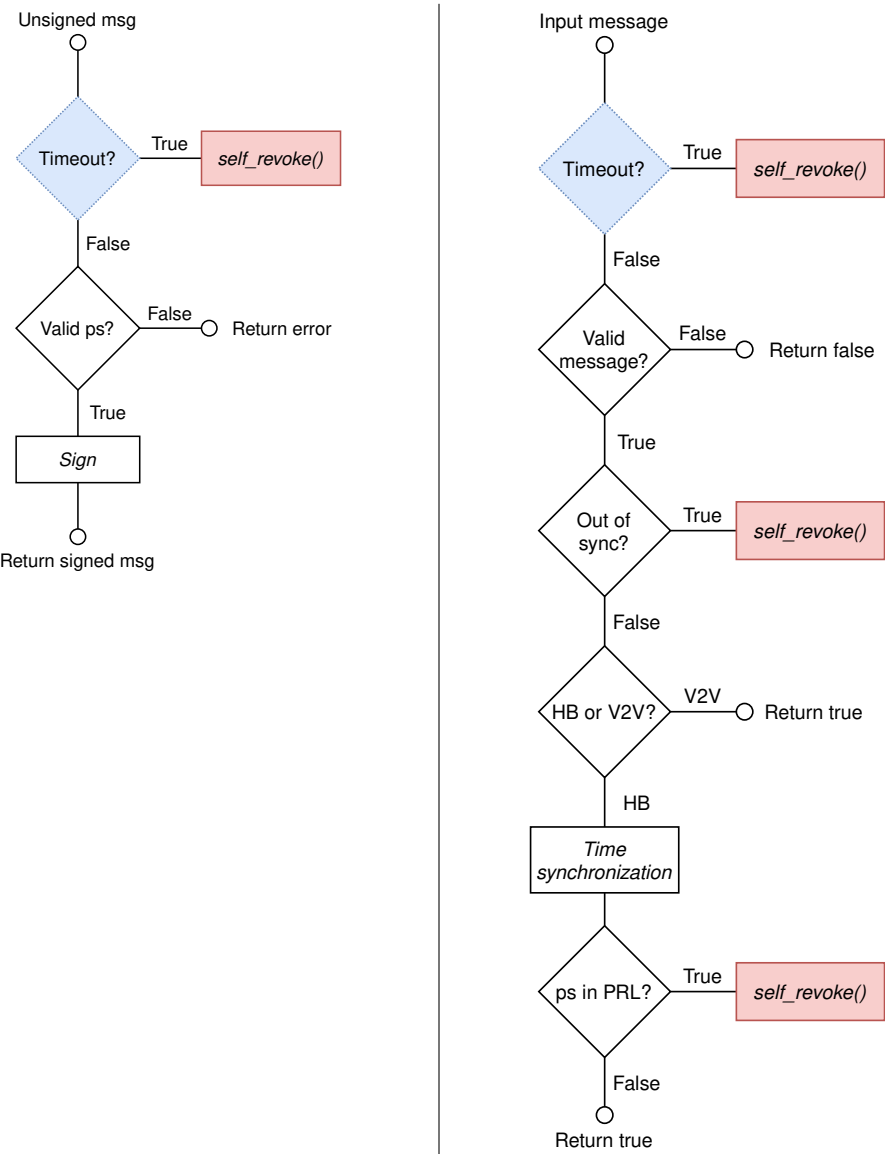
Appendix B

Additional Resources for Efficient and Timely Revocation of V2X Credentials

B.1 Flowcharts

Fig. B.1 depicts flowcharts for the Trusted Component (TC) behavior described in our design. In particular, Fig. B.1a illustrates the steps taken to sign a Vehicle-to-Vehicle (V2V) message, while Fig. B.1b shows the logic to verify a network message, which can be either a heartbeat (HB) or a V2V message.

According to Req. 4 and Eqs. (5.1) and (5.3), external messages should be verified with respect to authenticity (via digital signature verification) and freshness (according to the validity window T_v). Furthermore, the TC should automatically perform self-revocation if it detects de-synchronization (Eq. (5.6)). For every valid HB, the TC should synchronize its internal time *now* ((Eq. (5.4)), redundant if a local trusted time source is used), and then inspect the Pending Revocation List (PRL) to check whether self-revocation should be executed or not (Eq. (5.5)). For signing a message, the TC should possess valid pseudonyms in order to make a signature, which includes the input message and context metadata such as timestamps (Req. 3).



(a) Receiving a sign request.

(b) Receiving a message.

Figure B.1: Flowcharts of the TC logic. Blue (dotted) nodes are only relevant when a local trusted time source is available in TCs.

In Fig. B.1, the *timeout* condition in blue only applies to the extension that takes into account a local trusted time source in TC, and checks if the current time is bigger than the biggest timestamp received in a HB plus T_v (Eq. (5.12)).

B.2 Tamarin Models

This appendix extends Sect. 5.6, providing more details on how we translated our design to TAMARIN and how we captured the properties of our design. Here, we also discuss our variant where a trusted time source in the TC is available (cf. Sect. 5.5.2).

B.2.1 Mapping our Design to Tamarin

To map our design to Tamarin, we used only six main rules:

- **RA_generate_heartbeat:** This rule takes as input the current time and PRL and produces a new HB as output, signed with the Revocation Authority (RA)'s private key. The HB is also sent out to the network channel, and an action fact `HeartbeatGenerated(HB)` is produced. This rule can only be executed once per PRL and time step to reduce the number of states, since multiple execution with the same parameters would generate the same HB, which is not relevant in our model since TAMARIN already allows replays of network messages.
- **RA_issue_revocation:** Taking as input the current time, PRL, and a pseudonym's public key, this rule generates a new PRL by adding the pseudonym to the list and incrementing the sequence number by one. We added a restriction to revoke each pseudonym only once. After execution, the action fact `RevocationIssued(ps, t)` is generated.
- **TC_get_pseudonym:** This rule allows the TC to obtain a new pseudonym credential. How exactly this credential is obtained (e.g., via a communication to the infrastructure) is outside of our scope; What is important in our model is that the TC can obtain and use an arbitrary number of pseudonyms. The only restriction of this rule is to ensure that only non-revoked TCs can obtain new pseudonyms (cf. Req. 2).
- **TC_process_heartbeat:** The TC processes a HB taken as input from the channel. Restrictions ensure that the received HB has a valid signature and timestamp. In addition, the TC has to check whether to perform self-revocation or not. To do so, we divided this rule in two mutually-exclusive

```

// property (i)
lemma effective_revocation [heuristic=o "oracle.py"]:
  " All msg ps t #i . MessageAccepted(msg, ps, t)@i ==>
    Ex tv #j . SystemInitialized(tv)@j & j<i
      & not (
        Ex ps2 t_rev #k . RevocationIssued(ps2, t_rev)@k
          & k<i
            & GreaterThan(t, t_rev + tv + tv)
        )"

// property (ii)
lemma no_operations_after_timeout [heuristic=o "oracle.py"]:
  "All t #i . NewOperation(t)@i ==>
    Ex tv #j . SystemInitialized(tv)@j & j<i
      & not (
        Ex ps t_rev #k . RevocationIssued(ps, t_rev)@k
          & k<i
            & GreaterThan(t, t_rev + tv)
        )"

```

Figure B.2: Properties of our design with a trusted time source in TC translated to TAMARIN lemmas.

rules that produce different results: if any of the TC pseudonyms is in the PRL, the action fact `Revoked(t)` is produced, otherwise it is not. Either way, a `HeartbeatProcessed(HB,t)` action fact is produced, along with a new `!Time(t_hb)` persistent fact to synchronize the TC time.

- **TC_sign_message:** The TC generates a new V2V message signed with a pseudonym's private key. It takes the latest time as input, while restrictions ensure that the TC has not been previously revoked (i.e., the `Revoked(t)` action fact does not exist). The rule produces a signed timestamped message and sends it to the out channel. A `Signed(msg,ps)` action fact is produced as well.
- **process_message:** This rule models a generic third entity that receives a V2V message from network and verifies it. If the signature is valid and the message is fresh, a `MessageAccepted(msg,ps,t)` action fact is generated.

B.2.2 Trusted Time Source in TCs

The design that models a trusted time source in TC comes with a major difference, i.e., that TC and RA use the same `!Time` facts. That is, this means that their clock is perfectly synchronized, and the TC does not need to process HBs to advance its internal time.

Table B.1 provides the full list of lemmas used in this variant, most of which are analogous to the main model. The main difference is that we added three *reusable lemmas*, i.e., lemmas that TAMARIN can leverage to prove other lemmas.

Lemma	Type	Oracle	Steps	Description
<code>sign_possible</code>	S		6	Ensures that the TC can sign V2V messages
<code>generate_hb_possible</code>	S		5	Ensures that the RA can generate and distribute HBs
<code>issue_revocation_possible</code>	S		5	Ensures that the RA can revoke pseudonyms, adding them to the PRL
<code>processing_hb_possible</code>	S		9	Ensures that the TC can receive and process a HB
<code>revocation_possible</code>	S		13	Ensures that the TC can self-revoke when receiving a HB
<code>process_message_possible</code>	S		7	Ensures that a third entity can process V2V messages signed by the TC
<code>exists_par_tv_2</code>	S		5	Used to verify that T_v is arbitrary
<code>exists_par_tv_4</code>	S		7	Used to verify that T_v is arbitrary
<code>no_signing_after_timeout</code>	F		205	Ensures that the TC cannot make signatures after performing automatic revocation
<code>no_signing_after_revocation</code>	F		2	Ensures that the TC cannot make signatures after performing self-revocation
<code>all_heartbeats_processed_within_tolerance</code>	F	✓	260	Ensures that outdated HB are correctly discarded by the TC
<code>all_messages_accepted_signed_exists</code>	R	✓	62	Ensures that, for each V2V message verified by a third entity, the same message was previously signed by the TC
<code>all_messages_accepted_within_tolerance</code>	R	✓	282	Ensures that outdated V2V messages are correctly discarded by the third entity
<code>no_messages_accepted_after_revocation</code>	R	✓	74	Ensures that, if revocation is mandated at time t_{rev} , all TC messages accepted by a third entity cannot contain timestamp greater than $t_{rev} + T_v$
<code>effective_revocation</code>	P	✓	4	Verifies property (i) in Sect. 5.5.1
<code>no_operations_after_timeout</code>	P	✓	76	Verifies property (ii) in Sect. 5.5.1

Table B.1: Full list of lemmas proven with TAMARIN on our model with trusted time. *Type* is either *S* (sanity), *F* (functional), *R* (reuse) or *P* (proof). *Oracle* indicates whether an oracle was needed, while *Steps* the number of steps required by TAMARIN to prove the lemma.

Scenario	Time?	T_v (s)	T_{eff} (s)	T_{prl} (s)
A1	no	30	60	30
A2	no	150	300	150
B1	yes	30	60	30
B2	yes	150	300	150

Table B.2: Scenarios evaluated in our simulation. Parameters are chosen according to the desired T_{eff} . The column named *Time?* indicates whether a local trusted time source is used in TCs or not.

This was helpful since we experienced that the two proof lemmas were harder to verify efficiently than in our main model. By defining additional lemmas marked as **reuse**, we could instead build efficient proofs, allowing TAMARIN to terminate in only a few minutes.

Fig. B.2 shows the proof properties of this model. Two are the main differences compared to the main model: First, the **effective_revocation** lemma can now directly prove our first property to calculate the exact value for T_{eff} . Indeed, we now assume that all TCs are perfectly synchronized with the RA time, hence the revocation time is now absolute and comes at $t_{\text{rev}} + 2T_v$. Second, thanks to internal timeouts in TC, we can have a stronger proof for the second property, saying that not only does the TC not process HBs generated after $t_{\text{rev}} + T_v$, but also that the TC cannot perform *any* operations after that time because the automatic revocation timeout would be triggered. To prove this, we defined an additional rule called **TC_do_operation**, which models a generic operation performed by the TC via the **NewOperation(t)** action fact. This is then used in **no_operations_after_timeout** to prove that there cannot be **NewOperation(t)** action facts with a value for t bigger than $t_{\text{rev}} + T_v$.

B.3 Complete Simulation Results

We ran simulations for each of the scenarios described in Tab. B.2. Results are depicted in Figs. B.3 to B.6.

Recall from Sect. 5.7.1 that each value of the boxes represents the time between the revocation of ps (REVOKE event) and the last V2V message signed with ps that was verified by a non-malicious TC (VERIFY event). The latter time represents the *effective revocation time* for that particular pseudonym ps . The

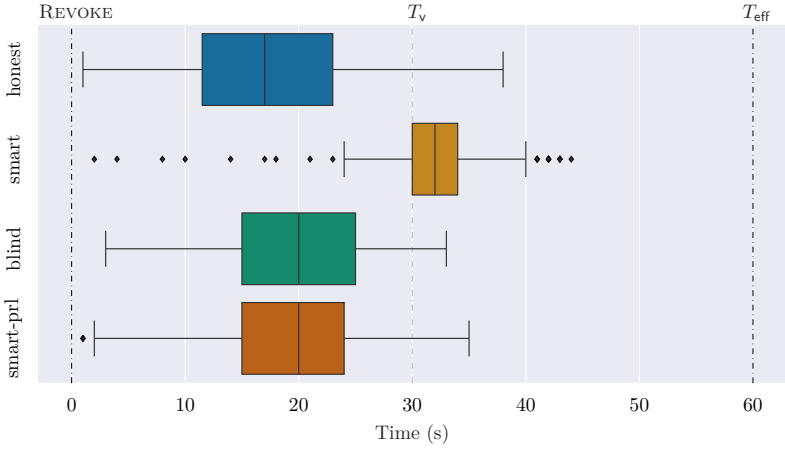


Figure B.3: Simulation results for scenario A1.

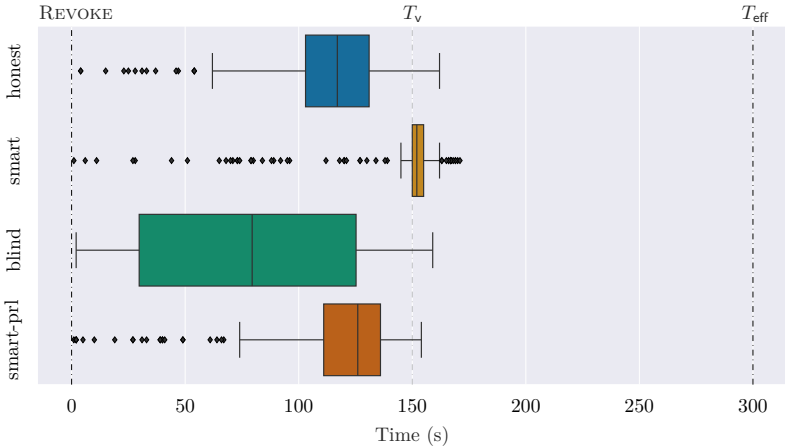


Figure B.4: Simulation results for scenario A2.

box plots give the distribution of such values, obtained aggregating more than 600 revocations, filtering out negative values.

The figures show one significant difference between the main design (A1 and A2) and the extension with a local trusted time source (B1 and B2): While in the former most revocations are effective around or before T_v , in the latter attackers are able to postpone revocation up until T_{eff} in some cases, i.e., it appears easier to reach the upper bound, especially for powerful attackers such as the *smart*

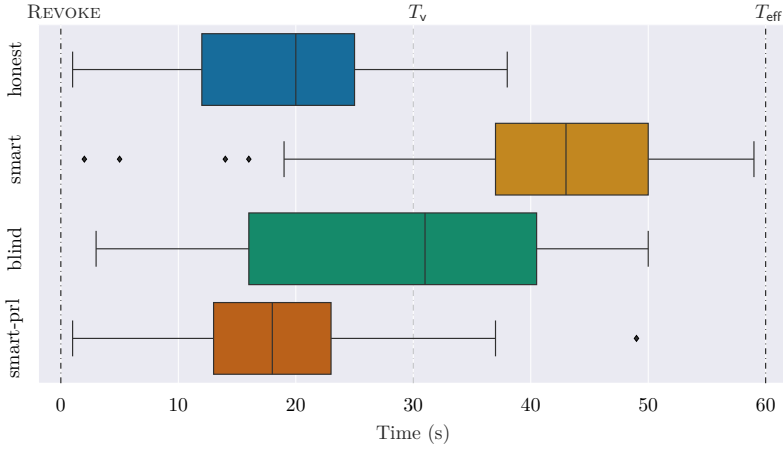


Figure B.5: Simulation results for scenario B1.

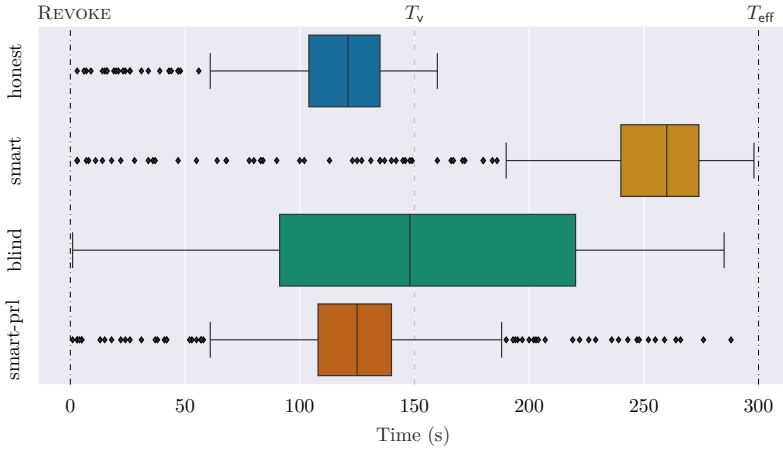


Figure B.6: Simulation results for scenario B2.

one. This is due to the fact that, while in the main design revoked TCs cannot synchronize their time since t_{rev} , with a local time source TCs can still advance their internal time up until $t_{rev} + T_v$, before triggering the automatic revocation logic. That is, in the latter case a revoked TC is able to generate “more fresh” V2V messages, which can be processed by other TCs later in time.

This peculiarity suggests that a local trusted time source negatively affects the revocation time. While this may be true in the average case, it still does not affect the worst-case effective revocation time T_{eff} , as shown in the figures.

B.4 Calculating Expected Size of the PRL

This appendix covers the basics of the utilized probabilities to then explain the Markov matrix. We also describe the calculation of the stationary distribution as a means to calculate the expected PRL size. Finally, we discuss the computation of the probabilities and expected values used in Sect. 5.7.2.

B.4.1 Utilized Probabilities

Recall from Sect. 5.7.2 that we model the processes of gaining and losing pseudonyms from the PRL as two individual probabilities, then we combine them in the next step into a Markov chain. The two probabilities shown in Eq. (5.13) and Eq. (5.14) are denoted as G_{il} and L_{il} respectively and can be seen as probability matrices that at location il have the probability for being in state i and gaining or losing l pseudonyms from the list. The core assumption for these two probabilities is that both adding and removing pseudonyms from the revocation list can be seen as a binomial distribution that only depends on: (1) the current size of the list, i.e., how many certificates there are left to be revoked or removed from the list, (2) the probability for each pseudonym to be revoked at any given time, and (3) the time that a pseudonym stays in the list. As such, this model assumes that pseudonyms are renewed as soon as they are evicted from the PRL, i.e., the list can never hold more than the total number of n pseudonyms, which is a realistic assumption for large n as it is unlikely that so many pseudonyms in the system would be revoked in a short time during regular operation. Furthermore, in this way we model the process of losing pseudonyms from the list as a probabilistic process that has the expected value at T_{prl} . We, however, can not model a precise eviction of the pseudonym from the PRL after T_{prl} time steps. In our view this is an acceptable tradeoff as on average and over the lifetime of the PRL, the modeled behavior comes very close to the actual behavior of evicting pseudonyms from the PRL after exactly T_{prl} time steps.

B.4.2 Markov Model

We can now combine these two probabilities into a matrix that at location p_{ij} has the probability of moving from state i to state j . Starting with the simple example of $n = 3$ pseudonyms, multiple entries of this matrix are straightforward:

- Starting at any state i and stepping into state $j = 0$ requires to not revoke any new pseudonyms and to lose all existing pseudonyms from the list. This is the combination of the two probabilities $L_{ii}G_{i0}$. For example, going from state $i = 2$ to $j = 0$ requires to lose 2 pseudonyms when there are 2 in the PRL, and requires to gain no new pseudonyms when there are already 2, leading to the two probabilities $L_{22}G_{20}$.
- Starting at state $i = 0$ and stepping into a state j implies that no pseudonyms can be lost from the PRL and j pseudonyms have to be added.
- Similarly, starting in state $i = 3$ and stepping into any state j implies that no pseudonyms can be gained, but $j - i$ pseudonyms are removed from the PRL.

The most interesting state transitions in this probability matrix are the transitions that consist of multiple combinations of events. One example is the transition from $i = 2$ to $j = 1$. Here, we first have the obvious possibility that we reach the state $j = 1$, i.e., one pseudonym in the PRL, by losing one pseudonym and gaining none. However, we also have the possibility to lose both pseudonyms in the PRL and gain one, leading to still one remaining pseudonym in the PRL. This is because the pseudonyms in the PRL and outside of the PRL are independent and pseudonyms can still be revoked while others are removed from the list. Thus, the state transition consists of the following parts: $p_{21} = L_{21}G_{20} + L_{22}G_{21}$.

The full state probability matrix for $n = 3$ pseudonyms can be seen below. Note that a matrix for n pseudonyms is a $n + 1 \times n + 1$ matrix to accommodate the state of the empty PRL.

$$P = \begin{bmatrix} L_{00}G_{00} & L_{00}G_{01} & L_{00}G_{02} & L_{00}G_{03} \\ L_{11}G_{10} & L_{10}G_{10} + L_{11}G_{11} & L_{10}G_{11} + L_{11}G_{12} & L_{10}G_{12} \\ L_{22}G_{20} & L_{21}G_{20} + L_{22}G_{21} & L_{20}G_{20} + L_{21}G_{21} & L_{20}G_{21} \\ L_{33}G_{30} & L_{32}G_{30} & L_{31}G_{30} & L_{30}G_{30} \end{bmatrix}$$

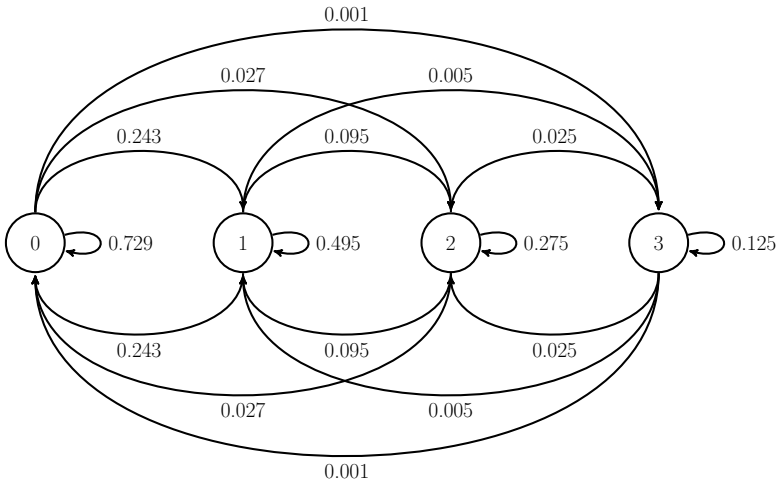


Figure B.7: A graph of the Markov chain with the exemplary parameters of $n = 3$, $p = 0.1$, and $T_{prl} = 2$.

Such probability matrices are also called discrete time Markov chains [97]. Fig. B.7 depicts the transition graph for the above Markov chain when we assume the parameters of $p = 0.1$ and $T_{prl} = 2$. Based on this first small example, we can approach a closed formula for an arbitrary number of pseudonyms (n). This closed formula is shown in Eq. (5.15). The core idea is that each entry depends on a combination of gaining and losing pseudonyms based on the maximum of $i - j$ and 0 and ranges up to i . Then, any state combination that is not possible will be 0 through the binomial coefficient. However, this range is necessary to catch the complicated combinations in the center of the matrix. Each combination is then based on losing l pseudonyms from the list and gaining $l - i + j$ pseudonyms, which basically iterates through the combinations with l as a stepping variable.

B.4.3 Calculating the Expected PRL Size

In discrete time Markov chains, a probability state vector π can be multiplied with the Markov matrix P to gain the probabilities for π after a single transition in the model [97] as follows: $\pi' = \pi \cdot P$. This, however, is only sufficient to gain the state probabilities after specific amounts of steps. In contrast to this approach, Markov chains may approach a so-called state equilibrium which is a stationary probability distribution that does not change even after taking

a step in the Markov chain. The stationary distribution follows the equation $\pi = \pi \cdot P$ [97].

To gain this stationary distribution, we solve the following equation system:

$$\begin{aligned}\pi &= \pi P \\ \Leftrightarrow \pi - \pi P &= 0 \\ \Leftrightarrow \pi(I - P) &= 0 \\ \Leftrightarrow (I - P)^T \pi^T &= 0\end{aligned}$$

Lastly, since there may be several solutions to this linear equation, we add the specific constraint to this equation system that the sum of π is equal to 1. This is the case for each probability vector in Markov models [97], and follows the intuition that, from any state probability, the probability to reach any other state is 1. To add this constraint, we add a row of 1 to $(I - P)^T$ and append a 1 to the zero vector. Solving this linear equation yields a stationary distribution that is stable. Fig. 5.6 summarizes this stationary distribution by showing the list sizes that occur to 75%, 90%, and to 99.99%. The graph shows these percentiles under different scenarios, i.e., with different shares of attackers in the network.

In the following, we expand this evaluation with Figures B.8 and B.9. Fig. B.8 depicts only the 99th percentile for a fixed T_{prl} and only focuses on the four extreme cases: For each of the two baseline scenarios, 0% and 20% of attackers in the network. The horizontal axis then depicts a growing number of pseudonyms (and, therefore, vehicles) in the network. This is to show that a larger number of vehicles does not exponentially grow the expected size of the PRL. Instead, the 99th percentile of the list size grows linearly with n . Similarly, Fig. B.9 shows the same situation for a fixed number of pseudonyms at $n = 800$ but a varying window for T_{prl} . This second graph shows that the PRL size also grows linearly with a linear increase in the time that a pseudonym stays in the PRL.

B.4.4 Probabilities and Expected Revocations

Above, p is the probability of revocation in each time step. As time steps are at the granularity of seconds, and our system runs for a long time, it turns out that the per-step probabilities leading to realistic revocation rates are very small. Instead, the probabilities used in the PRL size evaluation were described in terms of a compound probability q that a revocation occurs at least once over

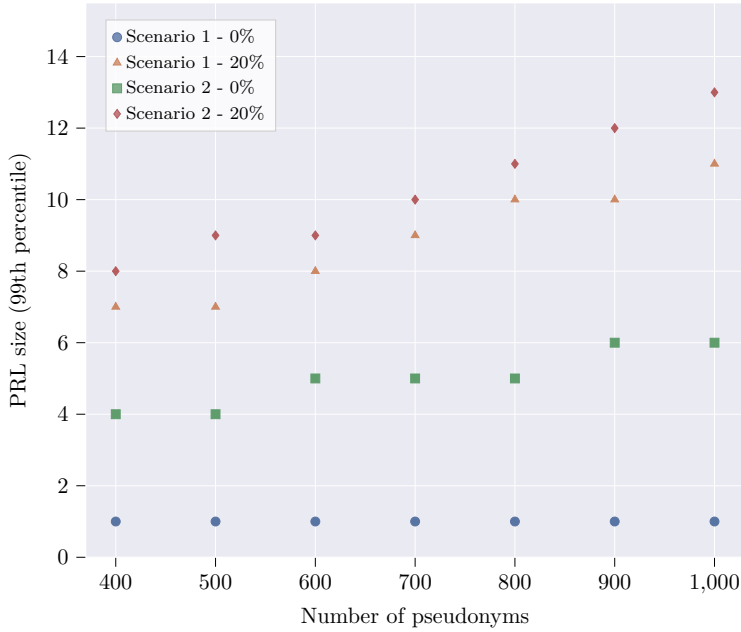


Figure B.8: 99th percentile PRL sizes for fixed $T_{prt} = 30$ and varying number of pseudonyms (n) under four scenarios.

a number of s steps. This probability can be computed via the geometric series

$$q(p, s) = \sum_{i=1}^s p \cdot (1-p)^{i-1} = 1 - (1-p)^s$$

which sums up the probabilities that the first revocation occurs the first time in step $i \in [1 : s]$. Note that this is equal to 1 minus the probability that in all s steps no revocation occurs. Solving for p we obtain

$$p(q, s) = 1 - \sqrt[s]{1-q}.$$

We used this formula to compute the baseline per-step probabilities underlying our scenarios for honest vehicle and attacker separately. The final probabilities p for the Markov models were then obtained by averaging the probabilities weighted according to the share of attackers.

For estimating the expected numbers of revocations in our scenarios we needed to calculate the expected value for n pseudonyms over s steps. If each step was independent this would simply be $n \cdot s \cdot p$, however, as explained above,

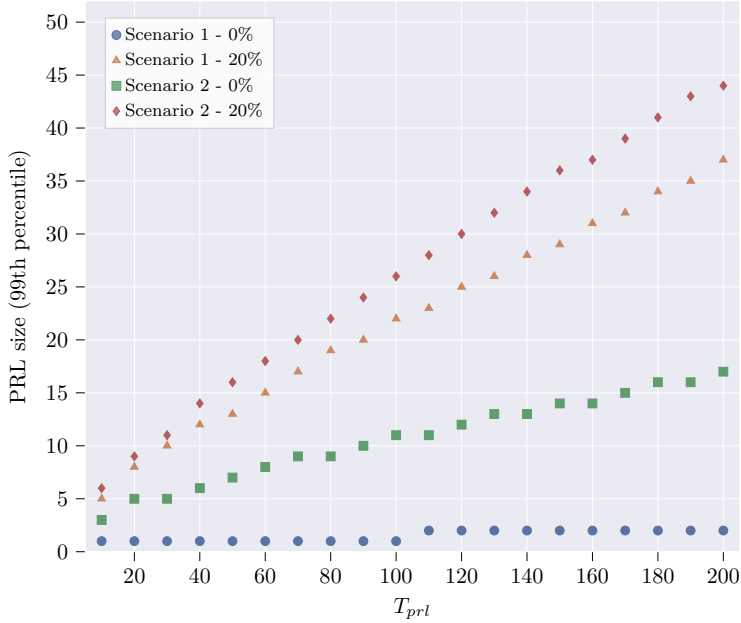


Figure B.9: 99th percentile PRL sizes for fixed $n = 800$ pseudonyms and varying T_{prl} under four scenarios.

in our Markov model at most n pseudonyms can be revoked at a time. We compensated for this fact by taking into account the chance p_{prl} that a given pseudonym is on the PRL at any given time and computed the desired estimate E_{rev} :

$$E_{rev} = n \cdot s \cdot p \cdot (1 - p_{prl})$$

We set p_{prl} to the median PRL size, as obtained by the Markov model for each scenario, divided by n .

Bibliography

- [1] ABADI, M. Protection in Programming-Language Translations. In *Secure Internet Prog., Security Issues for Mobile and Distributed Objects*, J. Vitek and C. D. Jensen, Eds. Springer, 1999, ch. 2, pp. 19–34.
- [2] ABATE, C., BLANCO, R., GARG, D., HRITCU, C., PATRIGNANI, M., AND THIBAULT, J. Journey beyond full abstraction: Exploring robust property preservation for secure compilation. In *2019 IEEE 32nd Computer Security Foundations Symposium (CSF)* (2019), IEEE, pp. 256–25615.
- [3] ACAR, A., AKSU, H., ULUAGAC, A. S., AND CONTI, M. A Survey on Homomorphic Encryption Schemes: Theory and Implementation. *ACM Comput. Surv.* 51, 4 (July 2018).
- [4] AHMED, K. J., AND LEE, M. J. Secure LTE-Based V2X Service. *IEEE Internet of Things Journal* 5, 5 (2018), 3724–3732.
- [5] AKRAM, A., GIANNAKOU, A., AKELLA, V., LOWE-POWER, J., AND PEISERT, S. Performance Analysis of Scientific Computing Workloads on General Purpose TEEs. In *2021 IEEE International Parallel and Distributed Processing Symposium (IPDPS)* (2021), pp. 1066–1076.
- [6] ALAM, A. K. M. M., AND CHEN, K. Making Your Program Oblivious: A Comparative Study for Side-channel-Safe Confidential Computing. In *2023 IEEE 16th International Conference on Cloud Computing (CLOUD)* (2023), pp. 282–289.
- [7] ALDER, F., ASOKAN, N., KURNIKOV, A., PAVERD, A., AND STEINER, M. S-faas: Trustworthy and accountable function-as-a-service using intel SGX. In *Cloud Computing Security Workshop* (2019).
- [8] ALDER, F., DANIEL, L., OSWALD, D., PIESSENS, F., AND BULCK, J. V. Pandora: Principled Symbolic Validation of Intel SGX Enclave Runtimes.

- In *2024 IEEE Symposium on Security and Privacy (SP)* (May 2024), IEEE Computer Society, pp. 93–93.
- [9] ALDER, F., KURNIKOV, A., PAVERD, A., AND ASOKAN, N. Migrating SGX enclaves with persistent state. In *2018 48th Annual IEEE/IFIP International Conference on Dependable Systems and Networks (DSN)* (2018), IEEE, pp. 195–206.
 - [10] ALDER, F., SCOPELLITI, G., VAN BULCK, J., AND MÜHLBERG, J. T. About Time: On the Challenges of Temporal Guarantees in Untrusted Environments. In *Proceedings of the 6th Workshop on System Software for Trusted Execution* (2023), SysTEX '23, Association for Computing Machinery, p. 27–33.
 - [11] ALDER, F., VAN BULCK, J., PIESSENS, F., AND MÜHLBERG, J. T. Aion: Enabling Open Systems through Strong Availability Guarantees for Enclaves. In *CCS '21* (Seoul, South Korea, 2021), ACM.
 - [12] ALVES, T., AND FELTON, D. TrustZone: Integrated hardware and software security. *Information Quarterly* 3, 4 (2004), 18–24.
 - [13] AMBIEL, S. The Case for Confidential Computing. Technical Report, The Linux Foundation, July 2024.
 - [14] AMD. AMD Secure Encrypted Virtualization. <https://github.com/AMDESE/AMDSEV/tree/snp-latest>. Accessed on June 3, 2024.
 - [15] AMD. AMD SEV-SNP: Strengthening VM Isolation with Integrity Protection and More. White Paper, Jan. 2020.
 - [16] AMD. AMD Memory Encryption. White Paper, Oct. 2021.
 - [17] AMD. AMD SEV-TIO: Trusted I/O for Secure Encrypted Virtualization. White Paper, Mar. 2023.
 - [18] AMD. AMD64 Architecture Programmer’s Manual Volume 2: System Programming. Specification, Jan. 2023. Revision 3.40.
 - [19] AMD. SEV Secure Nested Paging Firmware ABI Specification. Specification, Sept. 2023. Version 1.55.
 - [20] ANGELOGIANNI, A., Krontiris, I., AND GIANNETSOS, T. Comparative Evaluation of PKI and DAA-based Architectures for V2X Communication Security. In *2023 IEEE Vehicular Networking Conference (VNC)* (2023), pp. 199–206.
 - [21] ANJUNA. Anjuna Seaglass. <https://www.anjuna.io/product>. Accessed on June 3, 2024.

- [22] ANTONAKAKIS, M., APRIL, T., BAILEY, M., BERNHARD, M., BURSZEIN, E., COCHRAN, J., DURUMERIC, Z., HALDERMAN, J. A., INVERNIZZI, L., KALLITSIS, M., KUMAR, D., LEVER, C., MA, Z., MASON, J., MENSCHER, D., SEAMAN, C., SULLIVAN, N., THOMAS, K., AND ZHOU, Y. Understanding the Mirai Botnet. In *26th USENIX Security Symposium (USENIX Security 17)* (Aug. 2017), USENIX Association, pp. 1093–1110.
- [23] ANWAR, F. M., GARCIA, L., HAN, X., AND SRIVASTAVA, M. Securing time in untrusted operating systems with timeseal. In *2019 IEEE Real-Time Systems Symposium (RTSS)* (2019), IEEE, pp. 80–92.
- [24] ANWAR, F. M., AND SRIVASTAVA, M. Applications and Challenges in Securing Time. In *12th USENIX Workshop on Cyber Security Experimentation and Test (CSET 19)* (Santa Clara, CA, Aug. 2019), USENIX Association.
- [25] ARM. ARM CCA Security Model 1.0. Security model (sm), Aug. 2021. Version DEN0096.
- [26] ARM. ARM Realm Management Monitor specification. Specification, Mar. 2023. Version DEN0137, 1.0-eac1.
- [27] ARM. Learn the architecture - Introducing Arm Confidential Compute Architecture. Specification, June 2023. Version 3.0.
- [28] ARM-SOFTWARE. Trusted Board Boot. <https://github.com/ARM-software/arm-trusted-firmware/blob/master/docs/design/trusted-board-boot.rst>. Accessed on June 3, 2024.
- [29] AWS. Importing a VM as an image using VM Import/Export. <https://web.archive.org/web/20240427135926/https://docs.aws.amazon.com/vm-import/latest/userguide/vmimport-image-import.html>. Archived on April 27, 2024.
- [30] AWS. NitroTPM on AWS to measure kernel. <https://github.com/aws/uefi/issues/13>. Accessed on June 3, 2024.
- [31] AWS. UEFI. <https://github.com/aws/uefi>. Accessed on June 3, 2024.
- [32] BACCELLI, E., HAHM, O., GÜNES, M., WÄHLISCH, M., AND SCHMIDT, T. C. RIOT OS: Towards an OS for the Internet of Things. In *2013 IEEE Conf. on Computer Communications Workshops (INFOCOM)* (2013), IEEE, pp. 79–80.

- [33] BARBOSA, M., PORTELA, B., SCERRI, G., AND WARINSCHI, B. Foundations of Hardware-Based Attested Computation and Application to SGX. In *2016 IEEE European Symposium on Security and Privacy (Euro S&P)* (Mar. 2016), IEEE Computer Society, pp. 245–260.
- [34] BEERMAN, J., BERENT, D., FALTER, Z., AND BHUNIA, S. A Review of Colonial Pipeline Ransomware Attack. In *2023 IEEE/ACM 23rd International Symposium on Cluster, Cloud and Internet Computing Workshops (CCGridW)* (2023), pp. 8–15.
- [35] BERTONI, G., DAEMEN, J., PEETERS, M., AND ASSCHE, G. V. Duplexing the Sponge: Single-pass authenticated encryption and other applications. In *SAC* (2011), Springer, pp. 320–337.
- [36] BERTSCHI, A., SRIDHARA, S., GROSCHUPP, F., KUHNE, M., SCHLÜTER, B., THORENS, C., DUTLY, N., CAPKUN, S., AND SHINDE, S. Devlore: Extending Arm CCA to Integrated Devices. *arXiv preprint arXiv:2408.05835* (2024).
- [37] BIRKHOLZ, H., FOSSATI, T., PAN, W., AND BORMANN, C. Epoch Markers. Active internet-draft, Internet Engineering Task Force (IETF), Mar. 2023.
- [38] BIRKHOLZ, H., THALER, D., RICHARDSON, M., SMITH, N., AND PAN, W. Remote ATtestation procedureS (RATS) Architecture. Rfc 9334, Internet Engineering Task Force (IETF), Jan. 2023.
- [39] BOGDANOV, A., KNEZEVIC, M., LEANDER, G., TOZ, D., VARICI, K., AND VERBAUWHEDE, I. SPONGENT: The Design Space of Lightweight Cryptographic Hashing. *IEEE Transactions on Computers* 99, PrePrints (2012), 1.
- [40] BOHLING, F., MUELLER, T., ECKEL, M., AND LINDEMANN, J. Subverting Linux’ integrity measurement architecture. In *Proceedings of the 15th International Conference on Availability, Reliability and Security* (2020), ARES ’20, Association for Computing Machinery.
- [41] BRASSER, F., MÜLLER, U., DMITRIENKO, A., KOSTIAINEN, K., CAPKUN, S., AND SADEGHI, A.-R. Software grand exposure: SGX cache attacks are practical. In *11th USENIX WS on Offensive Technologies (WOOT 17)* (2017).
- [42] BRECHT, B., THERRIAULT, D., WEIMERSKIRCH, A., WHYTE, W., KUMAR, V., HEHN, T., AND GOUDY, R. A Security Credential Management System for V2X Communications. *IEEE Transactions on Intelligent Transportation Systems* 19, 12 (2018), 3850–3871.

- [43] BRICKELL, E., CAMENISCH, J., AND CHEN, L. Direct Anonymous Attestation. In *Proceedings of the 11th ACM Conference on Computer and Communications Security* (2004), CCS '04, Association for Computing Machinery, p. 132–145.
- [44] BUSI, M., NOORMAN, J., VAN BULCK, J., GALLETTA, L., DEGANO, P., MÜHLBERG, J. T., AND PIESSENS, F. Securing Interruptible Enclaved Execution on Small Microprocessors. *Transactions on Programming Languages and Systems (TOPLAS)* 43, 3 (2021), 1–77.
- [45] CANELLA, C., VAN BULCK, J., SCHWARZ, M., LIPP, M., VON BERG, B., ORTNER, P., PIESSENS, F., EVTYUSHKIN, D., AND GRUSS, D. A systematic evaluation of transient execution attacks and defenses. In *28th USENIX Security Symposium (USENIX Security 19)* (2019), pp. 249–266.
- [46] CANONICAL. cloud-init. <https://cloud-init.io/>. Accessed on June 3, 2024.
- [47] CAULIGI, S., DISSELKOEN, C., GLEISSENTHALL, K. v., TULLSEN, D., STEFAN, D., REZK, T., AND BARTHE, G. Constant-time foundations for the new spectre era. In *Proceedings of the 41st ACM SIGPLAN Conference on Programming Language Design and Implementation* (2020), PLDI 2020, Association for Computing Machinery, p. 913–926.
- [48] CEN, S., AND ZHANG, B. Trusted time and monotonic counters with intel software guard extensions platform services, 2017.
- [49] CHE TSAI, C., PORTER, D. E., AND VIJ, M. Graphene-SGX: A Practical Library OS for Unmodified Applications on SGX. In *2017 USENIX Annual Technical Conference (USENIX ATC 17)* (July 2017), USENIX Association, pp. 645–658.
- [50] CHEN, Z., VASILAKIS, G., MURDOCK, K., DEAN, E., OSWALD, D., AND GARCIA, F. D. VoltPillager: Hardware-based fault injection attacks against Intel SGX Enclaves using the SVID voltage scaling interface. In *30th USENIX Security Symposium (USENIX Security 21)* (2021), pp. 699–716.
- [51] CHENG, Y., DING, X., AND DENG, R. DriverGuard: A Fine-Grained Protection on I/O Flows. In *ESORICS*, vol. 6879 of *LNCS*. Springer, 2011.
- [52] CLOUD NATIVE COMPUTING FOUNDATION. Kubernetes. <https://kubernetes.io>. Accessed on June 3, 2024.
- [53] COCONUT. COCONUT-SVSM Github Repository. <https://github.com/coconut-svsm/svsm>. Accessed on June 3, 2024.

- [54] COHEN, C., AND EADS, J. Securing the Unseen: Vulnerability Research in Confidential Computing. <https://hardware.io/usa-2023/presentation/securing-the-unseen-vulnerability-research-in-confidential-computing.pdf>. Hardware.io USA 2023. Accessed on November 14, 2024.
- [55] COMPUTING, S. C. 2FA with Time-based One-time Passwords. https://sconedocs.github.io/CAS_session_lang_0_3. Accessed on June 3, 2024.
- [56] CONSORTIUM, C. C. Confidential Computing: Hardware-Based Trusted Execution for Applications and Data, Nov. 2022. Version 1.3.
- [57] CONSTABLE, S., BULCK, J. V., CHENG, X., XIAO, Y., XING, C., ALEXANDROVICH, I., KIM, T., PIESSENS, F., VIJ, M., AND SILBERSTEIN, M. AEX-Notify: Thwarting Precise Single-Stepping Attacks through Interrupt Awareness for Intel SGX Enclaves. In *32nd USENIX Security Symposium (USENIX Security 23)* (Aug. 2023), USENIX Association, pp. 4051–4068.
- [58] COOPER, D., SANTESSON, S., FARRELL, S., BOEYEN, S., HOUSLEY, R., AND POLK, W. Internet X.509 public key infrastructure certificate and certificate revocation list (CRL) profile. RFC 5280, Internet Engineering Task Force (IETF), 2008.
- [59] CORPORATION, I. Asynchronous Enclave Exit Notify and the EDECCSSA User Leaf Function. White Paper, June 2022.
- [60] COSTAN, V., AND DEVADAS, S. Intel SGX Explained. <https://eprint.iacr.org/2016/086>, 2016. Accessed on June 3, 2024.
- [61] CUI, J., YU, J. Z., SHINDE, S., SAXENA, P., AND CAI, Z. SmashEx: Smashing SGX Enclaves Using Exceptions. In *Proceedings of the 2021 ACM SIGSAC Conference on Computer and Communications Security* (2021), CCS '21, Association for Computing Machinery, p. 779–793.
- [62] DANISEVSKIS, J. Android Protected Confirmation: Taking transaction security to the next level. <https://android-developers.googleblog.com/2018/10/android-protected-confirmation.html>. Accessed on June 3, 2024.
- [63] DAVIS, B. D., MASON, J. C., AND ANWAR, M. Vulnerability studies and security postures of IoT devices: A smart home case study. *IEEE Internet of Things Journal* 7, 10 (2020), 10102–10110.
- [64] DEBIAN. Debian 11 (bullseye) statistics. <https://sources.debian.org/stats/bullseye/>. Accessed on June 3, 2024.

- [65] DEHAES, J. Evaluation of revocation schemes in vehicle-to-everything (V2X) applications. Master's thesis, KU Leuven, 2024.
- [66] DENG, Y., WANG, C., YU, S., LIU, S., NING, Z., LEACH, K., LI, J., YAN, S., HE, Z., CAO, J., ET AL. StrongBox: A GPU TEE on Arm Endpoints. In *Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security* (2022), pp. 769–783.
- [67] DESMOULINS, N., DIOP, A., RAFFLÉ, Y., TRAORÉ, J., AND GRATESAC, J. Practical anonymous attestation-based pseudonym schemes for vehicular networks. In *2019 IEEE Vehicular Networking Conference (VNC)* (2019), IEEE, pp. 1–8.
- [68] DOLEV, D., AND YAO, A. On the security of public key protocols. *IEEE Transactions on information theory* 29, 2 (1983), 198–208.
- [69] DOUCEUR, J. R. The sybil attack. In *International workshop on peer-to-peer systems* (2002), Springer, pp. 251–260.
- [70] DOV MURIK AND HUBERTUS FRANKE. Securing Linux VM boot with AMD SEV measurement. https://static.sched.com/hosted_files/kvmforum2021/ed/securing-linux-vm-boot-with-amd-sev-measurement.pdf. Accessed on June 3, 2024.
- [71] EDGELESS SYSTEMS. Constellation. <https://www.edgeless.systems/products/constellation>. Accessed on June 3, 2024.
- [72] EDGELESS SYSTEMS. Continuum AI. <https://www.edgeless.systems/products/continuum>. Accessed on September 4, 2024.
- [73] ENARX. Enarx: Confidential Computing with WebAssembly. <https://github.com/enarx/enarx/>. Accessed on June 3, 2024.
- [74] ENCLAIVE. Buckypaper. <https://www.enclaive.io/buckypaper-confidential-virtual-machines>. Accessed on June 3, 2024.
- [75] ENCLAVE, O. Open Enclave SDK. <https://openenclave.io/sdk>. Accessed on June 3, 2024.
- [76] ESKANDARIAN, S., COGAN, J., BIRNBAUM, S., BRANDON, P. C. W., FRANKE, D., FRASER, F., GARCIA, G., GONG, E., NGUYEN, H. T., SETHI, T. K., SUBBIAH, V., BACKES, M., PELLEGRINO, G., AND BONEH, D. Fidelius: Protecting User Secrets from Compromised Browsers. In *2019 IEEE Symposium on Security and Privacy (SP)* (2019), pp. 264–280.

- [77] ETSI. LTE; 5G; Security aspect for LTE support of Vehicle-to-Everything (V2X) services. Technical Specification (TS) TS 133 185, European Telecommunications Standard Institute (ETSI), July 2017. Version 14.0.0.
- [78] ETSI. Network Functions Virtualisation (NFV); NFV Security; Report on use cases and technical approaches for multi-layer host administration. Group Report (GR) GR NFV-SEC 009, European Telecommunications Standard Institute (ETSI), Jan. 2017. Version 1.2.1.
- [79] ETSI. Intelligent Transport Systems (ITS); Security; Pre-standardization study on pseudonym change management. Technical Report (TR) TR 103 415, European Telecommunications Standard Institute (ETSI), Apr. 2018. Version 1.1.1.
- [80] ETSI. 5G; Service requirements for enhanced V2X scenarios. Technical Specification (TS) TS 122 186, European Telecommunications Standard Institute (ETSI), Nov. 2020. Version 16.2.0.
- [81] ETSI. Intelligent Transport Systems (ITS); Security; Pre-standardization study on Misbehaviour Detection. Technical Report (TR) TR 103 460, European Telecommunications Standard Institute (ETSI), Oct. 2020. Version 2.1.1.
- [82] ETSI. Intelligent Transport Systems (ITS); Security, ITS communications security architecture and security management. Technical Specification (TS) TS 102 940, European Telecommunications Standard Institute (ETSI), July 2021. Version 2.1.1.
- [83] ETSI. Intelligent Transport Systems (ITS); Security; Trust and Privacy Management. Technical Specification (TS) TS 102 941, European Telecommunications Standard Institute (ETSI), Nov. 2022. Version 2.2.1.
- [84] FÖRSTER, D., LÖHR, H., ZIBUSCHKA, J., AND KARGL, F. REWIRE – Revocation Without Resolution: A Privacy-Friendly Revocation Mechanism for Vehicular Ad-Hoc Networks. In *Trust and Trustworthy Computing* (Cham, 2015), Springer International Publishing, pp. 193–208.
- [85] FORTANIX. Fortanix Enclave Development Platform. <https://edp.fortanix.com/>. Accessed on June 3, 2024.
- [86] FORTANIX. Idiomatic Rust wrapper for MbedTLS. <https://github.com/fortanix/rust-mbedtls>. Accessed on June 3, 2024.
- [87] GALANOU, A., BINDLISH, K., PREIBSCH, L., PIGNOLET, Y.-A., FETZER, C., AND KAPITZA, R. Trustworthy confidential virtual machines for the masses. In *Proceedings of the 24th International Middleware Conference* (2023), Middleware ’23, Association for Computing Machinery, p. 316–328.

- [88] GLOBALPLATFORM. Trust & Security in Automotive Systems. *White Paper* (Oct. 2023). <https://globalplatform.org/resource-publication/trust-security-in-automotive-systems/>.
- [89] GOOGLE. gce-tcb-verifier. <https://github.com/google/gce-tcb-verifier>. Accessed on September 4, 2024.
- [90] GOOGLE. Google Asylo. <https://asylo.dev/>. Accessed on June 3, 2024.
- [91] GOOGLE. Verify a Confidential VM instance's firmware. <https://cloud.google.com/confidential-computing/confidential-vm/docs/verify-firmware>. Accessed on November 12, 2024.
- [92] GOOGLE CLOUD. Confidential VM attestation. <https://web.archive.org/web/20240513094653/https://cloud.google.com/confidential-computing/confidential-vm/docs/attestation>. Archived on May 13, 2024.
- [93] GOOGLE CLOUD. Create custom Confidential VM images. <https://web.archive.org/web/20240513095702/https://cloud.google.com/confidential-computing/confidential-vm/docs/create-custom-confidential-vm-images>. Archived on May 13, 2024.
- [94] GOOGLE CLOUD. What is Shielded VM? - Secure Boot. <https://web.archive.org/web/20240425181344/https://cloud.google.com/compute/shielded-vm/docs/shielded-vm#vtpm>. Archived on April 25, 2024.
- [95] GÖTZFRIED, J., ECKERT, M., SCHINZEL, S., AND MÜLLER, T. Cache attacks on Intel SGX. In *Proceedings of the 10th European Workshop on Systems Security* (2017), pp. 1–6.
- [96] GUETTE, G., AND HEEN, O. A TPM-based architecture for improved security and anonymity in vehicular ad hoc networks. In *2009 IEEE Vehicular Networking Conference (VNC)* (2009), pp. 1–7.
- [97] HERMANN, H. Markov Chains. In *Interactive Markov Chains*. Springer, 2002, pp. 35–55.
- [98] HICKS, C., AND GARCIA, F. D. A Vehicular DAA Scheme for Unlinkable ECDSA Pseudonyms in V2X. In *2020 IEEE European Symposium on Security and Privacy (EuroS&P)* (2020), pp. 460–473.
- [99] HOLMES, B., WATERMAN, J., AND WILLIAMS, D. SEVeriFast: Minimizing the root of trust for fast startup of SEV microVMs. ASPLOS '24, Association for Computing Machinery, p. 1045–1060.

- [100] HUND, R., WILLEMS, C., AND HOLZ, T. Practical Timing Side Channel Attacks Against Kernel Space ASLR. In *Symp. S&P* (2013), IEEE.
- [101] (IETF), I. E. T. F. TOTP: Time-Based One-Time Password Algorithm. RFC- Proposed Standard 6238, 2011.
- [102] (IETF), I. E. T. F. OAuth 2.0 Threat Model and Security Considerations. RFC- Informational 6819, 2013.
- [103] INFINEON. OPTIGA TPM - Trusted Platform Module. <https://www.infineon.com/cms/en/product/security-smart-card-solutions/optiga-embedded-security-solutions/optiga-tpm/>. Accessed on June 3, 2024.
- [104] INOUE, Y. Satellite- and drone-based remote sensing of crops and soils for smart farming – a review. *Soil Science and Plant Nutrition* 66, 6 (2020), 798–810.
- [105] INTEL. Virtual TPM in Intel TDX. <https://github.com/intel/vtpm-td>. Accessed on June 3, 2024.
- [106] INTEL. Architecture Specification: Intel® Trust Domain Extensions (Intel® TDX) Module. Specification, June 2022. Version 344425-004US.
- [107] INTEL. Intel TDX Connect Architecture Specification. Specification, Mar. 2023. Version 354629 001-US.
- [108] INTEL. Intel Trust Domain Extensions. White Paper, Feb. 2023.
- [109] INTEL. Intel Trust Domain Extensions (Intel TDX) Module Base Architecture Specification. Specification, Nov. 2023. Version 348549-003US.
- [110] INTEL. Put Zero Trust Within Reach and Get Public Cloud Flexibility with Private Cloud Security. White paper, Sept. 2023. Version 355888-001US.
- [111] INTEL. Intel TDX Module Architecture Specification: TD Migration. Specification, Mar. 2024. Version 348550-004US.
- [112] JANG, I., TANG, A., KIM, T., SETHUMADHAVAN, S., AND HUH, J. Heterogeneous isolated execution for commodity gpus. In *Proceedings of the Twenty-Fourth International Conference on Architectural Support for Programming Languages and Operating Systems* (2019), pp. 455–468.
- [113] JANGID, M. K., AND LIN, Z. Towards A TEE-based V2V Protocol For Connected And Autonomous Vehicles. In *Workshop on Automotive and Autonomous Vehicle Security (AutoSec)* (2022), vol. 2022, p. 27.

- [114] JOHNSON, S., SCARLATA, V., ROZAS, C., BRICKELL, E., AND MCKEEN, F. Intel software guard extensions: EPID provisioning and attestation services. *White Paper 1*, 1-10 (2016), 119.
- [115] KERNEL.ORG. dm-crypt. <https://www.kernel.org/doc/html/latest/admin-guide/device-mapper/dm-crypt.html>. Accessed on June 3, 2024.
- [116] KERNEL.ORG. dm-integrity. <https://docs.kernel.org/admin-guide/device-mapper/dm-integrity.html>. Accessed on June 3, 2024.
- [117] KERNEL.ORG. dm-verity. <https://docs.kernel.org/admin-guide/device-mapper/verity.html>. Accessed on June 3, 2024.
- [118] KLEIN, G., ELPHINSTONE, K., HEISER, G., ANDRONICK, J., COCK, D., DERRIN, P., ELKADUWE, D., ENGELHARDT, K., KOLANSKI, R., NORRISH, M., SEWELL, T., TUCH, H., AND WINWOOD, S. seL4: Formal verification of an OS kernel. In *Proceedings of the ACM SIGOPS 22nd symposium on Operating systems principles* (2009), pp. 207–220.
- [119] KRAWIECKA, K., KURNIKOV, A., PAVERD, A., MANNAN, M., AND ASOKAN, N. Safekeeper: Protecting web passwords using trusted execution environments. In *WWW Conference* (2018).
- [120] KUMAR, V., PETIT, J., AND WHYTE, W. Binary Hash Tree Based Certificate Access Management for Connected Vehicles. In *Proceedings of the 10th ACM Conference on Security and Privacy in Wireless and Mobile Networks* (2017), WiSec '17, Association for Computing Machinery, p. 145–155.
- [121] KUVAISKII, D., STAVRAKAKIS, D., QIN, K., XING, C., BHATOTIA, P., AND VIJ, M. Gramine-TDX: A Lightweight OS Kernel for Confidential VMs. In *Proceedings of the 2024 ACM SIGSAC Conference on Computer and Communications Security (CCS'24)* (2024).
- [122] LARSEN, B., GIANNETSOS, T., Krontiris, I., AND GOLDMAN, K. Direct Anonymous Attestation on the Road: Efficient and Privacy-Preserving Revocation in C-ITS. In *Proceedings of the 14th ACM Conference on Security and Privacy in Wireless and Mobile Networks* (2021), WiSec '21, Association for Computing Machinery, p. 48–59.
- [123] LEE, D., KOHLBRENNER, D., SHINDE, S., ASANOVIĆ, K., AND SONG, D. Keystone: An open framework for architecting trusted execution environments. In *Proc. of the Fifteenth European Conf. on Computer Systems* (2020), pp. 1–16.

- [124] LI, M., WILKE, L., WICHELMANN, J., EISENBARTH, T., TEODORESCU, R., AND ZHANG, Y. A Systematic Look at Ciphertext Side Channels on AMD SEV-SNP. In *2022 IEEE Symposium on Security and Privacy (SP)* (2022), pp. 337–351.
- [125] LI, W., LI, H., CHEN, H., AND XIA, Y. Adattester: Secure online mobile advertisement attestation using trustzone. In *Proceedings of the 13th annual international conf. on mobile systems, applications, and services* (2015), pp. 75–88.
- [126] LI, W., LUO, S., SUN, Z., XIA, Y., LU, L., CHEN, H., ZANG, B., AND GUAN, H. Vbutton: Practical attestation of user-driven operations in mobile apps. In *Proceedings of the 16th annual international conf. on mobile systems, applications, and services* (2018), pp. 28–40.
- [127] LI, W., MA, M., HAN, J., XIA, Y., ZANG, B., CHU, C.-K., AND LI, T. Building trusted path on untrusted device drivers for mobile devices. In *Proceedings of 5th Asia-Pacific WS on Systems* (2014), pp. 1–7.
- [128] LI, X., LI, X., DALL, C., GU, R., NIEH, J., SAIT, Y., AND STOCKWELL, G. Design and Verification of the Arm Confidential Compute Architecture. In *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22)* (2022), pp. 465–484.
- [129] LIANG, H., AND LI, M. Bring the Missing Jigsaw Back: TrustedClock for SGX Enclaves. EuroSec’18.
- [130] LIANG, H., LI, M., CHEN, Y., JIANG, L., XIE, Z., AND YANG, T. Establishing Trusted I/O Paths for SGX Client Systems With Aurora. *IEEE Trans. Inf. Forensics Secur.* 15 (2020), 1589–1600.
- [131] LINARO. OP-TEE Documentation. <https://optee.readthedocs.io/>. Accessed on June 3, 2024.
- [132] LINDELL, Y. Secure multiparty computation. *Commun. ACM* 64, 1 (Dec. 2020), 86–96.
- [133] LIPP, M., KOGLER, A., OSWALD, D., SCHWARZ, M., EASDON, C., CANELLA, C., AND GRUSS, D. PLATYPUS: Software-based Power Side-Channel Attacks on x86. In *2021 IEEE Symposium on Security and Privacy (SP)* (2021), pp. 355–371.
- [134] LIU, D., AND COX, L. P. Veriui: Attested login for mobile devices. In *Proceedings of the 15th workshop on mobile computing systems and applications* (2014), pp. 1–6.

- [135] LIU, F., YAROM, Y., GE, Q., HEISER, G., AND LEE, R. B. Last-Level Cache Side-Channel Attacks are Practical. In *Symp. S&P* (2015), IEEE, pp. 605–622.
- [136] LIU, J., ZHANG, S., SUN, W., AND SHI, Y. In-Vehicle Network Attacks and Countermeasures: Challenges and Future Directions. *IEEE Network* 31, 5 (2017), 50–58.
- [137] LOU, X., ZHANG, T., JIANG, J., AND ZHANG, Y. A survey of microarchitectural side-channel vulnerabilities, attacks, and defenses in cryptography. *ACM Computing Surveys (CSUR)* 54, 6 (2021), 1–37.
- [138] MAENE, P., GÖTZFRIED, J., DE CLERCQ, R., MÜLLER, T., FREILING, F., AND VERBAUWHEDE, I. Hardware-Based Trusted Computing Architectures for Isolation and Attestation. *IEEE Transactions on Computers* 67, 9 (2018), 361–374.
- [139] MAURER, U. Modelling a public-key infrastructure. In *Computer Security — ESORICS 96* (1996), Springer Berlin Heidelberg, pp. 325–350.
- [140] McCUNE, J. M., PARNO, B. J., PERRIG, A., REITER, M. K., AND ISOZAKI, H. Flicker: An Execution Infrastructure for TCB Minimization. In *Eurosys* (2008), ACM, pp. 315–328.
- [141] McCUNE, J. M., PERRIG, A., AND REITER, M. K. Bump in the Ether: A Framework for Securing Sensitive User Input. In *ATEC* (2006), USENIX.
- [142] McCUNE, J. M., PERRIG, A., AND REITER, M. K. Safe Passage for Passwords and Other Sensitive Data. In *Proceedings of the 2009 Network and Distributed System Security (NDSS) Symposium* (2009), vol. 96 of *NDSS’09*, Internet Society, pp. 121–130.
- [143] McKEEN, F., ALEXANDROVICH, I., BERENZON, A., ROZAS, C. V., SHAFI, H., SHANBHOGUE, V., AND SAVAGAONKAR, U. R. Innovative Instructions and Software Model for Isolated Execution. In *HASP* (2013), ACM, pp. 10:1–10:1.
- [144] MEIER, S., SCHMIDT, B., CREMERS, C., AND BASIN, D. The TAMARIN Prover for the Symbolic Analysis of Security Protocols. In *Computer Aided Verification* (2013), Springer Berlin Heidelberg, pp. 696–701.
- [145] MICROSOFT. az_snp_vtpm Rust crate: function get_report_with_report_data. https://docs.rs/az-snp-vtpm/0.7.1/az_snp_vtpm/vtpm/fn.get_report_with_report_data.html. Accessed on November 18, 2024.

- [146] MICROSOFT. OpeVMM and OpenHCL. <https://github.com/microsoft/openvmm>. Accessed on November 18, 2024.
- [147] MICROSOFT AZURE. About Azure confidential VMs - Attestation and TPM. <https://web.archive.org/web/20240513093454/https://learn.microsoft.com/en-us/azure/confidential-computing/confidential-vm-overview#attestation-and-tpm>. Archived on May 13, 2024.
- [148] MICROSOFT AZURE. Azure Linux VM Agent overview. <https://learn.microsoft.com/en-us/azure/virtual-machines/extensions/agent-linux>. Accessed on June 3, 2024.
- [149] MICROSOFT AZURE. DCas_cc_v5 sizes series. <https://web.archive.org/web/20241112142520/https://learn.microsoft.com/en-us/azure/virtual-machines/sizes/general-purpose/dcasccv5-series>. Archived on November 12, 2024.
- [150] MICROSOFT AZURE. How to create a custom image for Azure confidential VMs. <https://web.archive.org/web/20240513095534/https://learn.microsoft.com/en-us/azure/confidential-computing/how-to-create-custom-image-confidential-vm>. Archived on May 13, 2024.
- [151] MICROSOFT AZURE. Virtual TPMs in Azure confidential VMs. <https://web.archive.org/web/20240503012205/https://learn.microsoft.com/en-us/azure/confidential-computing/virtual-tpms-in-azure-confidential-vm>. Archived on May 3, 2024.
- [152] MORBITZER, M., HUBER, M., HORSCH, J., AND WESSEL, S. SEVered: Subverting AMD’s Virtual Machine Encryption. In *Proceedings of the 11th European Workshop on Systems Security* (2018), EuroSec’18, Association for Computing Machinery.
- [153] MÜHLBERG, J. T., CLEEMPUT, S., MUSTAFA, A. M., VAN BULCK, J., PRENEEL, B., AND PIESSENS, F. An Implementation of a High Assurance Smart Meter using Protected Module Architectures. In *WISTP ’16* (Heidelberg, 2016), vol. 9895 of *LNCS*, Springer, pp. 53–69.
- [154] MÜHLBERG, J. T., AND VAN BULCK, J. Tutorial: Building Distributed Enclave Applications with Sancus and SGX. In *DSN ’18* (2018), IEEE.
- [155] MURDOCK, K., OSWALD, D., GARCIA, F. D., VAN BULCK, J., GRUSS, D., AND PIESSENS, F. Plundervolt: Software-based fault injection attacks against Intel SGX. In *2020 IEEE Symposium on Security and Privacy (SP)* (2020), IEEE, pp. 1466–1482.

- [156] NARAYANAN, V., CARVALHO, C., RUOCCO, A., ALMASI, G., BOTTOMLEY, J., YE, M., FELDMAN-FITZTHUM, T., BUONO, D., FRANKE, H., AND BURTSEV, A. Remote attestation of confidential VMs using ephemeral vTPMs. In *Proceedings of the 39th Annual Computer Security Applications Conference* (New York, NY, USA, 2023), ACSAC '23, Association for Computing Machinery, p. 732–743.
- [157] NGABONZIZA, B., MARTIN, D., BAILEY, A., CHO, H., AND MARTIN, S. TrustZone Explained: Architectural Features and Use Cases. *2016 IEEE 2nd Int. Conf. Collab. and Internet Computing (CIC)* (2016), 445–451.
- [158] (NHTSA), N. H. T. S. A. Automated Vehicles for Safety. <https://www.nhtsa.gov/vehicle-safety/automated-vehicles-safety>. Accessed on August 22, 2024.
- [159] NIST. Digital Signature Standard (DSS). Technical Standard FIPS 186-5, National Institute of Standards and Technology (NIST), 2023.
- [160] NIST. The NIST Cybersecurity Framework (CSF) 2.0. Technical Report, National Institute of Standards and Technology (NIST), Feb. 2024.
- [161] NOORMAN, J., AGTEN, P., DANIELS, W., STRACKX, R., VAN HERREWEGE, A., HUYGENS, C., PRENEEL, B., VERBAUWHEDE, I., AND PIESENS, F. Sancus: Low-cost trustworthy extensible networked devices with a zero-software trusted computing base. In *USENIX Sec. Symp.* (2013), USENIX, pp. 479–494.
- [162] NOORMAN, J., MÜHLBERG, J. T., AND PIESENS, F. Authentic Execution of Distributed Event-Driven Applications with a Small TCB. In *STM '17* (Heidelberg, 2017), LNCS, Springer.
- [163] NOORMAN, J., VAN BULCK, J., MÜHLBERG, J. T., PIESENS, F., MAENE, P., PRENEEL, B., VERBAUWHEDE, I., GÖTZFRIED, J., MÜLLER, T., AND FREILING, F. Sancus 2.0: A Low-Cost Security Architecture for IoT Devices. *ACM Transactions on Privacy and Security (TOPS)* 20, 3 (2017), 7:1–7:33.
- [164] NVIDIA. NVIDIA H100 Tensor Core GPU Architecture. White paper, 2023. Version v1.04.
- [165] NXP SEMICONDUCTORS. Automotive-Qualified Embedded Secure Element (SE). <https://www.nxp.com/products/security-and-authentication/secure-car-access/automotive-qualified-embedded-secure-element-se:NCJ38A>. Accessed on June 3, 2024.

- [166] ORBAN, M. Evaluation of Secure Certificate Revocation Schemes in Vehicle-to-Everything (V2X) Applications. Master's thesis, Université Libre de Bruxelles (ULB), 2024.
- [167] PCI-SIG. TEE Device Interface Security Protocol (TDISP). Specification, Aug. 2022. Revision 5.x.
- [168] PEI, M., TSCHOFENIG, H., THALER, D., AND WHEELER, D. Trusted Execution Environment Provisioning (TEEP) Architecture. Rfc 9397, Internet Engineering Task Force (IETF), July 2023.
- [169] PENNEKAMP, J., ALDER, F., BADER, L., SCOPELLITI, G., WEHRLE, K., AND MÜHLBERG, J. T. Securing Sensing in Supply Chains: Opportunities, Building Blocks, and Designs. *IEEE Access* 12 (2024), 9350–9368.
- [170] PETERS, T., LAL, R., VARADARAJAN, S., PAPPACHAN, P., AND KOTZ, D. BASTION-SGX: bluetooth and architectural support for trusted I/O on SGX. In *Proceedings of the 7th International Workshop on Hardware and Architectural Support for Security and Privacy, HASP@ISCA 2018, Los Angeles, CA, USA, June 02-02, 2018* (2018), J. Szefer, W. Shi, and R. B. Lee, Eds., ACM, pp. 3:1–3:9.
- [171] PINTO, S., AND SANTOS, N. Demystifying Arm TrustZone: A Comprehensive Survey. *ACM Computing Surveys (CSUR)* 51, 6 (2019), 1–36.
- [172] QEMU. Direct Linux Boot. <https://qemu-project.gitlab.io/qemu/system/linuxboot.html>. Accessed on June 3, 2024.
- [173] RAGHAVAN, B., NARDI, B., LOVELL, S. T., NORTON, J., TOMLINSON, B., AND PATTERSON, D. J. Computational Agroecology: Sustainable Food Ecosystem Design. In *Proceedings of the 2016 CHI Conference Extended Abstracts on Human Factors in Computing Systems* (May 2016), ACM, pp. 423–435.
- [174] RAJ, H., SAROIU, S., WOLMAN, A., AIGNER, R., COX, J., ENGLAND, P., FENNER, C., KINSHUMANN, K., LÖSER, J., MATTOON, D., NYSTRÖM, M., ROBINSON, D., SPIGER, R., THOM, S., AND WOOTEN, D. fTPM: A Software-Only Implementation of a TPM Chip. In *USENIX Security Symposium* (2016).
- [175] RANDAZZO, A., AND TINNIRELLO, I. Kata Containers: An Emerging Architecture for Enabling MEC Services in Fast and Secure Way. In *2019 Sixth International Conference on Internet of Things: Systems, Management and Security (IOTSMS)* (2019), pp. 209–214.

- [176] RIEBL, R., GÜNTHER, H.-J., FACCHI, C., AND WOLF, L. Artery: Extending Veins for VANET applications. In *2015 International Conference on Models and Technologies for Intelligent Transportation Systems (MT-ITS)* (2015), pp. 450–456.
- [177] SAID MOHAMED, E., BELAL, A., KOTB ABD-ELMABOD, S., EL-SHIRBENY, M. A., GAD, A., AND ZAHRAN, M. B. Smart farming for improving agricultural management. *The Egyptian Journal of Remote Sensing and Space Science* 24, 3, Part 2 (2021), 971–981.
- [178] SAILER, R., ZHANG, X., JAEGER, T., AND VAN DOORN, L. Design and implementation of a TCG-based integrity measurement architecture. In *USENIX Security symposium* (2004), vol. 13, pp. 223–238.
- [179] SASY, S., GORBUNOV, S., AND FLETCHER, C. W. ZeroTrace : Oblivious Memory Primitives from Intel SGX. In *Proceedings of the 2018 Network and Distributed System Security (NDSS) Symposium* (2018), NDSS’18, Internet Society.
- [180] SCHLUTER, B., SRIDHARA, S., BERTSCHI, A., AND SHINDE, S. WeSee: Using Malicious #VC Interrupts to Break AMD SEV-SNP . In *2024 IEEE Symposium on Security and Privacy (SP)* (May 2024), IEEE Computer Society, pp. 4220–4238.
- [181] SCHLÜTER, B., SRIDHARA, S., KUHNE, M., BERTSCHI, A., AND SHINDE, S. Heckler: Breaking Confidential VMs with Malicious Interrupts. In *33th USENIX Security Symposium (USENIX Security 24)* (2024).
- [182] SCHNEIDER, M., MASTI, R. J., SHINDE, S., CAPKUN, S., AND PEREZ, R. Sok: Hardware-supported trusted execution environments. *arXiv preprint arXiv:2205.12742* (2022).
- [183] SCOPELLITI, G. Securing Smart Environments with Authentic Execution. Master’s thesis, Politecnico Di Torino, 2020. <https://distrinet.cs.kuleuven.be/software/sancus/publications/scopelliti2020.pdf> Accessed on June 3, 2024.
- [184] SCOPELLITI, G., BAUMANN, C., ALDER, F., TRUYEN, E., AND MUEHLBERG, J. T. Artifacts for Efficient and Timely Revocation of V2X Credentials. <https://doi.org/10.5281/zenodo.10210425>, Nov. 2023.
- [185] SCOPELLITI, G., BAUMANN, C., ALDER, F., TRUYEN, E., AND MÜHLBERG, J. T. Demo: Efficient and Timely Revocation of V2X Credentials. In *Proceedings of the Symposium on Vehicles Security and Privacy (VehicleSec) 2024* (2024), VehicleSec’24, Internet Society.

- [186] SCOPELLITI, G., AND POUYANRAD, S. An Open-Source Framework for Developing Heterogeneous Distributed Enclave Applications. https://archive.fosdem.org/2021/schedule/event/tee_sancus/, 2021. Accessed on June 3, 2024.
- [187] SCOPELLITI, G., POUYANRAD, S., NOORMAN, J., ALDER, F., PIESSENS, F., AND MÜHLBERG, J. T. POSTER: An Open-Source Framework for Developing Heterogeneous Distributed Enclave Applications. In *Proceedings of the 2021 ACM SIGSAC Conference on Computer and Communications Security* (2021), CCS '21, Association for Computing Machinery, p. 2393–2395.
- [188] SEDAR, R., KALALAS, C., VÁZQUEZ-GALLEGO, F., ALONSO, L., AND ALONSO-ZARATE, J. A Comprehensive Survey of V2X Cybersecurity Mechanisms and Future Research Paths. *IEEE Open Journal of the Communications Society* 4 (2023), 325–391.
- [189] SHA, Z., SHEPHERD, C., RAFI, A., AND MARKANTONAKIS, K. Control-Flow Attestation: Concepts, Solutions, and Open Challenges. *arXiv preprint arXiv:2408.06304* (2024).
- [190] SHEN, C., CHEN, C., AND ZHANG, J. Micro-architectural Cache Side-Channel Attacks and Countermeasures. In *Proceedings of the 26th Asia and South Pacific Design Automation Conference* (2021), ASPDAC '21, Association for Computing Machinery, p. 441–448.
- [191] SHIRAZI, F., SIMEONOVSKI, M., ASGHAR, M. R., BACKES, M., AND DIAZ, C. A Survey on Routing in Anonymous Communication Protocols. *ACM Comput. Surv.* 51, 3 (June 2018).
- [192] SIGNAL. Technology Deep Dive: Building a Faster ORAM Layer for Enclaves. <https://signal.org/blog/building-faster-oram/>. Accessed on November 14, 2024.
- [193] SIMPLICIO, M. A., COMINETTI, E. L., KUPWADE PATIL, H., RICARDINI, J. E., AND SILVA, M. V. M. ACPC: Efficient revocation of pseudonym certificates using activation codes. *Ad Hoc Networks* 90 (2019), 101708.
- [194] SRIDHARA, S., BERTSCHI, A., SCHLÜTER, B., KUHN, M., ALIBERTI, F., AND SHINDE, S. ACAI: Protecting Accelerator Execution with Arm Confidential Computing Architecture. In *33rd USENIX Security Symposium (USENIX Security 24)* (Aug. 2024), USENIX Association, pp. 3423–3440.
- [195] SRIDHARA, S., BERTSCHI, A., SCHLÜTER, B., AND SHINDE, S. SIGY: Breaking Intel SGX Enclaves with Malicious Exceptions & Signals. *arXiv preprint arXiv:2404.13998* (2024).

- [196] STEFANOV, E., DIJK, M. V., SHI, E., CHAN, T.-H. H., FLETCHER, C., REN, L., YU, X., AND DEVADAS, S. Path ORAM: An Extremely Simple Oblivious RAM Protocol. *J. ACM* 65, 4 (Apr. 2018).
- [197] STRACKX, R., NOORMAN, J., VERBAUWHEDE, I., PRENEEL, B., AND PIESENS, F. Protected software module architectures. In *ISSE 2013 Securing Electronic Business Processes* (2013), Springer, pp. 241–251.
- [198] STREED, A., KANTAR, M., TOMLINSON, B., AND RAGHAVAN, B. How Sustainable is the Smart Farm? In *Seventh Workshop on Computing within Limits 2021* (June 2021), LIMITS.
- [199] SUN, H., SUN, K., WANG, Y., AND JING, J. TrustOTP: Transforming smartphones into secure one-time password tokens. In *Proceedings of the 22nd ACM SIGSAC Conference on Computer and Communications Security* (2015), pp. 976–988.
- [200] SYSTEMS, E. Marblerun: The control plane for confidential computing. <https://marblerun.sh/>. Accessed on June 3, 2024.
- [201] TCG. Trusted Platform Module Library Part 1: Architecture. Technical Documentation Level 00 Revision 01.59, Trusted Computing Group (TCG), Nov. 2019.
- [202] TOM DOHRMANN. Mushroom: Run integrity protected workloads in a hardware based Trusted Execution Environment. <https://github.com/Freax13/mushroom>. Accessed on November 15, 2024.
- [203] TRUSTED COMPUTING GROUP. TCG Vehicle Services Working Group. <https://trustedcomputinggroup.org/work-groups/vehicle-services/>. Accessed on June 3, 2024.
- [204] TRUSTED COMPUTING GROUP. TCG EFI Platform Specification. Specification, Jan. 2014. Version 1.22.
- [205] VAN BULCK, J., MINKIN, M., WEISSE, O., GENKIN, D., KASIKCI, B., PIESENS, F., SILBERSTEIN, M., WENISCH, T. F., YAROM, Y., AND STRACKX, R. Foreshadow: Extracting the Keys to the Intel {SGX} Kingdom with Transient {Out-of-Order} Execution. In *27th USENIX Security Symposium (USENIX Security 18)* (2018), pp. 991–1008.
- [206] VAN BULCK, J., MÜHLBERG, J. T., AND PIESENS, F. VulCAN: Efficient Component Authentication and Software Isolation for Automotive Control Networks. In *ACSAC '17* (New York, NY, USA, 2017), ACM, pp. 225–237.

- [207] VAN BULCK, J., PIESENS, F., AND STRACKX, R. SGX-Step: A Practical Attack Framework for Precise Enclave Execution Control. In *Proceedings of the 2nd Workshop on System Software for Trusted Execution* (2017), SysTEX'17, Association for Computing Machinery.
- [208] VAN BULCK, J., PIESENS, F., AND STRACKX, R. Nemesis: Studying microarchitectural timing leaks in rudimentary CPU interrupt logic. In *Proceedings of the 2018 ACM SIGSAC Conf, on Computer and Comm. Security* (2018), pp. 178–195.
- [209] VAN SCHAIK, S., KWONG, A., GENKIN, D., AND YAROM, Y. SGAXe: How SGX fails in practice. <https://sgaxe.com/files/SGAXe.pdf>, 2020. Accessed on June 3, 2024.
- [210] VAN SCHAIK, S., SETO, A., YUREK, T., BATORI, A., ALBASSAM, B., GENKIN, D., MILLER, A., RONEN, E., YAROM, Y., AND GARMAN, C. SoK: SGX.Fail: How Stuff Gets eXposed. In *2024 IEEE Symposium on Security and Privacy (SP)* (2024), pp. 4143–4162.
- [211] VERHEUL, E., HICKS, C., AND GARCIA, F. D. IFAL: Issue First Activate Later Certificates for V2X. In *2019 IEEE European Symposium on Security and Privacy (EuroS&P)* (2019), pp. 279–293.
- [212] VOLOS, S., VASWANI, K., AND BRUNO, R. Graviton: Trusted Execution Environments on GPUs. In *OSDI* (2018), pp. 681–696.
- [213] WAGAN, A. A., MUGHAL, B. M., AND HASBULLAH, H. VANET Security Framework for Trusted Grouping Using TPM Hardware. In *2010 Second International Conference on Communication Software and Networks* (2010), pp. 309–312.
- [214] WANG, J., LI, A., LI, H., LU, C., AND ZHANG, N. RT-TEE: Real-time System Availability for Cyber-physical Systems using ARM TrustZone. In *2022 IEEE Symposium on Security and Privacy* (2022).
- [215] WANG, Q., GAO, D., AND CHEN, D. Certificate Revocation Schemes in Vehicular Networks: A Survey. *IEEE Access* 8 (2020), 26223–26234.
- [216] WEISER, S., AND WERNER, M. SGXIO: Generic Trusted I/O Path for Intel SGX. In *Proceedings of the Seventh ACM on Conference on Data and Application Security and Privacy* (2017), Association for Computing Machinery, pp. 261–268.
- [217] WEISSE, O., VAN BULCK, J., MINKIN, M., GENKIN, D., KASIKCI, B., PIESENS, F., SILBERSTEIN, M., STRACKX, R., WENISCH, T. F., AND YAROM, Y. Foreshadow-NG: Breaking the virtual memory abstraction with transient out-of-order execution. Technical report, 2018.

- [218] WHITEFIELD, J., CHEN, L., GIANNETSOS, T., SCHNEIDER, S., AND TREHARNE, H. Privacy-enhanced capabilities for VANETs using direct anonymous attestation. In *2017 IEEE Vehicular Networking Conference (VNC)* (2017), IEEE, pp. 123–130.
- [219] WHITEFIELD, J., CHEN, L., KARGL, F., PAVERD, A., SCHNEIDER, S., TREHARNE, H., AND WESEMEYER, S. Formal Analysis of V2X Revocation Protocols. In *Security and Trust Management* (Cham, 2017), G. Livraga and C. Mitchell, Eds., Springer International Publishing, pp. 147–163.
- [220] WICHELMANN, J., RABICH, A., PATSCHKE, A., AND EISENBARTH, T. Obelix: Mitigating Side-Channels Through Dynamic Obfuscation . In *2024 IEEE Symposium on Security and Privacy (SP)* (May 2024), IEEE Computer Society, pp. 4182–4199.
- [221] WILKE, L., AND SCOPELLITI, G. SNPGuard: Remote Attestation of SEV-SNP VMs Using Open Source Tools. In *Proceedings of the 7th Workshop on System Software for Trusted Execution* (2024), SysTEX '24, IEEE.
- [222] WILKE, L., SIECK, F., AND EISENBARTH, T. TDXdown: Single-Stepping and Instruction Counting Attacks against Intel TDX. In *Proceedings of the 2024 ACM SIGSAC Conference on Computer and Communications Security (CCS'24)* (2024).
- [223] WILKE, L., WICHELMANN, J., RABICH, A., AND EISENBARTH, T. SEV-Step A Single-Stepping Framework for AMD-SEV. *IACR Transactions on Cryptographic Hardware and Embedded Systems 2024*, 1 (Dec. 2024).
- [224] WINDERIX, H., MÜHLBERG, J. T., AND PIESSENS, F. Compiler-Assisted Hardening of Embedded Software Against Interrupt Latency Side-Channel Attacks. In *EuroS&P '21* (2021), IEEE, pp. 667 – 682.
- [225] WU, L., CAI, H., AND LI, H. SGX-UAM: A secure unified access management scheme with one time passwords via intel sgx. *IEEE Access* 9 (2021), 38029–38042.
- [226] XU, Y., CUI, W., AND PEINADO, M. Controlled-Channel Attacks: Deterministic Side Channels for Untrusted Operating Systems. In *2015 IEEE Symposium on Security and Privacy* (2015), pp. 640–656.
- [227] YEONG JIN, J. *Building trust in the user I/O in computer systems*. PhD thesis, Georgia Institute of Technology, 2017.

- [228] YING, K., AHLAWAT, A., ALSHARIFI, B., JIANG, Y., THAVAI, P., AND DU, W. Truz-droid: Integrating trustzone with mobile operating system. In *Proceedings of the 16th annual international conference on mobile systems, applications, and services* (2018), pp. 14–27.
- [229] YING, K., THAVAI, P., AND DU, W. Truz-view: Developing trustzone user interface for mobile os using delegation integration model. In *Proc. 9th ACM Conf. on Data and Application Security and Privacy* (2019), pp. 1–12.
- [230] YOSHIZAWA, T., SINGELÉE, D., MÜHLBERG, J. T., DELBRUEL, S., TAHERKORDI, A., HUGHES, D., AND PRENEEL, B. A Survey of Security and Privacy Issues in V2X Communication Systems. *ACM Computing Surveys* (Aug. 2022), 3558052.
- [231] ZHENG, H., AND ARDEN, O. Secure Distributed Applications the Decent Way. In *Proceedings of the 2021 International Symposium on Advanced Security on Software and Systems* (2021), pp. 29–42.
- [232] ZHENG, X., YANG, L., MA, J., SHI, G., AND MENG, D. TrustPAY: Trusted mobile payment on security enhanced ARM TrustZone platforms. In *2016 IEEE Symposium on Computers and Comm. (ISCC)* (2016), IEEE, pp. 456–462.
- [233] ZHOU, Z., GLIGOR, V., NEWSOME, J., AND MCCUNE, J. Building Verifiable Trusted Path on Commodity x86 Computers. In *Proc. IEEE Symposium on Security and Privacy* (May 2012), pp. 616–630.
- [234] ZHOU, Z., YU, M., AND GLIGOR, V. Dancing with Giants: Wimpy Kernels for On-Demand I/O Isolation. *Proc. IEEE Symposium on Security and Privacy 13* (Mar. 2015), 38–46.
- [235] ZHU, J., HOU, R., WANG, X., WANG, W., CAO, J., ZHAO, B., WANG, Z., ZHANG, Y., YING, J., ZHANG, L., ET AL. Enabling rack-scale confidential computing using heterogeneous trusted execution environment. In *2020 IEEE Symposium on Security and Privacy (S&P)* (2020), IEEE, pp. 1450–1465.

Statement on the use of Generative AI

I did not use generative AI assistance tools during the research/writing process of my thesis, except for mere language assistance.

The text, code, and images in this thesis are my own (unless otherwise specified). Generative AI has only been used in accordance with the KU Leuven guidelines and appropriate references have been added. I have reviewed and edited the content as needed and I take full responsibility for the content of the thesis.

Curriculum Vitae

Gianluca Scopelliti was born on 5th of September 1993 in Parma, Italy. From 2015 to 2020 he studied Computer Engineering at Politecnico di Torino, where he obtained first the bachelor's degree in 2018 and then the master's degree in 2020, both degrees obtained with the maximum grade (110 with Laude). During the academic year 2019-2020, he spent an exchange year in Belgium at KU Leuven, part of the Erasmus+ mobility program. There, he pursued his master's thesis with the DistriNet group with title "Securing Smart Environments with Authentic Execution", under the supervision of Frank Piessens, Jan Tobias Mühlberg and Fritz Alder. After completing his master's degree, he started his Ph.D. with the MSCA ITN EID project 5GhOSTS, a collaboration between Ericsson Research (Sweden) and KU Leuven. Thus, between 2020 and 2024 he spent approximately half the time at Ericsson premises and half the time with the DistriNet group at KU Leuven.

List of Publications

Peer-reviewed international conference papers

- **G. Scopelliti**, C. Baumann and J. T. Mühlberg. “Understanding Trust Relationships in Cloud-Based Confidential Computing”. In: *7th Workshop on System Software for Trusted Execution (SysTEX’24)*. 2024
- L. Wilke and **G. Scopelliti**. “SNPGuard: Remote Attestation of SEV-SNP VMs Using Open Source Tools”. In: *7th Workshop on System Software for Trusted Execution (SysTEX’24)*. 2024
- **G. Scopelliti**, C. Baumann, F. Alder, E. Truyen and J. T. Mühlberg. “Efficient and Timely Revocation of V2X Credentials”. In: *Network and Distributed System Security (NDSS) Symposium 2024*. Internet Society. 2024
- F. Alder, **G. Scopelliti**, J. Van Bulck and J. T. Mühlberg. “About Time: On the Challenges of Temporal Guarantees in Untrusted Environments”. In: *6th Workshop on System Software for Trusted Execution (SysTEX’23)*. 2023

Peer-reviewed international journal articles

- J. Pennekamp, F. Alder, L. Bader, **G. Scopelliti**, K. Wehrle and J. T. Mühlberg. “Securing Sensing in Supply Chains: Opportunities, Building Blocks, and Designs”. In: *IEEE Access*. Volume 12, 2024
- **G. Scopelliti**, S. Pouyanrad, J. Noorman, F. Alder, C. Baumann, F. Piessens and J. T. Mühlberg. “End-to-End Security for Distributed Event-Driven Enclave Applications on Heterogeneous TEEs”. In: *ACM*

Transactions on Privacy and Security (TOPS). Volume 26, Issue 3, pp. 1–46. 2023

Peer-reviewed international posters and demos

- **G. Scopelliti**, C. Baumann, F. Alder, E. Truyen and J. T. Mühlberg. “Demo: Efficient and Timely Revocation of V2X Credentials”. In: *2nd Symposium on Vehicles Security and Privacy (VehicleSec’24)*. 2024
- **G. Scopelliti**, S. Pouyanrad, J. Noorman, F. Alder, C. Baumann, F. Piessens and J. T. Mühlberg. “POSTER: An Open-Source Framework for Developing Heterogeneous Distributed Enclave Applications”. In: *2021 ACM SIGSAC Conference on Computer and Communications Security (CCS’21)*. 2021

FACULTY OF ENGINEERING SCIENCE
DEPARTMENT OF COMPUTER SCIENCE
Celestijnenlaan 200A box 2402
B-3001 Leuven

